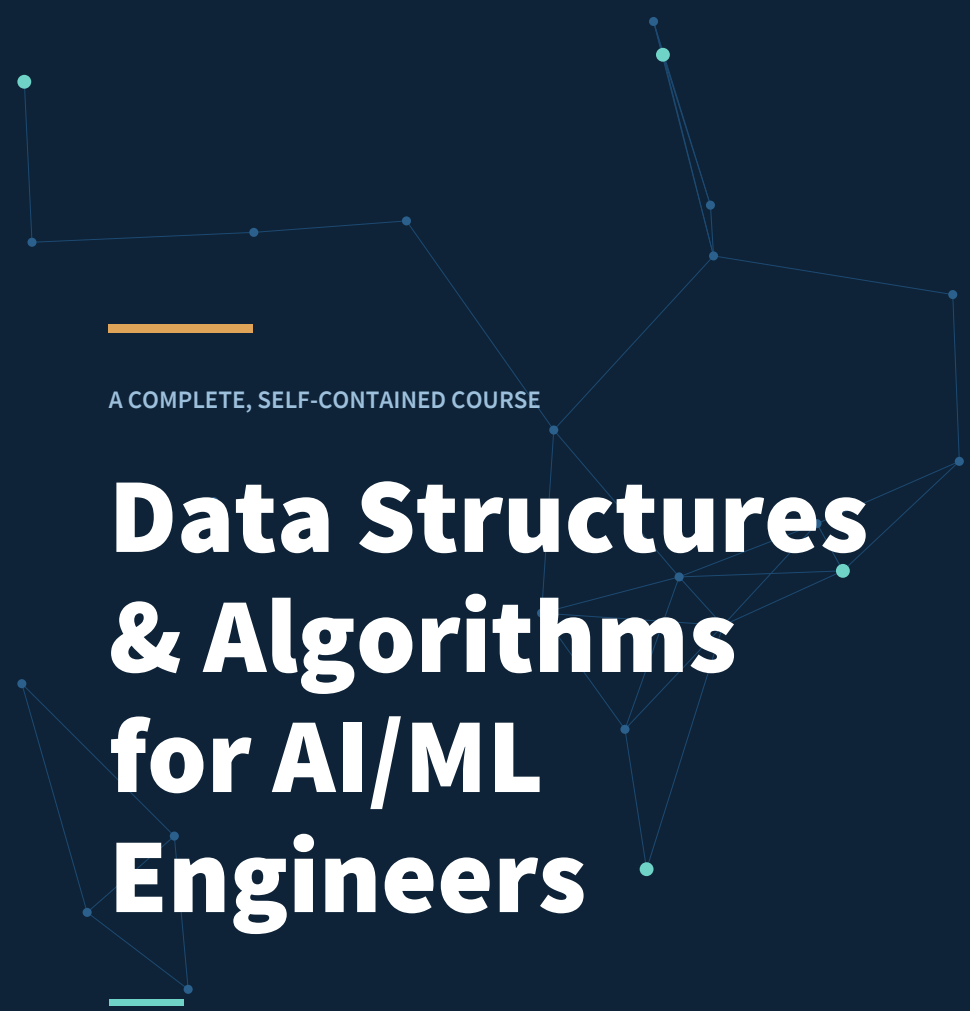
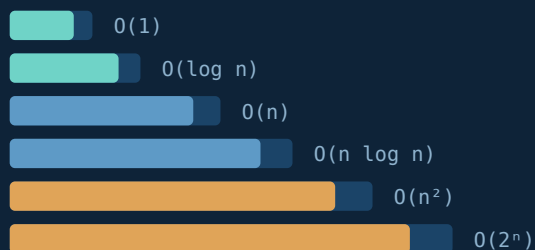




A COMPLETE, SELF-CONTAINED COURSE

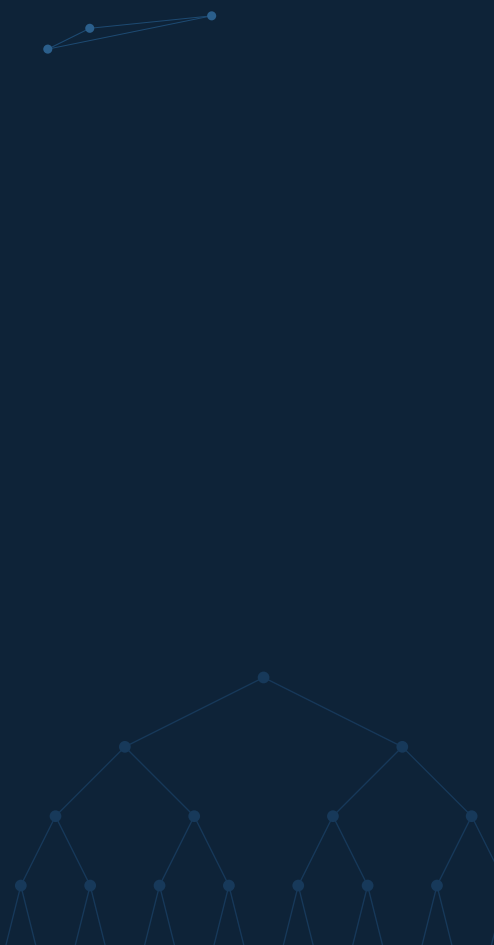
Data Structures & Algorithms for AI/ML Engineers

From first principles to production systems
Forty-seven chapters, from counting operations
to serving a trillion-parameter model.



Written for practitioners who train and serve models

First edition · 2026



Data Structures & Algorithms for AI/ML Engineers

From first principles to production systems
Forty-seven chapters, from counting operations
to serving a trillion-parameter model.

Most algorithms books teach you to pass an interview. This one teaches you to make a training run finish, a retrieval index fit in memory, and a decoder answer in under a second — and, along the way, to pass the interview anyway.

Written for practitioners who train and serve models

First edition · 2026

How to Read This Book

This is a long book, and you should not read it the way you read a novel. What follows is an honest account of who it is for, how it is organised, and how to extract the most from it per hour spent.

Who this is for

You write code that touches machine learning. Maybe you train models, maybe you serve them, maybe you build the pipelines that feed them. You are comfortable in Python. You know what a list and a dictionary are. Perhaps you have never formally studied algorithms; perhaps you did, years ago, and it has faded into a vague memory of Big-O notation and a red-black tree you never used again.

The premise of this book is that the second group is larger than anyone admits, and that both groups are being underserved. General algorithms textbooks are rigorous but speak a different dialect: their examples are schedulers and compilers, not dataloaders and vector indexes. Interview-prep books are practical but shallow: they teach you to recognise a pattern, not to reason about a machine. Meanwhile the actual algorithmic content of modern machine learning — paged attention, product quantisation, ring all-reduce, histogram-based tree splitting, speculative decoding — appears in neither.

KEY INSIGHT | What this book assumes

Python fluency, comfort with arrays and dictionaries, and the ability to read a mathematical expression without panic. It does **not** assume a computer science degree, prior exposure to formal proofs, or knowledge of C++. Where the book needs mathematics, it derives it.

The shape of the book

The forty-seven chapters are arranged in nine parts, and the difficulty is genuinely monotonic. Part I is about counting operations and understanding what a memory hierarchy is; Part VIII is about designing algorithms that saturate a thousand GPUs. A reader who starts at Chapter 1 with no background should be able to reach Chapter 47 without ever hitting a wall.

- **Parts I–III** build the foundation: cost models, memory, arrays, hashing, sorting, searching, trees, heaps. If you have never studied algorithms, this is your course. If you have, read Chapters 3 and 4 anyway — almost nobody was taught the memory-hierarchy story properly, and it silently governs everything else.
- **Parts IV–VI** are the classical algorithmic core, taught with an ML accent: dynamic programming through the lens of dynamic time warping and Viterbi decoding, graphs through the lens of autograd and GNN sampling, sketches through the lens of deduplicating a web-scale corpus.
- **Part VII** is the part you cannot find elsewhere: the algorithms inside the systems you use every day. Vector search, samplers, decoders, KV caches, gradient-boosted trees.

- **Part VIII** takes the whole toolkit to parallel and distributed hardware, where the cost model changes shape entirely.
- **Part IX** is for consolidation: a problem-solving method, a catalogue of patterns, a graded problem set with complete solutions, and a study plan.

How each chapter works

Every chapter follows the same rhythm, and knowing it lets you skim deliberately rather than accidentally.

Element	What it is	How to use it
Opening blurb and objectives	The chapter's promise	Read first; return to it to self-test
Main exposition	Idea, then mechanism, then code	Read linearly; do not skip the derivations
KEY INSIGHT boxes	The one sentence worth memorising	If you remember nothing else, remember these
WHERE THIS SHOWS UP IN ML	The concrete production connection	Skim on first pass, study on second
COMMON PITFALL	Mistakes practitioners actually make	Read every one; they are cheap experience
PERFORMANCE NOTE	Constants, cache behaviour, real numbers	Read if you ship code
Figures	Mechanism made visible	Trace them with a finger; they repay it
EXERCISES	Graded practice, hardest last	Do at least the first three
CHAPTER RECAP	Compressed summary	Use for review; never as a substitute

Three ways to read it

NOTE | Choose your path

The course (14–16 weeks). Chapters 1 to 47 in order, three to five hours a week, doing the exercises. This is what the book is optimised for, and Chapter 47 contains a week-by-week plan.

The interview sprint (5–6 weeks). Chapters 2, 4, 6–13, 16–20, 22–24, then Part IX. Skip Parts VI–VIII on the first pass. Do every exercise marked with a dagger.

The reference. Read Chapter 2 and Chapter 3 once, properly. After that, jump to whichever chapter names the thing you are fighting. Each chapter is written to survive being read alone, and cross-references are explicit.

About the code

Every implementation in this book is real, runnable Python. It is written for clarity first, but never at the cost of being wrong about complexity: where the clear version is asymptotically worse than the production version, the book says so and shows both. Type hints are used where they clarify and omitted where they only add noise.

Python is not a fast language, and some of the structures here would be written in C++ or CUDA in a real system. That is not a reason to avoid Python for teaching: the algorithm is the idea, and the idea transfers. Where the constant factor matters enough to change the decision — and it often does — the book gives you the measured numbers, not hand-waving.

COMMON PITFALL | The trap of reading without writing

Algorithms are a motor skill. Reading a description of a Fenwick tree produces a warm feeling of comprehension that evaporates within days. Typing one out, breaking it, and fixing it produces a memory that lasts years. Budget half your time for the keyboard.

A note on rigour

This book proves things when the proof teaches you something and states results plainly when it does not. You will find a full amortised analysis of the dynamic array, because that analysis is a reusable way of thinking. You will not find a proof that comparison sorting requires $\Omega(n \log n)$ comparisons — you will find the information-theoretic argument in two paragraphs, which is the part you can actually reuse.

Where a claim depends on hardware, the book names the hardware. Where a constant matters, it gives an order of magnitude. Where the honest answer is “measure it”, it says so.

Contents at a Glance

How to Read This Book	iii
Part I — Foundations: The Cost of Everything	1
1. Why an ML Engineer Needs Algorithms	2
2. Complexity: Counting What Actually Costs You	9
3. Memory, Cache and the Machine You Actually Run On	19
4. Arrays, Strides and the NumPy Memory Model	27
5. Strings, Bytes and Tokens	35
Part II — The Core Toolkit	42
6. Linked Structures: Lists, Stacks, Queues, Ring Buffers	43
7. Hashing: Dictionaries, Sets and the Hashing Trick	51
8. Recursion, Memoisation and the Call Stack	59
9. Sorting and Order Statistics	65
10. Binary Search and Searching the Answer Space	73
11. Two Pointers and Sliding Windows	79
Part III — Hierarchies and Priorities	86
12. Trees: Binary, Balanced, and B-Trees	87
13. Heaps, Priority Queues and Top-k	94
14. Tries, Automata and the Anatomy of a Tokenizer	101
15. Union-Find and the Algebra of Merging	109
Part IV — Algorithm Design Paradigms	114
16. Divide and Conquer	115
17. Greedy Algorithms	121

18. Dynamic Programming I: The Method	127
19. Dynamic Programming II: Sequence Algorithms in Machine Learning	134
20. Backtracking and Combinatorial Search	142
21. Randomised Algorithms and Reservoir Sampling	148
Part V — Graphs	154
22. Graph Representations and Traversal	155
23. Shortest Paths, Minimum Spanning Trees and A	162
24. DAGs: Topological Order, Autograd and Computation Graphs	168
25. Flows, Matchings and Assignment	174
26. Graphs in Machine Learning	180
Part VI — Data Structures for Scale	187
27. Segment Trees and Fenwick Trees	188
28. Probabilistic Structures: Bloom, Count-Min, HyperLogLog	195
29. MinHash, SimHash and LSH: Deduplicating a Training Corpus	202
30. Suffix Structures, Substring Search and Contamination	209
31. On-Disk Structures: B-Trees, LSM Trees and Feature Stores	215
Part VII — Algorithms at the Heart of Modern ML	222
32. Nearest-Neighbour Search and the Curse of Dimensionality	223
33. Vector Search in Production: IVF, PQ, HNSW, DiskANN	230
34. Sampling Algorithms	238
35. Decoding Algorithms	245
36. KV Caches as Data Structures	252
37. Tree Learners: Decision Trees, Histograms and GBDT	259
38. Streaming and Online Algorithms	265
Part VIII — Parallel, Distributed and Hardware-Aware	272

39. Parallel Primitives: Scan, Reduce, Sort	273
40. GPU Algorithm Design	278
41. Distributed Algorithms: All-Reduce, Sharding, Consistent Hashing	284
42. External Memory and the Data Loading Pipeline	290
Part IX — Mastery	296
43. A Problem-Solving Framework	297
44. The Pattern Catalogue	303
45. Problem Set I: Foundations	310
46. Problem Set II: Advanced and ML-Flavoured	327
47. Study Plan and Reference	340

Contents

How to Read This Book	iii
------------------------------	------------

Part I — Foundations: The Cost of Everything	1
---	----------

1. Why an ML Engineer Needs Algorithms	2
---	----------

A story that happens every year	2
Where the algorithms actually are	3
Why asymptotics start mattering later than you think, and then all at once	5
The four failure modes	5
What “world class” means here	6
Setting up	7

2. Complexity: Counting What Actually Costs You	9
--	----------

The question complexity answers	9
Counting without counting	10
The three notations, stated properly	10
The hierarchy you must know cold	12
Reading complexity off code	13
Best, average and worst case	14
Amortised analysis: the dynamic array	14
Space complexity	15
The complexity of Python itself	16
What complexity hides, and when that matters	16
A worked example: three ways to find the top k	17

3. Memory, Cache and the Machine You Actually Run On	19
---	-----------

The lie in the RAM model	19
Cache lines: the unit that actually moves	20
Three principles	22
Why linked lists lose	22
Array of structs versus struct of arrays	23
Measuring it yourself	24
Arithmetic intensity and the roofline	24
A checklist you can apply today	25
4. Arrays, Strides and the NumPy Memory Model	27
The array, precisely	27
The dynamic array from scratch	28
Two dimensions: shape, strides, dtype	29
Views versus copies	29
Fancy indexing, masks, and the argsort family	30
Broadcasting is a stride trick	31
Contiguity, and when to pay for it	32
The patterns that show up everywhere	32
5. Strings, Bytes and Tokens	35
Four different things called “length”	35
The cost of string operations	36
Rolling hashes: fingerprinting a window in $\Theta(1)$	37
What a tokenizer actually is	38
Normalisation, the unglamorous prerequisite	40
Part II — The Core Toolkit	42
6. Linked Structures: Lists, Stacks, Queues, Ring Buffers	43

The linked list, done once	43
Stacks and queues: the same array, different ends	45
Converting recursion to iteration	46
The ring buffer	46
The LRU cache: hash map plus doubly linked list	48
Monotonic stacks and queues	49
7. Hashing: Dictionaries, Sets and the Hashing Trick	51
What a hash table is	51
Separate chaining	52
Open addressing	53
Python's dict, concretely	54
Hashing your own objects	54
The hashing trick	55
Deduplication, the canonical application	56
Consistent hashing, in brief	57
8. Recursion, Memoisation and the Call Stack	59
Three parts of a recursion	59
Recursion trees	60
Analysing recursive cost	61
The call stack, and its limits	61
Recursion patterns worth having at hand	62
From memoisation to tabulation	63
9. Sorting and Order Statistics	65
Sorting is preprocessing	65
The comparison lower bound	66
The three classical algorithms	66

Stability, and why ranking depends on it	68
Escaping the lower bound	68
When you do not need a full sort	69
Timsort, and what your language actually runs	70
10. Binary Search and Searching the Answer Space	73
The algorithm, written correctly	73
The two boundary variants	74
The real idea: binary search over answers	75
Exponential (galloping) search	77
When not to binary search	77
11. Two Pointers and Sliding Windows	79
The observation	79
Sliding windows	80
Windows with a counter	82
Fixed windows and streaming statistics	82
Choosing between the patterns	83
Part III — Hierarchies and Priorities	86
12. Trees: Binary, Balanced, and B-Trees	87
The binary search tree	87
Traversals	88
Balance is the whole problem	89
B-trees: branching for the memory hierarchy	91
Choosing between tree, hash and sorted array	91
13. Heaps, Priority Queues and Top-k	94

A tree with no pointers	94
Using heapq without fighting it	96
k-way merge	98
Two useful variants	99
14. Tries, Automata and the Anatomy of a Tokenizer	101
The trie	101
Compressed tries (radix trees)	103
Aho–Corasick: searching for everything at once	104
Building a tokenizer, properly	105
15. Union–Find and the Algebra of Merging	109
The problem	109
The implementation	110
The four standard applications	111
Variants worth knowing	112
Part IV — Algorithm Design Paradigms	114
16. Divide and Conquer	115
The structure	115
The master theorem	116
Karatsuba: why reducing a matters	117
Two classic applications	118
Divide and conquer is the parallel paradigm	118
17. Greedy Algorithms	121
The pattern and the danger	121
Proving greedy correct	122
The canonical correct greedies	123

Where greedy fails	124
Matroids: when greedy is always right	124
18. Dynamic Programming I: The Method	127
When dynamic programming applies	127
The five-step procedure	128
Worked example: 0/1 knapsack	128
Top-down or bottom-up?	129
Recovering the solution, not just its value	130
The standard shapes	130
19. Dynamic Programming II: Sequence Algorithms in Machine Learning	134
One template, four algorithms	134
Edit distance	135
Dynamic time warping	136
Viterbi: the most likely state sequence	138
CTC: summing over all alignments	139
The complexity summary	140
20. Backtracking and Combinatorial Search	142
The template	142
Pruning is the algorithm	143
Backtracking versus its neighbours	145
21. Randomised Algorithms and Reservoir Sampling	148
Two kinds of randomised algorithm	148
The analysis toolkit	149
Reservoir sampling	149
Shuffling, correctly	151

Random projection	151
Reproducibility	152

Part V — Graphs 154

22. Graph Representations and Traversal 155

Definitions, quickly	155
Three representations	156
The universal traversal	157
Connected components and bipartiteness	159

23. Shortest Paths, Minimum Spanning Trees and A 162

Why BFS stops working	162
Dijkstra	163
Bellman–Ford and negative weights	164
A: Dijkstra with a conscience	165
Minimum spanning trees	166

24. DAGs: Topological Order, Autograd and Computation Graphs 168

Topological order	168
Linear-time DP on a DAG	169
Automatic differentiation	170
Graph transformations	172

25. Flows, Matchings and Assignment 174

Flow networks	174
Bipartite matching	176
The assignment problem	177

26. Graphs in Machine Learning 180

Everything is a sparse matrix product	180
PageRank	181
Spectral methods	182
Message passing and the neighbour explosion	183
Part VI — Data Structures for Scale	187
27. Segment Trees and Fenwick Trees	188
The three-way tradeoff	188
Segment trees	188
Fenwick trees	191
Dynamic weighted sampling	192
28. Probabilistic Structures: Bloom, Count-Min, HyperLogLog	195
Why sketches exist	195
Bloom filters	196
Count-Min sketch	198
HyperLogLog	199
29. MinHash, SimHash and LSH: Deduplicating a Training Corpus	202
The problem	202
Step 1: shingling	203
Step 2: MinHash	203
Step 3: banding into an LSH index	204
Step 4: clustering	206
SimHash: LSH for cosine similarity	206
30. Suffix Structures, Substring Search and Contamination	209
Exact substring search: KMP	209
Suffix arrays	210

Contamination and memorisation	212
Suffix automata and the alternatives	212
31. On-Disk Structures: B-Trees, LSM Trees and Feature Stores	215
The external-memory model	215
B+ trees	216
LSM trees	216
The amplification triangle	218
Skip lists, and why memtables use them	219
Part VII — Algorithms at the Heart of Modern ML	222
32. Nearest-Neighbour Search and the Curse of Dimensionality	223
The problem and the honest baseline	223
k-d trees	224
The curse of dimensionality, quantitatively	225
Intrinsic dimension	226
Metrics and their traps	227
33. Vector Search in Production: IVF, PQ, HNSW, DiskANN	230
The four ideas	230
IVF: partition the space	231
Product quantisation: compress the vectors	232
HNSW: navigate a graph	234
Choosing an index	236
34. Sampling Algorithms	238
The core problem	238
The alias method	239

The Gumbel trick	240
Negative sampling	241
Samplers that appear in every dataloader	242
35. Decoding Algorithms	245
Decoding is bounded search	245
Beam search	246
Sampling: top-k, top-p, min-p	247
Speculative decoding	248
Constrained decoding	249
36. KV Caches as Data Structures	252
The memory arithmetic	252
Fragmentation, and why contiguity is the enemy	253
Paged attention	253
Prefix sharing with a radix tree	255
Continuous batching	256
37. Tree Learners: Decision Trees, Histograms and GBDT	259
Decision-tree induction	259
Histogram-based split finding	260
Leaf-wise versus level-wise growth	262
Categorical features and missing values	262
38. Streaming and Online Algorithms	265
Why streaming is a different discipline	265
Stable moments	266
Quantiles: t-digest	267
Drift detection	269

Online algorithms and competitive ratios	269
Part VIII — Parallel, Distributed and Hardware-Aware	272
39. Parallel Primitives: Scan, Reduce, Sort	273
Work and depth	273
Reduction	274
Scan	274
Amdahl, Gustafson, and where speedup stops	276
40. GPU Algorithm Design	278
The execution model in one page	278
The four rules	279
The roofline	279
FlashAttention	280
What to reconsider on a GPU	281
41. Distributed Algorithms: All-Reduce, Sharding, Consistent Hashing	284
The cost model	284
Ring all-reduce	285
The four parallelisms	286
Sharding and placement	286
Gradient compression, asynchrony, and stragglers	288
42. External Memory and the Data Loading Pipeline	290
External-memory algorithms, briefly	290
Storage layout for training data	291
Diagnosing an input-bound job	292
Part IX — Mastery	296

43. A Problem-Solving Framework	297
The procedure	297
Signals and what they mean	299
Verification before execution	300
Communicating a solution	301
44. The Pattern Catalogue	303
How to use this chapter	303
Array and pointer patterns	303
Search and ordering patterns	305
Hashing and set patterns	306
Tree and graph patterns	307
Dynamic programming patterns	307
Choosing between similar patterns	308
45. Problem Set I: Foundations	310
Arrays, hashing and windows	310
Sorting, searching and selection	315
Linked structures, stacks and heaps	319
Trees and recursion	322
Dynamic programming and greedy	324
46. Problem Set II: Advanced and ML-Flavoured	327
Graphs	327
Advanced dynamic programming	332
Systems and ML-flavoured problems	335
47. Study Plan and Reference	340
The plan	340

Complexity of data structures	341
Complexity of algorithms	342
Python cost reference	343
Hardware cost reference	344
ML system cost formulas	344
The pitfalls index	345
Where to go next	346

PART I

Foundations: The Cost of Everything

Before any data structure, a mental model of what a computer actually charges you for — operations, memory traffic, and the distance between a number and the arithmetic unit.

-
- 01 Why an ML Engineer Needs Algorithms
 - 02 Complexity: Counting What Actually Costs You
 - 03 Memory, Cache and the Machine You Actually Run On
 - 04 Arrays, Strides and the NumPy Memory Model
 - 05 Strings, Bytes and Tokens

Why an ML Engineer Needs Algorithms

The parts of your system that no amount of model quality can fix

FOUNDATION

PREREQS: Python

There is a widespread and comfortable belief that machine learning engineering is about models, and that data structures belong to a previous generation of software. This chapter dismantles that belief with specifics. We will look at where time and memory actually go in a modern ML system, why the answer is almost never “the matrix multiply”, and what it costs a team when nobody on it can reason about an algorithm.

WHAT YOU WILL LEARN

- Locate the algorithmic content hiding inside a “pure ML” system
- Read a latency budget and identify which line is an algorithm choice
- Understand why asymptotics start mattering exactly when a prototype becomes a product
- Recognise the four failure modes that algorithmic ignorance produces on ML teams
- Set up a working environment and a habit for the rest of the book

A story that happens every year

A team ships a retrieval-augmented question answering service. In the prototype it is beautiful: 40,000 documents, a sentence-transformer embedding, cosine similarity computed with a single NumPy matrix multiply, and a language model on top. Latency is 180 milliseconds. Everyone is delighted.

Six months later the corpus is 40 million documents. The same matrix multiply now needs 40 million dot products of dimension 768 per query. That is about 61 billion multiply-accumulate operations, and even at a generous 50 GFLOP/s of achieved throughput on the serving box it takes over two seconds — per query, before the language model has produced a single token. The index no longer fits in the pod’s memory either: 40M x 768 float32 values is 123 GB.

The team’s first instinct is to buy bigger machines. Their second is to shard the corpus across twenty replicas and fan out. Both work, expensively. What neither does is notice that the operation they are performing — “find the twenty nearest vectors” — has been studied for fifty years, that an HNSW graph answers it in roughly 2 milliseconds with 98% recall, and that product quantisation would have compressed those 123 GB down to 4 GB with a measurable and *tunable* accuracy loss.

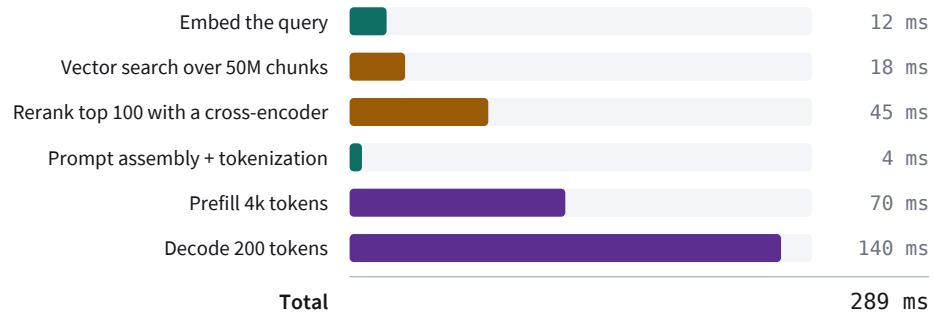


Figure 1.1 A realistic latency budget for a retrieval-augmented generation request. Five of the six lines are determined by an algorithm or a data structure; only the model weights themselves are fixed.

The gap between two seconds and two milliseconds is a factor of a thousand. No amount of model tuning produces a factor of a thousand. No framework upgrade does either. This is what algorithms buy you, and it is why the distinction between “ML work” and “algorithms work” is a fiction that the people who write the systems you depend on never believed in.

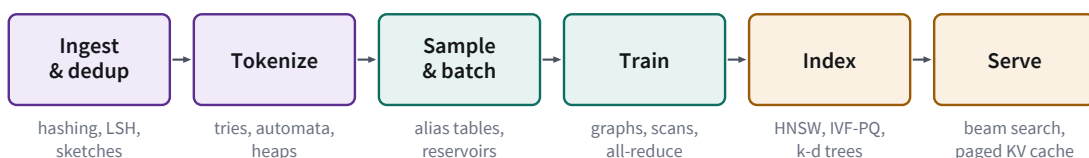
KEY INSIGHT | The core claim of this book

In a modern ML system, the model is usually the part whose cost you cannot change. Everything around it — how data is deduplicated, sampled, indexed, batched, cached, and decoded — is an algorithm choice, and those choices routinely span three orders of magnitude.

Where the algorithms actually are

It is worth being concrete rather than rhetorical. Here is a modern training-and-serving stack with the algorithmic content named.

DATA PATH



The structures below each stage are not optional extras — they are the stage.

Figure 1.2 The canonical pipeline. Under each stage is the family of data structures that stage is actually built from.

Ingestion and deduplication

Training corpora are full of near-duplicates, and near-duplicates measurably hurt models: they waste gradient steps, they inflate evaluation scores through contamination, and they cause verbatim memorisation. Finding exact duplicates is a hashing problem. Finding *near* duplicates in a corpus of ten billion documents is a MinHash and locality-sensitive hashing problem, solvable in a few CPU-hours; the naive pairwise comparison is $(10^{10}/2) \approx 5 \times 10^{19}$ document pairs, which is not solvable at all. Chapter 29 does this properly.

Tokenization

Every token your model ever sees passed through a tokenizer. Byte-pair encoding training is a repeated “find the most frequent adjacent pair” operation, which is a priority queue with lazy deletion; a naive implementation over a 300 GB corpus runs for days, and the standard one runs in hours. Inference-time tokenization is a longest-match search over a trie, or an Aho–Corasick automaton if you also need to detect forbidden strings. Chapter 14 builds both.

Sampling and batching

Curriculum learning, importance sampling, class-balanced sampling and prioritized experience replay all reduce to the same question: *sample an index from a distribution whose weights keep changing*. Done naively that is $O(n)$ per sample, and it will dominate your dataloader. Done with a Fenwick tree it is $O(\log n)$; done with an alias table — when the weights are static — it is $O(1)$. Chapters 27 and 34.

Training

Automatic differentiation is a topological sort over a directed acyclic graph followed by a reverse traversal. Gradient checkpointing is a space–time tradeoff computed by dynamic programming. Distributed data parallelism is ring all-reduce, an algorithm whose communication cost is $2(P-1)/P \times$ the parameter size regardless of how many workers P you add — a genuinely beautiful result that Chapter 41 derives.

Indexing and serving

Vector search is nearest-neighbour search in high dimensions: k-d trees, inverted file indexes, product quantisation, small-world graphs. Serving a language model is a decoding algorithm (greedy, beam, nucleus, speculative) on top of a memory manager for the key-value cache, and the best-known memory manager — paged attention — is a straight port of operating-system virtual memory. Chapters 32 through 36.

WHERE THIS SHOWS UP IN ML | None of this is exotic

Every algorithm named above is inside a library you already import. The question is not whether you will use them; you already do. The question is whether you can reason about them when the defaults stop working — which is precisely the moment your project becomes important enough to have users.

Why asymptotics start mattering later than you think, and then all at once

The most common objection to studying algorithms is empirical and fair: “my code is fast enough”. It usually is. The reason to learn this material anyway is that the transition is not gradual.

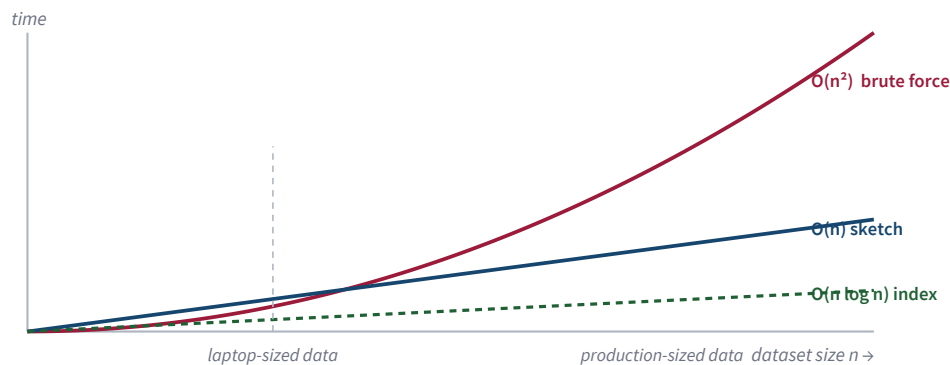


Figure 1.3 Three algorithms for one task. At prototype scale the quadratic algorithm is *faster* — it has the smallest constant and no index to build. The crossover is invisible until you cross it.

Consider a quadratic algorithm with a small constant versus a linearithmic one with a large constant. At $n=1,000$ the quadratic version may well win; it has no setup cost, no index, no memory allocator pressure. At $n=100,000$ it loses by a factor of a hundred. Nothing in the code changed. Nothing in the *profile* looked alarming at $n=1,000$, either; the function was 3% of runtime.

This is why experienced engineers react to an $O(n^2)$ loop the way a structural engineer reacts to a cracked weld. It is not currently a problem. It is a problem *shaped* like a countdown.

THE MATH | The rule of thumb worth memorising

A modern CPU core performs roughly 10^9 simple operations per second in optimised native code, and roughly 10^7 per second in interpreted Python. So for a one-second budget:

- $O(n^2)$ is fine to about $n=30,000$ in C, $n=3,000$ in Python.
- $O(n \log n)$ is fine to about $n=3 \times 10^7$ in C.
- $O(n)$ streaming over data you must read anyway is essentially free.
- $O(2^n)$ is fine to about $n=30$, and never beyond, ever.

These four lines will correctly predict the feasibility of most things you will be asked to build.

The four failure modes

Teams without algorithmic fluency fail in recognisable ways. Naming them makes them easier to catch in review.

1. The invisible quadratic. Someone writes a loop over documents, and inside it a lookup that is itself a scan — `if doc_id in list_of_ids`. In Python that is $O(n)$ inside an $O(n)$ loop. The fix is one character: make it a `set`. The bug survives review because the inner operation looks atomic. Chapter 7 will make you allergic to this.

2. The premature index. The inverse error. Someone builds an HNSW index over 4,000 vectors because “vector search needs a vector database”. The brute-force matrix multiply would have taken 0.2 milliseconds and been exact. Complexity has a cost too, and it is paid in bugs and on-call pages.

3. The right algorithm on the wrong machine. A linked list is asymptotically superb for insertion, and empirically catastrophic, because every node is a cache miss and a cache miss costs the time of roughly 200 arithmetic operations. Chapter 3 is entirely about this, because it is the most common place where textbook knowledge produces slow code.

4. The unreasoned default. `sort()` is called in a hot loop that only needs the largest element; `pandas.apply` is used where a vectorised operation exists; a Python `dict` holds 400 million integer keys and the pod is killed for memory. None of these is exotic. Each is a default that nobody thought about.

COMMON PITFALL | “The library handles it”

Libraries handle the common case. FAISS will happily let you build an index that needs 200 GB, PyTorch will happily let your dataloader become the bottleneck, and a vector database will happily return 60% recall while reporting success. Libraries expose knobs; knobs require a model of what they do. That model is this book.

What “world class” means here

A great deal of algorithm education optimises for reproducing a known solution under time pressure. That is a real skill — it is how you pass interviews, and Part IX takes it seriously — but it is not the skill that makes systems fast. The skill that makes systems fast is a three-part habit:

- 1. Name the operation.** “I need the twenty nearest vectors” is a named problem with a literature. “I need to make this faster” is not.
- 2. Write down the cost model.** How many times will this run? On what n ? Is the bottleneck operations, memory traffic, or network? Chapter 2 and Chapter 3 give you the vocabulary.
- 3. Know the shape of the solution space.** Between brute force and the state of the art there are usually four or five options with different accuracy, memory and build-time tradeoffs. Knowing they exist is 90% of the value; this book’s job is to put them in your head.

KEY INSIGHT | Recognition beats derivation

You will almost never derive a new algorithm at work. You will constantly *recognise* that a problem in front of you is an instance of one you have seen. This is why breadth matters, why the ML-specific chapters in Part VII matter, and why you should read even the chapters you will not immediately use.

Setting up

Everything in this book runs on a plain Python 3.10+ installation. A handful of chapters use NumPy; a few use `matplotlib` for you to plot your own measurements. Nothing requires a GPU, though Part VIII will make more sense if you have ever seen one.

Environment check

```
1 import sys, timeit, platform
2
3 print(sys.version.split()[0], platform.machine())
4
5 # How many simple Python operations does this machine do per second?
6 n = 1_000_000
7 t = timeit.timeit("total += 1", setup="total = 0", number=n)
8 print(f"{n/t:,.0f} simple statements/sec")
9
10 # ... and how many with NumPy doing the loop for us?
11 import numpy as np
12 a = np.ones(10_000_000, dtype=np.float32)
13 t = timeit.timeit(lambda: a.sum(), number=10) / 10
14 print(f"{a.size/t:,.0f} float adds/sec in NumPy")
```

Run it. On a typical 2026 laptop you will see something like 30 million Python statements per second and 3 billion float additions per second in NumPy — a hundredfold gap that explains most of what you will ever need to know about why vectorisation matters. Write the two numbers down; you will use them as a sanity check for the rest of the book.

EXERCISES

1. Take a piece of code you have written in the last month. Identify the single loop with the largest n . Write down its complexity in terms of that n . If you cannot, mark it and come back after Chapter 2.
2. Estimate, without running anything, how long a brute-force nearest neighbour search over one million 768-dimensional float32 vectors takes on one CPU core. Then measure it with NumPy. Explain the difference between your estimate and reality — Chapter 3 contains the answer.
3. Find one place in a codebase you work in where a `list` is used for membership tests. Estimate the speedup from changing it to a `set` at your current data size and at 100x that size.
4. Read the documentation page for `faiss.IndexFlatL2` and `faiss.IndexHNSWFlat`. Write one sentence on what you would need to know to choose between them. Keep the sentence; compare it to what you write after Chapter 33.

CHAPTER RECAP

- The expensive parts of an ML system are usually not the model: they are deduplication, sampling, indexing, caching and decoding, all of which are data-structure problems.
- The cost difference between a naive and a good choice is routinely 10^2 – 10^3 , far beyond what tuning or hardware provides.
- Asymptotic behaviour is invisible at prototype scale and decisive at production scale; the crossover gives no warning.
- Four recognisable failure modes: the invisible quadratic, the premature index, the right algorithm on the wrong machine, and the unreasoned default.
- The skill to build is *recognition*: naming the operation, writing down the cost model, and knowing the shape of the solution space.

Complexity: Counting What Actually Costs You

Asymptotic analysis as a working tool, not an exam topic

FOUNDATION

PREREQS: Chapter 1

Asymptotic analysis has a reputation for being the dull, formal part of algorithms. It is neither dull nor especially formal once you see what it is for: it is a device for throwing away the parts of a cost that depend on your laptop, so that what remains depends only on the algorithm. This chapter builds that device from scratch, then shows exactly where it lies to you.

WHAT YOU WILL LEARN

- Count the operations an algorithm performs without counting them one by one
- State and use the definitions of O , Ω and Θ correctly
- Read complexity off a piece of code in seconds, including nested and recursive code
- Perform an amortised analysis using the aggregate, accounting and potential methods
- Know the complexity of every common Python operation by heart
- Say precisely what asymptotic notation hides, and when that matters more than the notation

The question complexity answers

Suppose you have two implementations of the same function and you want to know which is better. You could time them. That answer is contaminated by your CPU, your Python version, whether a background process was running, how warm the cache was, and the specific input you happened to use.

Asymptotic analysis asks a narrower question with a more durable answer: **as the input grows, how does the cost grow?** A function that takes $3n+40$ microseconds and one that takes $0.5n+2000$ microseconds are both *linear*: double the input and both roughly double. A function that takes $0.001n^2$ microseconds is not, and there is some input size beyond which it loses to both, permanently, on every machine that will ever be built.

KEY INSIGHT | What the notation is actually for

Asymptotic notation deliberately discards constant factors and lower-order terms because those are properties of your machine and your compiler, while the growth rate is a property of your algorithm. It is a change of coordinates that isolates the part you can control by thinking.

The discipline this imposes is more useful than the notation. When you write $O(n \log n)$, you are committing to a claim about behaviour at scales you have not tested, and the claim is falsifiable.

Counting without counting

Start concretely. Here is a function that finds the two closest values in a list.

Closest pair, brute force

```

1 def closest_pair(xs):
2     best = float("inf")
3     pair = None
4     for i in range(len(xs)):
5         for j in range(i + 1, len(xs)):
6             d = abs(xs[i] - xs[j])
7             if d < best:
8                 best, pair = d, (xs[i], xs[j])
9     return pair, best

```

How many times does the inner body run? For $i=0$ it runs $n-1$ times, for $i=1$ it runs $n-2$ times, and so on down to zero:

$$\sum_{i=0}^{n-1} (n-1-i) = n(n-1)/2 = n^2/2 - n/2$$

So the exact count is $1/2n^2 - 1/2n$ iterations, each doing a fixed amount of work. We call this $\Theta(n^2)$, and we mean: the leading term is n^2 , the $1/2$ is a machine-dependent detail, and the $-1/2n$ is irrelevant once n is large.

Notice what we did *not* do: we did not count Python bytecodes, or worry that `abs` is cheaper than `**`, or measure anything. We identified the dominant repeated operation and counted how many times it happens.

NOTE | The recipe

1. Find the operation that dominates — usually the innermost statement in the deepest loop, or the recursive call.
2. Count how many times it executes as a function of the input size.
3. Keep only the fastest-growing term and drop its constant.

The three notations, stated properly

Practitioners use O for everything, and that is usually harmless, but the three symbols mean genuinely different things and there are moments when the difference decides an argument.

DEFINITION | Asymptotic bounds

Let $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$.

- $f(n) = O(g(n))$ (“ f grows no faster than g ”) if there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
- $f(n) = \Omega(g(n))$ (“ f grows at least as fast as g ”) if there exist $c > 0$ and n_0 with $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.
- $f(n) = \Theta(g(n))$ (“ f grows exactly like g ”) if both hold; equivalently there are $c_1, c_2 > 0$ and n_0 with $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for $n \geq n_0$.

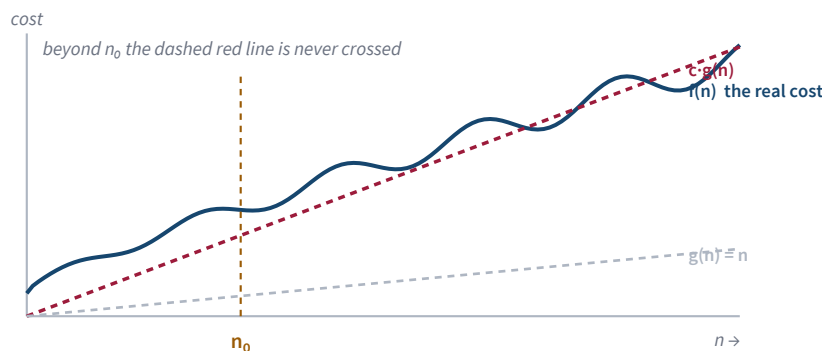


Figure 2.1 The definition of $f(n) = O(g(n))$ made visible. We are allowed to scale g by any constant we like, and we are allowed to ignore everything before some threshold n_0 . All that must hold is that beyond that point, the scaled g is an upper envelope.

Two consequences are worth internalising because they are the source of most confusion.

First, O is an *upper bound*, not a tight one. It is true that binary search is $O(n^2)$. It is a useless statement, but it is true, and the fact that it is true is why Θ exists. When you say “this is $O(n \log n)$ ” you almost always mean $\Theta(n \log n)$; say what you mean when precision matters.

Second, the equals sign is an abuse of notation. $O(g)$ is a *set* of functions, and $f = O(g)$ really means $f \in O(g)$. This is why you can write $n^2 + n = O(n^2)$ but not $O(n^2) = n^2 + n$. Nobody will correct you in a code review, but knowing it stops you writing nonsense in a design doc.

COMMON PITFALL | “Big-O of the worst case” is two independent choices

$O/\Omega/\Theta$ describe how a function grows. Worst/average/best case describe *which* function you are talking about. They are orthogonal. It is perfectly meaningful to say “quicksort’s best case is $\Theta(n \log n)$ ” and “quicksort’s worst case is $\Theta(n^2)$ ”. Saying “quicksort is $O(n \log n)$ ” without qualification is simply wrong.

The hierarchy you must know cold

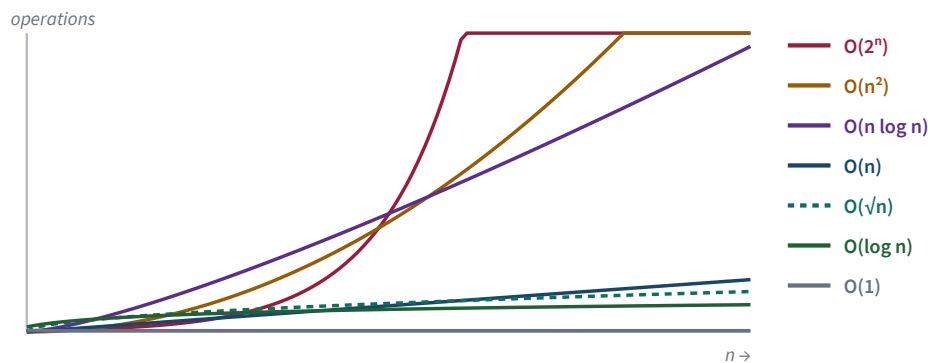


Figure 2.2 The standard growth hierarchy. Each curve is eventually and permanently below the one above it. The only interesting question for a given problem is which band you are in.

Growth	Name	$n=10$	$n=1000$	$n=10^6$	Typical source
$O(1)$	constant	1	1	1	hash lookup, array index
$O(\log n)$	logarithmic	3	10	20	binary search, balanced tree
$O(\sqrt{n})$	root	3	32	1000	block decomposition
$O(n)$	linear	10	1000	10^6	one pass over the data
$O(n \log n)$	linearithmic	33	10^4	2×10^7	sorting, FFT, divide and conquer
$O(n^2)$	quadratic	100	10^6	10^{12}	all pairs, nested loop
$O(n^3)$	cubic	1000	10^9	10^{18}	naive matrix multiply
$O(2^n)$	exponential	1024	$\approx 10^{301}$	—	subset enumeration
$O(n!)$	factorial	3.6×10^6	—	—	permutation enumeration

Two rows deserve comment because ML engineers meet them constantly.

$O(n \log n)$ is, for practical purposes, *almost linear*. The logarithm of a billion is thirty. A sorting-based solution to a billion-element problem costs about thirty times a single pass, and a single pass over a billion elements is something you were going to do anyway. Do not contort an algorithm to avoid a sort.

$O(n^2)$ with a small constant beats $O(n \log n)$ with a large one for surprisingly long. Insertion sort beats merge sort below roughly 32 elements, which is why every production sort (including CPython’s Timsort) switches to insertion sort for small runs. Asymptotics tell you who wins eventually; they do not tell you when eventually starts.

Reading complexity off code

With practice this becomes instant. The rules:

Sequential blocks add. Two loops one after another, each $\Theta(n)$, give $\Theta(n) + \Theta(n) = \Theta(n)$. Addition means “take the max”.

Nested loops multiply, but only if the inner bound is independent of the outer variable. A loop to n containing a loop to n is $\Theta(n^2)$. A loop to n containing a loop to i is $\Theta(n^2)$ as well — we computed it above — but a loop to n containing a loop to $\log n$ is $\Theta(n \log n)$.

Halving gives a logarithm. Any loop whose variable is multiplied or divided by a constant each iteration runs $\Theta(\log n)$ times.

Recursion needs a recurrence. $T(n) = 2T(n/2) + O(n)$ is $\Theta(n \log n)$; $T(n) = T(n-1) + O(1)$ is $\Theta(n)$; $T(n) = 2T(n-1) + O(1)$ is $\Theta(2^n)$. Chapter 16 gives you the master theorem to do these mechanically.

Four loops, four complexities

```

1  def a(xs):                                # theta(n)
2      return sum(x * x for x in xs)
3
4  def b(xs):                                # theta(n^2): inner bound independent of
5      i                                     i
6      return [[x * y for y in xs] for x in xs]
7
8  def c(n):                                # theta(log n): k doubles each step
9      k, steps = 1, 0
10     while k < n:
11         k *= 2
12         steps += 1
13     return steps
14
15 def d(xs):                                # theta(n log n): outer n, inner log n
16     total = 0
17     for x in xs:
18         k = len(xs)
19         while k > 1:
20             total += x
21             k //= 2
22     return total

```

COMMON PITFALL | The hidden loop

The single most common complexity bug in Python is a library call that looks atomic but is not. `x in some_list` is $\Theta(n)$. `list.insert(0, x)` is $\Theta(n)$. `str += piece` inside a loop is $\Theta(n^2)$ overall because strings are immutable and each concatenation copies.

`df.append(row)` in a pandas loop is $\Theta(n^2)$ for the same reason. None of these looks like a loop, and all of them are.

Best, average and worst case

The cost of an algorithm generally depends on which input of size n you give it, so “the” complexity is ambiguous until you say which input.

- **Worst case** is the maximum over all inputs of size n . It is what you quote for anything with a latency SLA, because it is a promise.
- **Best case** is the minimum. It is rarely useful except to show an algorithm is adaptive: insertion sort is $\Theta(n)$ on sorted input, and that is genuinely why Timsort exploits existing runs.
- **Average case** is the expectation over some distribution of inputs. This is where people get sloppy: *the average is meaningless without naming the distribution*. Quicksort’s $\Theta(n \log n)$ average assumes a uniformly random pivot, which is why real implementations randomise rather than trust the input.

WHERE THIS SHOWS UP IN ML | Why worst case matters more in serving than in training

A training job that occasionally takes 20% longer on one batch is invisible. An inference service at p99 latency is not: the 1% of requests that hit the worst case are the ones that page you. This is why production retrieval systems prefer structures with bounded worst-case behaviour (or a hard timeout and a fallback) over structures with a better average.

Amortised analysis: the dynamic array

Some operations are usually cheap and occasionally expensive, and reporting their worst case is misleading. Python’s `list.append` is the canonical example, and understanding it properly is worth the page it takes.

A Python list is a contiguous block of pointers with some spare capacity. Appending writes into the spare slot: $\Theta(1)$. When the spare capacity runs out, the list allocates a larger block, copies everything across, and frees the old block: $\Theta(n)$. So the worst case of `append` is $\Theta(n)$ — and yet a million appends take a million cheap steps, not a million expensive ones.

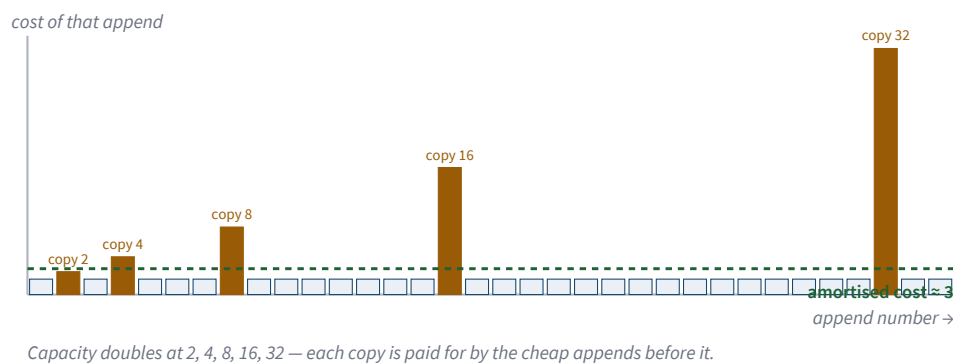


Figure 2.3 The cost of each individual append when capacity doubles. The spikes are real, but they are rare in exactly the right proportion: each spike of size k is preceded by $k/2$ appends that cost 1.

THE MATH | Three ways to prove it is $\Theta(1)$ per append

Aggregate method. Over n appends, the resizes happen at capacities $1, 2, 4, \dots$ up to n , costing $1+2+4+\dots+n < 2n$ copies in total. Add the n cheap writes: total work $< 3n$, so the average per operation is at most 3. Constant.

Accounting method. Charge every append 3 credits: 1 to write its own value, and 2 saved on the object. When the array of size k doubles, the $k/2$ items appended since the last resize have saved $2 \times k/2 = k$ credits, exactly paying for the k copies. No operation is ever in debt, so the true total never exceeds the charged total of $3n$.

Potential method. Define $\Phi = 2 \cdot \text{size} - \text{capacity}$ (non-negative once the array is at least half full). A cheap append raises Φ by 2, so its amortised cost is $1+2=3$. A doubling append costs size real work but drops Φ from size to 2, releasing size-2: amortised cost is again about 3.

All three give the same answer, and each generalises differently. The potential method is the one you will use again — it reappears in Fibonacci heaps, splay trees, and the analysis of union-find in Chapter 15.

KEY INSIGHT | Growth factor is the whole trick

The magic is not doubling specifically; it is growing by a constant *factor*. Growing by a constant *amount* — say, 1000 slots at a time — gives $\Theta(n^2)$ total work for n appends. CPython actually grows by about 1.125x for large lists, trading a bigger constant for less wasted memory. Go's slices double up to 1024 elements then grow by 1.25x. The factor is a memory-time knob, and any factor > 1 preserves the $\Theta(1)$ amortised bound.

WATCH OUT | Amortised is not average, and not worst case

Amortised $\Theta(1)$ means: *any sequence* of n operations costs $\Theta(n)$ — a guarantee, with no probability in it. But a single operation can still take $\Theta(n)$. If you have a hard per-request latency bound, amortised analysis does not save you; you need a structure with worst-case guarantees, or you need to do the resize incrementally.

Space complexity

Every statement above has a memory analogue, and in ML work memory is usually the binding constraint rather than time. Two distinctions matter.

Total space includes the input. **Auxiliary space** excludes it. An in-place sort uses $\Theta(1)$ auxiliary space even though it touches n elements; merge sort uses $\Theta(n)$ auxiliary space, which is precisely why NumPy's `np.sort` defaults to quicksort rather than mergesort.

The recursion stack counts. A recursive function of depth d uses $\Theta(d)$ space even if it allocates nothing. Naive recursion over a linked list of a million nodes does not merely run slowly — it raises `RecursionError`, and in a compiled language it segfaults.

Same output, very different memory

```
1 # theta(n) extra memory: the whole list is materialised.
2 squares = [x * x for x in range(10**8)]           # ~4 GB, will be killed
3
4 # theta(1) extra memory: one value at a time.
```

```
5 total = sum(x * x for x in range(10**8)) # fine
```

WHERE THIS SHOWS UP IN ML | The memory ladder you actually live on

For a transformer of P parameters trained in mixed precision with Adam, the optimiser state alone is roughly $12P$ bytes (fp32 master weights, first and second moments), plus $2P$ for bf16 weights and $2P$ for gradients: about $16P$ bytes before a single activation is stored. For a 7-billion-parameter model that is 112 GB — which is why sharding, offloading, and activation checkpointing (a dynamic-programming space–time tradeoff, see Chapter 18) exist at all.

The complexity of Python itself

You cannot reason about your code without knowing the cost of the primitives you build on. These are the ones worth memorising.

Operation	list	deque	dict/set	heapq list
index / access by key	$\Theta(1)$	$\Theta(n)$	$\Theta(1)^*$	—
append / push right	$\Theta(1)^{\text{dagger}}$	$\Theta(1)$	$\Theta(1)^*$	$\Theta(\log n)$
pop right	$\Theta(1)$	$\Theta(1)$	$\Theta(1)^*$	—
insert / pop left	$\Theta(n)$	$\Theta(1)$	—	—
membership <code>in</code>	$\Theta(n)$	$\Theta(n)$	$\Theta(1)^*$	$\Theta(n)$
min / pop min	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$
sort	$\Theta(n \log n)$	—	—	—
+ concatenate	$\Theta(n+m)$	—	—	—
slice <code>a[i:j]</code>	$\Theta(j-i)$	—	—	—

Table 2.1 How the standard growth rates behave at three scales. The empty cells are not omissions; those numbers exceed the number of atoms in the observable universe.

COMMON PITFALL | Three real bugs these tables prevent

1. `while queue: item = queue.pop(0)` — a BFS written with a list instead of a `deque` is $\Theta(n^2)$. Every ML engineer writes this once.
2. `if candidate in seen_list` inside a dedup loop — $\Theta(n^2)$. Use a `set`; the change takes four characters and turns hours into seconds.
3. `sorted(scores)[-k:]` to get the top k — $\Theta(n \log n)$ where `heapq.nlargest(k, scores)` is $\Theta(n \log k)$. For $n=10^7$ and $k=10$ that is a 20x difference, and Chapter 13 explains why.

What complexity hides, and when that matters

This is the part most courses omit, and it is the part that decides real performance.

Constants are not small. A hash table lookup and an array index are both $\Theta(1)$, and the array index is about 20 times faster. A B-tree and a balanced binary search tree are both $\Theta(\log n)$, and on disk the B-tree is a hundred times faster because it matches the page size. Chapter 31.

Memory traffic is invisible to the operation count. Two $\Theta(n)$ loops over the same array can differ by 10x depending on access order, because one streams cache lines and the other thrashes. Chapter 3 is entirely about this and it is, empirically, the highest-value chapter in Part I.

The model of computation may be wrong. On a GPU, the cost of an algorithm is dominated by memory movement and by how uniformly its threads branch, not by its operation count. FlashAttention performs *more* arithmetic than naive attention and is several times faster, because it moves far less data. Chapter 40 derives this.

n may not be what you think. For a nearest-neighbour query over n vectors of dimension d , the honest complexity is $\Theta(nd)$, and in practice d dominates the constant. Always name every parameter in the bound: $\Theta(nd)$, $\Theta(VE)$, $\Theta(bnhd)$.

HARDWARE REALITY | A cost model that fits on a line

$\text{time} \approx \max\left(\frac{\text{FLOPs}}{\text{peak FLOP/s}}, \frac{\text{bytes moved}}{\text{memory bandwidth}}\right)$

The ratio FLOPs/bytes is called *arithmetic intensity*. If it is below your hardware's ratio of peak-FLOPs to bandwidth (about 100 on a modern accelerator, about 10 on a CPU), you are memory-bound and reducing operation count will do nothing at all. Batched matrix multiply is compute-bound; elementwise activation, layer norm, and attention decoding are memory-bound. Chapter 40.

A worked example: three ways to find the top k

The value of all this is that it turns a vague “make it faster” into a decision you can make on paper. Suppose you need the k highest-scoring items out of n candidates in a reranking stage.

Approach	Time	Auxiliary space	When it wins
Sort, take last k	$\Theta(n \log n)$	$\Theta(n)$	k close to n ; you need full order anyway
Min-heap of size k	$\Theta(n \log k)$	$\Theta(k)$	$k \ll n$; streaming input; bounded memory
Quickselect	$\Theta(n)$ expected	$\Theta(1)$	Data in memory, k arbitrary, order not needed
<code>np.argpartition</code>	$\Theta(n)$	$\Theta(n)$	NumPy array; wins on constants by 10–50x

Table 2.2 Complexity of Python's core containers. * expected, amortised, assuming well-distributed hashes. ^{dagger} amortised.

The heap version is the one to reach for when the candidates arrive as a stream and you cannot hold them all, which is the normal situation in a retrieval pipeline. Chapter 13 builds it.

EXERCISES

1. Give the complexity of each: (a) a loop that runs n times containing a `sorted()` of a fixed-size 100-element list; (b) a loop that runs n times containing `sorted()` of the full n elements; © a recursive function that calls itself twice on inputs of size $n/2$ and does $\Theta(1)$ work.
2. Prove that $\log_a n = \Theta(\log_b n)$ for any constants $a, b > 1$. Explain why this justifies writing $O(\log n)$ without a base.
3. An array grows by a fixed 1024 slots whenever it fills. Show that n appends cost $\Theta(n^2)$ in total, and find the crossover n at which this becomes slower than doubling for a specific copy cost.
4. Using the potential method, show that a stack supporting `push`, `pop` and `multipop(k)` (pop up to k items) has $\Theta(1)$ amortised cost per operation even though `multipop` is $\Theta(k)$.
5. You profile a training step at batch size 32 and at batch size 64 and find times of 41 ms and 79 ms. At 512 you measure 690 ms rather than the predicted 630 ms. Give two distinct explanations and an experiment that distinguishes them.
6. `str.join` on a list of n pieces is $\Theta(\text{total length})$, while repeated `s += piece` is $\Theta(n \cdot \text{total length})$ in the worst case. Explain why, then explain why CPython sometimes makes the second one fast anyway — and why you should not rely on that.

CHAPTER RECAP

- Asymptotic notation exists to discard machine-dependent constants and isolate the growth rate, which is the part you control by design.
- O is an upper bound, Ω a lower bound, Θ both. Best, average and worst case are a separate axis; state both.
- Read complexity by finding the dominant repeated operation and counting its executions: blocks add, independent nested loops multiply, halving gives a logarithm, recursion needs a recurrence.
- Amortised analysis (aggregate, accounting, potential) proves that dynamic arrays give $\Theta(1)$ appends. Amortised is a guarantee over sequences, not a probability, and not a worst-case bound on one operation.
- Memorise the Python container table; three of the most common performance bugs in ML code are direct consequences of not knowing it.
- Complexity hides constants, memory traffic, the machine model, and sometimes the real parameters. On accelerators, bytes moved usually matters more than operations performed.

Memory, Cache and the Machine You Actually Run On

Why two algorithms with identical operation counts differ tenfold

FOUNDATION

PREREQS: Chapter 2

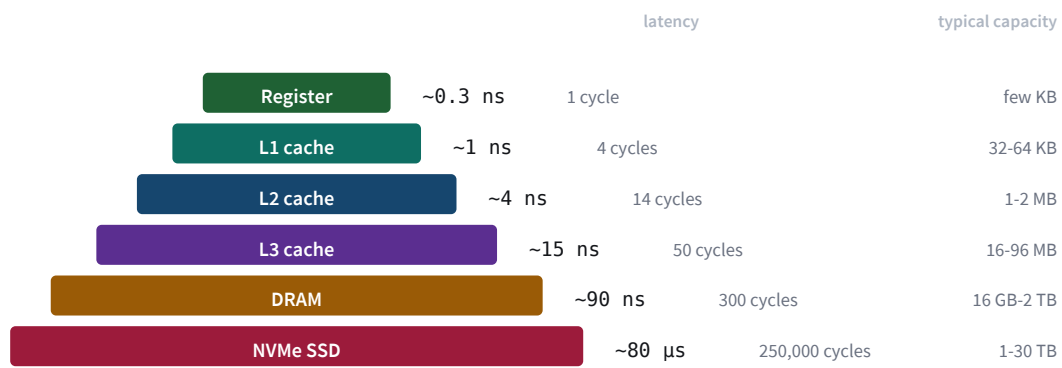
Asymptotic analysis assumes every memory access costs the same. On real hardware that assumption is wrong by a factor of three hundred, and it has been wrong since about 1990. This chapter replaces the flat-memory fiction with an accurate mental model, and shows why it is the single most useful piece of systems knowledge an ML engineer can acquire.

WHAT YOU WILL LEARN

- Explain the memory hierarchy and quote the latency of each level from memory
- Predict which of two loops with identical complexity will be faster, and why
- Use the three principles — locality, contiguity, predictability — as design constraints
- Choose between array-of-structs and struct-of-arrays for a given access pattern
- Understand why linked structures underperform their asymptotics so badly
- Reason about arithmetic intensity and the roofline model

The lie in the RAM model

Chapter 2 counted operations as if each one costs the same. That model — called the *RAM model* — was accurate in 1975, when a memory access and an addition both took roughly one cycle. Since then processors have got about 50,000 times faster and main memory has got about 10 times faster in *latency*. The gap is now enormous and it defines modern performance work.



One DRAM miss costs about as much as 300 additions. This single fact reshapes most data-structure choices.

Figure 3.1 The memory hierarchy on a 2026 server CPU, with honest numbers. Each level is roughly 4–8x slower and 10–50x larger than the one above it. The jump from L3 to DRAM is the cliff most code falls off.

Read the figure again and fix one number in your head: **a main-memory access costs about 300 cycles**, during which the core could have performed around 300 additions or, with vector instructions, several thousand. Your program spends most of its life waiting for data, not computing.

KEY INSIGHT | The real cost model

Cost is not “operations performed”. Cost is max (operations/throughput, bytes moved/bandwidth), and for most code that touches more data than fits in cache, the second term wins. Algorithms should therefore be designed to minimise *data movement*, not just operation count.

Cache lines: the unit that actually moves

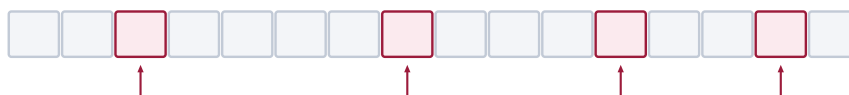
Memory does not move a byte at a time. It moves a **cache line**, typically 64 bytes (16 float32 values, 8 float64 values, or 8 pointers on a 64-bit machine). When you read `x[0]`, the hardware fetches the entire 64-byte line containing it and places it in L1.

ONE 64-BYTE CACHE LINE = 16 float32 values



↓
you asked for `x[0]` — the other 15 arrive free

RANDOM ACCESS: every touch is a new line



4 misses ≈ 1200 cycles wasted

Figure 3.2 Top: a sequential scan pays one miss and then gets fifteen free hits. Bottom: four random accesses into different lines pay four full misses.

This single mechanism explains the majority of performance differences between data structures:

- **Sequential access is nearly free after the first element.** Scanning an array of a million floats costs about $10^6/16=62,500$ misses, not 10^6 .
- **The hardware prefetcher makes it better still.** Modern CPUs detect a constant stride and fetch lines ahead of the loop, hiding the latency almost entirely. A predictable sequential scan can run at full memory bandwidth — tens of gigabytes per second.
- **Random access defeats both.** Every touch is a miss, and the prefetcher cannot help because there is no pattern to detect.

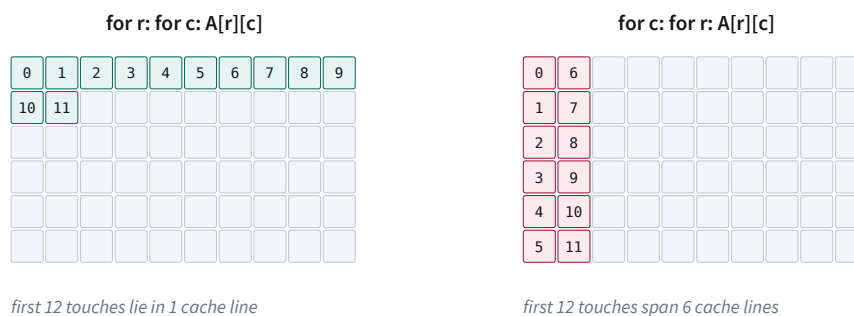
Same work, different order

```

1  import numpy as np, time
2
3  n = 8192
4  A = np.zeros((n, n), dtype=np.float32)
5
6  def by_rows(A):                # stride 4 bytes: perfectly sequential
7      s = 0.0
8      for r in range(A.shape[0]):
9          s += A[r, :].sum()
10     return s
11
12  def by_cols(A):                # stride 32 KB: a new cache line every
13     touch                        touch
14     s = 0.0
15     for c in range(A.shape[1]):
16         s += A[:, c].sum()
17     return s

```

Both functions touch exactly n^2 elements and perform exactly n^2 additions. On a typical machine `by_rows` is five to ten times faster. The operation count did not change; the number of cache lines touched changed by a factor of sixteen.



Identical operation count. On a 4096×4096 float32 matrix the left loop runs 5–10× faster.

Figure 3.3 The first twelve memory touches of each loop over a row-major matrix. Row order stays inside one or two cache lines; column order pulls a new line for every single element.

COMMON PITFALL | The transposed matrix trap

NumPy's `A.T` is a *view* with reversed strides, not a copy — it is free. But every subsequent operation on that view now runs with the bad access pattern. If you are going to touch a transposed array more than once, call `np.ascontiguousarray(A.T)` and pay the copy upfront. Chapter 4 makes the stride mechanics explicit.

Three principles

Everything in this chapter reduces to three properties, and every fast data structure has at least two of them.

Temporal locality. If you touch a datum, touch it again soon. Caches keep recently used lines; reuse is free. This is why blocked (tiled) matrix multiplication is faster than the naive triple loop even though it performs the same multiplications: it arranges for each loaded tile to be used b times before eviction.

Spatial locality. If you touch a datum, touch its neighbour next. This is contiguity, and it is why arrays beat linked lists and why struct-of-arrays beats array-of-structs for column-wise work.

Predictability. If your access pattern is a constant stride, the prefetcher will hide the latency. Random access, pointer chasing and data-dependent branches all destroy predictability. A branch mispredict costs 15–20 cycles; a data-dependent load chain costs the full latency every time because nothing can be overlapped.

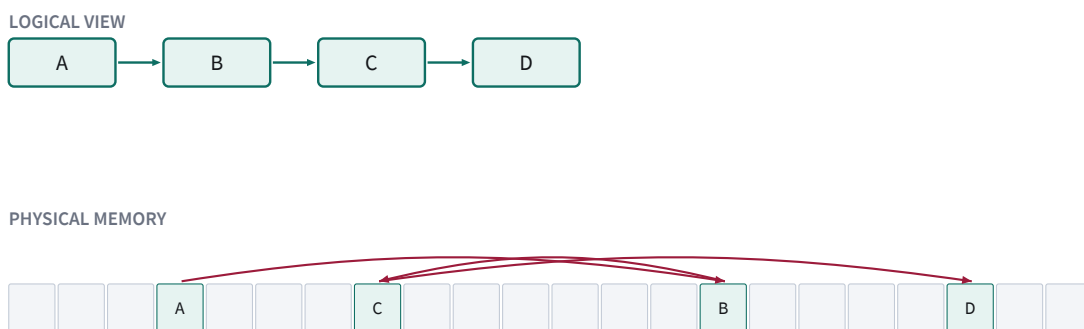
HARDWARE REALITY | Why sorted data is faster to filter

Branch prediction is a real, measurable effect. A loop that sums values above a threshold runs roughly 3x faster on sorted input than on shuffled input — identical work, identical memory pattern, but on sorted data the branch is taken in one long run and then not taken in another, which the predictor gets right almost every time. On shuffled data it is a coin flip and the pipeline stalls constantly.

Why linked lists lose

The classic analysis says: array insertion in the middle is $\Theta(n)$ because you must shift elements; linked-list insertion is $\Theta(1)$ because you just rewire two pointers. Therefore linked lists win for insertion-heavy workloads.

In practice, for anything below roughly a hundred thousand elements, the array wins — often by an order of magnitude — even for insertion-heavy workloads. Here is why.



Four logical steps, four cache misses, four TLB lookups — roughly 1200 wasted cycles for 4 useful ones.

Figure 3.4 A four-node linked list. Logically it is a straight line; physically the nodes are wherever the allocator put them, and traversing it is four dependent cache misses in a row.

1. **Every node is a separate allocation.** After a few thousand insertions and deletions the nodes are scattered across the heap, so traversal is pointer chasing: each `node.next` is a cache miss, and worse, the misses are *serially dependent* — you cannot start fetching node 3 until node 2 has arrived. No parallelism, no prefetching.
2. **Memory overhead is brutal.** A Python list of a million ints stores a million pointers plus the objects. A linked list adds a node object per element: in CPython that is roughly 56 bytes of object header and pointers *per element* before the payload.
3. **Shifting is vectorised.** Moving n contiguous bytes in an array is a `memmove`, which runs at tens of gigabytes per second. Shifting ten thousand 8-byte elements moves 80 KB — about 3 microseconds. One cache miss is 0.09 microseconds, so the array’s “expensive” $\Theta(n)$ shift is worth about 33 pointer dereferences.

KEY INSIGHT | When linked structures do earn their keep

Not never — just less often than the textbook implies. They win when: nodes are large (so the pointer overhead is proportionally small); you hold long-lived references *into* the middle and need stable addresses; you need $O(1)$ splice of entire sublists; or the structure is intrusive and allocation-free, as in a kernel scheduler’s run queue or an LRU cache’s recency list (Chapter 6).

Array of structs versus struct of arrays

This is the most actionable layout decision in data-intensive code, and it appears constantly in ML pipelines.

ARRAY OF STRUCTS — one record per slot



STRUCT OF ARRAYS — one field per run



Figure 3.5 The same five-field records in two layouts. If your loop reads one field across many records, struct-of-arrays moves five times fewer bytes.

Suppose you have ten million records, each with an `id`, three coordinates and a weight. If your hot loop computes the mean of `x`:

- **Array of structs** (`[{id, x, y, z, w}, ...]`, or a list of objects, or a pandas row-oriented view) loads every field of every record because they share cache lines. You move 200 MB to read 40 MB of useful data.
- **Struct of arrays** (`{id: [...], x: [...], ...}`, which is exactly what a NumPy structured layout, a pandas DataFrame column, or an Arrow record batch gives you) loads only the `x` array. You move 40 MB, and the loop vectorises.

WHERE THIS SHOWS UP IN ML | This is why the whole ML data stack is columnar

Parquet, Arrow, pandas, Polars, and every feature store you will meet are column-oriented for exactly this reason. It is also why a **Dataset** that returns Python dicts of scalars is slow and one that returns batched NumPy arrays is fast: the first is array-of-structs with an interpreter on top. Embedding tables are the same story — they are stored as one contiguous $V \times d$ matrix, and a lookup is a contiguous d -element read.

Measuring it yourself

Do not take any of this on faith. The following measures your machine’s actual cliff, and takes about ten seconds.

Find your cache sizes empirically

```

1  import numpy as np, time
2
3  def bench(size_kb, steps=4_000_000):
4      n = size_kb * 1024 // 4          # float32 elements
5      a = np.random.rand(n).astype(np.float32)
6      idx = np.random.randint(0, n, size=1 << 20) # random, defeats
7          prefetch
8      t0 = time.perf_counter()
9      for _ in range(steps // (1 << 20)):
10         a[idx].sum()
11     return (time.perf_counter() - t0) / steps * 1e9 # ns per access
12
13 for kb in [16, 64, 256, 1024, 4096, 16384, 65536]:
14     print(f"{kb:>6} KB   {bench(kb):6.2f} ns/access")

```

You will see a roughly flat region, then a step, then another step. The steps are your L1, L2 and L3 boundaries. Knowing where your L2 ends turns “make the batch smaller” from folklore into arithmetic.

PERFORMANCE NOTE | Tiling, in one paragraph

If a loop touches a working set larger than a cache level, restructure it to work on blocks that fit. Naive $n \times n$ matrix multiply reads $\Theta(n^3)$ words from memory; blocked multiply with block size b reads $\Theta(n^3/b + n^2)$, and choosing b so that three $b \times b$ tiles fit in L2 reduces traffic by a factor of b — typically 30–60x. This is the same idea that makes FlashAttention fast (Chapter 40), and it is a *restructuring*, not a different algorithm.

Arithmetic intensity and the roofline

To decide whether an optimisation can possibly help, compute the arithmetic intensity: FLOPs performed per byte moved from memory.

Operation	FLOPs	Bytes moved	Intensity	Verdict
Vector add $c = a + b$ (n fp32)	n	$12n$	0.08	memory-bound

Operation	FLOPs	Bytes moved	Intensity	Verdict
softmax over n	$\approx 5n$	$8n$	0.6	memory-bound
Matrix–vector $n \times n$	$2n^2$	$4n^2$	0.5	memory-bound
Matrix–matrix $n \times n$	$2n^3$	$12n^2$	$n/6$	compute-bound for large n
Attention, decode (batch 1)	$\approx 4nd$	$\approx 4nd$	1	memory-bound
Attention, prefill	$\approx 4n^2d$	$\approx 8nd$	$n/2$	compute-bound

The practical consequence is blunt and worth stating as a rule: **for a memory-bound kernel, reducing the operation count does nothing**. You must reduce the bytes moved — by fusing operations so intermediates never leave registers, by using a smaller dtype, or by restructuring to reuse loaded data. This is why `torch.compile` and Triton kernels chase *fusion* rather than arithmetic, and why quantising a model from fp16 to int8 roughly doubles decode throughput even though the arithmetic is the same shape.

WHERE THIS SHOWS UP IN ML | Token generation is a bandwidth problem

Generating one token from a 7-billion-parameter model in fp16 requires reading all 14 GB of weights from HBM, and performing about 14 GFLOPs. On a device with 2 TB/s of bandwidth and 500 TFLOP/s of compute, the read takes 7 ms and the arithmetic takes 0.03 ms. Decoding is 200x memory-bound. Every serving trick you have heard of — batching, speculative decoding, quantisation, paged KV caches — exists to improve that ratio. Chapters 35 and 36.

A checklist you can apply today

CHECKLIST | Before optimising anything that touches data

1. Is the working set larger than L2? If not, stop — memory is not your problem and you should count operations after all.
2. Is the access pattern sequential? If not, can it be made sequential by sorting, by reordering the loop, or by changing the layout?
3. Are you moving fields you do not use? Switch to columnar, or use a narrower dtype.
4. Is the kernel memory-bound? Compute the intensity before you optimise arithmetic.
5. Are you allocating per item? Per-element allocation is the hidden cost in most slow Python data pipelines.
6. Only now: is there an asymptotically better algorithm?

EXERCISES

1. Predict, then measure, the ratio between summing a 4096×4096 float32 NumPy array along axis 0 versus axis 1. Explain the direction of the difference in terms of strides.
2. A structure holds 50 million records of 40 bytes each. Your query reads one 4-byte field of each. How many bytes does an array-of-structs layout move? How many does struct-of-arrays? At 50 GB/s, how long does each take?
3. Implement a singly linked list and a Python list, and time (a) appending a million elements, (b) iterating and summing, © inserting at position 0 a hundred thousand times. Explain each result using this chapter.
4. Compute the arithmetic intensity of a batched matrix multiply with $M=N=4096$, $K=4096$, in fp16. Is it compute-bound on a device with 1.5 TB/s bandwidth and 400 TFLOP/s of fp16 throughput? What if $M=1$?
5. A colleague proposes replacing a `dict` lookup in a hot loop with a sorted array plus binary search, arguing that the array is contiguous. For one million int keys, which is faster and why? Does your answer change at one billion keys?
6. Explain why, in a batched inference server, doubling the batch size often increases throughput almost twofold while barely changing latency per request. Frame your answer entirely in terms of arithmetic intensity.

CHAPTER RECAP

- The flat-memory RAM model is wrong: DRAM latency is roughly 300 cycles, so most programs are waiting, not computing.
- Memory moves in 64-byte cache lines. Sequential access amortises a miss over sixteen float32 values; random access does not.
- The three properties that make code fast are temporal locality, spatial locality and predictability. Design for them explicitly.
- Linked structures lose to arrays at practical sizes because traversal is a chain of dependent cache misses and every node carries allocation overhead.
- Struct-of-arrays moves only the fields you read; this is the reason the entire modern data stack is columnar.
- Compute arithmetic intensity before optimising. For memory-bound kernels, only reducing bytes moved helps — which is what fusion, tiling and quantisation all do.

Arrays, Strides and the NumPy Memory Model

The one data structure you already use for everything

FOUNDATION

PREREQS: Chapters 2--3

The array is the simplest data structure and by far the most important one in machine learning: every tensor, every embedding table, every batch is an array. This chapter takes it seriously — the dynamic array’s growth policy, the stride arithmetic that makes views free, when an operation copies, and the handful of NumPy idioms that replace loops you should never write.

WHAT YOU WILL LEARN

- Explain how an ndarray’s shape, strides and dtype turn indexing into arithmetic
- Predict whether a NumPy operation returns a view or a copy, and why it matters
- Use fancy indexing, boolean masks and `argsort` / `argpartition` fluently
- Understand broadcasting as a stride-0 trick rather than as magic
- Implement a dynamic array and reason about its growth factor
- Recognise the array patterns that appear in every dataloader and inference path

The array, precisely

An array is a contiguous block of memory holding n equal-sized elements. That definition buys you exactly one superpower: the address of element i is $\text{base} + i \times \text{itemsize}$, computable in one multiply and one add. No search, no indirection, no branch.

Operation	Cost	Why
<code>a[i]</code> read or write	$\Theta(1)$	address arithmetic
scan all elements	$\Theta(n)$, cache-friendly	one miss per 64 bytes
append (amortised)	$\Theta(1)$	doubling, Chapter 2
insert or delete at i	$\Theta(n-i)$	<code>memmove</code> of the tail
search unsorted	$\Theta(n)$	no structure to exploit
search sorted	$\Theta(\log n)$	binary search, Chapter 10

The `memmove` in row four is worth a note: it is not a Python loop, it is a single vectorised instruction sequence running at tens of gigabytes per second. Shifting ten thousand elements costs a few microseconds. This is why “arrays are bad at insertion” is much weaker advice than it sounds.

The dynamic array from scratch

Python’s `list` is a dynamic array, and writing one makes its behaviour concrete.

```
A dynamic array with an explicit growth policy

1  class DynArray:
2      def __init__(self, growth=2.0):
3          self._buf = [None] * 1
4          self._n = 0
5          self._growth = growth
6          self.copies = 0                # instrumentation
7
8      def __len__(self):
9          return self._n
10
11     def __getitem__(self, i):
12         if not 0 <= i < self._n:
13             raise IndexError(i)
14         return self._buf[i]           # theta(1)
15
16     def append(self, x):
17         if self._n == len(self._buf):
18             self._resize(max(1, int(len(self._buf) * self._growth)))
19         self._buf[self._n] = x
20         self._n += 1                   # theta(1) amortised
21
22     def insert(self, i, x):
23         self.append(None)              # make room
24         for j in range(self._n - 1, i, -1):
25             self._buf[j] = self._buf[j - 1]
26         self._buf[i] = x               # theta(n - i)
27
28     def _resize(self, cap):
29         new = [None] * cap
30         for j in range(self._n):       # the expensive part
31             new[j] = self._buf[j]
32         self.copies += self._n
33         self._buf = new
```

Run it with different growth factors and count `copies` after a million appends. You will find roughly $2n$ copies at factor 2, $5n$ at factor 1.25, and $n^2/2000$ at a fixed increment of 1000 — the difference between linear and quadratic, decided by one parameter.

PERFORMANCE NOTE | What real implementations choose

CPython grows by roughly $n \gg 3$ extra slots (about 1.125x) for large lists, prioritising memory over copy count. Go doubles up to 1024 elements then uses 1.25x. C++ `std::vector` typically doubles; MSVC uses 1.5x so that freed blocks can be coalesced and reused. Every one of these is a point on the same memory–time curve, and all preserve $\Theta(1)$ amortised append.

Two dimensions: shape, strides, dtype

A NumPy array is not a nested list. It is a flat buffer plus a small header that says how to interpret it.

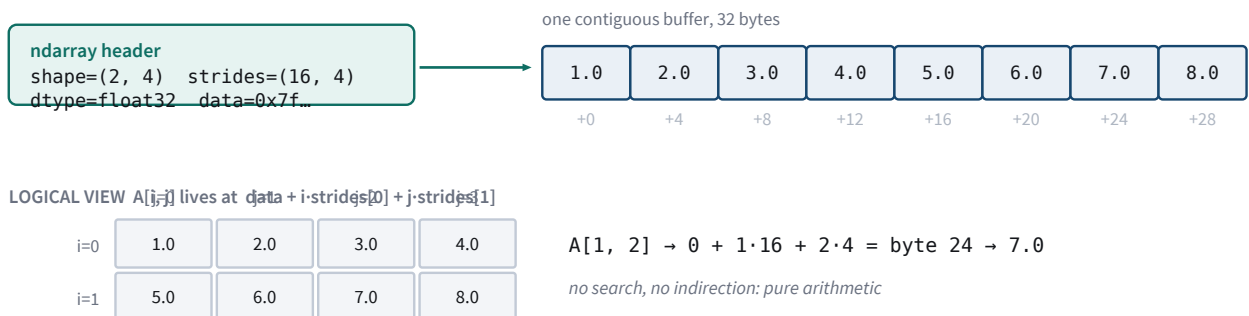


Figure 4.1 An `ndarray` is a pointer to a flat buffer plus a shape and a stride vector. Indexing is a dot product between the index tuple and the strides, then a single memory read.

The rule is one line:

$$\text{address}(i_0, i_1, \dots) = \text{base} + \sum_k i_k \cdot \text{strides}[k]$$

For a C-contiguous (row-major) $m \times n$ float32 array, `strides = (4n, 4)`: moving one step along the last axis moves 4 bytes; moving one step along the first axis skips a whole row. Fortran-contiguous (column-major, as in MATLAB and R) is the transpose of that convention.

This is the whole secret behind NumPy’s speed *and* behind the surprising performance cliffs in Chapter 3. `A[i, :]` is a contiguous run; `A[:, j]` is m reads separated by $4n$ bytes. Same element count, different worlds.

Views versus copies

Because indexing is stride arithmetic, an enormous number of operations can be implemented by *changing the header and reusing the buffer*. Those are views: $\Theta(1)$ time, zero memory. Everything else must copy: $\Theta(n)$ time and memory.

expression	shape	strides (bytes)	what happens
A	(3, 4)	(16, 4)	the original, C-contiguous
A.T	(4, 3)	(4, 16)	swap shape and strides — free
A[:, :2]	(2, 4)	(32, 4)	double the row stride — free
A[:, 1]	(3,)	(16,)	offset the base pointer — free
A.reshape(2,6)	(2, 6)	(24, 4)	free only if contiguous
A.T.reshape(-1)	(12,)	—	must COPY: no stride works

Figure 4.2 Which array expressions are free and which are not. Anything expressible as “same buffer, different shape and strides” costs nothing; anything else allocates.

KEY INSIGHT | The one question to ask

“Can the result be described by a new (offset, shape, strides) triple over the same buffer?” If yes, it is a view and it is free. Basic slicing, transpose, **reshape** on contiguous data, and broadcasting all pass this test. Fancy indexing, boolean masking, and **reshape** across a transpose do not.

Views bite, and views save

```

1  import numpy as np
2  a = np.arange(12).reshape(3, 4)
3
4  b = a[1:, :2]          # VIEW -> writing to b writes to a
5  b[0, 0] = 999
6  assert a[1, 0] == 999
7
8  c = a[[0, 2]]          # COPY (fancy indexing)
9  c[0, 0] = -1
10 assert a[0, 0] == 0
11
12 print(a.T.flags["C_CONTIGUOUS"])    # False: a view with swapped
   strides
13 print(a[:, :2].base is a)           # True: slicing keeps the parent
   alive

```

COMMON PITFALL | Two production bugs that come from views

Silent aliasing. A dataloader does `batch = buffer[i:i+B]` and hands the view to a worker that normalises it in place. The next batch overlaps and sees pre-normalised data. Symptoms: loss that depends on `num_workers`. Use `.copy()` at the boundary.

Accidental memory retention. `small = big_array[:10]` keeps the *entire* `big_array` buffer alive, because the view holds a reference to the base. If `big_array` was 8 GB, you still have 8 GB. `small = big_array[:10].copy()` releases it.

Fancy indexing, masks, and the argsort family

These four idioms replace almost every explicit loop you would otherwise write over a batch.

The idioms worth memorising

```

1  import numpy as np
2  scores = np.random.rand(1_000_000).astype(np.float32)
3
4  # 1. boolean mask: select by predicate.  theta(n), allocates.
5  keep = scores[scores > 0.9]
6
7  # 2. fancy index: gather by position.  theta(len(idx)).
8  idx = np.array([7, 3, 99, 3])
9  picked = scores[idx]                # duplicates allowed
10
11 # 3. full ordering when you need ranks
12 order = np.argsort(-scores)[:10]    # theta(n log n)
13
14 # 4. partial ordering when you only need the top k <- almost always this
15 top = np.argpartition(-scores, 10)[:10] # theta(n), 10-40x faster
16 top = top[np.argsort(-scores[top])]    # then sort just those 10
17
18 # 5. scatter-add: the primitive behind embedding-gradient accumulation
19 grads = np.zeros(1000, dtype=np.float32)
20 np.add.at(grads, idx, 1.0)            # note: slow; see bincount
21 counts = np.bincount(idx, minlength=1000) # the fast path for this case

```

WHERE THIS SHOWS UP IN ML | Gather and scatter are the two primitives of deep learning

An embedding lookup is a gather: `table[token_ids]`, a fancy index producing a (B, L, d) tensor from a (V, d) table. Its backward pass is a scatter-add into the same table, which is why sparse gradients and `EmbeddingBag` exist. Attention's KV cache read is a gather. A `Dataset`'s `__getitem__` over shuffled indices is a gather. Recognising an operation as gather or scatter tells you immediately whether it is random-access (slow, bandwidth-bound) or sequential.

Broadcasting is a stride trick

Broadcasting looks like magic and is not. When NumPy needs an axis of length 1 to act as length n , it sets that axis's stride to **zero**. Every index along it maps to the same memory. No data is copied and no memory is allocated.

Broadcasting, demystified

```

1  import numpy as np
2  a = np.arange(3).reshape(3, 1)    # shape (3,1) strides (8,8)
3  b = np.arange(4).reshape(1, 4)    # shape (1,4) strides (32,8)
4  print(a + b)                      # shape (3,4): a's axis1 stride -> 0
5
6  v = np.broadcast_to(a, (3, 4))
7  print(v.strides)                  # (8, 0) <- the zero is the whole
8  print(v.base is a)                # True: no allocation

```

The rules follow directly: align shapes from the right; an axis matches if the lengths are equal or one of them is 1; a length-1 axis is stretched by giving it stride 0.

COMMON PITFALL | The broadcast that eats your memory

Broadcasting is free only while it stays virtual. The moment you materialise the result it is fully allocated. `(x[:, None] - y[None, :]) ** 2` on two vectors of length 50,000 creates a $50,000^2$ float32 array: 10 GB. The identity $|x-y|^2 = |x|^2 - 2x^{\text{top}}y + |y|^2$ computes the same pairwise distances with one matrix multiply and two $\Theta(n)$ vectors. This is the single most common out-of-memory bug in retrieval code.

Contiguity, and when to pay for it

When to force a copy

```
1 import numpy as np
2 A = np.random.rand(4096, 4096).astype(np.float32)
3
4 s1 = A.T.sum(axis=0)                # strided reads, slow
5 B = np.ascontiguousarray(A.T)      # one big memmove
6 s2 = B.sum(axis=0)                 # sequential reads, fast
```

The copy costs one full pass over the data. If you will touch the transposed array more than about twice, the copy pays for itself. This is exactly the reasoning behind `contiguous()` calls in PyTorch code and behind layout conversions (`NCHW` to `NHWC`) in inference engines: a one-off $\Theta(n)$ cost to make every subsequent kernel run at full bandwidth.

PERFORMANCE NOTE | The dtype is a bandwidth decision

Changing `float64` to `float32` halves every byte moved, and for a memory-bound kernel that is a 2x speedup for free. `float16` or `bfloat16` halves it again. `int8` again. This is why quantisation is such a large win in serving even though it does not reduce the operation count at all (Chapter 3), and it is why you should never let a pipeline silently promote to `float64` — which `np.mean` on an integer array will happily do.

The patterns that show up everywhere

Pattern	Array expression	Where you meet it
Prefix sums	<code>np.cumsum(x)</code>	sampling by cumulative weight, IoU, ragged offsets
Run-length / grouping	<code>np.diff</code> , <code>np.searchsorted</code>	variable-length sequence batching
Ragged batch	values + offsets arrays	nested tensors, <code>EmbeddingBag</code> , CSR graphs
One-hot	<code>np.eye(V)[ids]</code> (never do this)	use a gather instead; <code>V</code> is 50,000

Pattern	Array expression	Where you meet it
Sliding window	<code>stride_tricks.sliding_window_view</code>	n-gram features, conv without a copy
Top-k	<code>argpartition</code> then <code>argsort</code>	reranking, beam search, nucleus sampling
Masking	boolean mask or <code>np.where</code>	padding, attention masks, filtering

Table 4.1 The array’s cost profile. Everything good follows from contiguity; everything bad follows from the fact that inserting in the middle must move the tail.

The *ragged* row deserves emphasis because it is the representation behind a surprising number of systems. When sequences have different lengths, the efficient layout is not a padded matrix but a flat `values` array plus an `offsets` array where sequence i occupies `values[offsets[i]:offsets[i+1]]`. This is the CSR format for sparse matrices (Chapter 22), the layout of PyTorch nested tensors, the layout of Arrow list columns, and the layout of variable-length KV caches (Chapter 36). Padding to the maximum length wastes memory in proportion to the length variance, which for real text is often 60–70%.

A ragged batch without padding

```

1  import numpy as np
2  seqs = [[1, 2, 3], [4], [5, 6, 7, 8, 9]]
3
4  lengths = np.array([len(s) for s in seqs])
5  offsets = np.concatenate([[0], np.cumsum(lengths)]) # [0, 3, 4, 9]
6  values = np.concatenate([np.asarray(s) for s in seqs])
7
8  # Mean of each sequence, no loop, no padding:
9  sums = np.add.reduceat(values, offsets[:-1])
10 means = sums / lengths

```

EXERCISES

1. Instrument `DynArray` and plot total copies against n for growth factors 1.1, 1.5, 2.0 and for a fixed increment of 512. Fit the curves.
2. For a C-contiguous array of shape (B, L, d) with float32 elements, give the strides. Which of `x[:, 0]`, `x[0]`, `x[..., 0]` is contiguous?
3. Write a function that decides, for a given NumPy expression, whether the result shares memory with its input. Verify with `np.shares_memory`.
4. Compute pairwise squared distances between two sets of 20,000 128-dimensional vectors, first with broadcasting and then with the $|x|^2 - 2x^{\text{top}}y + |y|^2$ identity. Compare peak memory and time.
5. Using only `sliding_window_view`, compute a 5-token moving average over a one-million-token array without allocating a copy. Verify the strides.
6. Given a ragged batch as `values` and `offsets`, write a vectorised function returning the maximum of each sequence. (Hint: `np.maximum.reduceat`.)
7. A colleague’s dataloader returns `torch.from_numpy(buf[i:i+B])` where `buf` is a ring buffer that is overwritten by a background thread. Describe the failure mode precisely and give two fixes with different cost profiles.

CHAPTER RECAP

- An array's power is that addressing is arithmetic: $\Theta(1)$ access, perfect spatial locality, vectorisable scans.
- Dynamic arrays give amortised $\Theta(1)$ append by growing geometrically; the growth factor trades memory against copy count.
- An ndarray is a buffer plus (shape, strides, dtype). Indexing is a dot product with the strides.
- An operation is a free view exactly when its result can be described by a new (offset, shape, strides) triple; otherwise it copies. Views cause aliasing bugs and retain their parent's memory.
- Broadcasting is implemented by setting a stride to zero, which is why it is free until you materialise the result — and why materialising it is the most common OOM in retrieval code.
- Gather and scatter are the two access primitives of deep learning; recognising which one you are doing predicts its cost.
- Ragged (values + offsets) layouts beat padding whenever lengths vary, and are the basis of CSR, nested tensors and paged KV caches.

Strings, Bytes and Tokens

The data structure between your text and your model

FOUNDATION

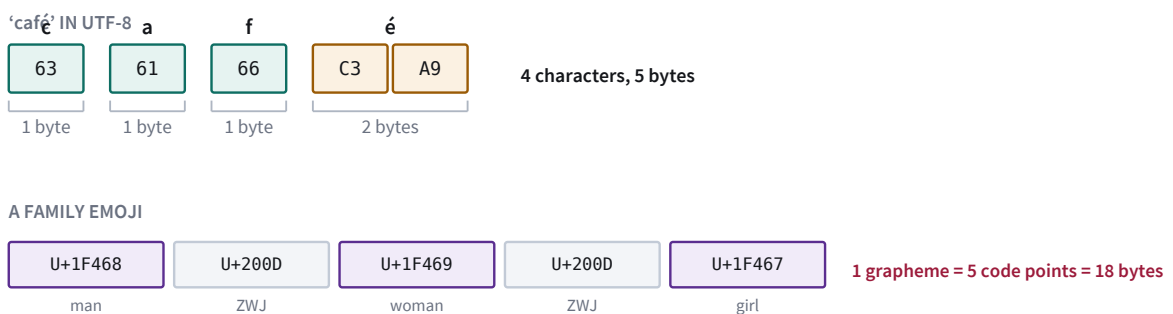
PREREQS: Chapter 4

Text is the input to most modern models, and it is the one data type where almost everyone carries a wrong mental model. A string is not a list of characters; a character is not a byte; and the token your model consumes is a third thing again. This chapter fixes the model, then covers the string algorithms that actually appear in data pipelines.

WHAT YOU WILL LEARN

- Distinguish bytes, code points, grapheme clusters and tokens, and know which one your code means
- Predict the cost of common string operations, including the quadratic traps
- Implement efficient string building, searching and normalisation
- Understand what a tokenizer is doing and why its data structures matter
- Build a rolling hash and use it for substring search and chunking

Four different things called “length”



len() answers a different question in every language. Decide which unit your algorithm means.

Figure 5.1 The same text measured four ways. Nearly every text-processing bug comes from an algorithm that assumes two of these units are the same.

- A **byte** is 8 bits. UTF-8 encodes ASCII in one byte, most European and Middle Eastern scripts in two, most CJK in three, and emoji in four.
- A **code point** is a Unicode scalar value, U+0041 to U+10FFFF. Python 3 strings are sequences of code points, so `len("café") == 4`.

- A **grapheme cluster** is what a reader calls a character. "é" may be one code point (U+00E9) or two (e + combining acute U+0301); a family emoji is five code points joined by zero-width joiners but one glyph.
- A **token** is whatever your tokenizer emits: typically a subword of one to six characters, drawn from a vocabulary of 30,000 to 300,000 entries.

COMMON PITFALL | The truncation bug

`text[:512]` truncates to 512 *code points*, which may be 512 tokens, or 150, or 2000, depending on the language. Truncating to a byte limit can split a multi-byte sequence and produce invalid UTF-8. Truncating a grapheme cluster separates a base character from its combining mark. If you need “512 tokens”, truncate the token ids, not the string — and if you must truncate bytes, back off to a code-point boundary (a byte whose top bits are not 10).

WHERE THIS SHOWS UP IN ML | Why the unit mismatch costs money

Context windows, pricing and positional encodings are all counted in tokens. English averages about 4 characters per token; code averages about 3; Hindi, Thai and Japanese often exceed 1 token per character under a predominantly English BPE vocabulary. A “4000-character” document can be 1000 tokens or 4000 depending on language, which changes both cost and whether it fits. Always measure with the actual tokenizer.

The cost of string operations

Python strings are immutable, contiguous arrays of fixed-width code units (1, 2 or 4 bytes per code point depending on the largest one present). Two consequences dominate.

Operation	Cost	Note
<code>len(s)</code> , <code>s[i]</code>	$\Theta(1)$	fixed-width representation
<code>s + t</code>	$\Theta(\text{len}(s) + \text{len}(t))$	allocates new memory
<code>s += t</code> in a loop	$\Theta(n^2)$ total	the classic trap
<code>"".join(parts)</code>	$\Theta(\text{total})$	one allocation, always use this
<code>s.find(t)</code>	$\Theta(nm)$ worst, near $\Theta(n)$ typical	CPython uses a Boyer-Moore-Horspool/two-way hybrid
<code>s.split(separator)</code>	$\Theta(n)$	allocates one object per piece
<code>s.replace(a, b)</code>	$\Theta(n)$	single pass
<code>re match</code>	depends	backtracking engines can be exponential
<code>s.encode("utf-8")</code>	$\Theta(n)$	allocates

Operation	Cost	Note
<code>hash(s)</code>	$\Theta(n)$ first time	cached on the object afterwards

Building a large string: three ways

```

1  # theta(n^2): each += copies everything built so far
2  out = ""
3  for piece in pieces:
4      out += piece
5
6  # theta(n): one allocation at the end. Always this.
7  out = "".join(pieces)
8
9  # theta(n) and streaming: when the result is too big to hold
10 import io
11 buf = io.StringIO()
12 for piece in pieces:
13     buf.write(piece)
14 out = buf.getvalue()

```

WATCH OUT | Catastrophic backtracking

A regular expression like `(a+)+b` against a string of 30 `a` s takes exponential time in a backtracking engine, which includes Python's `re`. This is a real denial-of-service vector when patterns touch user text. Fixes: avoid nested quantifiers, use possessive/atomic groups via the `regex` module, or switch to `re2`-style automaton matching, which is $\Theta(nm)$ worst case with no backtracking. Chapter 14 explains the automaton.

Rolling hashes: fingerprinting a window in $\Theta(1)$

A surprising amount of text infrastructure rests on one idea: maintain a hash of a sliding window so that moving the window by one character costs constant time.

Treat the window as a number in base B modulo a large prime M :

$$H(s_i \dots s_{i+m-1}) = \sum_{j=0}^{m-1} s_{i+j} \cdot B^{m-1-j} \bmod M$$

Sliding one step removes the leading term and appends a new one:

$$H' = ((H - s_i \cdot B^{m-1}) \cdot B + s_{i+m}) \bmod M$$

Rabin-Karp: substring search in expected linear time

```

1  def rabin_karp(text: str, pat: str, B: int = 257, M: int = (1 << 61) - 1):
2      n, m = len(text), len(pat)
3      if m == 0 or m > n:
4          return []
5      high = pow(B, m - 1, M)
6      hp = ht = 0
7      for i in range(m):
8          hp = (hp * B + ord(pat[i])) % M
9          ht = (ht * B + ord(text[i])) % M
10     hits = []

```

```

11     for i in range(n - m + 1):
12         if hp == ht and text[i:i + m] == pat:      # verify: hashes can
13             collide
14             hits.append(i)
15         if i + m < n:
16             ht = ((ht - ord(text[i]) * high) * B + ord(text[i + m])) % M
17     return hits

```

Expected $\Theta(n + m)$; worst case $\Theta(nm)$ if an adversary forces collisions, which is why M should be large and, in adversarial settings, randomly chosen per process.

WHERE THIS SHOWS UP IN ML | Three places rolling hashes appear in ML pipelines

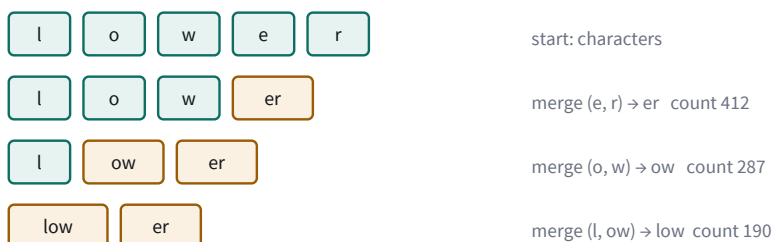
Content-defined chunking. To split documents into chunks that stay stable when the document is edited, cut wherever the rolling hash of the last 48 bytes has k low zero bits. An insertion near the start no longer shifts every downstream boundary, so deduplication and incremental re-embedding both keep working. This is how `rsync`, `restic` and most vector-store ingestion pipelines chunk.

Exact deduplication and contamination checks. Hash every 50-token window of every document; any test-set window appearing in training data is contamination. Chapter 30 does this at corpus scale.

The hashing trick. Map an unbounded feature space to m buckets with `hash(feature) % m`, avoiding a vocabulary entirely. Collisions act as weight sharing and are usually harmless; add a sign hash to keep the inner-product estimate unbiased. Chapter 7.

What a tokenizer actually is

A tokenizer is two algorithms wearing a trench coat: a training procedure that builds a vocabulary, and an encoding procedure that applies it.



Each step needs argmax over pair counts — a priority queue with lazy deletion (Chapter 14).

Figure 5.2 Byte-pair encoding training. Repeatedly find the most frequent adjacent pair across the corpus and merge it into a new symbol. The vocabulary is the ordered list of merges.

Training (BPE). Start with every byte as a symbol. Count all adjacent pairs across the corpus; merge the most frequent into a new symbol; repeat until the vocabulary reaches the target size. The naive implementation recounts every pair after each merge: $\Theta(V \cdot N)$, which is days on a large corpus. The real implementation keeps a priority queue of pair counts and an index from each pair to the positions where it occurs, so a merge only updates the pairs adjacent to the merged positions — roughly $\Theta(N + V \log V)$. Chapter 13 builds the queue and Chapter 14 the index.

Encoding. Given the merge list, apply merges to a word in rank order. The straightforward implementation is $\Theta(m^2)$ per word for a word of m characters; using a linked list of symbols plus a heap of candidate merges makes it $\Theta(m \log m)$. WordPiece instead does greedy longest-match against a trie, which is $\Theta(m)$ amortised. Unigram (SentencePiece) runs Viterbi over a lattice to find the highest-probability segmentation — a dynamic program, Chapter 19.

Greedy longest-match encoding against a trie

```

1  class TrieNode:
2      __slots__ = ("children", "token_id")
3      def __init__(self):
4          self.children = {}
5          self.token_id = None
6
7  def build(vocab):                                # vocab: token string -> id
8      root = TrieNode()
9      for tok, tid in vocab.items():
10         node = root
11         for ch in tok:
12             node = node.children.setdefault(ch, TrieNode())
13         node.token_id = tid
14     return root
15
16 def encode(root, text, unk):
17     out, i, n = [], 0, len(text)
18     while i < n:
19         node, j, best = root, i, None
20         while j < n and text[j] in node.children:
21             node = node.children[text[j]]
22             j += 1
23             if node.token_id is not None:
24                 best = (node.token_id, j)    # remember the longest match
25         if best is None:
26             out.append(unk)
27             i += 1
28         else:
29             out.append(best[0])
30             i = best[1]
31     return out

```

This is $\Theta(\text{total matched length})$ and is exactly what WordPiece does after its `##` continuation handling. Chapter 14 extends the trie to an Aho–Corasick automaton, which finds *all* occurrences of *all* vocabulary entries in one pass — the basis of profanity filters, PII scrubbers and constrained decoding.

COMMON PITFALL | Tokenizers are not reversible in the way you expect

`decode(encode(s))` may not equal `s`: normalisation (NFKC, lowercasing, whitespace collapsing) is lossy, and byte-level BPE round-trips bytes but not necessarily the original Unicode normalisation form. If you are computing character-level spans — for extractive QA, for highlighting, for alignment — you must carry an explicit offset mapping from tokens back to byte ranges. Every serious tokenizer library exposes one; use it rather than re-searching the string.

Normalisation, the unglamorous prerequisite

Before any of this, text needs normalising, and the choices are consequential.

- **Unicode normalisation.** NFC composes `e + ´` into `é`; NFD decomposes. NFKC additionally folds compatibility characters — full-width Latin, ligatures, superscripts. Pick one and apply it everywhere, including at query time in a retrieval system, or identical-looking strings will fail to match.
- **Case folding.** `str.lower()` is not the same as `str.casefold()` (which handles German `ß` to `ss`), and neither is safe for Turkish dotted/dotless `i` without locale awareness.
- **Whitespace and control characters.** Zero-width spaces, non-breaking spaces, and bidirectional control characters are invisible and will silently split your tokens. Strip the `Cf` category unless you have a reason not to.
- **Homoglyphs.** Cyrillic `а` and Latin `a` are different code points that render identically; this matters for deduplication, and for anyone trying to slip past a filter.

KEY INSIGHT | Normalise once, at the boundary

The rule that prevents most text bugs: normalise on ingestion, store the normalised form, and never normalise again. Systems that normalise lazily in several places end up with different normalisations in the index and in the query path, which produces silent recall loss that no test catches.

EXERCISES

1. Write a function that truncates a UTF-8 byte string to at most k bytes without splitting a code point. Test it on text containing emoji.
2. Measure characters-per-token for the same paragraph translated into English, German, Hindi and Japanese using any BPE tokenizer. Explain the spread.
3. Implement content-defined chunking with a 48-byte rolling hash and a 13-bit boundary condition. Show that inserting a character at the start of a document changes only the first chunk.
4. Build the greedy trie encoder above for a 5,000-entry vocabulary and compare its throughput to a regex-based tokenizer.
5. Construct a regex and an input for which Python's `re` takes more than ten seconds on a 40-character string. Then rewrite the regex so it does not.
6. Given a token sequence and an offset mapping, write a function that maps a character span to the minimal covering token span. This is the core of extractive QA evaluation.

CHAPTER RECAP

- Bytes, code points, grapheme clusters and tokens are four different units; almost every text bug is a confusion between two of them.
- Python strings are immutable: `+=` in a loop is $\Theta(n^2)$, `join` is $\Theta(n)$. Regex backtracking can be exponential.
- A rolling hash makes a sliding-window fingerprint $\Theta(1)$ per step, and underpins substring search, content-defined chunking, deduplication and the hashing trick.
- A tokenizer is a training algorithm (priority queue plus position index for BPE; Viterbi for Unigram) and an encoding algorithm (trie longest-match, or ranked merges).
- Carry an explicit offset mapping if you need character spans; do not re-search the string.
- Normalise once, on ingestion, and use the same normalisation in the index and the query path.

PART II

The Core Toolkit

The seven structures and four techniques that appear in every pipeline you will ever write, taught properly: what they cost, when they break, and how they are really used.

-
- 06 Linked Structures: Lists, Stacks, Queues, Ring Buffers
 - 07 Hashing: Dictionaries, Sets and the Hashing Trick
 - 08 Recursion, Memoisation and the Call Stack
 - 09 Sorting and Order Statistics
 - 10 Binary Search and Searching the Answer Space
 - 11 Two Pointers and Sliding Windows

Linked Structures: Lists, Stacks, Queues, Ring Buffers

Four structures that differ only in which end you touch

FOUNDATION

PREREQS: Chapter 4

Stacks, queues, deques and ring buffers are the same idea with different access rules, and that rule is often the entire difference between two algorithms — swapping a stack for a queue turns depth-first search into breadth-first search and nothing else changes. This chapter builds all of them, then uses them to build an LRU cache and a replay buffer.

WHAT YOU WILL LEARN

- Implement singly and doubly linked lists and know exactly when they earn their overhead
- Use stacks and queues fluently, including the iterative rewrite of any recursion
- Build a deque and understand why `list.pop(0)` is the most common Python performance bug
- Implement a ring buffer with $\Theta(1)$ push and eviction and zero allocation
- Build an $O(1)$ LRU cache from a hash map plus a doubly linked list
- Recognise these structures inside dataloaders, replay buffers and serving queues

The linked list, done once

A linked list stores each element in its own node together with a pointer to the next node. Its virtue is that inserting or removing a node you already hold a reference to costs $\Theta(1)$ with no shifting. Its vice is Chapter 3: every traversal step is a dependent cache miss.

Singly linked list with $O(1)$ push-front and $O(1)$ append

```
1 class Node:
2     __slots__ = ("val", "next")          # __slots__ saves ~50% of the
3     memory
4     def __init__(self, val, nxt=None):
5         self.val, self.next = val, nxt
6
7 class LinkedList:
8     def __init__(self):
9         self.head = self.tail = None
```



```

9         self.n = 0
10
11     def push_front(self, v):                # theta(1)
12         self.head = Node(v, self.head)
13         if self.tail is None:
14             self.tail = self.head
15         self.n += 1
16
17     def append(self, v):                    # theta(1) thanks to the tail
18         pointer
19         node = Node(v)
20         if self.tail is None:
21             self.head = self.tail = node
22         else:
23             self.tail.next = node
24             self.tail = node
25         self.n += 1
26
27     def pop_front(self):                   # theta(1)
28         if self.head is None:
29             raise IndexError("empty")
30         v, self.head = self.head.val, self.head.next
31         if self.head is None:
32             self.tail = None
33         self.n -= 1
34         return v
35
36     def reverse(self):                     # theta(n), theta(1) extra space
37         prev, cur = None, self.head
38         self.tail = cur
39         while cur:
40             cur.next, prev, cur = prev, cur, cur.next
41             self.head = prev
42
43     def __iter__(self):
44         cur = self.head
45         while cur:
46             yield cur.val
47             cur = cur.next

```

The `reverse` method deserves attention because it is the archetype of a whole family of pointer manipulations: three variables, one simultaneous assignment, $\Theta(1)$ extra space. Cycle detection is the other archetype.

Floyd's cycle detection: constant space, linear time

```

1  def has_cycle(head):
2      slow = fast = head
3      while fast and fast.next:
4          slow, fast = slow.next, fast.next.next
5          if slow is fast:
6              return True
7      return False

```

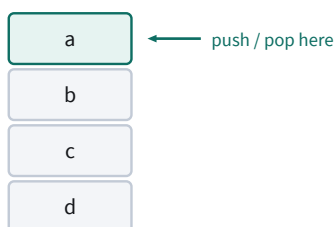
Why it works: if there is a cycle, the fast pointer enters it and gains one position per step on the slow pointer, so it must eventually coincide with it — within $\Theta(n)$ steps. The same two-speed idea finds the middle of a list in one pass and detects cycles in a functional graph, which is how Pollard’s rho factorisation works.

COMMON PITFALL | When you actually want a linked list in Python

Almost never on its own. You want one *inside* another structure: the recency list of an LRU cache, the free list of an allocator, the collision chain of a hash table, the symbol list during BPE encoding. In each case the list is not traversed from the head — you always arrive at a node via a hash map — so the cache-miss penalty never materialises and the $\Theta(1)$ splice is the whole point.

Stacks and queues: the same array, different ends

STACK — LIFO



depth-first search

QUEUE — FIFO



breadth-first search

`list.pop(0)` is $\Theta(n)$; `deque.popleft()` is $\Theta(1)$

Figure 6.1 A stack pushes and pops at one end; a queue pushes at one end and pops at the other. This is the only difference between depth-first and breadth-first search.

A stack is a Python list used with `append` and `pop` : both $\Theta(1)$ amortised, both cache-friendly, nothing more is needed. A queue is *not* a Python list, because `pop(0)` shifts every remaining element.

Structure	Push	Pop	Python
Stack (LIFO)	<code>append</code> $\Theta(1)$	<code>pop()</code> $\Theta(1)$	<code>list</code>
Queue (FIFO)	<code>append</code> $\Theta(1)$	<code>popleft</code> $\Theta(1)$	<code>collections.deque</code>
Queue, wrongly	<code>append</code> $\Theta(1)$	<code>pop(0)</code> $\Theta(n)$	<code>list</code> — never
Deque	both ends $\Theta(1)$	both ends $\Theta(1)$	<code>collections.deque</code>

KEY INSIGHT | One line separates DFS from BFS

```

1 frontier.pop()      # stack -> depth-first
2 frontier.popleft()  # queue -> breadth-first
3 heapq.heappop()     # heap -> best-first / Dijkstra / A*
```

The traversal skeleton is identical. Chapter 22 makes this explicit, and Chapter 35 shows that beam search is the same skeleton with a bounded priority queue.

Converting recursion to iteration

Every recursion is a loop plus an explicit stack — the machine does exactly this, and doing it manually is how you avoid `RecursionError` on deep structures.

Recursive and iterative in-order traversal

```

1  def inorder_rec(node, out):
2      if node:
3          inorder_rec(node.left, out)
4          out.append(node.val)
5          inorder_rec(node.right, out)
6
7  def inorder_iter(root):
8      out, stack, node = [], [], root
9      while stack or node:
10         while node:                # push the whole left spine
11             stack.append(node)
12             node = node.left
13         node = stack.pop()
14         out.append(node.val)
15         node = node.right          # then go right and repeat
16     return out

```

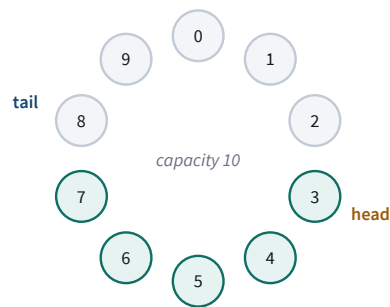
The iterative version uses the same $\Theta(h)$ memory as the recursive one, but on the heap rather than the C stack, so it survives a million-node degenerate tree. This matters in practice for deep JSON, deep expression graphs, and linked lists of parsed records.

WHERE THIS SHOWS UP IN ML | Where the stack is the algorithm

Reverse-mode automatic differentiation is a stack discipline: the forward pass pushes the values needed by each operation's backward function, and the backward pass pops them in reverse order. That stack is the *activation memory* that dominates training memory, and gradient checkpointing is the decision to push fewer things and recompute them instead — a direct space–time trade, analysed in Chapter 18 and applied in Chapter 24.

The ring buffer

When you need a bounded FIFO with no allocation — the common case in streaming systems — use a fixed array and two indices modulo its length.



<code>push(x)</code>	<code>buf[tail] = x; tail = (tail+1) % N</code>	$\Theta(1)$
<code>full?</code>	overwrite oldest — no allocation ever	$\Theta(1)$
<code>sample(k)</code>	k random indices into the live range	$\Theta(k)$
<code>memory</code>	exactly N slots, forever	$\Theta(N)$

This is the DQN replay buffer, the streaming-metrics window, and the token ring in a decoding server.

Figure 6.2 A ring buffer. The array never grows, pushes never allocate, and when it is full the oldest element is silently overwritten.

Ring buffer with uniform sampling, as used for experience replay

```

1  import numpy as np
2
3  class ReplayBuffer:
4      def __init__(self, capacity, obs_dim):
5          self.cap = capacity
6          self.obs = np.zeros((capacity, obs_dim), dtype=np.float32)
7          self.act = np.zeros(capacity, dtype=np.int64)
8          self.rew = np.zeros(capacity, dtype=np.float32)
9          self.done = np.zeros(capacity, dtype=bool)
10         self.pos = 0
11         self.full = False
12
13         def push(self, o, a, r, d):          # theta(1), never allocates
14             i = self.pos
15             self.obs[i], self.act[i], self.rew[i], self.done[i] = o, a, r, d
16             self.pos = (i + 1) % self.cap
17             self.full |= self.pos == 0
18
19         def __len__(self):
20             return self.cap if self.full else self.pos
21
22         def sample(self, batch):            # theta(batch)
23             idx = np.random.randint(0, len(self), size=batch)
24             return self.obs[idx], self.act[idx], self.rew[idx], self.done[idx]
```

Note the layout: struct-of-arrays, not an array of transition objects (Chapter 3). Sampling a batch is then four contiguous gathers rather than `batch` pointer dereferences, and the whole buffer is four flat arrays that can be memory-mapped or moved to a GPU as-is.

PERFORMANCE NOTE | Preallocate, then never allocate again

The ring buffer's real virtue in a training loop is not its $\Theta(1)$ push — it is that steady-state memory is exactly **capacity** and the allocator is never touched. A growing list of transition objects fragments the heap, triggers garbage collection pauses in the middle of training, and makes memory usage a function of how long the job has been running. Bounded structures make performance predictable, which is worth more than raw speed.

The LRU cache: hash map plus doubly linked list

An LRU cache must answer three questions in $\Theta(1)$: does key k exist, what is its value, and which key is least recently used? A hash map answers the first two. A doubly linked list in recency order answers the third. The trick is to store the list *node* as the map's value, so you can splice in $\Theta(1)$ without traversing.

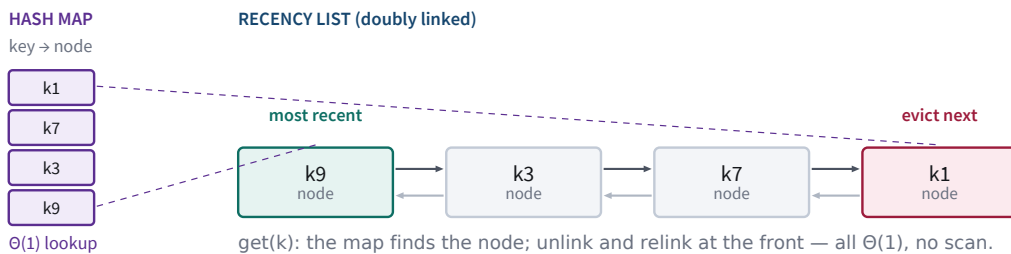


Figure 6.3 The classic $O(1)$ LRU. The hash map gives random access to a node; the doubly linked list gives ordering. Neither structure alone is enough.

```
O(1) LRU cache

1 class _N:
2     __slots__ = ("k", "v", "prev", "next")
3     def __init__(self, k=None, v=None):
4         self.k, self.v, self.prev, self.next = k, v, None, None
5
6 class LRUCache:
7     def __init__(self, cap):
8         self.cap = cap
9         self.map = {}
10        self.head, self.tail = _N(), _N()    # sentinels remove edge
11        cases
12        self.head.next, self.tail.prev = self.tail, self.head
13
14    def _unlink(self, n):
15        n.prev.next, n.next.prev = n.next, n.prev
16
17    def _push_front(self, n):
18        n.next, n.prev = self.head.next, self.head
19        self.head.next.prev = self.head.next = n
20
21    def get(self, k, default=None):
22        n = self.map.get(k)
23        if n is None:
```

```

23         return default
24     self._unlink(n)
25     self._push_front(n)           # mark as most recent
26     return n.v
27
28     def put(self, k, v):
29         n = self.map.get(k)
30         if n is not None:
31             n.v = v
32             self._unlink(n)
33             self._push_front(n)
34             return
35         if len(self.map) >= self.cap:
36             lru = self.tail.prev   # evict from the back
37             self._unlink(lru)
38             del self.map[lru.k]
39         n = _N(k, v)
40         self.map[k] = n
41         self._push_front(n)

```

The two sentinel nodes are not decoration: without them every operation needs four special cases for empty, single-element, front and back. Sentinels are a technique worth internalising — they reappear in skip lists, in segment trees, and in every balanced-tree implementation.

WHERE THIS SHOWS UP IN ML | Caching in ML systems, and where LRU is the wrong policy

LRU is the default for embedding caches, feature-store reads and tokenizer memoisation. It is the *wrong* default in three common situations. (1) A full scan of a dataset larger than the cache evicts everything useful — this is sequential flooding, and the fix is a scan-resistant policy such as 2Q, ARC or **SLRU**. (2) When items have very different costs (recomputing a 1024-dimension embedding versus a scalar), use GDSF or a simple cost-weighted priority queue. (3) When items have different sizes — variable-length KV-cache blocks — use a size-aware policy, which is exactly what vLLM's block manager implements (Chapter 36).

Monotonic stacks and queues

One more pattern, because it converts several quadratic algorithms into linear ones: maintain a stack (or deque) whose contents are always sorted.

Sliding-window maximum in O(n) with a monotonic deque

```

1  from collections import deque
2
3  def window_max(xs, k):
4      dq = deque()           # holds indices, values decreasing
5      out = []
6      for i, x in enumerate(xs):
7          while dq and xs[dq[-1]] <= x:
8              dq.pop()       # x dominates everything smaller
9          dq.append(i)
10         if dq[0] <= i - k:
11             dq.popleft()    # drop indices that left the window

```

```

12         if i >= k - 1:
13             out.append(xs[dq[0]])
14         return out

```

Every index is pushed once and popped once, so the total work is $\Theta(n)$ despite the inner `while`. The same idea — discard elements that can never be the answer again — gives the “next greater element” problem, the largest rectangle in a histogram, and online maximum drawdown. It is one of the highest-yield patterns in Part IX’s catalogue.

EXERCISES

1. Implement `reverse_k_group`: reverse every consecutive group of k nodes in a singly linked list, $\Theta(n)$ time, $\Theta(1)$ extra space.
2. Extend Floyd’s algorithm to return the *first node of the cycle*, and prove the pointer arithmetic that makes it work.
3. Implement a queue using two stacks with $\Theta(1)$ amortised `dequeue`. Give the potential-function argument (Chapter 2).
4. Measure `list.pop(0)` against `deque.popleft()` for $n=10^5$ and $n=10^6$. Confirm the quadratic.
5. Add a `sample` method to `LRUCache` that returns the k most recent keys in $\Theta(k)$.
6. Modify `ReplayBuffer` to store n -step returns, keeping $\Theta(1)$ push. What breaks when an episode boundary falls inside the n -step window?
7. Use a monotonic deque to compute, for each position of a 1M-element array, the maximum over the preceding 1000 elements. Compare to `np.lib.stride_tricks.sliding_window_view(...).max(axis=1)` in time and peak memory.

CHAPTER RECAP

- A linked list buys $\Theta(1)$ splicing at a node you already hold, at the cost of pointer chasing; it is almost always a component inside another structure rather than a standalone container.
- Two pointer idioms are worth memorising: three-variable reversal, and Floyd’s two-speed cycle detection.
- Stack versus queue is the only difference between DFS and BFS; swapping in a heap gives best-first search.
- Any recursion becomes a loop plus an explicit stack, which is how you avoid stack overflow on deep structures. Autograd is this pattern.
- Ring buffers give bounded memory, $\Theta(1)$ push and zero allocation — the right structure for replay buffers and streaming windows.
- A hash map plus a doubly linked list gives $\Theta(1)$ LRU; sentinels remove all the edge cases. LRU is wrong under sequential flooding, uneven costs, or variable item sizes.
- A monotonic stack or deque turns several quadratic scan problems into linear ones by discarding elements that can never be the answer again.

Hashing: Dictionaries, Sets and the Hashing Trick

Constant-time lookup, and the fine print that makes it true

FOUNDATION

PREREQS: Chapters 2–4

The hash table is the most-used non-trivial data structure in existence and the one most people use without a model of how it works. That model pays for itself the first time a lookup becomes slow, a dict eats forty gigabytes, or two runs of the same script produce different results. This chapter builds hash tables from scratch, then applies them to deduplication, feature hashing and vocabulary management.

WHAT YOU WILL LEARN

- Explain what a hash function must guarantee and what it must not
- Implement separate chaining and open addressing, and choose between them
- Reason about load factor, resizing and the amortised cost of insertion
- Predict and fix the memory cost of large Python dictionaries
- Apply the hashing trick, and know when collisions will and will not hurt
- Recognise the security and reproducibility traps in hashing

What a hash table is

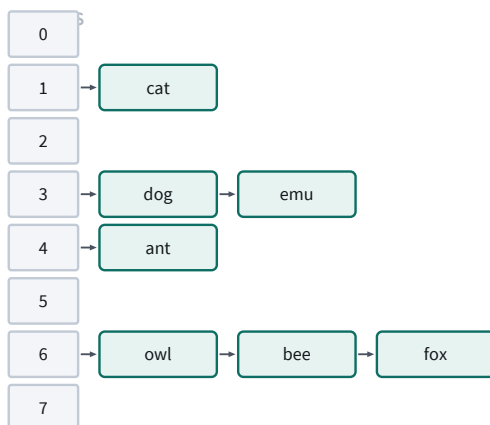
A hash table stores key–value pairs in an array of m buckets. A hash function h maps a key to a bucket index. If h spreads keys evenly, each bucket holds about n/m keys, and with m proportional to n that is a constant — hence $\Theta(1)$ expected lookup.

Everything interesting is in the two failure modes: *collisions* (two keys in one bucket) and *growth* (what happens when n outruns m).

DEFINITION | What a hash function must do

A hash function $h: K \rightarrow \{0, \dots, m-1\}$ should be **fast** (a few nanoseconds), **deterministic** within a process, and **uniform**: for keys drawn from any realistic distribution, the bucket indices should look uniformly distributed and independent. It does *not* need to be cryptographic; it needs to be hard to hit collisions *by accident*, and — when keys come from users — hard to hit them on purpose.

Separate chaining



load factor $\alpha = n / \text{buckets}$

expected chain length = α

expected probes $\approx 1 + \alpha/2$

Resize when α exceeds ~ 0.75

→ amortised $\Theta(1)$, like the dynamic array

Worst case: every key in one bucket → $\Theta(n)$.

Figure 7.1 Separate chaining: each bucket holds a list of entries. With load factor $\alpha = n/m$, an unsuccessful lookup scans α entries on average.

A hash map with separate chaining and resizing

```

1  class HashMap:
2      def __init__(self, capacity=8, max_load=0.75):
3          self._buckets = [[] for _ in range(capacity)]
4          self._n = 0
5          self._max_load = max_load
6
7      def _index(self, key):
8          return hash(key) & (len(self._buckets) - 1)  # power-of-2 mask
9
10     def get(self, key, default=None):
11         for k, v in self._buckets[self._index(key)]:
12             if k == key:  # == not 'is': equal keys must
13                 match
14                 return v
15         return default
16
17     def put(self, key, value):
18         bucket = self._buckets[self._index(key)]
19         for i, (k, _) in enumerate(bucket):
20             if k == key:
21                 bucket[i] = (key, value)
22                 return
23         bucket.append((key, value))
24         self._n += 1
25         if self._n > self._max_load * len(self._buckets):
26             self._resize(len(self._buckets) * 2)
27
28     def _resize(self, capacity):
29         old = self._buckets
30         self._buckets = [[] for _ in range(capacity)]
31         for bucket in old:
32             for k, v in bucket:
33                 self._buckets[hash(k) & (capacity - 1)].append((k, v))

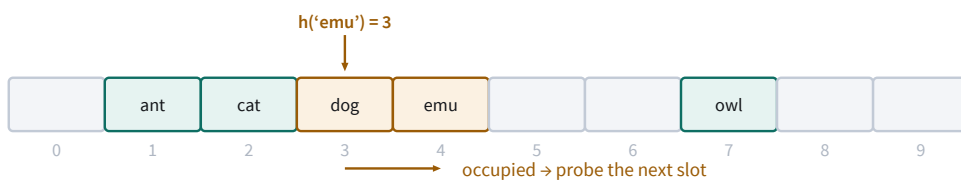
```

Two details in `_index` matter. Using a power-of-two capacity lets us replace a modulo (about 20 cycles) with a bitwise AND (one cycle). The cost is that we now depend on the *low* bits of the hash being well mixed — which is why Java’s `HashMap` XORs the high bits down before masking, and why using a power-of-two table with a weak hash such as Python’s identity hash for small ints can produce clustering.

The resize is the dynamic-array argument again: doubling makes rehashing $\Theta(1)$ amortised per insertion. But note that rehashing is $\Theta(n)$ in one operation, so a hash table gives amortised, not worst-case, $\Theta(1)$ insertion. In a latency-sensitive path this is a real spike; incremental resizing (Redis does this) spreads the work over subsequent operations.

Open addressing

The alternative is to keep everything in one array and, on collision, probe for another slot.



One contiguous array: a probe sequence is a cache-line scan, not a pointer chase.

Deletion needs a tombstone, and clustering makes performance collapse above $\alpha \approx 0.8$.

Figure 7.2 Linear probing. All entries live in a single contiguous array, so a probe sequence is a cache-line scan rather than a pointer chase.

Scheme	Probe sequence	Pros	Cons
Linear	$h, h+1, h+2, \dots$	perfect locality, simple	primary clustering
Quadratic	$h, h+1, h+4, h+9, \dots$	less clustering	worse locality
Double hashing	$h_1, h_1+h_2, h_1+2h_2, \dots$	near-uniform probing	two hashes, poor locality
Robin Hood	linear, but steal from the rich	low variance in probe length	more work per insert
Swiss / F14	SIMD scan of 16 control bytes	fastest in practice	complex

Open addressing wins on speed below a load factor of about 0.7 because the probe sequence is contiguous. It degrades sharply above that — the expected probe count for linear probing is roughly $1/2(1 + 1/(1-\alpha)^2)$, which at $\alpha=0.9$ is 50 probes. Chaining degrades gracefully but pays a pointer dereference on every lookup.

COMMON PITFALL | Deletion in open addressing

You cannot simply clear a slot: doing so breaks the probe sequences of entries that were displaced past it. You must write a *tombstone*, a marker meaning “empty for insertion, keep probing for lookup”. Tables that delete heavily accumulate tombstones and must be rehashed periodically. If you ever write your own table, this is the bug you will have.

Python’s dict, concretely

CPython’s `dict` is open-addressed with a perturbation-based probe sequence, and since version 3.6 it is *split*: a compact array of `(hash, key, value)` entries in insertion order, plus an index array of small integers pointing into it. That layout is why dicts preserve insertion order and why they got about 25% smaller in 3.6.

Structure	Bytes per entry (CPython 3.11, 64-bit)
<code>dict</code> of int to int	≈100 including the boxed ints
<code>set</code> of int	≈60
<code>list</code> of int	≈36 (8 pointer + 28 int object)
<code>array.array("q")</code>	8
<code>np.int64</code> array	8
<code>dict</code> of str to int, 12-char keys	≈170

Table 7.1 Open-addressing schemes. Modern high-performance tables (Abseil’s `flat_hash_map`, Rust’s `hashbrown`, Folly’s `F14`) all use a variant of the last row: a separate array of one-byte tags scanned sixteen at a time with SIMD.

COMMON PITFALL | The forty-gigabyte vocabulary

A `dict` mapping 200 million integer user ids to integer row indices needs roughly 20 GB in CPython and will be killed by the OOM reaper. Three fixes, in increasing order of effort: (1) sort the ids into a NumPy array and use `np.searchsorted` — 1.6 GB, $\Theta(\log n)$ lookup, and vectorised over a whole batch at once; (2) use a perfect hash or a minimal perfect hash if the key set is static; (3) drop the mapping entirely and use the hashing trick, below.

Hashing your own objects

Correct hash and eq

```

1 class Doc:
2     __slots__ = ("doc_id", "shard")
3     def __init__(self, doc_id, shard):
4         self.doc_id, self.shard = doc_id, shard
5
6     def __eq__(self, other):
7         return (isinstance(other, Doc)
8                 and (self.doc_id, self.shard) == (other.doc_id,
9                 other.shard))

```

```

9
10     def __hash__(self):
11         return hash((self.doc_id, self.shard)) # tuple hash mixes
        properly

```

Three invariants, and violating any of them produces a bug that is very hard to find:

1. **If $a == b$ then $\text{hash}(a) == \text{hash}(b)$.** Otherwise equal keys land in different buckets and your set contains duplicates.
2. **The hash must not change while the object is in a container.** Mutating a field used by `__hash__` makes the entry unreachable — it is still in the table, occupying memory, and `k in d` returns `False`.
3. **Hash only immutable, identity-defining fields.** Never hash a floating-point score or a timestamp.

WATCH OUT | `hash()` of a string is randomised per process

Python salts string and bytes hashes with a per-process random seed (`PYTHONHASHSEED`) to prevent collision-based denial of service. The consequence is that `hash("abc")` differs between runs, so anything you *persist* or *shard on* must use an explicit stable hash — `hashlib.blake2b`, `xxhash`, `mmh3` — not the builtin. Sharding a dataset with `hash(key) % n` will silently reassign every record on restart.

The hashing trick

Suppose you need feature indices for an unbounded, growing categorical space: user agents, URL n-grams, product ids in a live catalogue. The textbook answer is a vocabulary dict, which must be built, stored, versioned and synchronised between training and serving. The hashing trick removes it entirely.

$$\text{index}(f) = h(f) \bmod m, \text{sign}(f) = \pm 1 \text{ from a second hash}$$

Signed feature hashing

```

1  import xxhash, numpy as np
2
3  def hash_features(features, m=2 ** 20):
4      x = np.zeros(m, dtype=np.float32)
5      for f, value in features.items():
6          h = xxhash.xxh64(f).intdigest()
7          idx = h % m
8          sign = 1.0 if (h >> 63) & 1 else -1.0 # independent bit
9          x[idx] += sign * value
10     return x

```

The signed variant matters. Without signs, colliding features add constructively and the inner product between two hashed vectors is biased upward. With random signs the collisions cancel in expectation, so $\mathbb{E}[\langle \phi(a), \phi(b) \rangle] = \langle a, b \rangle$: the hashed inner product is an unbiased estimator of the true one, with variance falling as $1/m$.

WHERE THIS SHOWS UP IN ML | Where the hashing trick is the right call

- **Very high-cardinality categoricals** in ranking and ads models, where a vocabulary would be larger than the model.
- **Online / streaming features** where new values appear constantly and a frozen vocabulary would map them all to **UNK**.
- **Train-serve consistency**, because there is no artefact to keep in sync; the hash function is the vocabulary.

And where it is not: any setting where you must explain or inspect a feature (collisions destroy interpretability), where the vocabulary is small and stable (a dict is exact and just as fast), or where a single collision between two very high-signal features would be costly. In practice, choose m so that the expected number of colliding *frequent* features is small; $m=2^{20}$ to 2^{24} is typical.

Deduplication, the canonical application

Exact dedup of a document stream in one pass

```

1  import hashlib
2
3  def dedup(stream):
4      seen = set()                                # stores 16-byte digests, not
        documents
5      for doc in stream:
6          digest = hashlib.blake2b(doc.encode("utf-8"),
        digest_size=16).digest()
7          if digest not in seen:
8              seen.add(digest)
9          yield doc

```

Storing the digest rather than the document keeps memory at roughly 80 bytes per unique document instead of the document itself. With 16-byte digests the probability of a collision among n documents is about $n^2/2^{129}$, which for $n=10^{12}$ is around 10^{-15} — comfortably below your hardware’s error rate.

This is exact deduplication. *Near*-duplicate detection — documents that differ by a boilerplate header — cannot be done with a hash set, because hashes destroy similarity by design. That needs MinHash and LSH, Chapter 29.

KEY INSIGHT | The one-sentence summary of hashing

A hash function deliberately destroys structure, which is exactly why it gives you $\Theta(1)$ exact lookup and exactly why it can never give you “similar to”. Every approximate-similarity structure in Part VI is a hash that has been carefully weakened so that *some* structure survives.

Consistent hashing, in brief

Sharding with `hash(key) % N` has a fatal operational property: changing N remaps almost every key. Consistent hashing places both shards and keys on a circle and assigns each key to the next shard clockwise; adding a shard then moves only $1/N$ of the keys. With v virtual nodes per physical shard the load imbalance falls as $1/\sqrt{v}$; $v=100$ to 200 is standard.

This is how distributed caches, sharded vector indexes and parameter servers avoid a full reshuffle on every scaling event. Chapter 41 builds it.

EXERCISES

1. Implement the `HashMap` above with open addressing and linear probing, including tombstones. Measure the average probe count at load factors 0.5, 0.7, 0.9 and compare to $1/2(1+(1-\alpha)^{-2})$.
2. Construct a set of 10,000 Python strings that all collide modulo 2^{10} under a fixed `PYTHONHASHSEED`. Measure lookup time before and after. Explain why randomised hashing prevents this in the wild.
3. A dict maps 50 million 8-character strings to float scores. Estimate its memory. Design a replacement using sorted arrays plus `np.searchsorted` and measure both.
4. Show empirically that unsigned feature hashing biases inner products upward and that signed hashing does not. Plot the variance against m .
5. Write a `__hash__` that violates invariant 2, insert the object into a set, mutate it, and demonstrate that `obj in s` is now `False` while `len(s) == 1`.
6. Implement consistent hashing with virtual nodes and measure the fraction of keys that move when a shard is added, for $v \in \{1, 10, 100\}$.
7. You must deduplicate 4 billion documents on one machine with 64 GB of RAM. Exact dedup with a 16-byte digest set needs roughly 400 GB. Give two designs that fit, and state precisely what each one gives up.

CHAPTER RECAP

- A hash table trades a small probability of collision for $\Theta(1)$ expected lookup; load factor governs the trade and resizing keeps it bounded.
- Chaining degrades gracefully and tolerates high load; open addressing is faster below $\alpha \approx 0.7$ because probes are cache-line scans. Deleting from an open-addressed table requires tombstones.
- Insertion is amortised $\Theta(1)$, not worst-case: a resize is a real latency spike.
- Python dicts cost roughly 100 bytes per int-to-int entry; at tens of millions of keys switch to sorted arrays plus `searchsorted`, a perfect hash, or the hashing trick.
- `__hash__` must agree with `__eq__`, must never change while the object is in a container, and must use only identity-defining fields.
- Builtin string hashing is per-process randomised; never persist or shard on it.
- Signed feature hashing gives an unbiased estimate of inner products and removes the vocabulary artefact entirely — at the cost of interpretability.
- Consistent hashing moves only $1/N$ of keys when a shard is added.

Recursion, Memoisation and the Call Stack

Solving a problem by assuming you have already solved it

FOUNDATION

PREREQS: Chapter 6

Recursion is the technique that makes divide and conquer, dynamic programming, backtracking and tree algorithms possible, and it is also the technique most likely to produce an algorithm that is exponentially slower than necessary. This chapter covers both sides: how to write a correct recursion, how to analyse it, and how memoisation turns a recursion tree into a graph.

WHAT YOU WILL LEARN

- Write a recursion by identifying its base case, recursive case and measure of progress
- Draw the recursion tree of an algorithm and read its cost off the drawing
- Recognise overlapping subproblems and eliminate them with memoisation
- Convert any recursion into an iterative form with an explicit stack
- Understand tail calls, stack depth limits and why Python has no tail-call optimisation
- See how memoisation appears in ML systems as caching, not as an algorithm

Three parts of a recursion

A recursion is a function defined in terms of itself on smaller inputs. It is correct if and only if three things hold.

1. **A base case** that terminates without recursing.
2. **A recursive case** that is correct *assuming* the recursive calls are correct. This is the leap people find hard: you may assume the function already works, as long as you only call it on strictly smaller inputs.
3. **A measure that strictly decreases** on every call and is bounded below. Usually the size of the input; sometimes a subtler quantity.

The shape of every correct recursion

```
1 def solve(problem):
```



```

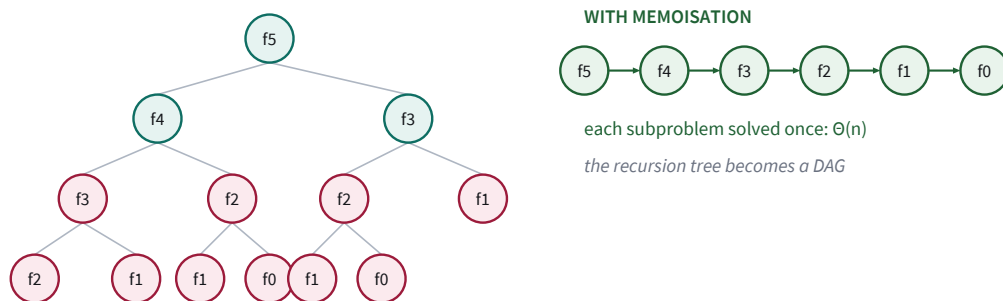
2     if is_trivial(problem):           # 1. base case
3         return trivial_answer(problem)
4     parts = decompose(problem)        # 3. each part is strictly smaller
5     answers = [solve(part) for part in parts]
6     return combine(answers)           # 2. assume the parts are right

```

The third condition is where real bugs live. `solve(n - 1)` obviously decreases; `solve(n // 2)` decreases until $n=1$ and then loops forever if your base case is `n == 0`. When recursing on a graph, the measure is not the input size but the number of unvisited nodes — which is why graph recursion needs a `visited` set and tree recursion does not (Chapter 22).

Recursion trees

The cost of a recursion is the total work across all nodes of its call tree. Drawing that tree is the fastest way to get the answer.



`fib(5)` naive: 15 calls, and `fib(3)` is computed twice. `fib(40)` makes 331 million calls.

Figure 8.1 Left: the call tree of naive `fib(5)`. Red nodes are recomputations. Right: with memoisation the tree collapses into a chain — each distinct subproblem is solved once.

For naive Fibonacci, $T(n) = T(n-1) + T(n-2) + \Theta(1)$, which grows as $\varphi^n \approx 1.618^n$. Concretely, `fib(40)` makes 331 million calls and takes about a minute in Python; `fib(50)` would take two hours.

The pathology is not recursion. It is that the tree contains the same subproblem many times. Count the *distinct* subproblems: there are only n of them. That gap — exponentially many nodes, linearly many distinct values — is the entire opportunity.

DEFINITION | Overlapping subproblems

A recursion has *overlapping subproblems* when the same argument occurs at more than one node of the call tree. When it does, caching results by argument reduces the cost from the number of tree nodes to the number of distinct arguments. This is **memoisation**, and it is the top-down half of dynamic programming (Chapter 18).

Three lines that change $O(1.618^n)$ to $O(n)$

```

1     from functools import lru_cache
2
3     @lru_cache(maxsize=None)

```

```

4 def fib(n):
5     return n if n < 2 else fib(n - 1) + fib(n - 2)

```

`fib(400)` now returns instantly. The cache turns the *tree* into a *directed acyclic graph*: each node is computed once and its result reused by every parent that needs it.

COMMON PITFALL | `lru_cache` on the wrong function

`@lru_cache` requires hashable arguments, so it cannot memoise a function taking a list or a NumPy array — convert to a tuple or pass an index. It also holds references to every argument and result forever with `maxsize=None`, which leaks memory in a long-running process, and it is per-process, so it does nothing across worker restarts. And memoising a function whose result depends on anything other than its arguments (a file, the clock, a random seed) silently produces wrong answers.

Analysing recursive cost

Three recurrence shapes cover most cases; Chapter 16 gives the general tool.

Recurrence	Shape	Solution	Example
$T(n) = T(n-1) + \Theta(1)$	linear chain	$\Theta(n)$	list traversal, factorial
$T(n) = T(n-1) + \Theta(n)$	shrinking work	$\Theta(n^2)$	selection sort, naive quicksort worst case
$T(n) = 2T(n-1) + \Theta(1)$	branching, size -1	$\Theta(2^n)$	subsets, Towers of Hanoi
$T(n) = T(n/2) + \Theta(1)$	halving	$\Theta(\log n)$	binary search
$T(n) = 2T(n/2) + \Theta(n)$	halving, linear merge	$\Theta(n \log n)$	merge sort, FFT
$T(n) = 2T(n/2) + \Theta(1)$	halving, cheap merge	$\Theta(n)$	tree traversal, max of array

KEY INSIGHT | Branching factor versus shrink rate

A recursion that makes a calls on inputs of size n/b has a tree of depth $\log_b n$ and $a^{\log_b n} = n^{\log_b a}$ leaves. Two calls on half the input gives n^1 leaves — linear. Two calls on $n-1$ gives 2^n leaves. The shrink rate is multiplicative and the branching factor is exponential; whenever they are not matched, one of them dominates completely.

The call stack, and its limits

Each call pushes a frame holding arguments, locals and the return address. Python's default recursion limit is 1000 frames, and the underlying C stack will segfault at around 10,000–50,000 even if you raise it.

When recursion depth is the bug

```

1 import sys
2 sys.setrecursionlimit(30_000)      # buys you a little; not a real fix
3

```

```

4  def depth_rec(node):                # fails on a 100k-node degenerate tree
5      return 0 if node is None else 1 + max(depth_rec(node.left),
6                                             depth_rec(node.right))
7
8  def depth_iter(root):                # fine at any depth: heap, not C stack
9      best, stack = 0, [(root, 1)]
10     while stack:
11         node, d = stack.pop()
12         if node is None:
13             continue
14         best = max(best, d)
15         stack.append((node.left, d + 1))
16         stack.append((node.right, d + 1))
17     return best

```

A *balanced* tree of a billion nodes has depth 30 and recursion is perfectly safe. A *degenerate* tree — a linked list in disguise, which is exactly what you get from inserting sorted data into an unbalanced BST — has depth n and recursion is fatal. This asymmetry is why Chapter 12 cares so much about balance.

NOTE | Why Python has no tail-call optimisation

A *tail call* is a recursive call in return position, which in principle can reuse the current frame instead of pushing a new one, making the recursion a loop. Scheme and most functional languages guarantee this. Python deliberately does not, because it would destroy the tracebacks that make debugging possible — the frames are the stack trace. The practical rule: in Python, if depth can exceed a few thousand, write the loop.

Recursion patterns worth having at hand

Four shapes that cover most recursive code

```

1  # 1. Linear: process head, recurse on tail.
2  def total(xs, i=0):
3      return 0 if i == len(xs) else xs[i] + total(xs, i + 1)
4
5  # 2. Binary divide and conquer: split, solve halves, combine.
6  def max_dc(xs, lo, hi):
7      if hi - lo == 1:
8          return xs[lo]
9      mid = (lo + hi) // 2
10     return max(max_dc(xs, lo, mid), max_dc(xs, mid, hi))
11
12 # 3. Tree recursion: recurse on children, combine upward.
13 def sum_tree(node):
14     if node is None:
15         return 0
16     return node.val + sum_tree(node.left) + sum_tree(node.right)
17
18 # 4. Backtracking: choose, recurse, un-choose. (Chapter 20)
19 def permutations(items, chosen=None, out=None):
20     chosen = chosen if chosen is not None else []
21     out = out if out is not None else []
22     if not items:

```

```

23         out.append(chosen[:])
24         return out
25     for i, x in enumerate(items):
26         chosen.append(x)
27         permutations(items[:i] + items[i + 1:], chosen, out)
28         chosen.pop()                # un-choose: restore the state
29     return out

```

Pattern 3 is the shape of nearly every tree algorithm, and pattern 4 is the shape of nearly every exhaustive search. Pattern 4's `pop` is the part people forget, and forgetting it produces answers that are all the same object.

WHERE THIS SHOWS UP IN ML | Memoisation by another name

Most of what an ML system calls “caching” is memoisation of a pure function. **Prefix caching** in an LLM server: the KV cache for a prompt prefix is the memoised result of running the model on it, and SGLang’s RadixAttention is literally a trie of memoised prefixes (Chapters 14 and 36). **Embedding caches**: `embed(text)` is pure, so the result can be cached by content hash. **Feature stores**: precomputed features are memoised aggregations keyed by entity and time. **`torch.compile`**: compiled kernels are memoised by input shape — which is why a new shape triggers a recompile and why dynamic shapes are set up specially.

In every case the question is the same as for `lru_cache`: is the function actually pure, what is the key, and what is the eviction policy?

From memoisation to tabulation

Memoisation is top-down: start from the goal and recurse, caching. The same computation can be done bottom-up, filling a table in an order where every dependency is already computed.

The same recurrence, two directions

```

1  from functools import lru_cache
2
3  def grid_paths_top_down(m, n):
4      @lru_cache(maxsize=None)
5      def go(i, j):
6          if i == 0 or j == 0:
7              return 1
8          return go(i - 1, j) + go(i, j - 1)
9      return go(m - 1, n - 1)
10
11 def grid_paths_bottom_up(m, n):
12     row = [1] * n                # theta(n) memory instead of
13     theta(mn)
14     for _ in range(1, m):
15         for j in range(1, n):
16             row[j] += row[j - 1]
17             # row[j] now holds the number of paths to (i, j)
18     return row[-1]

```

Both are $\Theta(mn)$ time. The bottom-up version uses $\Theta(n)$ memory instead of $\Theta(mn)$, has no recursion depth, no function-call overhead and no cache lookups — typically 10–50x faster in Python. The top-down version has one advantage that sometimes decides the matter: it only computes the subproblems that are actually reachable, which for sparse state spaces can be a tiny fraction. Chapter 18 makes this choice systematically.

EXERCISES

1. Write a recursion for the number of ways to climb n stairs taking 1, 2 or 3 steps. Give the recurrence, the naive complexity, and the memoised complexity. Then write the $\Theta(1)$ -space bottom-up version.
2. Draw the recursion tree of `max_dc` on eight elements. How many nodes? How many comparisons? Prove the general formula.
3. Implement `flatten` for an arbitrarily nested list both recursively and iteratively. Construct an input on which the recursive version fails.
4. Memoise a function of two arguments where one is a NumPy array. Show three ways to make the key hashable and compare their cost and their collision risk.
5. Ackermann's function $A(m, n)$ grows faster than any primitive recursive function. Compute $A(3, 3)$ with memoisation and explain why memoisation helps here but does not make it tractable for $A(4, 2)$.
6. Convert `permutations` to an iterative algorithm using an explicit stack. Then rewrite it as a generator that yields permutations lazily, and explain why the generator uses $\Theta(n)$ memory rather than $\Theta(n!)$.
7. A colleague memoises `score(model_version, document)` with `lru_cache`. Identify three distinct ways this can return a wrong answer in production.

CHAPTER RECAP

- A correct recursion needs a base case, a recursive case that may assume the calls are correct, and a strictly decreasing measure.
- Read a recursion's cost off its call tree. Branching on $n-1$ is exponential; branching on n/b is polynomial.
- Overlapping subproblems make the tree exponentially larger than the set of distinct arguments; memoisation collapses it to a DAG.
- `lru_cache` requires hashable arguments and purity, and leaks without a bound.
- Any recursion converts to a loop with an explicit stack, which is mandatory when depth can exceed a few thousand. Python has no tail-call optimisation, by design.
- Bottom-up tabulation is usually faster and more memory-efficient; top-down memoisation wins when the reachable state space is sparse.
- Most caching in ML systems — prefix caches, embedding caches, feature stores, compiled-kernel caches — is memoisation of a pure function.

Sorting and Order Statistics

The most-studied problem in computing, and what it buys you

CORE

PREREQS: Chapters 2, 8

You will rarely implement a sort. You will constantly decide whether to sort, what to sort by, and whether you need a full ordering at all — and those decisions are worth far more than the implementation. This chapter covers the algorithms because their ideas recur everywhere, then spends equal time on the questions you actually face: stability, keys, partial sorts, and the linear-time alternatives.

WHAT YOU WILL LEARN

- Explain why comparison sorting cannot beat $\Omega(n \log n)$, and how counting and radix sorts escape it
- Implement merge sort, quicksort and heapsort, and say when each is right
- Understand stability and why it matters for multi-key ranking
- Use quickselect and partial sorting to avoid a full sort
- Know what Timsort and `np.argsort` actually do
- Recognise sorting as a preprocessing step that enables other algorithms

Sorting is preprocessing

Sorting is rarely the answer to a question; it is what makes the answer cheap. Once data is sorted you can binary search it ($\Theta(\log n)$), find duplicates in one pass, compute rank statistics by indexing, merge two datasets in linear time, group by key without a hash table, and turn a range query into two binary searches. Most of Part II is unlocked by sorting.

KEY INSIGHT | The decision, in one line

Sorting costs $\Theta(n \log n)$ once and makes many later operations $\Theta(\log n)$ instead of $\Theta(n)$. So sort when you will query more than about $\log n$ times, and do not sort for a single query — scan it.

The comparison lower bound

THEOREM | Comparison sorting requires $\Omega(n \log n)$ comparisons

Any algorithm that sorts by comparing pairs of elements must, in the worst case, perform at least $\log_2(n!) = \Omega(n \log n)$ comparisons.

PROOF

A comparison sort's execution is a path down a binary decision tree: each internal node is a comparison with two outcomes, each leaf is a permutation the algorithm can output. To sort correctly the tree must have at least $n!$ leaves, one per possible input permutation. A binary tree with L leaves has height at least $\log_2 L$. Hence the worst-case number of comparisons is at least $\log_2(n!)$, which by Stirling's approximation is $n \log_2 n - n \log_2 e + \Theta(\log n) = \Omega(n \log n)$.

The argument is information-theoretic: there are $n!$ possible answers, each comparison yields one bit, so you need $\log_2 n!$ bits. It applies to *any* comparison-based method, including ones nobody has invented yet. It does not apply to methods that look at the *structure* of the keys — which is the loophole counting sort and radix sort walk through.

The three classical algorithms

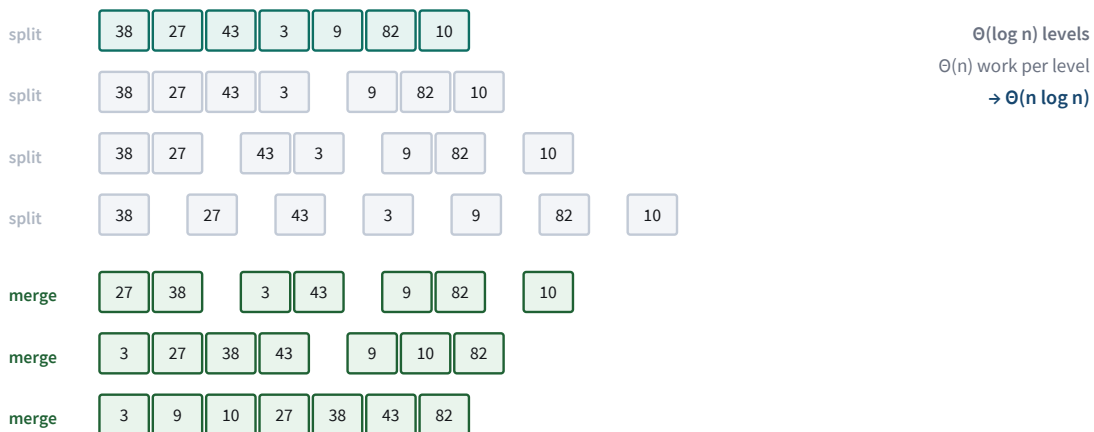


Figure 9.1 Merge sort. The recursion splits until every piece has one element, then merges upward. There are $\log n$ levels and each level does $\Theta(n)$ work.

Merge sort: stable, predictable, $O(n)$ extra space

```

1 def merge_sort(xs):
2     if len(xs) <= 1:
3         return xs
4     mid = len(xs) // 2
5     left, right = merge_sort(xs[:mid]), merge_sort(xs[mid:])
6     out, i, j = [], 0, 0
7     while i < len(left) and j < len(right):
8         if right[j] < left[i]:          # '<' not '<=' keeps it STABLE
9             out.append(right[j]); j += 1

```

```

10         else:
11             out.append(left[i]); i += 1
12         out.extend(left[i:]); out.extend(right[j:])
13     return out

```

The single character in the comparison is the difference between a stable and an unstable sort. With `<`, a tie keeps the left (earlier) element first.

Quicksort with Lomuto partition and a random pivot

```

1  import random
2
3  def quicksort(xs, lo=0, hi=None):
4      hi = len(xs) - 1 if hi is None else hi
5      while lo < hi:
6          k = random.randint(lo, hi)           # randomise: defeats
7          xs[k], xs[hi] = xs[hi], xs[k]         adversaries
8          pivot, i = xs[hi], lo
9          for j in range(lo, hi):
10             if xs[j] < pivot:
11                 xs[i], xs[j] = xs[j], xs[i]
12                 i += 1
13             xs[i], xs[hi] = xs[hi], xs[i]
14             if i - lo < hi - i:                 # recurse on the smaller
15                 quicksort(xs, lo, i - 1)       side,
16                 lo = i + 1                     # loop on the larger: O(
17             else:                             log n)
18                 quicksort(xs, i + 1, hi)       # stack depth guaranteed
19             hi = i - 1
20     return xs

```

Quicksort is the fastest comparison sort in practice — it sorts in place, its inner loop is a sequential scan, and it has excellent cache behaviour — but its worst case is $\Theta(n^2)$ and that worst case is *reachable by an adversary* if the pivot rule is deterministic. Randomising the pivot makes the expected time $\Theta(n \log n)$ regardless of input. Production implementations go further: `introsort` (C++ `std::sort`) counts its recursion depth and switches to heapsort beyond $2 \log n$, giving a hard $\Theta(n \log n)$ worst case while keeping quicksort's constants.

Heapsort, built on the binary heap of Chapter 13, sorts in place with a guaranteed $\Theta(n \log n)$ worst case but poor locality — it jumps around the array by powers of two — which is why it loses to quicksort by a factor of two or three despite identical asymptotics. A clean illustration of Chapter 3's thesis.

algorithm	best	worst	space	stable	when to choose it
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(1)$	yes	$n < 32$, nearly sorted
Merge	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	yes	stability required, external
Quick	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$	no	in-memory, no stability need
Heap	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(1)$	no	hard worst-case bound
Timsort	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n)$	yes	Python/Java default
Counting	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(k)$	yes	small integer range
Radix (LSD)	$\Theta(d(n + b))$	$\Theta(d(n + b))$	$\Theta(n + b)$	yes	fixed-width keys, GPU

Figure 9.2 The sorting landscape. The rightmost column is the one that decides real choices; the complexity columns rarely do.

Stability, and why ranking depends on it

A sort is **stable** if elements comparing equal keep their original relative order. This sounds like a technicality until you sort by multiple keys.

Multi-key ranking via stable sorts

```

1 docs = [...]                                     # (score, recency, source)
2 # Sort by the LEAST significant key first, then the most significant.
3 docs.sort(key=lambda d: d.recency, reverse=True)
4 docs.sort(key=lambda d: d.score, reverse=True)
5 # Now: ordered by score, and within equal scores, by recency.
```

This works only because `list.sort` is stable. The same idea is the entire mechanism of LSD radix sort, and it is how a search ranker applies tie-break rules without constructing a composite key. NumPy's default `np.argsort(kind="quicksort")` is **not** stable; pass `kind="stable"` when ties matter — a silent source of nondeterminism between runs and between CPU and GPU implementations.

COMMON PITFALL | Nondeterministic ranking

Two models with identical scores can produce different top-10 lists on different machines if the sort is unstable, because ties break by whatever order the partition happened to produce. In evaluation this shows up as metrics that shift by 0.1% for no reason. Fix: always sort by an explicit total order — append a deterministic tiebreaker such as the document id.

Escaping the lower bound

When keys are small integers, you can sort without comparing.

Counting sort: $\Theta(n + k)$, stable

```

1 def counting_sort(xs, k):                         # keys in [0, k)
2     count = [0] * k
3     for x in xs:
4         count[x] += 1
```

```

5     pos, total = [0] * k, 0
6     for v in range(k):                # prefix sums give output
7         positions
8         pos[v], total = total, total + count[v]
9     out = [0] * len(xs)
10    for x in xs:                        # forward pass preserves
11        stability
12        out[pos[x]] = x
13        pos[x] += 1
14    return out

```

Counting sort is $\Theta(n+k)$ time and $\Theta(k)$ space, so it wins when $k=O(n)$ and is useless when k is large. Radix sort applies counting sort to one digit at a time, least significant first, relying on stability to preserve the work of earlier passes: $\Theta(d(n+b))$ for d digits in base b . For 32-bit integers with $b=2^{16}$, that is two passes — which is why radix sort is the standard sort on GPUs and in columnar databases.

WHERE THIS SHOWS UP IN ML | Where non-comparison sorts show up

GPU sorting is almost always radix sorting: it is branch-free, its memory access is a sequence of scatters that can be made coalesced, and it parallelises through prefix sums (Chapter 39). CUB's `DeviceRadixSort`, the sort inside a top-k kernel, and the bucketing step in sparse attention all use it. Counting sort appears whenever you bucket by a small categorical: sorting a batch by sequence length before padding, grouping tokens by expert in a mixture-of-experts layer, or histogram binning in gradient-boosted trees (Chapter 37).

When you do not need a full sort

This is the practically important section. A full sort gives you a total order; most questions need far less.

Question	Full sort	Better	Gain at $n=10^7, k=100$
The k largest	$\Theta(n \log n)$	heap: $\Theta(n \log k)$	$\approx 3\times$
The k largest, in memory	$\Theta(n \log n)$	<code>argpartition</code> : $\Theta(n)$	$\approx 25\times$
The median	$\Theta(n \log n)$	quickselect: $\Theta(n)$ expected	$\approx 20\times$
The p -th percentile	$\Theta(n \log n)$	quickselect or t-digest	$\approx 20\times$
Any duplicates?	$\Theta(n \log n)$	hash set: $\Theta(n)$	$\approx 15\times$
Top k from a stream	impossible	bounded heap: $\Theta(n \log k)$, $\Theta(k)$ space	—

Quickselect: the k -th smallest in expected linear time

```
1 import random
```

```

2
3 def quickselect(xs, k):          # 0-indexed: k=0 is the minimum
4     lo, hi = 0, len(xs) - 1
5     while True:
6         if lo == hi:
7             return xs[lo]
8         p = random.randint(lo, hi)
9         xs[p], xs[hi] = xs[hi], xs[p]
10        pivot, i = xs[hi], lo
11        for j in range(lo, hi):
12            if xs[j] < pivot:
13                xs[i], xs[j] = xs[j], xs[i]
14                i += 1
15        xs[i], xs[hi] = xs[hi], xs[i]
16        if k == i:
17            return xs[i]
18        elif k < i:
19            hi = i - 1          # recurse into ONE side only
20        else:
21            lo = i + 1

```

The analysis is the geometric-series argument: each partition is expected to discard a constant fraction, so the expected work is $n + n/2 + n/4 + \dots = \Theta(n)$. Only one side is explored, which is the whole difference from quicksort. The median-of-medians variant achieves $\Theta(n)$ in the *worst* case, at a constant factor that makes it slower in practice — a classic case of a theoretically superior algorithm nobody uses.

Timsort, and what your language actually runs

Python's `list.sort` and `sorted` use Timsort, and so does Java's `Arrays.sort` for objects and Android's runtime. It is worth knowing what it does because it explains some surprising timings.

1. **Find natural runs.** Scan for maximal ascending or descending stretches; reverse the descending ones in place. Real data is full of runs — log files, time series, partially updated indexes — and Timsort's best case on such data is $\Theta(n)$.
2. **Extend short runs** to a minimum length (32–64) with binary insertion sort, which is fast and cache-friendly at that size.
3. **Merge runs** with a stack, maintaining invariants that keep merges balanced, plus *galloping mode*: when one run is consistently winning, switch to exponential search to skip ahead in $\Theta(\log n)$ instead of one element at a time.

The result is $\Theta(n)$ on nearly sorted data, $\Theta(n \log n)$ worst case, stable, and adaptive. `np.sort` by contrast defaults to an introsort variant which is faster on random numeric data but unstable and not adaptive.

PERFORMANCE NOTE | The `key=` function dominates

`sorted(items, key=f)` calls `f` exactly once per element (the decorate-sort-undecorate pattern, built in). But `f` is a Python function call, roughly 100 ns, so sorting a million items with a lambda spends 0.1 s in the key function and perhaps 0.5 s in comparisons. Sorting with `key=operator.itemgetter(1)` instead of `key=lambda t: t[1]` is measurably faster because it is C. Sorting NumPy data with a Python key is a catastrophe — use `np.argsort` on the key array and then a fancy index.

The NumPy idiom for sorting records by a key

```

1  import numpy as np
2  scores = np.random.rand(10_000_000).astype(np.float32)
3  ids = np.arange(10_000_000)
4
5  order = np.argsort(-scores, kind="stable")    # theta(n log n), all in C
6  top_ids = ids[order[:100]]                  # one gather
7
8  # Better still when you only need 100 of ten million:
9  part = np.argpartition(-scores, 100)[:100]    # theta(n)
10 top_ids = ids[part[np.argsort(-scores[part])]]

```

EXERCISES

1. Prove that any stable sort can be simulated by an unstable sort at the cost of $\Theta(n)$ extra space. (Hint: augment each key with its index.)
2. Implement bottom-up (iterative) merge sort with no recursion, and external merge sort that sorts a file larger than memory using k -way merging with a heap.
3. Construct an input on which a median-of-three quicksort is $\Theta(n^2)$. Explain why randomisation prevents this and why “shuffle then sort” is equivalent.
4. Implement LSD radix sort for 32-bit unsigned integers with base 2^8 and compare it to `sorted()` at $n=10^6$ and $n=10^8$. Then extend it to signed integers and to IEEE-754 floats. (Hint: for floats, flip the sign bit; for negatives, flip all bits.)
5. Measure Timsort’s adaptivity: sort an array that is 0%, 1%, 10% and 100% shuffled and plot the time.
6. You have 500 GB of `(user_id, timestamp, event)` records on disk and 32 GB of RAM, and you need them ordered by `(user_id, timestamp)`. Design the sort. How many passes over the data does it need?
7. A reranker scores 50,000 candidates and needs the top 50. Compare a full `argsort`, a `heapq.nlargest`, and an `argpartition` in time and in peak memory. Which would you choose for a p99 latency budget, and why might the answer differ from the mean-latency answer?

CHAPTER RECAP

- Sorting is preprocessing: it costs $\Theta(n \log n)$ once and makes later queries logarithmic. Sort when you will query more than $\log n$ times.
- Comparison sorting needs $\Omega(n \log n)$ comparisons by an information-theoretic argument; counting and radix sorts escape it by inspecting key structure rather than comparing.
- Merge sort is stable and predictable; quicksort is fastest in practice but needs a random pivot; heapsort has the best worst case and the worst locality. Introsort combines them.
- Stability is what makes multi-key sorting work, and instability is a common source of nondeterministic rankings. Add an explicit tiebreaker.
- Most questions need less than a total order: use quickselect, `argpartition`, or a bounded heap. The gains are 3–25x, not percentages.
- Timsort exploits natural runs and galloping, giving $\Theta(n)$ on nearly sorted data. NumPy's default sort is faster but unstable.
- In NumPy, sort the key array with `argsort` and gather; never pass a Python `key=` over array data.

Binary Search and Searching the Answer Space

The most under-generalised algorithm in the toolkit

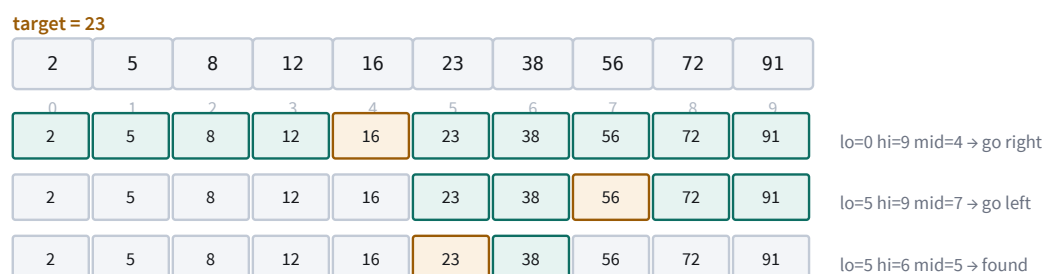
CORE PREREQS: Chapter 9

Everyone can describe binary search and most people cannot write it correctly on the first attempt. Worse, almost everyone under-uses it: binary search over a sorted array is the least interesting of its applications. The important idea is binary search over an answer space, which turns a large family of optimisation problems into a few lines of code.

WHAT YOU WILL LEARN

- Write binary search correctly, including the two boundary variants, without off-by-one errors
- Use `bisect` and `np.searchsorted` as they are meant to be used
- Recognise the monotone-predicate pattern and binary search over answers
- Apply binary search to real ML problems: calibration thresholds, batch sizing, quantile lookup
- Understand exponential (galloping) search and interpolation search
- Know when a hash table is the better answer

The algorithm, written correctly



10 elements, 3 probes. A billion elements would take 30.

Figure 10.1 Binary search on ten elements. Each probe eliminates half the remaining interval, so the number of probes is $\lceil \log_2 n \rceil$.

The classic version, with the two invariants that make it correct:

```

Binary search, half-open interval
1  def binary_search(xs, target):
2      """Return an index of target, or -1. Invariant: if target is present,
3      it lies in xs[lo:hi]."""
4      lo, hi = 0, len(xs)                # half-open: [lo, hi)
5      while lo < hi:
6          mid = (lo + hi) // 2           # lo <= mid < hi, so it always
7          # shrinks
8          if xs[mid] == target:
9              return mid
10         if xs[mid] < target:
11             lo = mid + 1                # target cannot be at mid
12         else:
13             hi = mid                    # target cannot be at mid
14     return -1

```

Three rules prevent every off-by-one error people make here:

1. **Use a half-open interval** `[lo, hi)`. The loop condition is `lo < hi`, the answer region is `xs[lo:hi]`, and `hi` starts at `len(xs)`, not `len(xs) - 1`. This makes the empty interval `lo == hi` natural rather than a special case.
2. **Make sure the interval strictly shrinks.** With `mid = (lo + hi) // 2` we have `mid < hi`, so `hi = mid` shrinks. If you use a closed interval and write `lo = mid`, you get an infinite loop when `hi == lo + 1`.
3. **Say what each branch rules out.** `xs[mid] < target` means index `mid` is definitely not the answer, hence `mid + 1`.

NOTE | The overflow that lived in production for twenty years

`mid = (lo + hi) // 2` overflows in fixed-width integer languages when `lo + hi` exceeds the integer range. The bug was in the JDK's `Arrays.binarySearch` from 1997 until 2006. The fix is `mid = lo + (hi - lo) // 2`. Python's arbitrary-precision integers make this a non-issue, but you will meet it the moment you write C++ or Rust.

The two boundary variants

In practice you almost never want “an index of the target”. You want one of two boundaries, and getting these right removes an entire class of bugs.

```

lower_bound and upper_bound
1  def lower_bound(xs, t):
2      """First index i with xs[i] >= t. == bisect.bisect_left"""
3      lo, hi = 0, len(xs)
4      while lo < hi:
5          mid = (lo + hi) // 2
6          if xs[mid] < t:
7              lo = mid + 1
8          else:

```

```

9         hi = mid
10    return lo
11
12    def upper_bound(xs, t):
13        """First index i with xs[i] > t.  == bisect.bisect_right"""
14        lo, hi = 0, len(xs)
15        while lo < hi:
16            mid = (lo + hi) // 2
17            if xs[mid] <= t:
18                lo = mid + 1
19            else:
20                hi = mid
21    return lo

```

With these two, everything else is a one-liner:

Query	Expression
Is t present?	<code>lower_bound(xs,t) < len(xs) and xs[lower_bound(xs,t)] == t</code>
Count of t	<code>upper_bound(xs,t) - lower_bound(xs,t)</code>
Number of elements $< t$	<code>lower_bound(xs, t)</code>
Number in range $[a, b]$	<code>upper_bound(xs,b) - lower_bound(xs,a)</code>
Insertion point keeping order	<code>lower_bound(xs, t)</code>
Predecessor of t	<code>lower_bound(xs,t) - 1</code>

PERFORMANCE NOTE | `np.searchsorted` is the batch version

`np.searchsorted(sorted_keys, query_keys)` performs m binary searches in one C call: $\Theta(m \log n)$ with no Python overhead. This is the idiomatic replacement for a large `dict` when keys are numeric (Chapter 7): 1.6 GB instead of 20 GB for 200 million int64 keys, and faster for batched lookups because it is vectorised and cache-friendly.

The real idea: binary search over answers

Binary search does not need an array. It needs a **monotone predicate**: a boolean function P over a range such that once P becomes true it stays true. Then binary search finds the boundary.

DEFINITION | **Binary search on a predicate**

Let $P: \{lo, \dots, hi\} \rightarrow \{false, true\}$ be monotone: $P(x) \Rightarrow P(x+1)$. Then the smallest x with $P(x)$ true can be found in $\Theta(\log(hi-lo))$ evaluations of P .

The template worth memorising

```

1    def first_true(lo, hi, pred):
2        """Smallest x in [lo, hi) with pred(x) true; hi if none."""
3        while lo < hi:
4            mid = lo + (hi - lo) // 2
5            if pred(mid):

```



```

6         hi = mid
7     else:
8         lo = mid + 1
9     return lo

```

Everything below is an instance of this template. The work is not in the search; it is in *finding the monotone predicate*.

Minimum capacity to finish in D days. Given daily workloads, find the smallest per-day capacity such that the work fits in D days. `pred(c)` = “capacity c suffices”, which is monotone because more capacity never hurts. Evaluating `pred` is one $\Theta(n)$ greedy pass, so the whole thing is $\Theta(n \log(\text{range}))$.

The square root, or any inverse. `pred(x)` = “ $x^2 \geq v$ ”.

The k -th smallest in a sorted matrix. `pred(v)` = “at least k elements are $\leq v$ ”, counted in $\Theta(n)$ by walking the staircase.

Binary search over a continuous answer space

```

1 def solve_real(pred, lo, hi, iters=60):
2     """Smallest real x in [lo, hi] with pred(x) true."""
3     for _ in range(iters):          # 60 halvings ~ 1e-18 relative
4         precision
5         mid = (lo + hi) / 2
6         if pred(mid):
7             hi = mid
8         else:
9             lo = mid
10    return hi

```

For real-valued answers, iterate a fixed number of times rather than testing for convergence: 60 iterations of bisection reduce the interval by 2^{-60} , which is below double-precision resolution. This avoids the infinite loops that `while hi - lo > eps` produces when `eps` is smaller than the representable gap.

WHERE THIS SHOWS UP IN ML | Five places you should reach for binary search over answers

Calibration thresholds. Find the classification threshold achieving exactly 95% precision. Precision is monotone in the threshold (up to ties), so this is `first_true` over the threshold, evaluated on a validation set. Much more robust than scanning a grid.

Largest batch size that fits in memory. Peak memory is monotone in batch size. Binary search it with a try/except around an actual forward-backward pass: about 12 probes to find the exact limit between 1 and 4096, instead of doubling until it crashes and halving once.

Temperature or top-p for a target entropy. Entropy is monotone in temperature; bisect to hit a target.

Quantiles of a sorted score array. `np.searchsorted` on the cumulative distribution.

Sampling from a discrete distribution. Build the prefix sums once, then each sample is `searchsorted(cumsum, u)` for a uniform u : $\Theta(\log n)$ per sample, and vectorised over a whole batch. Chapter 34 shows the $\Theta(1)$ alternative.

Exponential (galloping) search

When the array is unbounded, or when the target is likely near the start, first find a bracketing range by doubling, then binary search inside it.

Exponential search: $O(\log i)$ for a target at index i

```

1  def exponential_search(xs, t):
2      if not xs:
3          return -1
4      bound = 1
5      while bound < len(xs) and xs[bound] < t:
6          bound *= 2                # 1, 2, 4, 8, ...
7      lo, hi = bound // 2, min(bound + 1, len(xs))
8      i = lower_bound(xs[lo:hi], t) + lo
9      return i if i < len(xs) and xs[i] == t else -1

```

This is $\Theta(\log i)$ where i is the answer's index, rather than $\Theta(\log n)$. It matters when $i \ll n$ — which is exactly the situation inside Timsort's galloping merge (Chapter 9) and inside inverted-index intersection, where you repeatedly skip forward in a long posting list to match a short one.

KEY INSIGHT | Interpolation search, and why it is rarely used

If keys are close to uniformly distributed, guess the position by linear interpolation instead of taking the midpoint: expected $\Theta(\log \log n)$ probes. That sounds impressive and is almost never worth it. For $n=10^9$, $\log_2 n=30$ and $\log_2 \log_2 n \approx 5$ — but each probe is a cache miss either way, the arithmetic per probe is more expensive, and on skewed data it degrades to $\Theta(n)$. The lesson generalises: below about $n=10^{12}$, $\log n$ is already a small number, and constant factors decide.

When not to binary search

Situation	Better tool	Why
Exact-match lookup, many queries	hash table	$\Theta(1)$ beats $\Theta(\log n)$
Data changes frequently	balanced BST / skip list	keeping an array sorted costs $\Theta(n)$ per insert
Predicate is not monotone	ternary search (unimodal), or scan	the invariant fails
Evaluating the predicate is expensive and noisy	Bayesian optimisation	binary search assumes an exact oracle
$n < 64$	linear scan	branch-free SIMD scan beats branchy binary search

Table 10.1 Every ordered query in terms of two primitives. Python's `bisect` module provides both; `np.searchsorted(a, v, side="left"/"right")` provides them vectorised over an entire array of queries at once.

COMMON PITFALL | The noisy predicate

Binary searching a threshold on a *validation set* gives you the answer for that sample, not the population. If the metric is noisy — and with 500 positive examples it is very noisy — the located boundary has a confidence interval far wider than the bisection’s precision. Reporting a threshold to six decimal places because bisection converged is false precision. Bootstrap the search and report the spread.

EXERCISES

1. Write `lower_bound` and `upper_bound` from memory, then test them against `bisect` on 10,000 random arrays including empty arrays, all-equal arrays, and arrays with the target absent at both ends.
2. Find the smallest positive integer x with $x^3 \geq 10^{18}$ using `first_true`, without floating point.
3. Given daily page counts and D days, find the minimum number of pages per day to finish. State the predicate and prove it is monotone.
4. Implement “find the k -th smallest element of a row-and-column sorted $n \times n$ matrix” in $\Theta(n \log(\max - \min))$ by binary searching the value.
5. Write a routine that finds the largest batch size fitting in GPU memory by binary searching over a function that runs one training step and catches out-of-memory. How many probes for a range of 1 to 4096?
6. Replace a 50-million-entry `dict[int, int]` with sorted NumPy arrays and `searchsorted`. Measure memory and the time for one lookup and for a batch of 10,000 lookups. Explain the difference in the two timings.
7. Implement galloping search and use it to intersect two sorted posting lists of lengths 10 and 10,000,000. Compare to a linear merge.

CHAPTER RECAP

- Use half-open intervals `[lo, hi)`, ensure the interval strictly shrinks, and state what each branch rules out. These three rules eliminate off-by-one errors.
- `lower_bound` and `upper_bound` are the useful primitives; every ordered query is a combination of them. `np.searchsorted` is the vectorised form and often replaces a large dict.
- The general pattern is binary search over a *monotone predicate*, not over an array. Finding the predicate is the creative step; the search is a template.
- Real ML uses: calibration thresholds, maximum batch size, temperature for target entropy, quantiles, and sampling by inverse CDF.
- Exponential search costs $\Theta(\log i)$ and is what makes Timsort’s galloping and posting-list skipping fast.
- Prefer a hash table for repeated exact lookups, a balanced tree for frequently changing data, and a linear scan below about 64 elements.
- A binary search on a noisy metric converges to more precision than the metric deserves.

Two Pointers and Sliding Windows

Turning nested loops into single passes

CORE

PREREQS: Chapters 9--10

A large family of problems that look quadratic are linear if you notice that the two indices you are looping over never need to move backwards. This chapter develops that observation into two reliable techniques, proves why they are linear, and connects them to the batching, chunking and streaming code that sits in front of every model.

WHAT YOU WILL LEARN

- Recognise when two indices can be advanced monotonically instead of restarted
- Write the converging-pointers and sliding-window templates from memory
- Prove the amortised $\Theta(n)$ bound for both
- Combine windows with hash maps, monotonic deques and prefix sums
- Apply the pattern to sequence batching, chunking and streaming metrics
- Know the variants: fixed window, variable window, window with a counter

The observation

Consider finding two numbers in a *sorted* array that sum to a target. The naive algorithm tries all pairs: $\Theta(n^2)$. But sortedness means that if `xs[i] + xs[j]` is too large, every `j' > j` is also too large — so `j` can move left and never needs to come back.

Converging pointers: $O(n)$ after sorting

```
1 def two_sum_sorted(xs, target):
2     i, j = 0, len(xs) - 1
3     while i < j:
4         s = xs[i] + xs[j]
5         if s == target:
6             return i, j
7         if s < target:
8             i += 1           # only a larger left value can help
9         else:
10            j -= 1           # only a smaller right value can help
11    return None
```

Each iteration advances i or retreats j , and they meet after at most n steps: $\Theta(n)$. The correctness argument is the important part — it is an *exchange* argument. When $s < \text{target}$, no pair (i, j') with $j' < j$ can work either, because $xs[j'] \leq xs[j]$. So discarding i loses no solution.

KEY INSIGHT | The general condition

Two-pointer methods work when the problem has a *monotone structure*: moving one pointer changes the objective in a known direction, so one side of the search space can be discarded without examining it. Sorting is the usual way to create that structure, which is another reason Chapter 9 comes first.

This template generalises immediately: three-sum is a loop over the first element with two-sum inside ($\Theta(n^2)$ instead of $\Theta(n^3)$); merging two sorted lists is two pointers; the merge step of merge sort is two pointers; intersecting two sorted posting lists is two pointers; removing duplicates in place is two pointers where one is a read cursor and the other a write cursor.

Read and write cursors: in-place partition

```

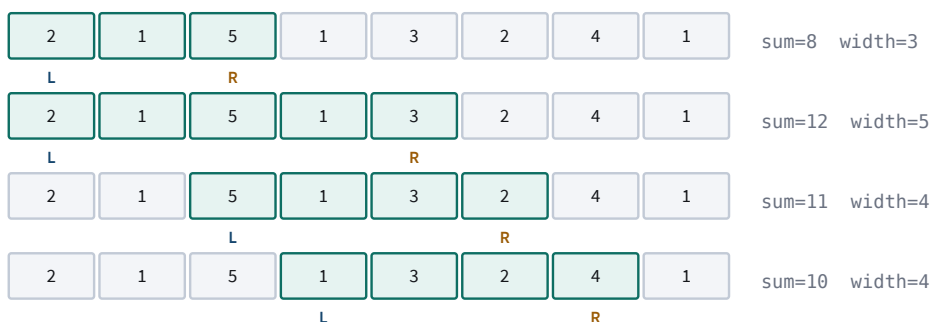
1 def remove_duplicates(xs):          # xs sorted; returns new length
2     w = 0
3     for r in range(len(xs)):
4         if w == 0 or xs[r] != xs[w - 1]:
5             xs[w] = xs[r]
6             w += 1
7     return w

```

The read/write cursor pattern is worth internalising: it is how in-place filtering, stable partitioning and compaction are written, and it is exactly what a GPU stream-compaction kernel does after a prefix sum (Chapter 39).

Sliding windows

longest window with sum ≤ 8



Each index enters the window once and leaves once: $\Theta(n)$ total, not $\Theta(n^2)$.

Figure 11.1 A variable-size window. The right edge advances once per element; the left edge advances only when the window becomes invalid. Both move at most n times in total.

The sliding-window template answers questions of the form “the longest (or shortest) contiguous subarray satisfying P ”, where P has the property that shrinking a valid window keeps it valid (or shrinking an invalid one eventually makes it valid).

The variable-window template

```

1  def longest_window(xs, is_valid_after_add, remove_left):
2      """Generic shape; specialise the two callbacks."""
3      left, best = 0, 0
4      for right in range(len(xs)):
5          add(xs[right])          # extend the window
6          while not valid():      # restore the invariant
7              remove_left(xs[left])
8              left += 1
9          best = max(best, right - left + 1)
10     return best

```

Concrete: longest substring with no repeated character

```

1  def longest_unique(s):
2      last, left, best = {}, 0, 0
3      for right, ch in enumerate(s):
4          if ch in last and last[ch] >= left:
5              left = last[ch] + 1          # jump past the previous
6              occurrence
7          last[ch] = right
8          best = max(best, right - left + 1)
9      return best

```

Concrete: shortest subarray with sum >= target (positive values)

```

1  def shortest_at_least(xs, target):
2      left, total, best = 0, 0, float("inf")
3      for right, x in enumerate(xs):
4          total += x
5          while total - xs[left] >= target: # shrink while still valid
6              total -= xs[left]
7              left += 1
8          if total >= target:
9              best = min(best, right - left + 1)
10     return best if best < float("inf") else 0

```

THE MATH | Why the inner while does not make it quadratic

The inner loop looks nested, but `left` only ever increases and is bounded by n . Across the whole run, the inner loop body executes at most n times *in total*, not n times per outer iteration. Formally: define the potential $\Phi = \text{right} - \text{left}$; each outer step raises it by 1 and each inner step lowers it by 1, and $\Phi \geq 0$ always, so the number of inner steps is at most the number of outer steps. Total: $\Theta(n)$. This is the accounting method from Chapter 2 in its simplest form.

COMMON PITFALL | The window that must not shrink

The template requires that the validity condition be *monotone in the window*. “Sum $\geq$ target” with **non-negative** values is monotone: removing an element cannot increase the sum. With negative values present it is not, and the sliding window is simply wrong — the answer needs prefix sums plus a monotonic deque, or a prefix-sum-plus-BIT approach. This is the single most common incorrect application of the technique, and it passes small tests.

Windows with a counter

When the condition involves multiplicities — “contains all characters of T ”, “at most k distinct values” — carry a hash map of counts plus a scalar summarising validity, so that checking validity stays $\Theta(1)$.

At most k distinct values: window plus counter

```

1  from collections import defaultdict
2
3  def longest_at_most_k_distinct(xs, k):
4      count, distinct, left, best = defaultdict(int), 0, 0, 0
5      for right, x in enumerate(xs):
6          if count[x] == 0:
7              distinct += 1
8          count[x] += 1
9          while distinct > k:
10             count[xs[left]] -= 1
11             if count[xs[left]] == 0:
12                 distinct -= 1
13             left += 1
14             best = max(best, right - left + 1)
15     return best

```

The scalar `distinct` is the trick: recomputing “how many distinct values are in the window” from the map would be $\Theta(k)$ per step. Maintaining it incrementally keeps the whole algorithm $\Theta(n)$. Every efficient window algorithm has some such incremental summary — a count, a running sum, a monotonic deque for the max (Chapter 6), or a heap.

Fixed windows and streaming statistics

For a fixed window of size k , the incremental update is even simpler, and this is the form that appears in every metrics pipeline.

Streaming mean and variance over a fixed window

```

1  from collections import deque
2
3  class RollingStats:
4      """O(1) per update, O(k) memory. Welford-style for stability."""
5      def __init__(self, k):
6          self.k, self.buf = k, deque(maxlen=k)
7          self.s = 0.0          # running sum

```

```

8         self.s2 = 0.0          # running sum of squares
9
10    def push(self, x):
11        if len(self.buf) == self.k:
12            old = self.buf[0]
13            self.s -= old
14            self.s2 -= old * old
15        self.buf.append(x)
16        self.s += x
17        self.s2 += x * x
18
19    def mean(self):
20        return self.s / len(self.buf)
21
22    def var(self):
23        n = len(self.buf)
24        return max(0.0, self.s2 / n - (self.s / n) ** 2)

```

WATCH OUT | Catastrophic cancellation in the rolling variance

The $s2/n - \text{mean}^2$ formula is numerically unstable: if the values are large and the variance small, you subtract two nearly equal large numbers and lose most of your significant digits, sometimes producing a negative variance. For a *streaming* (non-windowed) variance use Welford's algorithm, which is stable. For a *windowed* variance there is no exactly stable $O(1)$ update; the practical answer is to recompute from the buffer every few thousand updates, or to keep the values centred by subtracting a slowly updated running mean. Chapter 38 treats streaming statistics properly.

WHERE THIS SHOWS UP IN ML | Windows in the ML stack

Sequence chunking. Splitting a long document into overlapping windows of 512 tokens with a 64-token stride is a fixed sliding window; the stride is a recall-versus-cost knob.

Length bucketing. Sorting a batch by sequence length and cutting it into groups whose padding waste stays under a threshold is a variable window over the sorted lengths — the greedy window is optimal because length is monotone after sorting (Chapter 17).

Sliding-window attention. Longformer and Mistral-style local attention restrict each token to attend to the previous w tokens, which turns the $\Theta(n^2)$ attention matrix into $\Theta(nw)$. The KV cache then becomes a ring buffer of size w (Chapter 6), with eviction for free.

Online drift detection. Comparing a rolling window of recent predictions to a reference distribution is two windows and a divergence.

Deduplication by shingles. A k -token shingle is a fixed window, and hashing it incrementally is the rolling hash of Chapter 5.

Choosing between the patterns

Problem shape	Technique	Cost
Pair with property, sorted input	converging pointers	$\Theta(n)$ after sort
Longest/shortest contiguous run with monotone property	variable window	$\Theta(n)$

Problem shape	Technique	Cost
Statistic over a fixed window	fixed window + incremental update	$\Theta(1)$ per step
Max/min over a window	window + monotonic deque	$\Theta(1)$ amortised
Sum over an arbitrary range, static array	prefix sums	$\Theta(1)$ per query
Sum over an arbitrary range, with updates	Fenwick tree (Chapter 27)	$\Theta(\log n)$
Contiguous subarray with negative values	prefix sums + map or deque	$\Theta(n)$
Non-contiguous subsequence	dynamic programming (Chapter 18)	usually $\Theta(n^2)$ or more

EXERCISES

1. Implement three-sum in $\Theta(n^2)$ using sort plus converging pointers, handling duplicates so that each triple is reported once.
2. Prove that `longest_unique` is correct when `left` jumps forward rather than shrinking one step at a time. Why is the `last[ch] >= left` guard necessary?
3. Give a concrete array with negative numbers on which `shortest_at_least` returns the wrong answer, and then write a correct $\Theta(n \log n)$ version using prefix sums and a monotonic deque.
4. Implement “minimum window substring”: the shortest substring of S containing all characters of T with multiplicity, in $\Theta(|S| + |T|)$.
5. Use a sliding window plus a monotonic deque to compute the rolling maximum drawdown of a price series in one pass.
6. Given 100,000 sequences with lengths drawn from a log-normal distribution, implement length bucketing that keeps padding waste below 10%, and report the number of batches versus naive fixed-size batching.
7. Show that the windowed variance above can return a negative number, and fix it two ways: with a periodic recomputation, and by centring.

CHAPTER RECAP

- Two-pointer methods apply when moving a pointer changes the objective monotonically, so half the search space can be discarded without examination. Sorting is the usual way to create that structure.
- The read/write cursor is the in-place filtering idiom and the sequential analogue of GPU stream compaction.
- The sliding window is $\Theta(n)$ because each index enters and leaves the window once; the potential-function argument makes this precise.
- The window technique is only correct when validity is monotone in the window — non-negative values for sum conditions, for example.
- Efficient windows keep an incremental summary: a count, a running sum, a monotonic deque, or a heap. Recomputing from the window is what makes a window algorithm accidentally quadratic.
- Windows require contiguity. Non-contiguous subsequence problems belong to dynamic programming.

PART III

Hierarchies and Priorities

Trees give you ordered access in logarithmic time; heaps give you the best element now. Between them they power every index, every top-k, and every tokenizer in production.

-
- 12 Trees: Binary, Balanced, and B-Trees
 - 13 Heaps, Priority Queues and Top-k
 - 14 Tries, Automata and the Anatomy of a Tokenizer
 - 15 Union-Find and the Algebra of Merging

Trees: Binary, Balanced, and B-Trees

Ordered access in logarithmic time, and what it costs to keep it

CORE

PREREQS: Chapters 8--10

A hash table gives you $\Theta(1)$ lookup but no order; a sorted array gives you order but $\Theta(n)$ insertion. Trees are the structure that gives you both, and the entire difficulty is keeping them balanced. This chapter covers binary search trees, the rotations that rebalance them, and the B-trees that every database and file system is built on.

WHAT YOU WILL LEARN

- Implement a binary search tree and all four traversals
- Explain why an unbalanced BST degenerates and what balance guarantees buy
- Understand rotations and the invariants of AVL and red-black trees
- Know why B-trees dominate on disk and why the branching factor is the whole point
- Choose correctly between a tree, a hash table and a sorted array
- Recognise trees in ML systems: index structures, decision trees, prefix caches

The binary search tree

DEFINITION | Binary search tree property

Every node stores a key. For every node v , all keys in v 's left subtree are less than v 's key, and all keys in v 's right subtree are greater. This must hold for *every* node, not just the root — a distinction that catches people out.

The property means search is a sequence of decisions: compare, go left or right, repeat. The cost is the depth of the tree, and therein lies everything.

BST with insert, search and delete

```
1 class Node:
2     __slots__ = ("key", "val", "left", "right")
3     def __init__(self, key, val):
4         self.key, self.val = key, val
```

```

5         self.left = self.right = None
6
7     def search(node, key):
8         while node:                                # iterative: no stack depth
9             limit
10            if key == node.key:
11                return node.val
12            node = node.left if key < node.key else node.right
13        return None
14
15    def insert(node, key, val):
16        if node is None:
17            return Node(key, val)
18        if key < node.key:
19            node.left = insert(node.left, key, val)
20        elif key > node.key:
21            node.right = insert(node.right, key, val)
22        else:
23            node.val = val
24        return node
25
26    def delete(node, key):
27        if node is None:
28            return None
29        if key < node.key:
30            node.left = delete(node.left, key)
31        elif key > node.key:
32            node.right = delete(node.right, key)
33        else:
34            if node.left is None:
35                return node.right                    # 0 or 1 child: splice it out
36            if node.right is None:
37                return node.left
38            succ = node.right                         # 2 children: use the
39            successor
40            while succ.left:
41                succ = succ.left
42            node.key, node.val = succ.key, succ.val
43            node.right = delete(node.right, succ.key)
44        return node

```

Deletion with two children is the only interesting case: replace the node's key with its in-order successor (the smallest key in the right subtree), then delete that successor, which by construction has at most one child.

Traversals

The four traversals

```

1     def inorder(n, out):        # left, node, right    -> keys in sorted order
2         if n: inorder(n.left, out); out.append(n.key); inorder(n.right, out)
3
4     def preorder(n, out):       # node, left, right    -> serialisation, tree
5         copy

```

```

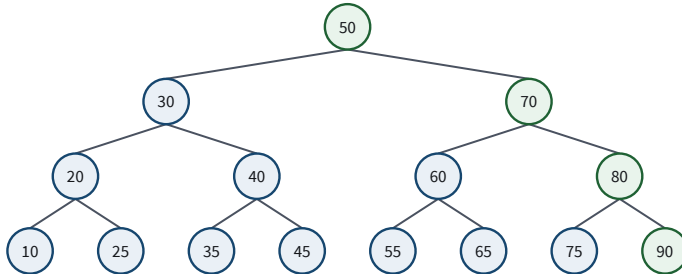
5     if n: out.append(n.key); preorder(n.left, out); preorder(n.right, out)
6
7     def postorder(n, out):      # left, right, node    -> deletion, expression
8         eval
9         if n: postorder(n.left, out); postorder(n.right, out); out.append(
10             n.key)
11
12     from collections import deque
13     def level_order(root):      # breadth-first        -> printing, level
14         statistics
15         out, q = [], deque([root] if root else [])
16         while q:
17             n = q.popleft()
18             out.append(n.key)
19             q.extend(c for c in (n.left, n.right) if c)
20         return out

```

Each has a characteristic use. In-order on a BST yields sorted keys — that is the defining property, and it is the reason **inorder** is how you verify a BST implementation. Post-order evaluates an expression tree bottom-up, which is exactly what the backward pass of autograd does (Chapter 24). Pre-order serialises a tree so it can be rebuilt by inserting in the same sequence.

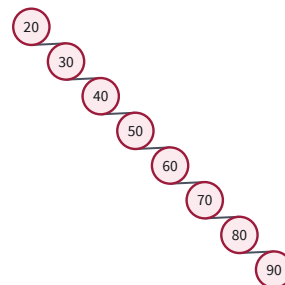
Balance is the whole problem

BALANCED — depth 3, 15 nodes, $\Theta(\log n)$ search



search for 90: 4 comparisons

DEGENERATE — depth 7, $\Theta(n)$ search

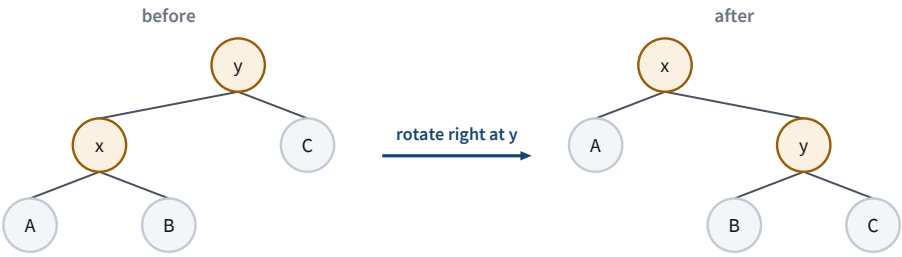


insert 20, 30, 40, ... in sorted order and you get a linked list with extra steps

Figure 12.1 The same eight keys in two trees. Both satisfy the BST property; one answers queries in 4 comparisons and the other in 8. Inserting sorted data — which is exactly what happens with timestamps, autoincrement ids, or an alphabetised vocabulary — always produces the right-hand shape.

A BST with n nodes has depth between $\lfloor \log_2 n \rfloor$ and $n-1$. Random insertion order gives expected depth $\approx 1.39 \log_2 n$, which is fine. But real data is very rarely randomly ordered, and sorted insertion produces a linked list. Self-balancing trees exist to make the bad case impossible.

Rotations



In-order traversal is $A \times B \times C$ on both sides — rotation changes the shape, never the order.

Figure 12.2 A rotation restructures three subtrees and two nodes in $\Theta(1)$ time. Crucially it preserves the in-order sequence, so the BST property survives.

Rotation: four pointer writes

```
1 def rotate_right(y):
2     x = y.left
3     y.left = x.right
4     x.right = y
5     return x
```

x is the new subtree root

Everything about balanced trees is built from this primitive. The variants differ in what invariant they maintain and how many rotations they need.

Tree	Invariant	Height	Rotations per insert	Notes
AVL	subtree heights differ by ≤ 1	$\leq 1.44 \log n$	≤ 2	shallowest; more rotations on update
Red-black	no two red in a row; equal black-height	$\leq 2 \log n$	≤ 3	fewer rotations; C++ <code>map</code> , Java <code>TreeMap</code> , Linux CFS
Treap	heap order on random priorities	$\Theta(\log n)$ expected	$\Theta(1)$ expected	simple to write; randomised
Splay	recently accessed near the root	amortised $\Theta(\log n)$	varies	adapts to skewed access
Skip list	probabilistic levels	$\Theta(\log n)$ expected	none	lock-free friendly; Chapter 31
B-tree	all leaves at equal depth	$\Theta(\log_B n)$	node split	the on-disk answer

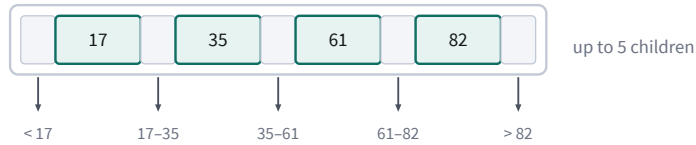
KEY INSIGHT | Why you will probably never implement one

Every language ships a balanced tree (`std::map`, `TreeMap`, `sortedcontainers` in Python), and Python’s `sortedcontainers` is actually a list of sorted lists — not a tree at all — which outperforms tree implementations in CPython by a wide margin because it moves contiguous memory instead of chasing pointers. Chapter 3’s thesis again. Know the invariants so you can reason about the guarantees; reach for the library.

B-trees: branching for the memory hierarchy

The deep reason binary trees lose on disk is arithmetic. A lookup in a binary tree of 10^9 keys takes 30 comparisons, and if each node is a separate disk page that is 30 disk reads at 80 microseconds each: 2.4 milliseconds. A B-tree with 200 keys per node has depth $\log_{200}(10^9) \approx 3.9$, so 4 reads: 0.3 milliseconds.

ONE B-TREE NODE (order 5) — one disk page



structure	keys/node	depth at 10^9 keys	cost of one lookup
binary search tree	1	~ 30	30 disk reads
B-tree, 4 KB pages	~ 200	3	3 disk reads
B-tree, 16 KB pages	~ 800	2–3	2–3 disk reads

Figure 12.3 A B-tree node holds many keys and many child pointers, sized so that one node is one disk page (or one cache line group). The branching factor turns $\log_2$ into $\log_{200}$.

DEFINITION | B-tree of order m

Every node holds between $\lceil m/2 \rceil - 1$ and $m - 1$ keys, and one more child pointer than keys. All leaves are at the same depth. Insertion splits a full node and pushes the median key up; deletion merges or borrows from a sibling. Height is $\Theta(\log_m n)$.

The **B+ tree** variant — keys in internal nodes, *all* values in the leaves, and the leaves linked in a list — is what databases actually use, because it makes range scans a sequential walk along the leaf chain rather than a tree traversal. This is why `SELECT ... WHERE x BETWEEN a AND b` is fast, and why the same structure indexes the metadata of a vector database.

HARDWARE REALITY | The branching-factor principle, generalised

Choose the node size to match the transfer unit of the level you are optimising for: 64 bytes for cache, 4 KB for a page, 4 MB for an object store. The same reasoning gives cache-conscious B-trees in memory (node = cache line, branching factor 8–16), gives the block size in an LSM tree, and gives the number of neighbours per node in an HNSW graph (Chapter 33). It is one idea wearing several costumes.

Choosing between tree, hash and sorted array

Need	Hash table	Balanced tree	Sorted array
Exact lookup	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Insert / delete	$\Theta(1)$ amortised	$\Theta(\log n)$	$\Theta(n)$
Min / max	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$
Range query $[a, b]$	$\Theta(n)$	$\Theta(\log n + k)$	$\Theta(\log n + k)$

Need	Hash table	Balanced tree	Sorted array
Ordered iteration	$\Theta(n \log n)$	$\Theta(n)$	$\Theta(n)$
Predecessor / successor	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$
Memory per entry	high	highest	lowest
Cache behaviour	good	poor	excellent

Table 12.1 Balanced structures. AVL is best for read-heavy workloads, red-black for update-heavy, treaps and skip lists when you want short, correct code, and B-trees whenever the data lives outside cache.

COMMON PITFALL | The tree you did not need

Reaching for a balanced tree when a sorted array would do is a common piece of over-engineering. If you build the structure once and query it many times — which describes nearly every index in an ML system — a sorted NumPy array with `searchsorted` is smaller, faster and simpler. Trees pay off when writes are interleaved with ordered reads.

WHERE THIS SHOWS UP IN ML | Trees in machine learning systems

Decision trees and GBDT. A learned decision tree is a binary tree whose internal nodes are thresholds. Training it is a greedy recursive partitioning (Chapter 37); inference is a root-to-leaf walk of depth 6–12, and the performance work is entirely about making that walk branch-free and cache-resident.

k-d trees and ball trees. Spatial trees for nearest-neighbour search, which work beautifully up to about 20 dimensions and fail completely beyond that. Chapter 32 explains why, precisely.

Merkle trees. Hash trees for verifying dataset integrity and for content-addressed storage of model checkpoints.

Prefix trees over KV caches. SGLang’s RadixAttention stores a tree of token prefixes so that conversations sharing a system prompt share its KV cache. This is a trie (Chapter 14) used as a cache index, and it is worth a 2–5x throughput improvement on workloads with shared prefixes (Chapter 36).

Hierarchical softmax. A Huffman tree over the vocabulary reduces the cost of the output layer from $\Theta(V)$ to $\Theta(\log V)$; historically important for word2vec, and still used when V is in the millions.

EXERCISES

1. Write a function that verifies the BST property. The naive version (checking each node against its children) is wrong — construct a tree that passes it and is not a BST, then write the correct version using (min, max) bounds.
2. Implement `kth_smallest(root, k)` in $\Theta(h+k)$ using an iterative in-order traversal, and then in $\Theta(h)$ by augmenting each node with its subtree size.
3. Implement AVL insertion with the four rebalancing cases (LL, LR, RR, RL). Verify that the height stays below $1.44\log_2 n$ for $n=10^6$ random insertions.
4. Build a BST from 100,000 sorted integers and measure search time; then shuffle the input and measure again. Explain the ratio.
5. Implement a B-tree of order 64 with insertion and search. Measure the number of node accesses for $n=10^6$ and compare to $\log_{64} n$.
6. Compare `sortedcontainers.SortedList` to a hand-written red-black tree for 1M inserts and 1M range queries. Explain the result using Chapter 3.
7. Serialise and deserialise a binary tree so that the reconstruction is exact, including the shape. Show that a pre-order traversal alone is insufficient and state the minimum extra information needed.

CHAPTER RECAP

- The BST property must hold for every node's entire subtree, not just its children — the source of the classic validation bug.
- Search cost is the depth. Random insertion gives $1.39\log n$; sorted insertion gives n , and real data is usually sorted.
- Rotations restructure in $\Theta(1)$ while preserving in-order sequence; AVL, red-black, treaps and splay trees are different policies over the same primitive.
- B-trees raise the branching factor to match the transfer unit of the memory level, turning $\log_2$ into $\log_{200}$. B+ trees put all values in linked leaves, making range scans sequential.
- Use a hash table when you never ask ordered questions, a sorted array when the data is static, and a tree only when you need order *and* mutation.
- Trees in ML: decision trees, k-d trees, Merkle trees, radix prefix caches, and hierarchical softmax.

Heaps, Priority Queues and Top-k

When you need the best element, not all of them in order

CORE

PREREQS: Chapters 9, 12

The heap answers exactly one question — what is the smallest element? — and answers it in constant time while supporting logarithmic insertion and removal. That narrow contract is enough to build top-k retrieval, beam search, Dijkstra’s algorithm, event simulation, BPE training and the scheduler in your inference server. It is the highest leverage-per-line data structure in the book.

WHAT YOU WILL LEARN

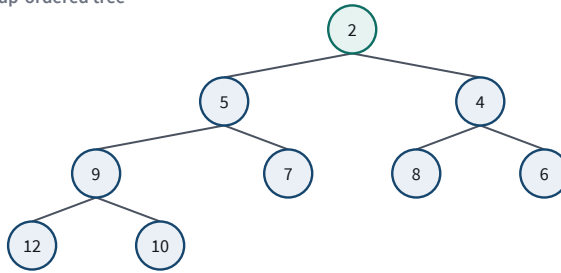
- Explain why a complete binary tree can be stored in a flat array with no pointers
- Implement sift-up, sift-down, push, pop and the $\Theta(n)$ heapify
- Use `heapq` correctly, including max-heaps, tuples and lazy deletion
- Solve top-k over a stream with $\Theta(k)$ memory
- Implement k -way merge, running median and a priority queue with `decrease-key`
- Recognise heaps inside beam search, BPE training and GPU schedulers

A tree with no pointers

DEFINITION | Binary heap

A **complete** binary tree (every level full except possibly the last, which is filled left to right) satisfying the **heap property**: every node’s key is $\leq$ both of its children’s keys (a min-heap). The root is therefore the global minimum. Note what is *not* required: siblings are unordered, and an in-order traversal is meaningless.

heap-ordered tree



the same thing, as an array



$\text{parent}(i) = (i-1)//2$ $\text{left}(i) = 2i+1$ $\text{right}(i) = 2i+2$

No pointers, perfect cache locality, and the root — the minimum — is always at index 0.

Figure 13.1 Because the tree is complete, it maps onto an array with no gaps, and parent/child relationships become index arithmetic. No pointers, no allocation, perfect cache locality.

The index arithmetic is the whole trick:

$$\text{parent}(i) = \lfloor i-1/2 \rfloor, \text{left}(i) = 2i+1, \text{right}(i) = 2i+2$$

A binary min-heap from scratch

```

1  class MinHeap:
2      def __init__(self, items=None):
3          self.a = list(items or [])
4          for i in range(len(self.a) // 2 - 1, -1, -1):
5              self._sift_down(i)          # heapify: theta(n), see
6              below
7
8      def push(self, x):
9          self.a.append(x)
10         self._sift_up(len(self.a) - 1)    # theta(log n)
11
12     def pop(self):
13         top = self.a[0]
14         last = self.a.pop()
15         if self.a:
16             self.a[0] = last              # move the last leaf to the
17             root                          # then restore the property
18             self._sift_down(0)            # theta(log n)
19         return top
20
21     def peek(self):
22         return self.a[0]                  # theta(1)
23
24     def _sift_up(self, i):
25         while i > 0:
26             p = (i - 1) >> 1
27             if self.a[i] >= self.a[p]:
28                 break
29             self.a[i], self.a[p] = self.a[p], self.a[i]
30             i = p

```

```

29
30     def _sift_down(self, i):
31         n = len(self.a)
32         while True:
33             smallest, l, r = i, 2 * i + 1, 2 * i + 2
34             if l < n and self.a[l] < self.a[smallest]:
35                 smallest = l
36             if r < n and self.a[r] < self.a[smallest]:
37                 smallest = r
38             if smallest == i:
39                 return
40             self.a[i], self.a[smallest] = self.a[smallest], self.a[i]
41             i = smallest

```

THE MATH | Why heapify is $\Theta(n)$ and not $\Theta(n \log n)$

Building a heap by n pushes costs $\Theta(n \log n)$. Building it by sifting down from the last internal node costs only $\Theta(n)$, and the reason is that most nodes are near the *bottom*, where sifting is cheap. At height h there are at most $\lceil n/2^{h+1} \rceil$ nodes, each costing $O(h)$:

$$\sum_{h=0}^{\lceil \log n \rceil} n/2^{h+1} \cdot O(h) = O(n \sum_{h=0}^{\infty} h/2^{h+1}) = O(n)$$

because $\sum_{h=0}^{\infty} h/2^h = 2$. Half the nodes are leaves and cost nothing; only the single root costs $\log n$.

Operation	Cost	Note
peek	$\Theta(1)$	the root
push	$\Theta(\log n)$	sift up
pop	$\Theta(\log n)$	sift down
heapify(list)	$\Theta(n)$	bottom-up
pushpop / replace	$\Theta(\log n)$, one sift	half the cost of push then pop
search for arbitrary key	$\Theta(n)$	heaps are not searchable
decrease-key	$\Theta(\log n)$ with an index map	otherwise $\Theta(n)$ to find it
merge two heaps	$\Theta(n)$	$\Theta(\log n)$ for pairing/leftist heaps

Using `heapq` without fighting it

Python's `heapq` operates on a plain list. It is a min-heap only, which leads to four idioms worth knowing.

The four `heapq` idioms

```

1  import heapq
2
3  # 1. Max-heap: negate the key.
4  h = []
5  for x in data:
6      heapq.heappush(h, -x)
7  largest = -heapq.heappop(h)
8

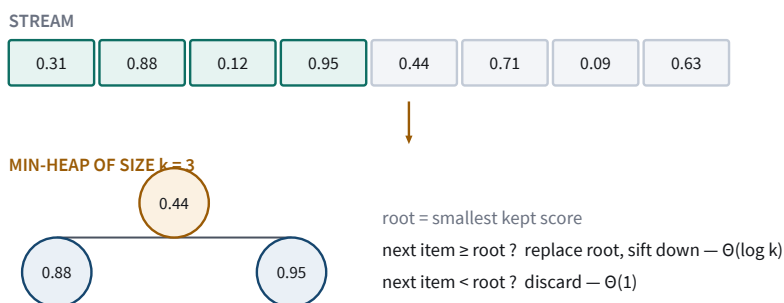
```

```

9  # 2. Priority with a payload: push tuples, and add a tiebreaker so that
10 #    Python never has to compare the payloads (which may be uncomparable).
11 import itertools
12 counter = itertools.count()
13 heapq.heappush(h, (priority, next(counter), payload))
14
15 # 3. Bounded top-k: keep a MIN-heap of size k of the LARGEST items.
16 def top_k(stream, k):
17     h = []
18     for x in stream:
19         if len(h) < k:
20             heapq.heappush(h, x)
21         elif x > h[0]:
22             heapq.heapreplace(h, x)      # one sift instead of two
23     return sorted(h, reverse=True)
24
25 # 4. Lazy deletion: you cannot remove an arbitrary element, so mark it
26 #    dead in a side table and skip it when it surfaces.
27 def pop_alive(h, dead):
28     while h:
29         item = heapq.heappop(h)
30         if item not in dead:
31             return item
32         dead.remove(item)
33     return None

```

Idiom 3 is the one that matters most. It is counter-intuitive — to find the k largest you keep a heap of *smallest* — and the reason is that the root then holds the weakest survivor, so deciding whether a new item belongs is a single $\Theta(1)$ comparison.



$\Theta(n \log k)$ time and $\Theta(k)$ memory, and it never needs the whole stream in memory.

Figure 13.2 Top- k over a stream with a bounded min-heap. Items below the root are discarded in constant time; only items that make the cut pay $\Theta(\log k)$.

PERFORMANCE NOTE | Which top-k should you actually use?

Situation	Use	Cost
Stream, cannot hold n	bounded min-heap	$\Theta(n \log k)$ time, $\Theta(k)$ space
Python list in memory	<code>heapq.nlargest(k, xs)</code>	same, but in C
NumPy array, $k \ll n$	<code>np.argpartition</code>	$\Theta(n)$, 10–40x faster
Torch tensor on GPU	<code>torch.topk</code>	radix-select, $\Theta(n)$
Need all k and their order	partition then sort k	$\Theta(n + k \log k)$
k close to n	just sort	$\Theta(n \log n)$

k -way merge

Merging k sorted streams is the canonical heap application, and it is how external sorting, log aggregation and multi-shard retrieval all work.

Merge k sorted streams with a heap of size k

```

1  import heapq
2
3  def merge_k(streams):
4      """streams: list of sorted iterables. Yields one merged sorted
5         stream."""
6      h = []
7      iters = [iter(s) for s in streams]
8      for i, it in enumerate(iters):
9          for first in it:
10             heapq.heappush(h, (first, i))
11             break
12     while h:
13         val, i = heapq.heappop(h)
14         yield val
15         for nxt in iters[i]:
16             heapq.heappush(h, (nxt, i))
17             break

```

Total cost: $\Theta(N \log k)$ for N elements across k streams, and the memory is $\Theta(k)$ regardless of N . That memory bound is why this is the merge step of external sort, and why a retrieval service can merge the results of 64 index shards without materialising them.

WHERE THIS SHOWS UP IN ML | Heaps in the ML stack, specifically

Top- k retrieval. Every vector index — flat, IVF, HNSW — maintains a bounded max-heap of candidates and a min-heap of results. HNSW’s search is literally two heaps (Chapter 33).

Beam search. Keep the b best partial sequences; at each step expand all of them and take the best b of $b \times V$ continuations, which is a bounded heap (or, faster, a **topk** over the flattened logits). Chapter 35.

BPE training. Repeatedly merge the most frequent adjacent pair. That is a max-heap of pair counts with lazy deletion, because a merge changes the counts of neighbouring pairs and you cannot efficiently find them in the heap. Chapter 14.

Schedulers. An inference server’s continuous batching loop picks which requests to run next by priority — deadline, sequence length, or fair share — which is a priority queue. So is the GPU work queue underneath it.

Sparse attention and MoE routing. Top- k over scores, per token, is a heap or a radix-select depending on hardware.

Two useful variants

Running median with two heaps. Keep a max-heap of the lower half and a min-heap of the upper half, with sizes differing by at most one. The median is the root of the larger heap, or the mean of the two roots.

Streaming median in $O(\log n)$ per element

```

1  import heapq
2
3  class RunningMedian:
4      def __init__(self):
5          self.lo = []      # max-heap via negation: the smaller half
6          self.hi = []      # min-heap: the larger half
7
8      def push(self, x):
9          heapq.heappush(self.lo, -heapq.heappushpop(self.hi, x))
10         if len(self.lo) > len(self.hi):      # rebalance
11             heapq.heappush(self.hi, -heapq.heappop(self.lo))
12
13     def median(self):
14         if len(self.hi) > len(self.lo):
15             return self.hi[0]
16         return (self.hi[0] - self.lo[0]) / 2.0

```

Indexed priority queue with **decrease-key.** Dijkstra’s algorithm (Chapter 23) wants to lower a vertex’s tentative distance. A plain heap cannot find that vertex. Two solutions: keep a **pos** dict from element to array index and update it on every swap; or — far simpler and what almost everyone does in practice — push a new entry and ignore stale ones on pop. The lazy version uses $\Theta(E)$ memory instead of $\Theta(V)$ and is faster in practice because the bookkeeping is free.

NOTE | Fibonacci heaps, and why nobody uses them

Fibonacci heaps achieve $\Theta(1)$ amortised **decrease-key**, improving Dijkstra from $\Theta(E \log V)$ to $\Theta(E + V \log V)$. They are a beautiful piece of amortised analysis (the potential method of Chapter 2 at its best) and essentially never used: the constant factors are large, the structure is a forest of pointer-linked trees with terrible locality, and pairing heaps get most of the benefit with a tenth of the code. A good reminder that an asymptotic improvement is a claim about large n , not about your n .

EXERCISES

1. Prove that a complete binary tree of n nodes has height $\lfloor \log_2 n \rfloor$, and that the last internal node is at index $n/2 - 1$.
2. Implement **heapify** both ways (repeated push, and bottom-up sift-down) and measure the ratio at $n=10^6$. Confirm the $\Theta(n)$ versus $\Theta(n \log n)$ prediction.
3. Implement a max-heap without negating keys, by parameterising the comparison. Then explain why **heapq** does not offer this.
4. Find the top 100 of 10^7 float32 scores four ways: **sorted**, **heapq.nlargest**, a hand-written bounded heap, and **np.argpartition**. Report times and peak memory.
5. Implement an indexed priority queue with **decrease-key** in $\Theta(\log n)$, and compare it to the lazy-deletion approach inside Dijkstra on a graph with 10^6 edges.
6. Implement k -way merge of 1000 sorted files totalling more than memory. How does the running time vary with the merge fan-in k , and where is the optimum?
7. Extend **RunningMedian** to an arbitrary quantile q , keeping the two-heap invariant. What changes, and why is this harder than the median?
8. A serving queue must run requests by (deadline, then arrival). Implement it with **heapq**, supporting cancellation of an arbitrary request.

CHAPTER RECAP

- A binary heap is a complete tree stored as a flat array; parent and child are index arithmetic, so there are no pointers and locality is perfect.
- **peek** is $\Theta(1)$; **push** and **pop** are $\Theta(\log n)$; bottom-up **heapify** is $\Theta(n)$ because most nodes are leaves.
- A heap cannot be searched, and **decrease-key** requires an auxiliary index map — or lazy deletion, which is usually better in practice.
- Top- k over a stream uses a *min*-heap of size k : the root is the weakest survivor, so rejection is $\Theta(1)$.
- k -way merge costs $\Theta(N \log k)$ with $\Theta(k)$ memory, which is what makes external sorting and multi-shard retrieval possible.
- Two heaps give a streaming median; the same trick gives any fixed quantile with more care.
- Heaps appear in retrieval, beam search, BPE training, schedulers and MoE routing. In NumPy or Torch, prefer **argpartition** / **topk**.

Tries, Automata and the Anatomy of a Tokenizer

Structures indexed by strings rather than by hashes of strings

CORE

PREREQS: Chapters 5, 7

A hash table can tell you whether a string is in a set. It cannot tell you which strings start with a prefix, which of them is the longest match at this position, or where in a document any of a million patterns occur. Those questions need a structure that indexes the string itself, and the answer is a trie — which, with one addition, becomes an automaton that scans text without ever backtracking.

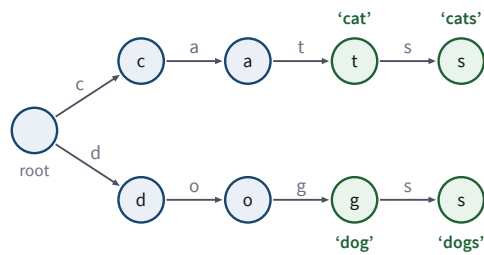
WHAT YOU WILL LEARN

- Implement a trie with insert, search, prefix enumeration and longest-match
- Compare tries with hash tables and know exactly which queries each supports
- Build an Aho–Corasick automaton and use it for multi-pattern search
- Understand compressed tries (radix trees) and their memory behaviour
- Implement a working BPE tokenizer with the right data structures
- See how radix trees index prefix caches in LLM serving

The trie

DEFINITION | Trie (prefix tree)

A rooted tree in which each edge is labelled with a symbol, so each node corresponds to the string spelled by the path from the root. A node is *terminal* if that string is a member of the stored set. Lookup, insertion and deletion all cost $\Theta(|\text{key}|)$, independent of how many keys the trie holds.



insert(word)	$\Theta(\text{word})$
lookup(word)	$\Theta(\text{word})$
all words with prefix P	$\Theta(P + \text{output})$
longest prefix of S in the set	$\Theta(S)$
memory	prefixes shared

Nothing here depends on how many words the trie holds.
That is the property a hash table cannot give you.

Figure 14.1 A trie over {cat, cats, dog, dogs}. Shared prefixes are stored once, and every operation's cost depends on the length of the query, not on the size of the set.

A trie with the four operations that matter

```

1  class Trie:
2      __slots__ = ("children", "terminal", "value")
3
4      def __init__(self):
5          self.children = {}
6          self.terminal = False
7          self.value = None
8
9      def insert(self, word, value=None):          # theta(|word|)
10         node = self
11         for ch in word:
12             node = node.children.setdefault(ch, Trie())
13         node.terminal, node.value = True, value
14
15     def _walk(self, prefix):
16         node = self
17         for ch in prefix:
18             node = node.children.get(ch)
19             if node is None:
20                 return None
21         return node
22
23     def __contains__(self, word):                  # theta(|word|)
24         node = self._walk(word)
25         return node is not None and node.terminal
26
27     def with_prefix(self, prefix):                  # theta(|prefix| +
28         output)
29         """Every stored word beginning with prefix, in lexicographic
30         order."""
31         node = self._walk(prefix)
32         out = []
33         if node is None:
34             return out
35         stack = [(node, prefix)]
36         while stack:
37             n, s = stack.pop()
38             if n.terminal:
39                 out.append(s)
40             for ch in sorted(n.children, reverse=True):
41                 stack.append((n.children[ch], s + ch))

```

```

40         return out
41
42     def longest_prefix_of(self, s, start=0):    # theta(|s| - start)
43         """Longest stored word that is a prefix of s[start:], or None."""
44         node, best, i = self, None, start
45         while i < len(s):
46             node = node.children.get(s[i])
47             if node is None:
48                 break
49             i += 1
50             if node.terminal:
51                 best = (i, node.value)
52         return best

```

`longest_prefix_of` is the operation that justifies the whole structure. It is how a Word-Piece tokenizer segments a word, how an IP router matches the longest routing prefix, and how a dictionary-based segmenter handles Chinese and Japanese — and no hash table can do it without trying every prefix length separately.

Query	Hash set	Trie
Is w present?	$\$ \backslash \Theta \backslash$	$w \backslash$
All words with prefix p	$\Theta(n)$ scan	$\$ \backslash \Theta \backslash$
Longest stored prefix of s	$\$ \backslash \Theta \backslash$	$s \backslash$
Lexicographic iteration	$\Theta(n \log n)$	$\Theta(\text{total length})$
Fuzzy match within edit distance 1	$\Theta(n)$	$\$ \backslash \Theta \backslash$
Memory	keys stored whole	prefixes shared, nodes have overhead

COMMON PITFALL | Trie memory in Python

A naive Python trie over a 200,000-word vocabulary can use hundreds of megabytes: every node is a `dict` (over 200 bytes) plus the `Trie` object. Three fixes, each roughly an order of magnitude: use `__slots__` and a plain dict (as above); compress single-child chains into a radix tree; or use a double-array trie or an LOUDS-encoded succinct trie, which store the whole structure in two flat integer arrays and query it with bit operations. Production tokenizers use the last option.

Compressed tries (radix trees)

A trie over long, sparse keys has many chains of single-child nodes. A **radix tree** (or Patricia trie) collapses each such chain into one edge labelled with a whole substring. The result has at most $2k-1$ nodes for k keys regardless of key length, which for URLs, file paths or token sequences is a huge saving.

Radix trees are what implement IP routing tables, the Linux kernel's page cache index, and — the reason they appear in this book — the prefix cache of an LLM server.

WHERE THIS SHOWS UP IN ML | RadixAttention: a radix tree over KV caches

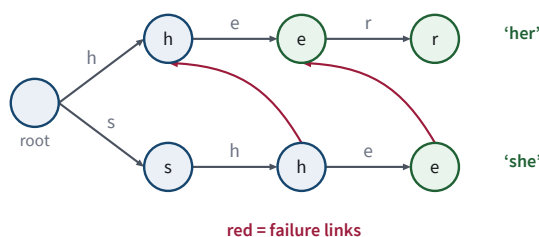
In a chat service most requests share a long system prompt, and many share a conversation prefix. Recomputing the KV cache for a shared prefix is pure waste. SGLang’s RadixAttention keeps a radix tree whose edges are token sequences and whose nodes point at the cached KV blocks for that prefix.

- A new request walks the tree to find its longest cached prefix, then computes only the remaining suffix.
- Eviction is LRU *over the tree*: only leaves may be evicted, because an internal node’s blocks are shared by its descendants.
- The hit rate on real chat traffic is 50–90%, and since prefill is compute-bound and proportional to prefix length, this translates almost directly into throughput.

The data structure is exactly the radix tree above, with token ids instead of characters and reference counts on nodes. Chapter 36 returns to it.

Aho–Corasick: searching for everything at once

Suppose you must find every occurrence of any of 50,000 patterns in a document — a profanity list, a PII detector, a set of blocked domains, a gazetteer of entity names. Running 50,000 separate searches is $\Theta(50000 \cdot n)$. Aho–Corasick does it in one pass.



Build the trie of all patterns.

Add a failure link from each node to the longest proper suffix that is also a node.

Then one pass over the text finds every occurrence of every pattern:

$\Theta(|\text{text}| + \text{total pattern length} + \text{hits})$,
with no backtracking, ever.

Figure 14.2 An Aho–Corasick automaton. The black edges are the trie of the patterns; the red failure links jump to the longest proper suffix that is still a prefix of some pattern, so the text pointer never moves backwards.

Aho–Corasick: build and scan

```

1  from collections import deque
2
3  class AhoCorasick:
4      def __init__(self, patterns):
5          self.next = [{}]
```

next[state][char] -> state

```

6          self.fail = [0]
7          self.out = [[]]
```

patterns ending at this state

```

8          for p in patterns:
9              self._add(p)
10             self._build_links()
11
12     def _add(self, p):
13         s = 0
14         for ch in p:
15             if ch not in self.next[s]:
```

```

16         self.next.append({}); self.fail.append(0);
17         self.out.append([])
18         self.next[s][ch] = len(self.next) - 1
19         s = self.next[s][ch]
20         self.out[s].append(p)
21
22     def _build_links(self):
23         q = deque()
24         for ch, s in self.next[0].items():
25             self.fail[s] = 0
26             q.append(s)
27         while q:
28             s = q.popleft()
29             for ch, t in self.next[s].items():
30                 q.append(t)
31                 f = self.fail[s]
32                 while f and ch not in self.next[f]:
33                     f = self.fail[f]
34                 self.fail[t] = self.next[f].get(ch, 0) if f or ch in
35                     self.next[0] else 0
36                 self.out[t] += self.out[self.fail[t]]      # inherit
37                     suffix hits
38
39     def find(self, text):
40         s = 0
41         for i, ch in enumerate(text):
42             while s and ch not in self.next[s]:
43                 s = self.fail[s]                      # follow
44                 failure links
45             s = self.next[s].get(ch, 0)
46             for p in self.out[s]:
47                 yield i - len(p) + 1, p

```

Total cost: $\Theta(\sum |p_i|)$ to build and $\Theta(|T| + \text{matches})$ to scan — independent of the number of patterns. The amortised argument is the same shape as the sliding window in Chapter 11: the state depth increases by at most one per character and each failure step decreases it, so the total number of failure steps is bounded by the text length.

KEY INSIGHT | The general principle behind failure links

The knowledge you gained from a partial match is not wasted when the match breaks; it tells you exactly how far you may skip. That is the idea in Knuth–Morris–Pratt (one pattern), Aho–Corasick (many patterns), and the Z-algorithm. Whenever an algorithm restarts from scratch after a mismatch, ask what it already knows.

Building a tokenizer, properly

Chapter 5 described what a tokenizer does. Here is what it is *made of*.

Training BPE. Maintain (a) a max-heap of candidate pairs keyed by count and (b) an index from each pair to the word positions where it occurs. Pop the best pair; merge it everywhere it occurs; for each affected position, decrement the counts of the two pairs that were destroyed and increment the two that were created, pushing the updated entries

onto the heap. Old heap entries are stale, so check the current count on pop and discard mismatches: lazy deletion again.

The core loop of BPE training

```

1  import heapq
2  from collections import defaultdict
3
4  def train_bpe(word_freqs, num_merges):
5      """word_freqs: {tuple_of_symbols: frequency}"""
6      pair_counts = defaultdict(int)
7      for word, f in word_freqs.items():
8          for a, b in zip(word, word[1:]):
9              pair_counts[(a, b)] += f
10
11     heap = [(-c, p) for p, c in pair_counts.items()]
12     heapq.heapify(heap)                                # theta(n)
13     merges = []
14     while heap and len(merges) < num_merges:
15         negc, pair = heapq.heappop(heap)
16         if -negc != pair_counts.get(pair, 0):
17             continue                                  # stale: skip
18         merges.append(pair)
19         new_sym = pair[0] + pair[1]
20         updated = {}
21         for word, f in word_freqs.items():
22             if pair[0] not in word:
23                 continue
24             out, i = [], 0
25             while i < len(word):
26                 if (i + 1 < len(word) and word[i] == pair[0]
27                     and word[i + 1] == pair[1]):
28                     out.append(new_sym); i += 2
29                 else:
30                     out.append(word[i]); i += 1
31             new_word = tuple(out)
32             if new_word != word:
33                 updated[word] = (new_word, f)
34         for old, (new, f) in updated.items():           # incremental
35             counts
36             for a, b in zip(old, old[1:]):
37                 pair_counts[(a, b)] -= f
38             for a, b in zip(new, new[1:]):
39                 pair_counts[(a, b)] += f
40             heapq.heappush(heap, (-pair_counts[(a, b)], (a, b)))
41         del word_freqs[old]
42         word_freqs[new] = word_freqs.get(new, 0) + f
43     return merges

```

This is still simplified — a production implementation keeps a position index so it does not scan every word — but it has the right shape: a heap for the argmax, incremental count updates, and lazy deletion for staleness. The naive alternative recomputes all pair counts after every merge and is $\Theta(V \cdot N)$ instead of roughly $\Theta(N + V \log V)$.

Encoding. WordPiece uses `longest_prefix_of` against a trie. BPE applies merges in rank order, which is best done with a linked list of symbols plus a heap of candidate merge positions: $\Theta(m \log m)$ per word instead of the naive $\Theta(m^2)$. Unigram/SentencePiece runs Viterbi over a lattice (Chapter 19).

WHERE THIS SHOWS UP IN ML | Constrained decoding is a trie walk

When you require a model's output to be valid JSON, to match a regex, or to name one of 100,000 allowed entities, you compile the constraint into a finite automaton — often literally a trie of allowed continuations — and at each decoding step mask the logits of tokens that would leave the automaton's valid transitions. The mask is precomputed per state, so the runtime cost is one lookup and one masked softmax per token. This is how `outlines`, `guidance` and `llama.cpp`'s grammar sampler work, and the data structure is the one in this chapter.

EXERCISES

1. `with_prefix` sorts each node's children on every visit. Replace the explicit stack with a recursive generator that yields lazily, and state the complexity of taking only the first ten results.
2. Implement a radix tree (compressing single-child chains) and measure the node count and memory against a plain trie for 100,000 URLs.
3. Implement fuzzy search over a trie: all words within edit distance 1 of a query, in $\Theta(|w| \cdot |\Sigma|)$ per candidate. (Hint: carry a dynamic-programming row down the trie — this previews Chapter 19.)
4. Build an Aho–Corasick automaton over 10,000 patterns and scan a 100 MB document. Compare to running `str.find` per pattern.
5. Explain why `out[t] += out[fail[t]]` is necessary, and give a pattern set and text where omitting it loses a match.
6. Implement BPE encoding with a linked list plus a heap and compare to the naive repeated-scan version on 100,000 words.
7. Design the radix tree for a prefix cache: nodes hold token spans and reference counts, eviction is LRU over leaves only. What invariant must hold before a node can be evicted, and how do you maintain it?

CHAPTER RECAP

- A trie indexes strings by their content, so cost depends on key length, not on set size, and prefix queries become natural.
- Tries beat hash sets on prefix enumeration, longest-prefix match, lexicographic iteration and fuzzy matching; hash sets win on plain membership and memory.
- Naive Python tries are memory-hungry; radix compression and succinct encodings are the production answers.
- Aho–Corasick adds failure links to a trie, giving $\Theta(|T| + \text{matches})$ multi-pattern search with no backtracking.
- BPE training is a max-heap of pair counts with incremental updates and lazy deletion; WordPiece encoding is a trie longest-match; Unigram is Viterbi over a lattice.
- Radix trees index prefix caches in LLM serving, and tries drive constrained decoding by masking invalid continuations.

Union-Find and the Algebra of Merging

Twenty lines that answer “are these two things the same thing?”

CORE

PREREQS: Chapter 12

Disjoint-set union is the smallest data structure in this book with a genuinely surprising analysis: two one-line optimisations take it from linear to effectively constant time per operation. It is also the structure you want whenever a pipeline merges things — connected components, clustering, deduplication groups, entity resolution — and it is routinely reimplemented badly.

WHAT YOU WILL LEARN

- Implement union-find with union by rank and path compression
- Understand the inverse Ackermann bound and why it is effectively constant
- Apply it to connected components, Kruskal’s MST and cycle detection
- Build transitive deduplication clusters from pairwise near-duplicate matches
- Know the variants: weighted union, union with rollback, and the offline trick
- Recognise when union-find is the wrong tool

The problem

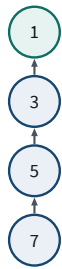
You have n elements, initially each in its own group, and a stream of operations:

- `union(a, b)` : merge the groups containing a and b .
- `find(a)` : return a canonical representative of a ’s group, so that `find(a) == find(b)` exactly when a and b are in the same group.

The obvious implementations are all bad. A list of sets makes `union` $\Theta(n)$. An array of group labels makes `union` $\Theta(n)$ (relabel everything). A dict of dicts makes both slow and the memory enormous.

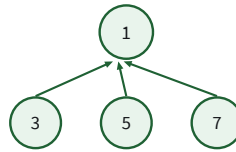
The right idea is to represent each group as a *tree*, where `find` walks to the root and `union` links one root under another. Then both operations cost the tree’s depth — and the entire art is keeping that depth small.

BEFORE find(7)



depth 3: find(7) walks 3 links

AFTER find(7) with path compression



every node on the path now points at the root
the next find is $\Theta(1)$

With union by rank and path compression, m operations cost $\Theta(m \alpha(n))$ — $\alpha(n) \leq 4$ for any n you will ever see.

Figure 15.1 Left: a chain, where **find** walks three links. Right: after path compression, every node on the path points directly at the root, so all subsequent finds are constant time.

The implementation

Union-find, complete

```

1  class DSU:
2      def __init__(self, n):
3          self.parent = list(range(n))      # parent[i] == i means i is a
4              root
5          self.rank = [0] * n               # upper bound on the tree
6              height
7          self.size = [1] * n               # component sizes, often useful
8          self.components = n
9
10     def find(self, x):
11         # iterative: no recursion limit
12         root = x
13         while self.parent[root] != root:
14             root = self.parent[root]
15         while self.parent[x] != root:      # second pass: path compression
16             self.parent[x], x = root, self.parent[x]
17         return root
18
19     def union(self, a, b):
20         ra, rb = self.find(a), self.find(b)
21         if ra == rb:
22             return False                    # already together
23         if self.rank[ra] < self.rank[rb]:  # union by rank: shallow under
24             deep
25             ra, rb = rb, ra
26         self.parent[rb] = ra
27         self.size[ra] += self.size[rb]
28         if self.rank[ra] == self.rank[rb]:
29             self.rank[ra] += 1
30         self.components -= 1
31         return True
32
33     def connected(self, a, b):
34         return self.find(a) == self.find(b)
  
```

Two optimisations, each one line:

Union by rank attaches the shallower tree under the deeper one, so the height only grows when two equal-height trees merge. By induction, a tree of rank r contains at least 2^r nodes, so the height is at most $\log_2 n$. (Union by *size* — attach the smaller component under the larger — gives the same bound and is often more convenient because you want the sizes anyway.)

Path compression makes every node visited by a `find` point directly at the root. It costs nothing extra asymptotically — you already walked the path — and it makes the structure flatter every time you query it.

THEOREM | Tarjan's bound

With both union by rank and path compression, any sequence of m operations on n elements takes $\Theta(m\alpha(n))$ time, where α is the inverse Ackermann function. $\alpha(n) \leq 4$ for every n below $2^{2^{65536}}$, so the amortised cost is constant for any input that exists.

The proof is a sophisticated potential-function argument and is not worth reproducing here. The takeaway is practical: **use both optimisations**. With only one you get $\Theta(\log n)$; with neither, $\Theta(n)$ worst case.

COMMON PITFALL | Rank is not height after compression

After path compression a tree's actual height may be far below its stored `rank`. That is fine — `rank` remains a valid *upper bound*, which is all the argument needs — but do not use `rank` as if it were the true height, and do not try to “fix it up”. Also note that union by rank and path compression interact: neither analysis alone gives the α bound.

The four standard applications

Connected components. Union every edge; the number of components is `dsu.components` and the component of a vertex is `find(v)`. This is $\Theta(E\alpha(V))$, which beats a BFS/DFS ($\Theta(V+E)$) only in the *incremental* setting — when edges arrive over time and you must answer queries in between. For a static graph, BFS is simpler and equally fast.

Kruskal's minimum spanning tree. Sort edges by weight, then add each edge whose endpoints are not already connected. Union-find is what makes the “already connected?” test cheap. Chapter 23.

Cycle detection in an undirected graph. `union(u, v)` returning `False` means u and v were already connected, so the edge closes a cycle.

Equivalence closure. Given pairwise “these two are the same” assertions — from a near-duplicate detector, an entity resolver, or a merge of user accounts — union-find computes the transitive closure in one pass.

Turning pairwise near-duplicates into clusters

```
1 def dedup_clusters(n_docs, pairs):
2     """pairs: iterable of (i, j) judged near-duplicate by LSH (Chapter
3     29)."""
4     dsu = DSU(n_docs)
5     for i, j in pairs:
```

```

5     dsu.union(i, j)
6     clusters = {}
7     for i in range(n_docs):
8         clusters.setdefault(dsu.find(i), []).append(i)
9     return list(clusters.values())

```

WHERE THIS SHOWS UP IN ML | Deduplicating a training corpus

This is the standard final step of corpus deduplication. MinHash and LSH (Chapter 29) produce *candidate pairs* of near-duplicate documents; those pairs form an implicit graph; union-find turns the graph into clusters; and you keep one representative per cluster.

Two things to know. First, this is the right place for the work: the pair list for a billion-document corpus can be tens of billions of pairs, and union-find processes them in a single streaming pass with one array of integers — 4 GB for a billion documents, no graph materialised.

Second, transitivity is an *assumption*, and it is a slightly false one. Near-duplicate is not a transitive relation: A resembles B, B resembles C, and A and C may be quite different.

Chaining through a dense region can therefore merge a large cluster of loosely related documents. Production pipelines cap cluster size, or cluster by connected components of a *thresholded, mutual k-nearest-neighbour* graph instead.

Variants worth knowing

Weighted / potential DSU. Store, along with each parent pointer, an offset relative to the parent — a difference, a ratio, or a group element. Then `find` accumulates the offset along the path, and you can answer “what is $x-y$?” as well as “are x and y related?”. This solves systems of difference constraints and the classic “evaluate division” problem.

DSU with rollback. Path compression makes undo impossible, so drop it, keep union by rank only, and push each parent change onto a stack. Each operation is $\Theta(\log n)$ and can be undone. This enables offline dynamic connectivity: process a query tree (segment tree on time) and roll back on the way out.

Persistent / partial DSU. Store, for each union, the timestamp at which it happened; then “were u and v connected at time t ?” is answered by walking upward while the recorded timestamps are $\leq t$. Useful for time-travel queries over a merge history.

WATCH OUT | What union-find cannot do

There is no efficient `split` or `un-union` in the plain structure — the relation is monotone, groups only ever merge. If your problem deletes edges, union-find is the wrong tool; you need a link-cut tree, or the offline segment-tree-on-time trick, or simply to rebuild.

Recognising this limitation early saves a great deal of wasted effort.

EXERCISES

1. Implement **DSU** without path compression and measure the average `find` depth after 10^6 random unions on 10^6 elements. Then add compression and measure again.
2. Prove that with union by rank, a tree of rank r has at least 2^r nodes, and conclude that the height is at most $\log_2 n$.

3. Implement path *halving* (`parent[x] = parent[parent[x]]` during the walk) and compare it to full two-pass compression in wall-clock time. Why is halving often faster in practice despite the same asymptotics?
4. Use union-find to implement Kruskal's algorithm and run it on a graph with 10^6 edges. Profile the split between sorting and union-find.
5. Implement a weighted DSU that answers $x-y$ for a stream of constraints $x-y=c$, and detects contradictions.
6. Given 100 million near-duplicate pairs over 20 million documents, build clusters with DSU. Report memory and time. Then cap cluster size at 1000 and describe the policy you used for choosing which merges to reject.
7. Show that the “offline dynamic connectivity” problem (edges are added and removed over time; answer connectivity queries) can be solved in $\Theta((n+q)\log q \log n)$ with DSU with rollback and a segment tree over time.

CHAPTER RECAP

- Union-find represents each group as a tree; `find` walks to the root and `union` links roots. Cost is the depth.
- Union by rank (or size) bounds the height at $\log_2 n$; path compression flattens the tree on every query. Together they give $\Theta(\alpha(n))$ amortised, which is at most 4 for any real input — use both.
- `rank` remains a valid upper bound after compression even though the true height may be smaller.
- Applications: connected components (especially incremental), Kruskal's MST, undirected cycle detection, and equivalence closure.
- Corpus deduplication uses DSU to turn LSH candidate pairs into clusters in one streaming pass; beware that near-duplicate is not really transitive.
- Variants: weighted DSU for offsets, rollback DSU for undo and offline dynamic connectivity, timestamped DSU for time-travel queries.
- There is no efficient split; if your problem removes edges, use a different structure.

PART IV

Algorithm Design Paradigms

Four ways to turn a problem you cannot solve into problems you can: split it, commit to the locally best move, remember subproblems, or search with the courage to undo.

-
- 16 Divide and Conquer
 - 17 Greedy Algorithms
 - 18 Dynamic Programming I: The Method
 - 19 Dynamic Programming II: Sequence Algorithms in Machine Learning
 - 20 Backtracking and Combinatorial Search
 - 21 Randomised Algorithms and Reservoir Sampling

Divide and Conquer

Break it, solve the pieces, pay for the glue

CORE

PREREQS: Chapter 8

Divide and conquer is the first of the four design paradigms, and the one with the cleanest analysis: split the problem into independent subproblems, solve them recursively, and combine. The master theorem turns that description into a complexity in about ten seconds, and the same structure is what makes an algorithm parallelisable — which is why this paradigm matters more on modern hardware than it did on a single core.

WHAT YOU WILL LEARN

- Recognise the divide-and-conquer structure and write down its recurrence
- Apply the master theorem, including all three cases and when it does not apply
- Derive the cost of merge sort, binary search, Karatsuba and Strassen
- Use divide and conquer for counting inversions and closest pairs
- Understand why divide and conquer maps onto parallel hardware
- Know when the combine step makes the whole approach pointless

The structure

The shape

```

1  def solve(problem):
2      if small(problem):
3          return base_case(problem)
4      subproblems = divide(problem)           # a pieces, each of size
      n/b
5      answers = [solve(s) for s in subproblems] # independent: can be
      parallel
6      return combine(answers)                 # cost f(n)

```

The resulting recurrence is

$$T(n) = a \cdot T(n/b) + f(n)$$

with a the number of subproblems, b the shrink factor, and $f(n)$ the cost of dividing and combining. Three numbers determine everything.

KEY INSIGHT | The key word is *independent*

The subproblems must not depend on one another's answers. When they do overlap, this paradigm recomputes them exponentially often and you should be doing dynamic programming instead (Chapter 18). “Do the subproblems overlap?” is the single question that separates the two chapters.

The master theorem

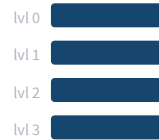
a) leaves dominate



$$T(n) = 4T(n/2) + n$$

$$\Theta(n^2)$$

b) evenly spread



$$T(n) = 2T(n/2) + n$$

$$\Theta(n \log n)$$

c) root dominates



$$T(n) = 2T(n/2) + n^2$$

$$\Theta(n^2)$$

Compare the work at the root, $n^{\log_b a}$, with $f(n)$. Whichever wins by a polynomial factor is the answer.

Figure 16.1 The three regimes. Compare the total work at the leaves, $n^{\log_b a}$, with the work at the root, $f(n)$. One dominates, or they balance.

THEOREM | Master theorem

Let $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$. Write $c = \log_b a$, so n^c is the number of leaves.

1. If $f(n) = O(n^{c-\epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^c)$. (Leaves dominate.)
2. If $f(n) = \Theta(n^c \log^k n)$ with $k \geq 0$, then $T(n) = \Theta(n^c \log^{k+1} n)$. (Balanced.)
3. If $f(n) = \Omega(n^{c+\epsilon})$ and $af(n/b) \leq df(n)$ for some $d < 1$ and large n (the regularity condition), then $T(n) = \Theta(f(n))$. (Root dominates.)

The intuition is worth more than the statement. The recursion tree has $\log_b n$ levels; level i contains a^i nodes each doing $f(n/b^i)$ work. The leaves number $a^{\log_b n} = n^{\log_b a}$. So you are comparing a geometric series' two ends: if the terms grow going down, the leaves dominate; if they shrink, the root dominates; if they are equal, multiply by the number of levels.

Recurrence	a	b	$n^{\log_b a}$	$f(n)$	Case	$T(n)$
Binary search	1	2	$n^0 = 1$	$\Theta(1)$	2	$\Theta(\log n)$
Merge sort	2	2	n	$\Theta(n)$	2	$\Theta(n \log n)$
Tree traversal	2	2	n	$\Theta(1)$	1	$\Theta(n)$
Karatsuba	3	2	$n^{1.585}$	$\Theta(n)$	1	$\Theta(n^{1.585})$
Strassen	7	2	$n^{2.807}$	$\Theta(n^2)$	1	$\Theta(n^{2.807})$
Naive matmul	8	2	n^3	$\Theta(n^2)$	1	$\Theta(n^3)$
Closest pair	2	2	n	$\Theta(n)$	2	$\Theta(n \log n)$

Recurrence	a	b	$n^{\log_b a}$	$f(n)$	Case	$T(n)$
Quickselect (expected)	1	2	1	$\Theta(n)$	3	$\Theta(n)$

WATCH OUT | When the master theorem does not apply

It requires subproblems of *equal* size and a polynomial f . It says nothing about $T(n)=T(n-1)+n$ (not a constant-factor shrink), about $T(n)=T(n/3)+T(2n/3)+n$ (unequal splits — draw the tree; the answer is $\Theta(n \log n)$), or about $T(n)=2T(n/2)+n/\log n$ (falls in the gap between cases 1 and 2; the Akra–Bazzi method handles it).

Karatsuba: why reducing a matters

Multiplying two n -digit numbers the schoolbook way is $\Theta(n^2)$. Split each into halves, $x = x_1 B + x_0$ and $y = y_1 B + y_0$:

$$xy = x_1 y_1 B^2 + (x_1 y_0 + x_0 y_1) B + x_0 y_0$$

That is four half-size multiplications: $T(n) = 4T(n/2) + \Theta(n)$, which by case 1 is $\Theta(n^2)$ — no gain. Karatsuba's observation is that

$$x_1 y_0 + x_0 y_1 = (x_1 + x_0)(y_1 + y_0) - x_1 y_1 - x_0 y_0$$

so three multiplications suffice. $T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.585})$.

Karatsuba multiplication

```

1  def karatsuba(x, y):
2      if x < 10 or y < 10:                # base case: schoolbook
3          return x * y
4      n = max(x.bit_length(), y.bit_length()) // 2
5      mask = (1 << n) - 1
6      x1, x0 = x >> n, x & mask
7      y1, y0 = y >> n, y & mask
8      z2 = karatsuba(x1, y1)
9      z0 = karatsuba(x0, y0)
10     z1 = karatsuba(x1 + x0, y1 + y0) - z2 - z0  # one multiply, not two
11     return (z2 << (2 * n)) + (z1 << n) + z0

```

Strassen's algorithm does the same thing for matrices: eight $n/2$ block multiplications become seven, giving $\Theta(n^{2.807})$.

HARDWARE REALITY | Why Strassen is rare in practice

Strassen is asymptotically faster and is used, but only above roughly $n=1000$ and with care. Three reasons. It is numerically less stable — the subtractions cause cancellation, so the error bound is weaker than for the standard algorithm. It needs $\Theta(n^2)$ temporary matrices, which hurts on memory-bound hardware. And standard matmul has been optimised into the ground: a tuned GEMM reaches 90%+ of peak FLOPs through tiling and vector instructions, so Strassen's 12% fewer operations can easily be outweighed by worse locality. The asymptotically best known exponent is around 2.37, and those algorithms are entirely theoretical — their constants are astronomical.

Two classic applications

Counting inversions. How far is a permutation from sorted? Count pairs (i, j) with $i < j$ and $a_i > a_j$. Brute force is $\Theta(n^2)$; merge sort counts them for free, because during the merge, when an element from the right half is taken, it is smaller than everything remaining in the left half.

Inversions in $O(n \log n)$, piggybacking on merge sort

```

1  def sort_count(a):
2      if len(a) <= 1:
3          return a, 0
4      m = len(a) // 2
5      left, cl = sort_count(a[:m])
6      right, cr = sort_count(a[m:])
7      out, i, j, cross = [], 0, 0, 0
8      while i < len(left) and j < len(right):
9          if left[i] <= right[j]:
10             out.append(left[i]); i += 1
11          else:
12             out.append(right[j]); j += 1
13             cross += len(left) - i      # all remaining left elements
14             invert
15      out += left[i:] + right[j:]
16      return out, cl + cr + cross

```

This is exactly how Kendall's τ rank correlation is computed efficiently — a metric you will meet when evaluating rankers.

Closest pair of points. Sort by x , split at the median, recurse on both halves to get the best distance δ , then check the strip of width 2δ around the dividing line. The non-obvious part is that within that strip, sorted by y , each point needs to be compared with at most the next seven — a geometric argument — so the combine step is $\Theta(n)$ and the total is $\Theta(n \log n)$ rather than $\Theta(n^2)$.

Divide and conquer is the parallel paradigm

The subproblems are independent, which means they can run on different cores, different threads, or different machines with no coordination until the combine step. This is the structural reason divide and conquer matters more now than when it was invented.

Sequential algorithm	Parallel form	Where you meet it
Sum of an array	tree reduction	<code>all_reduce</code> , <code>torch.sum</code>
Prefix sum	Blelloch scan (Chapter 39)	ragged offsets, sampling
Merge sort	parallel merge sort / sample sort	GPU sort, Spark shuffle
Matrix multiply	block decomposition	every GEMM ever written
Max / argmax	tree reduction	<code>topk</code> , softmax denominator
FFT	butterfly network	convolution, positional encodings

Table 16.1 The master theorem applied. Notice how Karatsuba and Strassen both buy their speedup by trading one multiplication for several additions — reducing a at the cost of a larger $f(n)$, which is worth it because a sits in an exponent.

WHERE THIS SHOWS UP IN ML | **Tree reduction is why `all_reduce` scales**

Summing gradients across P workers naively means every worker sends to one master: $\Theta(P)$ messages through one link. A tree reduction has depth $\Theta(\log P)$ and no hot spot. Ring all-reduce goes further and achieves optimal bandwidth: each worker sends exactly $2(P-1)/P$ times the parameter size, *independent of P* . Chapter 41 derives it. The structural idea — combine pairwise, in a tree — is this chapter's.

COMMON PITFALL | **When the combine step kills you**

Divide and conquer only wins if `combine` is cheaper than solving the problem directly. If combining is $\Theta(n^2)$, then $T(n) = 2T(n/2) + \Theta(n^2) = \Theta(n^2)$ by case 3: you did all that recursion for nothing. Always check which case you land in *before* implementing. Equally, if the division is expensive — copying slices in Python is $\Theta(n)$ per level and allocates — you may add a constant factor that erases the benefit. Pass indices, not slices.

EXERCISES

1. Solve with the master theorem: (a) $T(n) = 9T(n/3) + n$; (b) $T(n) = T(2n/3) + 1$; (c) $T(n) = 3T(n/4) + n \log n$; (d) $T(n) = 2T(n/2) + n \log n$.
2. Show that $T(n) = T(n/3) + T(2n/3) + n$ is $\Theta(n \log n)$ by drawing the recursion tree and bounding its depth.
3. Implement Karatsuba and find empirically the input size at which it beats schoolbook multiplication in Python. Explain the crossover.
4. Implement `sort_count` and verify it against a brute-force $\Theta(n^2)$ count on random permutations. Use it to compute Kendall's τ between two rankings of 100,000 items.
5. Implement the closest-pair algorithm and prove the “at most seven neighbours in the strip” claim.
6. Implement Strassen's algorithm with a cutoff to standard multiplication below size k . Tune k and explain why the optimum is well above 2.
7. A divide-and-conquer algorithm splits into three subproblems of size $n/2$ and combines in $\Theta(n)$. What is its complexity? Would you use it in place of an $\Theta(n \log n)$ alternative? At what n ?

CHAPTER RECAP

- Divide and conquer: split into a independent subproblems of size n/b , recurse, combine in $f(n)$. The recurrence $T(n) = aT(n/b) + f(n)$ follows directly.
- The master theorem compares $n^{\log_b a}$ (work at the leaves) with $f(n)$ (work at the root): whichever dominates polynomially wins; if they balance, multiply by $\log n$.
- Reducing a is worth a lot because it sits in an exponent — the insight behind Karatsuba and Strassen.
- Strassen is asymptotically better and practically marginal, because standard GEMM is heavily optimised and Strassen is less stable and more memory-hungry.
- Counting inversions and closest-pair both ride on the merge structure and become $\Theta(n \log n)$.
- Independence of subproblems is what makes this the parallel paradigm: tree reductions, scans, block matmul and FFT are all instances.
- If subproblems overlap, use dynamic programming; if combining is as expensive as the whole problem, the paradigm buys nothing.

Greedy Algorithms

Take the best-looking move, and be able to prove you may

CORE

PREREQS: Chapter 9

A greedy algorithm makes the locally best choice and never reconsiders. When that works it is the simplest and fastest approach available; when it does not, it produces confidently wrong answers that pass every small test. The whole skill is knowing which case you are in, and this chapter teaches the two proof techniques that tell you.

WHAT YOU WILL LEARN

- State the two structural properties a problem needs for greedy to be correct
- Prove greedy correctness with an exchange argument and with “greedy stays ahead”
- Solve interval scheduling, Huffman coding and fractional knapsack
- Recognise where greedy fails and what to do instead
- Understand matroids as the general theory behind greedy
- Apply greedy reasoning to batching, caching and beam search

The pattern and the danger

Every greedy algorithm

```
1 def greedy(items, key):
2     solution = []
3     for item in sorted(items, key=key):      # the choice is the sort
4         order                                order
5         if feasible(solution, item):
6             solution.append(item)           # never reconsidered
7     return solution
```

The algorithm is trivial. The *sort order* is the algorithm, and choosing it correctly is the entire problem. In interval scheduling, sorting by start time gives a wrong answer, sorting by duration gives a wrong answer, and sorting by finish time gives the optimum — with no visible difference in the code.

SIX JOBS



Greedy by earliest finish time picks jobs 1, 3, 5 — provably optimal.

Greedy by earliest start would pick job 1 then job 4 and stop at two.

Figure 17.1 Interval scheduling. Earliest-finish-time is optimal and earliest-start-time is not, on the same input. Nothing about the code distinguishes them; only a proof does.

DEFINITION | The two properties greedy needs

Greedy choice property. There exists an optimal solution containing the greedy first choice. (Equivalently: committing to the greedy choice never makes the remaining problem worse.)

Optimal substructure. After making that choice, an optimal solution to the remaining subproblem, combined with the choice, is optimal overall.

Optimal substructure alone gives you dynamic programming. The greedy choice property is the extra ingredient that lets you skip the search.

Proving greedy correct

Two techniques cover almost everything.

The exchange argument. Take any optimal solution O . Show that you can transform it, step by step, into the greedy solution G without making it worse. Therefore G is optimal too.

PROOF | Interval scheduling: earliest finish time is optimal

Let $G = g_1, g_2, \dots$ be the greedy selection (by earliest finish) and $O = o_1, o_2, \dots$ any optimal selection, both sorted by finish time. Suppose they agree on the first k jobs and differ at $k+1$. By the greedy rule, g_{k+1} finishes no later than o_{k+1} . So replacing o_{k+1} with g_{k+1} in O leaves it feasible — g_{k+1} starts after $g_k = o_k$ ends, and finishes no later than o_{k+1} did, so it cannot conflict with o_{k+2} — and the same size. Repeating this exchange turns O into a solution agreeing with G everywhere, with no loss. Hence $|G| = |O|$.

Greedy stays ahead. Define a measure of partial progress and show that after every step, the greedy solution is at least as far along as any other. In interval scheduling the measure is “the finish time of the i -th chosen job”, and greedy’s is always $\leq$ any other solution’s.

KEY INSIGHT | How to test a greedy idea in ninety seconds

Before proving anything, try to break it. Construct the smallest adversarial input you can: two items where the greedy criterion and the true objective disagree. If you find one, greedy is wrong and you have a counterexample for free. If you cannot find one after genuine effort, look for the exchange argument. Most incorrect greedy algorithms die in the first thirty seconds of this exercise.

The canonical correct greedies

Interval scheduling (maximise count): sort by finish time. $\Theta(n \log n)$.

Fractional knapsack: sort by value/weight ratio, take greedily, split the last item. $\Theta(n \log n)$. Note that the 0/1 version — where you cannot split — is *not* greedy-solvable and needs dynamic programming (Chapter 18). The difference is one word in the problem statement.

Huffman coding: repeatedly merge the two least frequent symbols.

Huffman coding: a greedy algorithm built on a heap

```

1  import heapq
2  from collections import Counter
3
4  def huffman(text):
5      freq = Counter(text)
6      if len(freq) == 1:                                # degenerate: one symbol
7          return {next(iter(freq)): "0"}
8      heap = [(f, i, sym) for i, (sym, f) in enumerate(freq.items())]
9      heapq.heapify(heap)
10     codes = {sym: "" for sym in freq}
11     groups = {sym: [sym] for sym in freq}
12     nxt = len(freq)
13     while len(heap) > 1:
14         f1, _, a = heapq.heappop(heap)                 # two rarest
15         f2, _, b = heapq.heappop(heap)
16         for s in groups[a]:
17             codes[s] = "0" + codes[s]                 # prepend: deepest gets
18             most
19         for s in groups[b]:
20             codes[s] = "1" + codes[s]
21         merged = a + b
22         groups[merged] = groups[a] + groups[b]
23         heapq.heappush(heap, (f1 + f2, nxt, merged))
24         nxt += 1
25     return codes

```

The exchange argument for Huffman: in an optimal prefix code the two least frequent symbols can be assumed to be siblings at maximum depth (otherwise swap them with whatever is down there — the cost cannot increase). So merging them first is safe, and induction does the rest. The result is the optimal prefix-free code, within one bit per symbol of the entropy.

Minimum spanning tree (Kruskal and Prim, Chapter 23) and **Dijkstra’s algorithm** are both greedy with exchange-argument proofs.

Where greedy fails

Problem	Greedy gives	Why it fails	Correct approach
0/1 knapsack	can be arbitrarily bad	taking the best ratio may block a better pair	DP, $\Theta(nW)$
Coin change, arbitrary denominations	may fail entirely	4, 3, 1 for target 6: greedy gives 4+1+1, optimal is 3+3	DP, $\Theta(nA)$
Longest path in a DAG	wrong	local best edge is not on the best path	DP over topological order
Travelling salesman	$\Theta(\log n)$ factor off	nearest-neighbour paints itself into corners	DP / approximation
Set cover	$\ln n$ factor off	unavoidable unless $P = NP$	greedy is the best known
Optimal BST	wrong	frequencies interact	DP, $\Theta(n^3)$ or $\Theta(n^2)$

COMMON PITFALL | The coin-change trap

Greedy coin change is optimal for the US and Euro denominations and wrong in general. This matters because it is the classic example of an algorithm that works on every input you are likely to test by hand and fails on inputs a system will actually produce.

Whenever a greedy rule “obviously” works, ask what property of the specific numbers makes it work — for coins it is that each denomination is at least twice the previous one — and whether your data guarantees it.

Matroids: when greedy is *always* right

There is a complete theory, and it is worth knowing it exists.

DEFINITION | Matroid

A matroid is a pair (E, I) where E is a finite set and I a family of “independent” subsets satisfying: (1) $\emptyset \in I$; (2) subsets of independent sets are independent; (3) the **exchange property** — if $A, B \in I$ and $|A| < |B|$, there is some $x \in B \setminus A$ with $A \cup \{x\} \in I$.

THEOREM | Rado–Edmonds

For any weight function on E , the greedy algorithm — sort by weight descending, add each element if it keeps the set independent — finds a maximum-weight independent set **if and only if** (E, I) is a matroid.

Kruskal's algorithm is greedy over the *graphic matroid* (independent = acyclic edge sets), which is why it is correct. Scheduling unit jobs with deadlines to maximise profit is greedy over a *transversal matroid*. When your problem's feasible sets satisfy the exchange property, greedy is guaranteed; when they do not, you need a proof or a different paradigm.

WHERE THIS SHOWS UP IN ML | Greedy decisions in ML systems

Length bucketing. Sort sequences by length, then greedily fill batches until the padding waste exceeds a threshold. Optimal because, after sorting, the cost is monotone — the same structure as interval scheduling.

Greedy decoding. Take the argmax token at each step. This is greedy in the technical sense and it is *not* optimal for sequence likelihood: the highest-probability token now can lead to a low-probability continuation. Beam search is the bounded-width relaxation, and it is still not optimal. Chapter 35.

Decision-tree induction. Choosing the split that maximises information gain at each node is greedy. Finding the globally optimal tree is NP-hard, so every practical learner is greedy and accepts the suboptimality. Chapter 37.

Cache admission. Deciding whether a newly computed embedding is worth caching, by value density (hit probability / size), is fractional knapsack — greedy and optimal if items are divisible, approximate otherwise.

Token budget allocation in RAG. Filling a context window with the highest-relevance-per-token chunks is fractional knapsack again; since chunks are not divisible, greedy by density is a 2-approximation, which in practice is entirely acceptable.

EXERCISES

1. Give a counterexample showing that sorting intervals by duration does not maximise the number of non-overlapping intervals.
2. Prove, by exchange, that fractional knapsack by value/weight ratio is optimal. Then give an instance where the same rule applied to 0/1 knapsack is off by a factor of nearly 2.
3. Implement Huffman coding and verify that the average code length is within one bit of the entropy for several distributions.
4. Find a set of coin denominations, all at least twice the previous, for which greedy change-making still fails — or prove no such set exists.
5. Show that the set of forests of a graph forms a matroid, and deduce Kruskal's correctness from Rado–Edmonds.
6. Implement length bucketing that minimises total padded tokens given a maximum batch size. Prove your greedy rule is optimal or give a counterexample.
7. Greedy set cover is a $\ln n$ approximation. Implement it and construct an instance where it is a factor of $\ln n$ worse than optimal.

CHAPTER RECAP

- Greedy makes a locally optimal choice and never revisits it; the sort order *is* the algorithm.
- Correctness needs the greedy choice property plus optimal substructure. Prove it with an exchange argument or by showing greedy stays ahead.
- Before proving, spend ninety seconds trying to break it — most wrong greedy ideas die immediately.
- Correct: interval scheduling by finish time, fractional knapsack by density, Huffman by merging rarest pairs, Kruskal, Prim, Dijkstra.
- Wrong: 0/1 knapsack, general coin change, longest path, TSP. For some NP-hard problems greedy is the best known approximation and its ratio is the result.
- Matroids characterise exactly when greedy always works.
- In ML: length bucketing (correct), greedy decoding (suboptimal by design), decision-tree splitting (necessarily suboptimal), context budgeting (approximate).

Dynamic Programming I: The Method

A repeatable procedure, not a flash of insight

CORE

PREREQS: Chapters 8, 16

Dynamic programming has a reputation for requiring inspiration. It does not. It requires a procedure, applied patiently: name the state, write the recurrence, identify the base cases, choose an evaluation order, and then optimise the memory. This chapter gives you that procedure and applies it to enough problems that the pattern becomes mechanical.

WHAT YOU WILL LEARN

- Apply a five-step procedure to turn a problem into a dynamic program
- Distinguish problems with optimal substructure and overlapping subproblems from those without
- Choose between top-down memoisation and bottom-up tabulation for a given problem
- Reduce a DP's memory from $\Theta(n^2)$ to $\Theta(n)$, and recover the solution anyway
- Recognise the standard DP shapes: knapsack, interval, tree, bitmask, digit
- Apply DP to real space-time tradeoffs such as gradient checkpointing

When dynamic programming applies

Two conditions, both necessary.

Optimal substructure. An optimal solution is built from optimal solutions to subproblems. (Shared with divide and conquer and greedy.)

Overlapping subproblems. The same subproblem is needed many times. (This is what divide and conquer lacks and what makes caching pay.)

KEY INSIGHT | The distinction, in one line

Divide and conquer: subproblems are *disjoint*, so recurse and combine. Dynamic programming: subproblems *overlap*, so solve each once and store it. Greedy: you can prove one choice is safe, so you need not solve subproblems at all.

The five-step procedure

1. Define the state. What does $dp[\dots]$ mean, in a full English sentence? This is 80% of the work and the step people skip. “ $dp[i][w]$ is the maximum value obtainable using the first i items with total weight at most w ” is a state definition. “ $dp[i][w]$ is the answer for i and w ” is not.

2. Write the recurrence. Express $dp[\text{state}]$ in terms of strictly smaller states, by enumerating the possible *last decisions*.

3. Identify the base cases. The states that need no recurrence. Getting these wrong is the most common source of off-by-one bugs.

4. Choose an evaluation order. Any order in which every state’s dependencies are computed first. Equivalently: a topological order of the dependency DAG (Chapter 24).

5. Read off the answer, and optimise. Which state holds the result? Can you drop dimensions from memory?

	1	1	1	1	1	1	1
	1	2	3	4	5	6	7
$dp[i][j] =$	1	3	6	10	15	21	28
$dp[i-1][j]$	1	4	10	20	35	56	84
$+ dp[i][j-1]$							

Fill in any order that respects the dependencies. Row by row works; so does column by column.

Only the previous row is ever needed $\rightarrow \Theta(n)$ memory, not $\Theta(mn)$.

Figure 18.1 A DP table is a DAG of dependencies. Once you know which earlier cells a cell needs, the evaluation order and the memory optimisation both follow mechanically.

Worked example: 0/1 knapsack

Problem. n items with weights w_i and values v_i , capacity W . Maximise total value; each item may be taken at most once.

State. $dp[i][c]$ = the best value using items $0 \dots i-1$ with capacity exactly c available.

Recurrence. The last decision is whether to take item $i-1$:

$$dp[i][c] = \max(dp[i-1][c], v_{i-1} + dp[i-1][c - w_{i-1}])$$

where the second term applies only if $w_{i-1} \leq c$.

Base case. $dp[0][c] = 0$ for all c : no items, no value.

Order. Increasing i ; c in any order.

Answer. $dp[n][W]$.

0/1 knapsack: table, then rolling array

```

1  def knapsack_table(weights, values, W):
2      n = len(weights)
3      dp = [[0] * (W + 1) for _ in range(n + 1)]
4      for i in range(1, n + 1):
5          w, v = weights[i - 1], values[i - 1]
6          for c in range(W + 1):
7              dp[i][c] = dp[i - 1][c]
8              if w <= c:
9                  dp[i][c] = max(dp[i][c], v + dp[i - 1][c - w])
10     return dp[n][W]                    # theta(nW) time, theta(nW)
    memory
11
12 def knapsack_rolling(weights, values, W):
13     dp = [0] * (W + 1)
14     for w, v in zip(weights, values):
15         for c in range(W, w - 1, -1):    # DESCENDING: see the pitfall
16             dp[c] = max(dp[c], v + dp[c - w])
17     return dp[W]                      # theta(nW) time, theta(W)
    memory

```

COMMON PITFALL | The direction of the inner loop

In the rolling version the inner loop must run **downwards**. Going upwards would let `dp[c - w]` already include item *i*, so the item could be used more than once — which is the *unbounded* knapsack, a different problem. Reversing one loop changes which problem you are solving, silently. (If you *want* unbounded knapsack, loop upwards; that is the trick, not a bug.)

NOTE | “Pseudo-polynomial” means what it says

$\Theta(nW)$ looks polynomial but *W* is a *value*, not an input length. A capacity written with 64 bits gives *W* up to 2^{64} , so the running time is exponential in the input *size*. 0/1 knapsack is NP-hard; the DP is efficient only when *W* is small. The same caveat applies to coin change and subset-sum.

Top-down or bottom-up?

	Top-down (memoised recursion)	Bottom-up (tabulation)
Written as	the recurrence, directly	nested loops
Computes	only reachable states	all states
Order	automatic	you must find it
Memory	recursion stack + cache	table only
Speed in Python	slower (call + hash per state)	10–50x faster
Memory optimisation	hard	easy (drop dimensions)
When it wins	sparse or hard-to-order state space	dense state space, big <i>n</i>

The same knapsack, top-down

```

1  from functools import lru_cache
2
3  def knapsack_topdown(weights, values, W):
4      @lru_cache(maxsize=None)
5      def best(i, c):
6          if i == 0:
7              return 0
8          skip = best(i - 1, c)
9          take = (values[i - 1] + best(i - 1, c - weights[i - 1])
10                 if weights[i - 1] <= c else 0)
11          return max(skip, take)
12  return best(len(weights), W)

```

Recovering the solution, not just its value

Memory optimisation destroys the table you would traceback through. Two standard fixes.

Keep a decision table. Store one bit per state (“did we take item i ?”), which is $\Theta(nW)$ bits rather than $\Theta(nW)$ integers — often a 32x saving and enough to fit.

Hirschberg’s divide-and-conquer traceback. For sequence alignment (Chapter 19) you can compute the optimal alignment in $\Theta(nm)$ time and $\Theta(\min(n, m))$ memory by recursively finding the midpoint of the optimal path: run the DP forwards to the middle row and backwards from the end, add the two rows, and the argmin is where the optimal path crosses. Then recurse on the two halves. The time doubles; the memory becomes linear. This is the standard trick in bioinformatics and it is worth knowing because the same idea reappears as gradient checkpointing.

The standard shapes

Shape	State	Typical recurrence	Examples
Linear / prefix	<code>dp[i]</code>	from <code>dp[i-1]</code> , <code>dp[i-2]</code> , ...	Fibonacci, house robber, LIS
Two sequences	<code>dp[i][j]</code>	from three neighbours	edit distance, LCS, DTW
Knapsack	<code>dp[i][c]</code>	take / skip	subset sum, coin change, partition
Interval	<code>dp[i][j]</code>	split at k in (i, j)	matrix chain, optimal BST, burst balloons
Tree	<code>dp[v][state]</code>	combine children	tree independent set, tree diameter
Bitmask	<code>dp[mask][i]</code>	add one element to the set	TSP, assignment, scheduling
Digit	<code>dp[pos][tight][...]</code>	choose next digit	counting numbers with a property

Shape	State	Typical recurrence	Examples
On a DAG	$dp[v]$	over incoming edges	longest path, autograd, Viterbi

Table 18.1 The two directions. Write top-down first — it is a transcription of the recurrence and therefore hard to get wrong — then convert to bottom-up if you need the speed or the memory.

Longest increasing subsequence: two ways	
1	<code>def lis_quadratic(a):</code>
2	<code> dp = [1] * len(a)</code> <i># dp[i]: LIS ending at i</i>
3	<code> for i in range(len(a)):</code>
4	<code> for j in range(i):</code>
5	<code> if a[j] < a[i]:</code>
6	<code> dp[i] = max(dp[i], dp[j] + 1)</code>
7	<code> return max(dp, default=0)</code> <i># theta(n^2)</i>
8	
9	<code>from bisect import bisect_left</code>
10	<code>def lis_nlogn(a):</code>
11	<code> tails = []</code> <i># tails[k]: smallest tail of</i>
12	<code> for x in a:</code> <i># increasing subsequence of</i>
13	<code> i = bisect_left(tails, x)</code>
14	<code> if i == len(tails):</code>
15	<code> tails.append(x)</code>
16	<code> else:</code>
17	<code> tails[i] = x</code>
18	<code> return len(tails)</code> <i># theta(n log n)</i>

The second version is not a faster DP; it is a different algorithm that maintains a cleverer invariant. It illustrates a general point: once you have a correct DP, look for structure in the recurrence — monotonicity, convexity, a matrix that can be exponentiated — that lets you evaluate it faster than state-by-state. Convex hull trick, divide-and-conquer optimisation, Knuth optimisation and matrix-power DP are all instances.

WHERE THIS SHOWS UP IN ML | Gradient checkpointing is a dynamic program

Training uses memory proportional to the number of stored activations. If you store none, you must recompute the entire forward pass for every backward step; if you store all, memory is $\Theta(L)$ for L layers. Checkpointing stores a subset and recomputes the rest. The optimal policy — which layers to store, given a memory budget — is a dynamic program over (layer range, budget), and it is exactly the same shape as knapsack. The famous $\Theta(\sqrt{L})$ result (store every $\sqrt{L}$ -th layer, recompute within a segment) is the solution for a uniform-cost network, giving $\Theta(\sqrt{L})$ memory for one extra forward pass. Real implementations (`torch.utils.checkpoint`, DeepSpeed) solve the non-uniform version numerically. This is a space–time tradeoff computed by DP, running inside your training loop.

WHERE THIS SHOWS UP IN ML | Two more dynamic programs you are already running

Beam search with length normalisation is a truncated DP over (position, partial hypothesis).

Optimal batch splitting for pipeline parallelism — how to divide a model into stages so that the slowest stage is as fast as possible — is an interval DP over layer ranges, and is what a pipeline partitioner solves.

EXERCISES

1. For each of these, write the state definition in one English sentence: (a) longest common subsequence; (b) minimum number of coins for amount A ; © maximum sum of non-adjacent elements; (d) the number of distinct ways to decode a digit string.
2. Implement `knapsack_rolling` with the inner loop ascending, and show empirically that it solves unbounded knapsack instead.
3. Implement coin change (minimum coins) both top-down and bottom-up, and measure the ratio at $A=10^5$.
4. Add a decision table to `knapsack_rolling` so that it also returns the chosen items, using $\Theta(nW)$ bits.
5. Implement matrix-chain multiplication ($\Theta(n^3)$ interval DP) and use it to find the cheapest order to multiply a chain of 15 matrices with random dimensions. Compare to left-to-right.
6. Implement the $\Theta(n \log n)$ LIS and extend it to *return* the subsequence, not just its length.
7. Implement bitmask DP for the travelling salesman problem on 18 cities, $\Theta(2^n n^2)$. Measure the memory and explain why 25 cities is out of reach.
8. Formulate gradient checkpointing as a DP: given L layers with forward costs f_i , recompute costs r_i and activation sizes s_i , and a memory budget M , minimise total recomputation. Write the recurrence.

CHAPTER RECAP

- DP needs optimal substructure *and* overlapping subproblems; without overlap use divide and conquer, and with a provable safe choice use greedy.
- The five steps: define the state in a sentence, write the recurrence by enumerating last decisions, fix the base cases, choose an order that respects dependencies, read off the answer and optimise memory.
- Write top-down first (it transcribes the recurrence), convert to bottom-up for speed and memory.
- Rolling arrays reduce memory by a dimension; the loop direction then encodes which problem you are solving.
- To recover the solution after memory optimisation, keep a bit-level decision table or use Hirschberg's divide-and-conquer traceback.
- Eight standard shapes cover most problems; identifying the shape gives you the state.
- Pseudo-polynomial bounds like $\Theta(nW)$ are exponential in input size.
- Gradient checkpointing and pipeline partitioning are dynamic programs running inside your training stack.

Dynamic Programming II: Sequence Algorithms in Machine Learning

Edit distance, DTW, Viterbi and CTC are one algorithm in four costumes

ADVANCED

PREREQS: Chapter 18

Four algorithms that look unrelated — spell-check distance, time-series alignment, hidden Markov decoding and the loss function of a speech model — are the same dynamic program with different semirings and different allowed moves. Seeing that unity is worth more than learning them separately, and this chapter builds all four from the same template.

WHAT YOU WILL LEARN

- Implement edit distance with traceback and its linear-memory variant
- Implement dynamic time warping with a band constraint
- Implement the Viterbi algorithm and see why it is edit distance in log-space
- Understand the CTC forward-backward algorithm and what it sums over
- Recognise the max-plus and sum-product structure shared by all of them
- Apply these to real tasks: WER, alignment, sequence labelling, forced alignment

One template, four algorithms

Every algorithm in this chapter fills a table indexed by a position in each of two sequences (or by time and state), where

$$dp[i][j] = \text{bigoplus}_{\text{moves } m} (dp[\text{prev}(i,j,m)] \otimes \text{cost}(i,j,m))$$

The four differ only in three choices: the set of allowed moves, the combining operator $\oplus$ (min, max, or sum), and the accumulating operator $\otimes$ (plus, or multiply).

Algorithm	$\oplus$	$\otimes$	Moves	Answers
Edit distance	min	+	insert, delete, substitute	fewest edits
DTW	min	+	right, down, diagonal (no skips)	best alignment cost
Viterbi	max	$\times$ (or max, + in log)	state transitions	most likely path

Algorithm	$\oplus$	$\otimes$	Moves	Answers
Forward algorithm	Σ	$\times$	state transitions	total probability
CTC loss	Σ	$\times$	stay, advance, skip blank	probability of a label sequence

KEY INSIGHT | Why this matters practically

Once you see the template, implementing a new variant is a matter of changing two operators and the move set — not designing a new algorithm. It also tells you immediately what the complexity is ($\Theta(\text{states} \times \text{moves})$), how to band or prune it, and how to make it numerically stable (work in log-space, use `logsumexp` for the sum case).

Edit distance

DEFINITION | Levenshtein distance

The minimum number of single-character insertions, deletions and substitutions that transform string A into string B .

State. $dp[i][j]$ = the edit distance between $A[:i]$ and $B[:j]$.

Recurrence.

$$dp[i][j] = \min \begin{cases} dp[i-1][j] + 1 & \text{delete } A_i \\ dp[i][j-1] + 1 & \text{insert } B_j \\ dp[i-1][j-1] + [A_i \neq B_j] & \text{match or substitute} \end{cases}$$

Base cases. $dp[i][0] = i$, $dp[0][j] = j$.

		S	I	T	T	I	N	G
	0	1	2	3	4	5	6	7
K	1	1	2	3	4	5	6	7
I	2	2	1	2	3	4	5	6
T	3	3	2	1	2	3	4	5
T	4	4	3	2	1	2	3	4
E	5	5	4	3	2	2	3	4
N	6	6	5	4	3	3	2	3

d(KITTEN, SITTING) = 3

substitute K→S
substitute E→I
insert G

Highlighted cells are the traceback path — the edit script, recovered by walking backwards from the corner.

Figure 19.1 The edit-distance table for KITTEN and SITTING. Each cell is the minimum over three neighbours; the highlighted cells are the traceback path, which is the edit script itself.

Edit distance with traceback

```

1 def edit_distance(a, b):
2     n, m = len(a), len(b)
```

```

3      dp = [[0] * (m + 1) for _ in range(n + 1)]
4      for i in range(n + 1):
5          dp[i][0] = i
6      for j in range(m + 1):
7          dp[0][j] = j
8      for i in range(1, n + 1):
9          for j in range(1, m + 1):
10             sub = dp[i - 1][j - 1] + (a[i - 1] != b[j - 1])
11             dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, sub)
12
13     ops, i, j = [], n, m                                # traceback from the corner
14     while i or j:
15         if i and j and dp[i][j] == dp[i-1][j-1] + (a[i-1] != b[j-1]):
16             ops.append(("match" if a[i-1] == b[j-1] else "sub", a[i-1],
17                 b[j-1]))
18             i, j = i - 1, j - 1
19         elif i and dp[i][j] == dp[i - 1][j] + 1:
20             ops.append(("del", a[i - 1], None)); i -= 1
21         else:
22             ops.append(("ins", None, b[j - 1])); j -= 1
23     return dp[n][m], ops[::-1]

```

$\Theta(nm)$ time and memory. The memory drops to $\Theta(\min(n, m))$ with a rolling row — but then the traceback is gone, which is where Hirschberg’s algorithm (Chapter 18) earns its keep.

WHERE THIS SHOWS UP IN ML | Word error rate is edit distance

WER, the standard metric for speech recognition, is Levenshtein distance between the reference and hypothesis *word* sequences, divided by the reference length. The traceback gives the per-word alignment, which is what lets you report substitutions, insertions and deletions separately and attribute errors to specific words. The same computation with weights tuned per operation gives the alignment used for translation post-editing effort.

COMMON PITFALL | Edit distance at scale is quadratic and nobody can fix it

Under the Strong Exponential Time Hypothesis, edit distance cannot be computed in $\Theta(n^{2-\epsilon})$. So for long sequences you must approximate: band the DP (only compute cells within $|i-j| \leq w$, giving $\Theta(nw)$ and an exact answer whenever the true distance is $\leq w$), filter with a cheap bound first (length difference, or a q -gram count bound), or use bit-parallel algorithms (Myers’ algorithm computes exact Levenshtein in $\Theta(nm/64)$ using bitwise operations on machine words — a 64x constant-factor win that is very much worth having).

Dynamic time warping

DTW aligns two sequences that have the same shape but different pacing: two utterances of the same word, two sensor traces of the same gesture, two time series with different sampling.

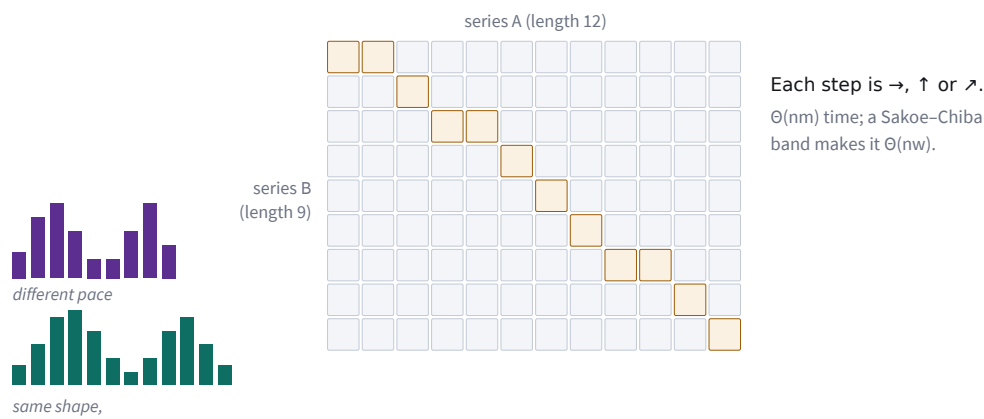


Figure 19.2 DTW's cost matrix and the warping path. Only monotone moves are allowed, which preserves ordering; the path may stretch one series against the other.

DTW with a Sakoe-Chiba band

```

1  import numpy as np
2
3  def dtw(a, b, band=None):
4      n, m = len(a), len(b)
5      band = band if band is not None else max(n, m)
6      INF = float("inf")
7      prev = np.full(m + 1, INF)
8      prev[0] = 0.0
9      for i in range(1, n + 1):
10         cur = np.full(m + 1, INF)
11         lo = max(1, i - band)
12         hi = min(m, i + band)
13         for j in range(lo, hi + 1):
14             cost = abs(a[i - 1] - b[j - 1])
15             cur[j] = cost + min(prev[j],          # insert in b
16                               cur[j - 1],        # insert in a
17                               prev[j - 1])        # align
18         prev = cur
19     return prev[m]
```

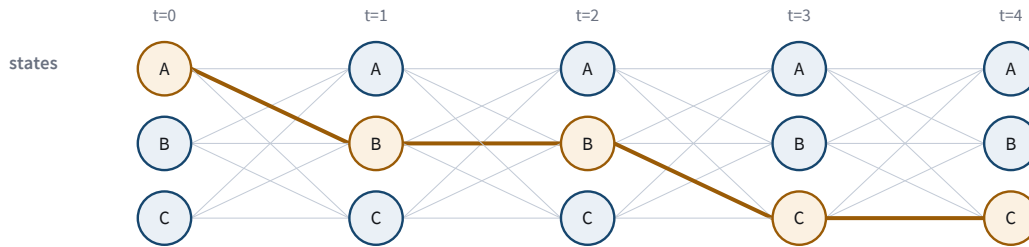
The band is not just an optimisation. Unconstrained DTW can produce pathological alignments that map one point of A onto half of B ; the Sakoe–Chiba band (allow $|i-j| \leq w$) or the Itakura parallelogram restricts the warp to something physically plausible and reduces the cost to $\Theta(nw)$.

WHERE THIS SHOWS UP IN ML | DTW in practice

DTW is the classical distance for time-series k -NN classification, and it still competes with deep models on small datasets. It is used for gesture recognition, for aligning sensor streams with different clocks, and — importantly — as **soft-DTW**, a differentiable relaxation that replaces `min` with `softmin` so it can be used as a training loss for sequence prediction. That relaxation is a change of the $\oplus$ operator in this chapter's template, nothing more.

Viterbi: the most likely state sequence

A hidden Markov model has states S , transition probabilities A_{ij} , emission probabilities $B_j(o)$ and an observation sequence $o_1 \dots o_T$. The Viterbi algorithm finds the state sequence maximising the joint probability.



At each step keep only the best path INTO each state: $\Theta(T \cdot S^2)$ instead of $\Theta(S^T)$.

Figure 19.3 The Viterbi lattice. At every time step and every state, keep only the best path into that state — $\Theta(TS^2)$ instead of S^T paths.

State. $\delta_t(j)$ = the probability of the most likely path that ends in state j at time t .

Recurrence. $\delta_t(j) = \max_i (\delta_{t-1}(i) \cdot A_{ij}) \cdot B_j(o_t)$.

Viterbi in log-space

```
1 import numpy as np
2
3 def viterbi(log_A, log_B, log_pi, obs):
4     """log_A: (S,S) log_B: (S,V) log_pi: (S,) obs: (T,) ints"""
5     T, S = len(obs), log_A.shape[0]
6     delta = log_pi + log_B[:, obs[0]]
7     back = np.zeros((T, S), dtype=np.int32)
8     for t in range(1, T):
9         scores = delta[:, None] + log_A          # (S_prev, S_next)
10        back[t] = np.argmax(scores, axis=0)
11        delta = scores.max(axis=0) + log_B[:, obs[t]]
12    path = np.zeros(T, dtype=np.int32)
13    path[-1] = int(np.argmax(delta))
14    for t in range(T - 1, 0, -1):
15        path[t - 1] = back[t, path[t]]
16    return path, float(delta.max())
```

Two things to note. Work in **log-space**: multiplying T probabilities underflows float64 after a few hundred steps. And the inner step is a matrix operation — `delta[:, None] + log_A` then a max — so it vectorises, which is why Viterbi over a 100-state model on a million frames is fast.

Replacing `max` with `logsumexp` turns Viterbi into the **forward algorithm**, which computes the total probability of the observations rather than the best path. Two operators, two algorithms.

WHERE THIS SHOWS UP IN ML | Where Viterbi runs in modern systems

Part-of-speech tagging and named-entity recognition with a CRF output layer use Viterbi for decoding: the neural network produces per-token emission scores, the CRF adds learned transition scores, and Viterbi finds the best tag sequence — which is how BIO tagging schemes are kept consistent. SentencePiece’s Unigram tokenizer runs Viterbi over a segmentation lattice. Forced alignment in speech — mapping a known transcript onto audio timestamps — is Viterbi over an HMM built from the transcript.

CTC: summing over all alignments

Connectionist Temporal Classification solves a problem that has no supervised alignment: the audio has T frames, the transcript has S characters, $T \gg S$, and nobody has labelled which frame corresponds to which character.

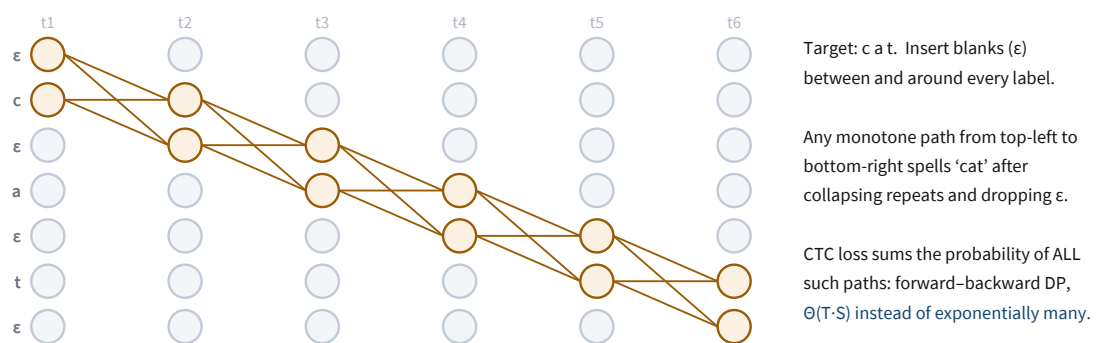


Figure 19.4 The CTC lattice for the target “cat”. Blanks are inserted between and around the labels; every monotone path through the lattice collapses to the target, and the loss is the total probability of all of them.

The construction: insert a blank symbol ϵ between every pair of labels and at both ends, giving an extended sequence of length $2S+1$. A path through the lattice is valid if it moves only forward, may stay in a state, and may skip a blank when moving between two *different* labels. The collapse rule — merge repeats, then delete blanks — maps every valid path to the target.

The CTC loss is $-\log$ of the *sum* over all valid paths, computed by the forward algorithm in $\Theta(T \cdot S)$:

$$\alpha_t(s) = (\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + [\text{skip allowed}] \alpha_{t-1}(s-2)) \cdot y_t^{(s)}$$

CTC forward pass in log-space (the core recursion)

```

1  import numpy as np
2
3  def logsumexp(*xs):
4      m = max(xs)
5      return m + np.log(sum(np.exp(x - m) for x in xs)) if m > -np.inf else
6      -np.inf
7
8  def ctc_log_prob(log_probs, target, blank=0):
9      """log_probs: (T, V) log-softmax outputs. target: list of label
10         ids."""
11      ext = [blank]
```



```

10     for lab in target:
11         ext += [lab, blank]                # eps l1 eps l2 eps ...
12         eps
13     S, T = len(ext), log_probs.shape[0]
14     NEG = -np.inf
15     a = np.full(S, NEG)
16     a[0] = log_probs[0, ext[0]]
17     if S > 1:
18         a[1] = log_probs[0, ext[1]]
19     for t in range(1, T):
20         nxt = np.full(S, NEG)
21         for s in range(S):
22             best = a[s]
23             if s > 0:
24                 best = logsumexp(best, a[s - 1])
25             if s > 1 and ext[s] != blank and ext[s] != ext[s - 2]:
26                 best = logsumexp(best, a[s - 2]) # skip the blank
27             nxt[s] = best + log_probs[t, ext[s]]
28         a = nxt
29     return logsumexp(a[S - 1], a[S - 2] if S > 1 else NEG)

```

The backward pass mirrors it, and the gradient with respect to the network outputs is $y_t^{(k)} - 1 / p(\text{ell}) \sum_{s: \text{ext}[s] = k} \alpha_t(s) \beta_t(s)$ — the difference between what the network predicted and the alignment-marginalised target. This is the same forward-backward structure as the Baum–Welch algorithm for HMMs, and the same structure as backpropagation itself (Chapter 24).

KEY INSIGHT | Why sum-over-alignments rather than best alignment

Training against the single best alignment (Viterbi training) is simpler and works less well: early in training the best alignment is essentially random, and committing to it creates a feedback loop. Summing over all alignments gives gradient signal to every plausible alignment weighted by its probability, which is much better behaved. The same argument recurs throughout ML — marginalise when you can, maximise when you must.

The complexity summary

Algorithm	Time	Memory	Banded / pruned
Edit distance	$\Theta(nm)$	$\Theta(\min(n, m))$ rolling	$\Theta(nw)$ banded; $\Theta(nm/64)$ bit-parallel
LCS	$\Theta(nm)$	$\Theta(\min(n, m))$	Hunt–Szymanski $\Theta((r+n)\log n)$
DTW	$\Theta(nm)$	$\Theta(m)$	$\Theta(nw)$ with a band; FastDTW $\Theta(n)$ approximate
Viterbi	$\Theta(T S^2)$	$\Theta(TS)$ for traceback	beam-pruned $\Theta(TbS)$
Forward-backward	$\Theta(T S^2)$	$\Theta(TS)$	same

Algorithm	Time	Memory	Banded / pruned
CTC	$\Theta(TS)$	$\Theta(TS)$	prefix beam search for decoding

Table 19.1 The same recursion with different algebra. In algebraic terms each row is the same computation over a different *semiring*; min-plus gives shortest paths, max-times gives most-likely paths, sum-times gives total probability.

EXERCISES

1. Modify `edit_distance` to support weighted operations (substitution cost depending on the character pair, as in a keyboard-distance model).
2. Implement the banded edit distance and verify that it returns the exact answer whenever the true distance is within the band, and a valid upper bound otherwise.
3. Implement Hirschberg’s algorithm for LCS and verify that its memory is $\Theta(\min(n, m))$ while its output matches the quadratic version.
4. Implement DTW with and without a band on two 10,000-point series. Compare time and the resulting alignment. Construct a case where the unbanded alignment is pathological.
5. Implement the forward algorithm by changing one operator in `viterbi`, and check that its result equals the log-sum over all paths on a tiny model by brute force.
6. Verify `ctc_log_prob` against brute-force enumeration of all valid paths for $T=6$ and target “ab”.
7. Implement CTC greedy decoding (argmax per frame, then collapse) and CTC prefix beam search. Compare their outputs on a synthetic model.
8. Express the LCS recurrence in the semiring notation at the top of this chapter, naming $\oplus$, $\otimes$ and the move set.

CHAPTER RECAP

- Edit distance, DTW, Viterbi, the forward algorithm and CTC are the same dynamic program with different semirings and move sets.
- Edit distance is $\Theta(nm)$ and provably cannot be much better; band it, filter first, or use bit-parallel Myers for a 64x constant.
- DTW aligns sequences with different pacing; a band is required for sane alignments as well as for speed. Soft-DTW makes it differentiable by replacing min with softmin.
- Viterbi keeps the best path into each state: $\Theta(TS^2)$. Work in log-space, and vectorise the inner step. Replacing max with logsumexp gives the forward algorithm.
- CTC sums over all monotone alignments of a blank-extended label sequence, which is why it trains without frame-level labels. Its gradient is the difference between prediction and alignment-marginalised target.
- Marginalise over alignments rather than committing to the best one when you can; it is better behaved during training.

Backtracking and Combinatorial Search

Exhaustive search that is not actually exhaustive

CORE

PREREQS: Chapters 8, 18

Some problems have no structure to exploit: no greedy rule, no overlapping subproblems, no polynomial algorithm at all. For those, the honest approach is to search — but to search intelligently, cutting off whole regions of the space as soon as they are provably hopeless. This chapter builds the template, then shows the pruning techniques that make the difference between seconds and centuries.

WHAT YOU WILL LEARN

- Write the choose / explore / un-choose template correctly
- Prune with feasibility checks, bounds and symmetry breaking
- Implement permutations, subsets, N -queens, Sudoku and graph colouring
- Understand branch and bound, and the role of a good heuristic ordering
- Distinguish backtracking from DP and from beam search
- Recognise where constrained search appears in ML systems

The template

Choose, explore, un-choose

```

1  def backtrack(state, partial, results):
2      if is_complete(partial):
3          results.append(snapshot(partial))      # copy! see the pitfall
4          return
5      for choice in candidates(state, partial):
6          if not feasible(partial, choice):
7              continue                          # prune
8          apply(partial, choice)                 # choose
9          backtrack(state, partial, results)     # explore
10         undo(partial, choice)                  # un-choose

```

The `undo` is the defining feature. Instead of copying the state at every node — which would cost $\Theta(n)$ per node and dominate everything — you mutate one shared state and

restore it on the way out. The saving is the difference between $\Theta(n \cdot \text{nodes})$ and $\Theta(\text{nodes})$.

COMMON PITFALL | Forgetting to copy at the leaf

`results.append(partial)` appends a *reference* to the mutable state, which will be undone on the way back up. Every entry in `results` then ends up identical (usually empty). Use `partial[:]`, `list(partial)`, or `tuple(partial)`. This is the single most common backtracking bug.

Permutations and subsets, the two skeletons

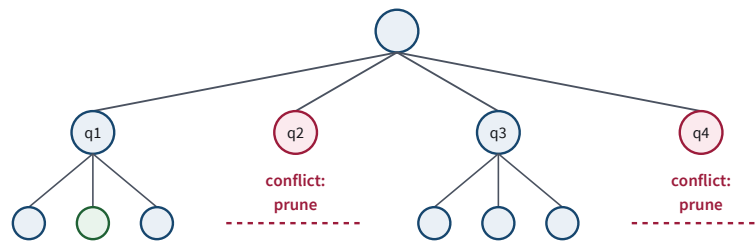
```

1  def permutations(nums):
2      out, used, cur = [], [False] * len(nums), []
3      def go():
4          if len(cur) == len(nums):
5              out.append(cur[:])          # copy
6              return
7          for i, x in enumerate(nums):
8              if used[i]:
9                  continue
10             used[i] = True; cur.append(x)
11             go()
12             cur.pop(); used[i] = False
13      go()
14      return out                          # theta(n * n!)
15
16  def subsets(nums):
17      out, cur = [], []
18      def go(i):
19          if i == len(nums):
20              out.append(cur[:])
21              return
22          go(i + 1)                        # exclude nums[i]
23          cur.append(nums[i]); go(i + 1); cur.pop()  # include nums[i]
24      go(0)
25      return out                          # theta(n * 2^n)

```

Pruning is the algorithm

Unpruned, N -queens for $N=8$ examines $8^8=16.7$ million placements. With three simple prunes it examines about two thousand nodes — a factor of eight thousand, from four lines of code.



Two of four subtrees are cut at depth 1. For 8 queens this turns 16.7 million placements into about 2,000 explored nodes.

Figure 20.1 Pruning cuts entire subtrees. Every node you never visit is an entire branch of the search space eliminated, so the savings compound multiplicatively with depth.

```

N-queens with O(1) conflict checks
1  def n_queens(n):
2      cols = [False] * n
3      diag1 = [False] * (2 * n)          # r + c is constant on this
4          diagonal
5      diag2 = [False] * (2 * n)          # r - c + n is constant on this one
6      placement, solutions = [], []
7
8      def go(r):
9          if r == n:
10             solutions.append(placement[:])
11             return
12         for c in range(n):
13             d1, d2 = r + c, r - c + n
14             if cols[c] or diag1[d1] or diag2[d2]:
15                 continue                # prune in theta(1)
16             cols[c] = diag1[d1] = diag2[d2] = True
17             placement.append(c)
18             go(r + 1)
19             placement.pop()
20             cols[c] = diag1[d1] = diag2[d2] = False
21
22     go(0)
23     return solutions

```

The three boolean arrays make the feasibility test $\Theta(1)$ rather than $\Theta(r)$. That is the general lesson: **maintain enough incremental state that pruning is cheap**, exactly as in the sliding window of Chapter 11.

The four pruning techniques

1. Feasibility pruning. Reject a partial solution that already violates a constraint. The earlier you detect it, the more you save.

2. Bound pruning (branch and bound). Compute an optimistic bound on the best completion of the current partial solution. If the bound is no better than the best solution found so far, abandon the branch. This turns exhaustive search into something usable for optimisation problems.

3. Symmetry breaking. If two branches produce equivalent solutions, explore only one. In N -queens, restricting the first queen to the left half halves the work. In subset problems, requiring indices to increase avoids generating each subset $k!$ times. In graph colouring, fixing the colour of the first vertex removes a factor of $k!$.

4. Ordering heuristics. Choose the *most constrained variable* next (fewest remaining legal values) and try the *least constraining value* first. For Sudoku this alone is the difference between minutes and milliseconds, because it forces contradictions to surface immediately.

Sudoku with most-constrained-variable ordering

```

1  def solve_sudoku(grid):
2      """grid: 9x9 list of ints, 0 = empty. Solves in place."""
3      def candidates(r, c):
4          used = set(grid[r]) | {grid[i][c] for i in range(9)}
5          br, bc = 3 * (r // 3), 3 * (c // 3)
6          used |= {grid[br + i][bc + j] for i in range(3) for j in range(3)}
7          return [v for v in range(1, 10) if v not in used]
8
9      best, best_opts = None, None
10     for r in range(9):
11         for c in range(9):
12             if grid[r][c] == 0:
13                 opts = candidates(r, c)
14                 if not opts:
15                     return False                # dead end, prune now
16                 if best_opts is None or len(opts) < len(best_opts):
17                     best, best_opts = (r, c), opts
18                     if len(opts) == 1:
19                         break                    # cannot do better
20     if best is None:
21         return True                            # no empty cells:
22         solved
23     r, c = best
24     for v in best_opts:
25         grid[r][c] = v
26         if solve_sudoku(grid):
27             return True
28         grid[r][c] = 0                        # un-choose
29     return False

```

KEY INSIGHT | Branch and bound in one sentence

Keep the best complete solution found so far; prune any branch whose optimistic bound cannot beat it. Everything else — the choice of bound, the exploration order, the data structures — is engineering around that one idea, and it is how integer programming solvers, TSP solvers and alpha-beta game search all work.

Backtracking versus its neighbours

Technique	Explores	Memory	Optimal?	Use when
Backtracking	a tree, with undo	$\Theta(\text{depth})$	yes, if exhaustive	no structure to exploit
Branch and bound	a pruned tree	$\Theta(\text{depth})$	yes	optimisation with a good bound
Dynamic programming	a DAG of states	$\Theta(\text{states})$	yes	subproblems overlap
Greedy	one path	$\Theta(1)$	only with proof	exchange argument holds
Beam search	b paths	$\Theta(b)$	no	state space too large, approximate is fine
MCTS	a sampled tree	$\Theta(\text{nodes kept})$	asymptotically	huge branching, simulation available

WHERE THIS SHOWS UP IN ML | Search in machine learning systems

Beam search (Chapter 35) is backtracking with a hard memory cap and no undo: at each depth, keep the b best partial sequences and discard the rest. It is not optimal and cannot be made so, but the failure mode is benign.

Constrained decoding is backtracking's feasibility pruning applied to token generation: a grammar or regex automaton says which tokens are legal in the current state, and the rest are masked out of the softmax. Unlike backtracking there is no undo — the model cannot take back an emitted token — so the automaton must guarantee that every legal prefix can be completed. That guarantee is what distinguishes a correct grammar-constrained sampler from one that paints itself into a corner.

Neural architecture search and hyperparameter search are branch and bound with a learned or statistical bound; successive halving and Hyperband are precisely “prune the branches whose optimistic bound is poor, cheaply”.

Alpha-beta pruning in game-playing systems is branch and bound with two bounds, and MCTS replaces the bound with a statistical estimate from rollouts.

COMMON PITFALL | Backtracking on an exponential space is still exponential

Pruning changes the constant and often the effective base, not the complexity class. N -queens is fine to $N \approx 30$; TSP by branch and bound is fine to perhaps 50 cities with a good bound; Boolean satisfiability solvers routinely handle a million variables *because industrial instances have structure*, not because the algorithm is polynomial. Know which regime you are in before promising a runtime.

EXERCISES

1. Implement `combinations(n, k)` by backtracking with index-increasing symmetry breaking. Verify the count is $\binom{n}{k}$.
2. Instrument `n_queens` to count visited nodes with and without the diagonal pruning, for $N=6, 8, 10$. Plot the ratio.
3. Add symmetry breaking to `n_queens` (restrict the first row to the left half, handling odd N) and measure the saving.
4. Implement the word-break problem two ways: backtracking with memoisation, and bottom-up DP. Explain when each is preferable.
5. Implement branch and bound for the 0/1 knapsack using the fractional relaxation as the upper bound (Chapter 17). Compare visited nodes to the DP's table size for $n=40$ and $W=10^6$.
6. Implement graph colouring with most-constrained-variable ordering and test it on random graphs of increasing density. Find the density at which it becomes intractable.
7. Write a constrained decoder that generates only strings matching a given regular expression, by compiling the regex to a DFA and masking logits. What must be true of the DFA for the decoder never to get stuck?

CHAPTER RECAP

- The template is choose / explore / un-choose; mutate shared state and restore it rather than copying, and copy only at the leaf.
- Pruning is the algorithm, not an optimisation: feasibility checks, bounds, symmetry breaking and ordering heuristics each cut whole subtrees, and their effects compound with depth.
- Maintain incremental state so that feasibility tests are $\Theta(1)$.
- Branch and bound keeps the best solution so far and prunes any branch whose optimistic bound cannot beat it.
- Most-constrained-variable ordering surfaces contradictions early and is often the single largest win.
- Beam search, constrained decoding, alpha-beta and MCTS are all the same tree explored under different budgets.
- Pruning improves the base of the exponential, not the complexity class.

Randomised Algorithms and Reservoir Sampling

When a coin flip is a better guarantee than a clever rule

CORE

PREREQS: Chapters 9, 17

Randomness in an algorithm is not a compromise. It buys simplicity, it defeats adversarial inputs, and for some problems it provides guarantees no deterministic algorithm can match at the same cost. This chapter covers the two kinds of randomised algorithm, the probabilistic tools you need to analyse them, and the sampling algorithms that every training pipeline depends on.

WHAT YOU WILL LEARN

- Distinguish Las Vegas from Monte Carlo algorithms and know when each is appropriate
- Analyse a randomised algorithm using linearity of expectation and concentration bounds
- Implement reservoir sampling, including the weighted and k -item variants
- Understand random projection and the Johnson–Lindenstrauss lemma
- Implement correct shuffling, stratified sampling and deterministic seeding
- Recognise the reproducibility traps randomness introduces in ML pipelines

Two kinds of randomised algorithm

DEFINITION | Las Vegas and Monte Carlo

A **Las Vegas** algorithm is always correct; its *running time* is a random variable.

Randomised quicksort is the canonical example: the answer is always a sorted array, and only the time varies.

A **Monte Carlo** algorithm always runs in the stated time; its *answer* may be wrong with bounded probability. A Bloom filter is the canonical example: it answers in $\Theta(k)$ always, and sometimes says “present” when the item is absent.

The choice is a design decision with real consequences. A Las Vegas algorithm inside a latency budget can blow the budget; a Monte Carlo one cannot, but you must be able to tolerate — or detect — a wrong answer. Most systems pick Monte Carlo with a failure probability driven below the hardware error rate, which is the engineering answer.

KEY INSIGHT | Why randomness beats cleverness against an adversary

A deterministic algorithm has a worst case, and if the input is chosen by something that knows your code — a user, an attacker, or simply a production data distribution you did not anticipate — you will meet it. Randomising makes the worst case depend on *your* coin flips rather than on the input, so no input is reliably bad. This is why quicksort randomises its pivot, why Python salts string hashes, and why load balancers randomise.

The analysis toolkit

Linearity of expectation. $\mathbb{E}[X+Y]=\mathbb{E}[X]+\mathbb{E}[Y]$ even when X and Y are dependent. This is the workhorse: decompose a complicated count into indicator variables and sum their probabilities.

THE MATH | Randomised quicksort in three lines

Let $X_{ij}=1$ if elements of rank i and j are ever compared. They are compared exactly when the first pivot chosen from the range of ranks $[i,j]$ is i or j itself — otherwise they are separated. That range has $j-i+1$ elements, so $\Pr[X_{ij}=1]=2/(j-i+1)$. Hence

$$\mathbb{E}[\text{comparisons}] = \sum_{i < j} 2/(j-i+1) = 2nH_n + O(n) \approx 1.39n \log_2 n$$

No recursion tree, no case analysis. This is the standard example of why linearity of expectation is worth learning properly.

Concentration. Expectation alone is not a guarantee. Markov, Chebyshev and Chernoff bounds say how unlikely large deviations are.

Bound	Requires	Gives
Markov	$X \geq 0$	$\Pr[X \geq a] \leq \mathbb{E}[X]/a$
Chebyshev	finite variance	$\Pr[X - \mathbb{E}[X] \geq a] \leq \text{var}[X]/a^2$
Chernoff / Hoeffding	sum of bounded independent variables	$\Pr[X - \mathbb{E}[X] \geq a] \leq 2e^{-a^2/n}$

Probability amplification. If a Monte Carlo algorithm is wrong with probability $p < 1/2$, running it k times independently and taking the majority reduces the error probability exponentially in k . This is why a one-sided-error test with $p=1/4$ becomes a certainty for practical purposes after 40 repetitions ($p < 10^{-24}$).

Reservoir sampling

The problem: draw a uniform sample of k items from a stream whose length you do not know in advance, in one pass, using $\Theta(k)$ memory.

STREAM OF UNKNOWN LENGTH

RESERVOIR ($k = 4$)

item i ($i > k$) replaces a random slot with probability k/i
 every item ends up in the reservoir with probability exactly k/n

One pass, $\Theta(k)$ memory, no need to know n in advance — the right tool for sampling a dataset you are still writing.

Figure 21.1 Reservoir sampling. The first k items fill the reservoir; item $i > k$ replaces a uniformly chosen slot with probability k/i . Every item ends up in the sample with probability exactly k/n .

Reservoir sampling, uniform k of n

```

1  import random
2
3  def reservoir(stream, k, rng=random):
4      res = []
5      for i, x in enumerate(stream, start=1):
6          if i <= k:
7              res.append(x)
8          else:
9              j = rng.randrange(i)          # uniform in [0, i)
10             if j < k:
11                 res[j] = x                # probability k/i
12     return res

```

PROOF | Every item is retained with probability exactly k/n

By induction on the stream length n . For $n=k$ the claim is trivial. Assume it holds after $n-1$ items. Item n is kept with probability k/n directly. An earlier item was in the reservoir with probability $k/(n-1)$ and survives item n unless it is the one evicted, which happens with probability $k/n \cdot 1/k = 1/n$. So it survives with probability $k/n - 1 \cdot (1-1/n) = k/n - 1 \cdot (n-1)/n = k/n$.

Weighted reservoir sampling (A-Res). For sampling without replacement proportional to weights w_i , assign each item the key u_i^{1/w_i} with u_i uniform in $(0,1)$, and keep the k largest keys in a min-heap.

Weighted reservoir sampling with a heap

```

1  import heapq, random
2
3  def weighted_reservoir(stream, k, rng=random):
4      heap = []                                # min-heap of (key, item)
5      for item, w in stream:
6          if w <= 0:
7              continue
8          key = rng.random() ** (1.0 / w)      # exponential-race trick
9          if len(heap) < k:
10             heapq.heappush(heap, (key, item))

```

```

11         elif key > heap[0][0]:
12             heapq.heapreplace(heap, (key, item))
13     return [item for _, item in heap]

```

The identity that makes this work: $\Pr[u^{1/w} > t] = 1 - t^w$, so larger weights produce larger keys in exactly the right proportion. This is the same “exponential race” argument behind the Gumbel-max trick (Chapter 34).

WHERE THIS SHOWS UP IN ML | Where reservoir sampling actually runs

Building a held-out evaluation set from a data stream you are still writing; sampling training examples for human review without a second pass; sampling log lines for monitoring at a fixed rate regardless of traffic; drawing a random subset of a dataset too large to shuffle. In each case the property that matters is *one pass, bounded memory, no need to know n* .

Shuffling, correctly

Fisher-Yates: the only correct in-place shuffle

```

1  import random
2
3  def shuffle(a, rng=random):
4      for i in range(len(a) - 1, 0, -1):
5          j = rng.randint(0, i)           # INCLUSIVE of i
6          a[i], a[j] = a[j], a[i]
7      return a

```

Two classic ways to get this wrong. Choosing j from the whole array (`randint(0, n-1)`) produces n^n equally likely execution paths mapping onto $n!$ permutations, and since $n! \ll n^n$ the distribution cannot be uniform — a bias large enough to detect with a few thousand trials on $n=4$. And `sorted(a, key=lambda _: random.random())` is uniform but $\Theta(n \log n)$ and allocates; it was also the source of a famous browser-ballot bias when implemented with a random *comparator* instead of a random key.

COMMON PITFALL | Shuffling a dataset larger than memory

Fisher–Yates needs random access. For a dataset on disk you cannot shuffle in place. The standard approaches: (1) *shard shuffle* — shuffle the file order, then shuffle a buffer of B examples in memory (what `tf.data.shuffle(B)` and `WebDataset` do); the randomness is only as good as B , and a buffer smaller than one class’s run length leaves correlated batches. (2) *Two-pass shuffle* — scatter records into m random files, then shuffle each file in memory; this is a true uniform shuffle and costs two passes. (3) Pre-shuffle once at write time and read sequentially with a random offset each epoch.

Random projection

THEOREM | Johnson–Lindenstrauss

For any $0 < \epsilon < 1$ and any set of n points in $\mathbb{R}^d$, there is a linear map $f: \mathbb{R}^d \rightarrow \mathbb{R}^m$ with $m = O(\epsilon^{-2} \log n)$ such that for all pairs u, v ,

$$(1 - \epsilon)|u - v|^2 \leq |f(u) - f(v)|^2 \leq (1 + \epsilon)|u - v|^2$$

Moreover, a matrix with i.i.d. Gaussian entries works with high probability.

Read the bound carefully: m depends on $\log n$ and **not at all on d** . You can project a million 4096-dimensional vectors into about 500 dimensions and preserve all pairwise distances to within 10%. The proof is a Chernoff bound on the squared length of a projected vector plus a union bound over $(n/2)$ pairs.

Random projection for cheap distance-preserving compression

```

1  import numpy as np
2
3  def project(X, m, seed=0):
4      rng = np.random.default_rng(seed)
5      d = X.shape[1]
6      R = rng.standard_normal((d, m)).astype(np.float32) / np.sqrt(m)
7      return X @ R                                # (n, m), distances preserved

```

WHERE THIS SHOWS UP IN ML | Random projection in the retrieval stack

It is the theoretical basis for LSH (Chapter 29), it is used to shrink embeddings before building an index when memory is tight, and the sparse variants (Achlioptas: entries in $\{-1, 0, +1\}$ with probability $\{1/6, 2/3, 1/6\}$) make the projection itself cheap. It also explains why reducing an embedding from 1536 to 384 dimensions often costs surprisingly little recall — the information you need for *relative* distances survives projection, even though the individual coordinates do not.

Reproducibility

Randomness that you cannot reproduce is a debugging catastrophe. Four rules.

1. **Seed explicitly, and seed everything:** Python's `random`, NumPy, the framework, and each dataloader worker (which by default gets a different seed derived from the base seed — verify what yours does).
2. **Prefer generator objects to global state.** `np.random.default_rng(seed)` passed explicitly makes the dependency visible; the global `np.random` is shared mutable state and breaks under concurrency.
3. **Derive per-item seeds from content, not from position.** Seeding an augmentation with `hash(example_id) ^ epoch_seed` makes it reproducible regardless of sharding, worker count or ordering. Seeding with the batch index does not.
4. **Know what is still nondeterministic.** Floating-point reduction order on a GPU, atomic scatter-adds, and non-deterministic cuDNN kernels all produce run-to-run differences that no seed controls. If you need bit-exact reproducibility, you must also request deterministic kernels and accept the slowdown.

EXERCISES

1. Implement the biased shuffle (`randint(0, n-1)`) and the correct one. For $n=4$, run 10^6 trials of each and plot the frequency of all 24 permutations. Quantify the bias with a chi-squared test.
2. Verify reservoir sampling empirically: run it 10^5 times on a stream of 100 items with $k=10$ and check that each item appears close to 10% of the time.
3. Implement weighted reservoir sampling and verify that the inclusion probability of each item is proportional to its weight.
4. Use linearity of expectation to compute the expected number of fixed points of a random permutation. (The answer is 1, for every n .)
5. Empirically verify Johnson–Lindenstrauss: project 10^4 random 1000-dimensional points to $m \in \{50, 100, 200, 500\}$ and plot the distribution of relative distance distortion. Compare to the bound.
6. Implement a two-pass shuffle of a 100 GB file using m intermediate shards. How large must m be for each shard to fit in 8 GB, and what is the total I/O?
7. Show that `tf.data.shuffle(buffer_size=B)` on a dataset sorted by class produces correlated batches when B is smaller than a class run. Quantify the correlation and propose a fix.

CHAPTER RECAP

- Las Vegas algorithms are always correct with variable time; Monte Carlo always finish on time but may be wrong with bounded probability.
- Randomisation defeats adversarial inputs by moving the worst case from the input to your own coin flips.
- Linearity of expectation decomposes complicated counts into indicators; Chernoff bounds turn expectations into guarantees and justify repetition.
- Reservoir sampling gives a uniform k -sample in one pass with $\Theta(k)$ memory and no knowledge of n ; the weighted version uses the key $u^{1/w}$ and a min-heap.
- Fisher–Yates is the only correct in-place shuffle; the off-by-one variant is measurably biased. Shuffling out-of-core needs a buffer shuffle or a two-pass scatter.
- Johnson–Lindenstrauss: $O(\epsilon^{-2} \log n)$ dimensions preserve all pairwise distances, independent of the original dimension.
- Seed explicitly, pass generators rather than using global state, derive per-item seeds from content, and know what remains nondeterministic on accelerators.

PART V

Graphs

A graph is what you get when relationships matter more than order. Computation graphs, knowledge graphs, retrieval graphs, and the scheduler that runs your training job are all graphs.

-
- 22 Graph Representations and Traversal
 - 23 Shortest Paths, Minimum Spanning Trees and A
 - 24 DAGs: Topological Order, Autograd and Computation Graphs
 - 25 Flows, Matchings and Assignment
 - 26 Graphs in Machine Learning

Graph Representations and Traversal

The structure that appears once relationships matter more than order

CORE

PREREQS: Chapters 6, 13

A graph is the most general of the structures in this book: arrays, trees and lists are all special cases. This chapter covers how to store one, how to walk one, and the handful of traversal variants that solve a remarkable share of real problems — including several you have already run today without noticing.

WHAT YOU WILL LEARN

- Choose between adjacency matrix, adjacency list and CSR for a given workload
- Implement BFS and DFS iteratively and know exactly what each one computes
- Find connected components, detect cycles and test bipartiteness
- Understand CSR as the format that makes graphs work on GPUs
- Recognise the graph inside a computation graph, a dependency tree and a retrieval index
- Estimate memory for a graph with a billion edges

Definitions, quickly

A graph $G=(V, E)$ is a set of vertices and a set of edges. Edges may be **directed** or **undirected**, **weighted** or not. A graph is **sparse** when $|E|=O(|V|)$ and **dense** when $|E|=\Theta(|V|^2)$; almost every real graph is sparse, and almost every asymptotic decision follows from that.

Term	Meaning	Where it matters
Degree	edges incident to a vertex	power-law degrees break load balancing
Path	sequence of adjacent vertices	routing, reachability
Cycle	path returning to its start	deadlock, invalid dependency
Connected component	maximal mutually reachable set	clustering, deduplication
DAG	directed, acyclic	schedules, autograd, build systems

Term	Meaning	Where it matters
Bipartite	vertices split so all edges cross	user–item, matching

Three representations

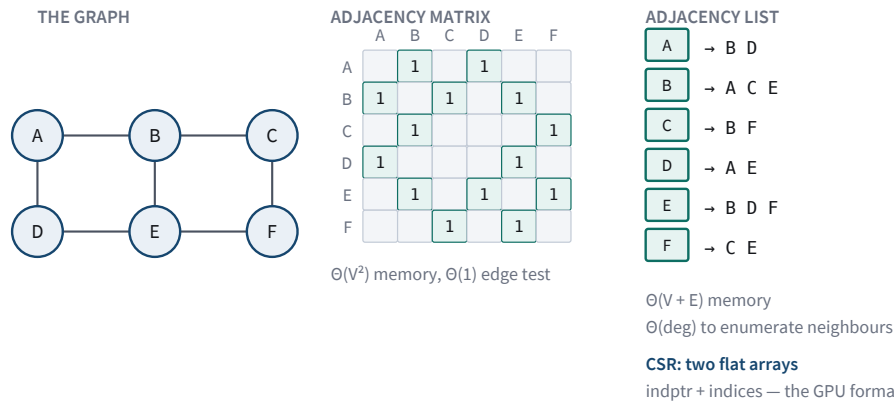


Figure 22.1 The same graph three ways. The matrix answers “is there an edge?” in $\Theta(1)$ but costs $\Theta(V^2)$ memory; the list costs $\Theta(V + E)$ but must scan to answer the same question; CSR is the list flattened into two arrays.

Representation	Memory	Edge test	Enumerate neighbours	Add edge
Adjacency matrix	$\Theta(V^2)$	$\Theta(1)$	$\Theta(V)$	$\Theta(1)$
Adjacency list	$\Theta(V + E)$	$\Theta(\deg)$	$\Theta(\deg)$	$\Theta(1)$
Adjacency set	$\Theta(V + E)$	$\Theta(1)$	$\Theta(\deg)$	$\Theta(1)$
CSR (compressed sparse row)	$\Theta(V + E)$, smallest constant	$\Theta(\log \deg)$	$\Theta(\deg)$, contiguous	rebuild
Edge list	$\Theta(E)$	$\Theta(E)$	$\Theta(E)$	$\Theta(1)$

Table 22.1 The vocabulary you need. Everything else in Part V is built from these six ideas.

KEY INSIGHT | CSR is the format that matters at scale

CSR stores a graph as two flat integer arrays: `indptr` of length $V + 1$ and `indices` of length E . The neighbours of vertex v are `indices[indptr[v]:indptr[v+1]]` — a contiguous slice. That is one sequential read per vertex, no pointer chasing, trivially memory-mappable, and directly usable by a GPU kernel. It is `scipy.sparse.csr_matrix`, it is PyTorch Geometric’s edge storage, and it is the layout of every serious graph engine. It is also exactly the ragged layout from Chapter 4.

Building CSR from an edge list

```

1  import numpy as np
2
3  def build_csr(n, edges, undirected=True):
4      src = np.asarray([u for u, v in edges], dtype=np.int64)
5      dst = np.asarray([v for u, v in edges], dtype=np.int64)
6      if undirected:
7          src, dst = np.concatenate([src, dst]), np.concatenate([dst, src])
8      order = np.argsort(src, kind="stable")  # group by source

```

```

8      vertex
9      src, dst = src[order], dst[order]
10     indptr = np.zeros(n + 1, dtype=np.int64)
11     np.add.at(indptr, src + 1, 1)           # degree of each
12     vertex
13     np.cumsum(indptr, out=indptr)           # prefix sum ->
14     offsets
15     return indptr, dst
16
17 def neighbours(indptr, indices, v):
18     return indices[indptr[v]:indptr[v + 1]] # contiguous, no copy

```

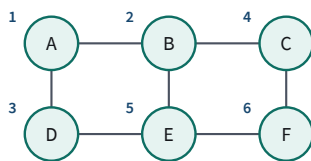
PERFORMANCE NOTE | The memory arithmetic for a billion-edge graph

CSR with int32 indices: $4(V+E)$ bytes. A graph with 10^8 vertices and 10^9 edges needs about 4.4 GB — it fits on one machine. The same graph as a Python dict of lists needs roughly $10^9 \times 40$ bytes for the list elements alone, plus per-vertex dict and list overhead: over 60 GB. The representation choice is the difference between one machine and a cluster.

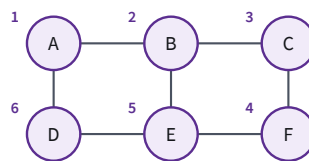
The universal traversal

BFS and DFS differ in one line. Writing them as the same function makes the relationship impossible to forget.

BFS — queue



DFS — stack



frontier.popleft() → shortest paths in an unweighted graph. *frontier.pop()* → cycle detection, topological order.

Figure 22.2 The same graph traversed two ways. Neither order is “better”; they compute different things.

One traversal skeleton

```

1  from collections import deque
2
3  def traverse(indptr, indices, start, mode="bfs"):
4      n = len(indptr) - 1
5      seen = bytearray(n)
6      parent = [-1] * n
7      order = []
8      frontier = deque([start])
9      seen[start] = 1
10     while frontier:
11         v = frontier.popleft() if mode == "bfs" else frontier.pop()
12         order.append(v)
13         for u in indices[indptr[v]:indptr[v + 1]]:
14             if not seen[u]:
15                 seen[u] = 1

```

```

16         parent[u] = v
17         frontier.append(u)
18     return order, parent

```

Both are $\Theta(V+E)$: every vertex is enqueued once and every edge examined once (twice for undirected graphs).

COMMON PITFALL | Mark when you enqueue, not when you dequeue

If you set `seen[u] = 1` only when you pop u , a vertex reachable by several edges gets pushed multiple times. On a dense graph that turns $\Theta(V+E)$ into $\Theta(V \cdot E)$ and can exhaust memory. Mark on push. (The one exception is Dijkstra with lazy deletion, Chapter 23, where you *must* allow duplicate pushes because distances improve.)

What BFS gives you

BFS visits vertices in non-decreasing distance from the source, so the `parent` array is a **shortest-path tree** for an unweighted graph. That single fact solves: minimum number of moves in a puzzle, degrees of separation, shortest word ladder, minimum hops in a network, and the “level” of each node in a dependency graph.

Two refinements are worth knowing. **Multi-source BFS** — seed the queue with every source at distance 0 — computes, for every vertex, the distance to the *nearest* source, in the same $\Theta(V+E)$. This is how you compute “distance to the nearest labelled node” for semi-supervised learning on a graph. **Bidirectional BFS** — search forward from the source and backward from the target, stopping when the frontiers meet — reduces the explored set from b^d to $2b^{d/2}$, which for a social graph with $b \approx 200$ and $d=6$ is a factor of 10^6 .

What DFS gives you

DFS is about *structure*, not distance. Its discovery and finish times expose:

- **Cycle detection.** In a directed graph, a back edge (to a vertex currently on the stack) means a cycle. This is how a build system detects a circular dependency and how an autograd engine detects an illegal in-place operation.
- **Topological order.** Reverse finishing order of a DFS on a DAG (Chapter 24).
- **Strongly connected components.** Tarjan’s and Kosaraju’s algorithms are DFS with extra bookkeeping.
- **Bridges and articulation points.** Edges and vertices whose removal disconnects the graph — single points of failure in a network.

Cycle detection in a directed graph, iteratively

```

1  WHITE, GREY, BLACK = 0, 1, 2
2
3  def has_cycle(indptr, indices):
4      n = len(indptr) - 1
5      colour = bytearray(n)
6      for s in range(n):
7          if colour[s] != WHITE:
8              continue

```

```

9      stack = [(s, indptr[s])]
10     colour[s] = GREY
11     while stack:
12         v, i = stack[-1]
13         if i < indptr[v + 1]:
14             stack[-1] = (v, i + 1)
15             u = indices[i]
16             if colour[u] == GREY:
17                 return True                # back edge: cycle
18             if colour[u] == WHITE:
19                 colour[u] = GREY
20                 stack.append((u, indptr[u]))
21         else:
22             colour[v] = BLACK                # finished
23             stack.pop()
24     return False

```

The three colours are the standard idiom: white = unvisited, grey = on the current path, black = fully explored. Grey is what distinguishes a back edge (a cycle) from a cross edge (not a cycle), and forgetting it is the classic bug.

Connected components and bipartiteness

Components by repeated BFS, and a 2-colouring test

```

1  def components(indptr, indices):
2      n = len(indptr) - 1
3      comp = [-1] * n
4      k = 0
5      for s in range(n):
6          if comp[s] != -1:
7              continue
8          stack = [s]
9          comp[s] = k
10         while stack:
11             v = stack.pop()
12             for u in indices[indptr[v]:indptr[v + 1]]:
13                 if comp[u] == -1:
14                     comp[u] = k
15                     stack.append(u)
16         k += 1
17     return comp, k
18
19 def is_bipartite(indptr, indices):
20     n = len(indptr) - 1
21     colour = [-1] * n
22     for s in range(n):
23         if colour[s] != -1:
24             continue
25         colour[s] = 0
26         stack = [s]
27         while stack:
28             v = stack.pop()
29             for u in indices[indptr[v]:indptr[v + 1]]:

```

```

30         if colour[u] == -1:
31             colour[u] = 1 - colour[v]
32             stack.append(u)
33         elif colour[u] == colour[v]:
34             return False          # odd cycle
35     return True

```

A graph is bipartite exactly when it has no odd cycle, and the 2-colouring is the proof. Bipartite structure is what makes user–item recommendation graphs, document–term matrices and bipartite matching (Chapter 25) tractable.

WHERE THIS SHOWS UP IN ML | Graphs you are already using

The computation graph. Every framework builds a DAG of operations; forward is a topological traversal and backward is its reverse (Chapter 24).

The dependency graph of a pipeline. Airflow, Dagster, `make`, Bazel: all topological sorts with cycle detection.

The retrieval graph. HNSW is a navigable small-world graph and its search is a greedy best-first traversal — literally the skeleton above with a priority queue (Chapter 33).

The user–item graph. Collaborative filtering is a bipartite graph; random walks on it (PageRank-style) are a strong baseline recommender (Chapter 26).

The knowledge graph. Multi-hop retrieval is bounded-depth BFS from the entities mentioned in a query.

The attention graph. Sparse attention patterns are adjacency structures; sliding-window attention is a band matrix, and the “graph” framing is how BigBird’s random+window+global pattern was designed and analysed.

COMMON PITFALL | Power-law degree distributions break everything

Real graphs are not regular. In a social or web graph the top 0.01% of vertices have millions of neighbours while the median has three. This breaks naive parallel BFS (one thread gets the celebrity vertex and everyone waits), breaks vertex-partitioned distributed graphs (one machine holds the hub), and breaks GNN mini-batching (one sampled neighbourhood explodes). The standard mitigations are edge-based rather than vertex-based partitioning, degree capping, and neighbour sampling (Chapter 26).

EXERCISES

1. Build CSR for a graph with 10^6 vertices and 10^7 edges and measure the memory against a `dict` of `list`s. Then measure BFS time for both.
2. Implement multi-source BFS and use it to compute, for each vertex, the distance to the nearest of 100 labelled vertices.
3. Implement bidirectional BFS and compare the number of vertices explored to ordinary BFS on a random graph with average degree 10 and 100,000 vertices.
4. Demonstrate the “mark on pop” bug: construct a graph where it causes the queue to grow quadratically.
5. Implement Tarjan’s strongly connected components algorithm iteratively and use it to find cycles in a package dependency graph.
6. Find all bridges of a graph in $\Theta(V+E)$ using DFS low-link values, and interpret the result for a network reliability question.
7. Given a power-law graph, measure the imbalance of a vertex-partitioned BFS across 8 workers. Then implement degree capping and measure again.

CHAPTER RECAP

- Real graphs are sparse, so $\Theta(V+E)$ representations dominate. CSR — `indptr` plus `indices` — is the layout for large static graphs and the one GPUs use.
- BFS and DFS are the same skeleton with a queue or a stack. Both are $\Theta(V+E)$; mark vertices when you push, not when you pop.
- BFS gives shortest paths in unweighted graphs; multi-source BFS gives distance to the nearest source; bidirectional BFS squares the reachable depth.
- DFS gives structure: cycles via grey back edges, topological order, strongly connected components, bridges and articulation points.
- Connected components and bipartiteness are two-line variations on the traversal.
- Computation graphs, pipeline DAGs, HNSW indexes, user-item graphs and sparse attention patterns are all graphs you already use.
- Power-law degree distributions break naive partitioning and sampling; cap degrees or partition by edges.

Shortest Paths, Minimum Spanning Trees and A*

Weighted traversal, and what a good heuristic buys you

CORE

PREREQS: Chapters 13, 22

Once edges have weights, BFS stops working and a family of algorithms takes over, each making a different assumption about the weights in exchange for a different running time. This chapter covers the four you need, the one that handles negative weights, the two minimum-spanning-tree algorithms, and A* — which is the bridge between classical search and the heuristic search inside modern systems.

WHAT YOU WILL LEARN

- Implement Dijkstra's algorithm with a priority queue and understand why it needs non-negative weights
- Use Bellman–Ford for negative weights and negative-cycle detection
- Compute all-pairs shortest paths with Floyd–Warshall and know when it wins
- Implement A* and state the admissibility and consistency conditions
- Implement Kruskal and Prim for minimum spanning trees
- Recognise these algorithms inside routing, scheduling and retrieval systems

Why BFS stops working

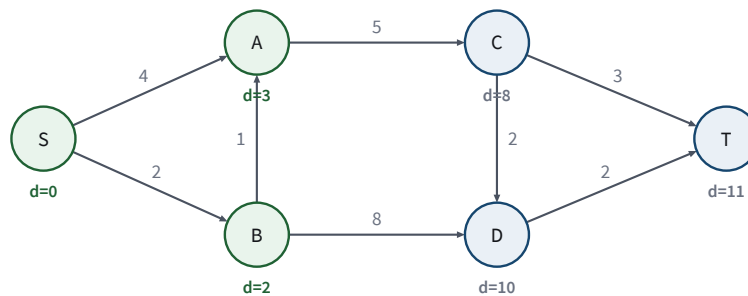
BFS explores in order of *hop count*. With weights, the fewest-hops path is often not the cheapest: a single edge of weight 100 versus two edges of weight 1 each. The fix is to explore in order of *accumulated cost*, which means replacing the queue with a priority queue — one line, again.

Algorithm	Weights	Time	Space	Gives
BFS	unweighted	$\Theta(V+E)$	$\Theta(V)$	single-source shortest paths
0-1 BFS	weights in $\{0,1\}$	$\Theta(V+E)$	$\Theta(V)$	same, using a deque
Dijkstra	non-negative	$\Theta((V+E)\log V)$	$\Theta(V)$	single-source
Bellman–Ford	any	$\Theta(VE)$	$\Theta(V)$	single-source + negative-cycle detection

Algorithm	Weights	Time	Space	Gives
SPFA	any	$\Theta(VE)$ worst, fast in practice	$\Theta(V)$	single-source
Floyd–Warshall	any (no negative cycle)	$\Theta(V^3)$	$\Theta(V^2)$	all pairs
Johnson	any	$\Theta(VE + V^2 \log V)$	$\Theta(V^2)$	all pairs, sparse graphs
A*	non-negative + heuristic	$\Theta((V+E) \log V)$ worst	$\Theta(V)$	single pair, much faster in practice
DAG relaxation	any, but acyclic	$\Theta(V+E)$	$\Theta(V)$	single-source, longest path too

Dijkstra

green = settled (distance final)



- 1 pop the closest unsettled vertex
- 2 relax every edge leaving it
- 3 repeat

Correct only with non-negative weights:
a settled distance must never improve.

Figure 23.1 Dijkstra settles vertices in increasing order of distance. Once a vertex is settled, its distance is final — a claim that is only true when weights are non-negative.

Dijkstra with lazy deletion

```

1  import heapq
2
3  def dijkstra(indptr, indices, weights, src):
4      n = len(indptr) - 1
5      dist = [float("inf")] * n
6      parent = [-1] * n
7      dist[src] = 0.0
8      pq = [(0.0, src)]
9      while pq:
10         d, v = heapq.heappop(pq)
11         if d > dist[v]:
12             continue # stale entry: skip it
13         for i in range(indptr[v], indptr[v + 1]):
14             u, w = indices[i], weights[i]
15             nd = d + w
16             if nd < dist[u]:
17                 dist[u] = nd
18                 parent[u] = v
19                 heapq.heappush(pq, (nd, u)) # lazy: no decrease-key
20     return dist, parent

```


PROOF | Why non-negative weights are required

The invariant is: when a vertex v is popped with distance d , d is the true shortest distance to v . Suppose not — some shorter path P to v exists. P must leave the settled set at some edge (x, y) where x is settled and y is not. Then $\text{dist}[y] \leq \text{dist}[x] + w(x, y) \leq \text{length of } P \leq d$. Since y is still in the queue with a key no larger than d , and we popped v instead, we have $\text{dist}[y] \geq d$, so the whole prefix has length exactly d and the remainder of P has length ≤ 0 . With non-negative weights that remainder is exactly 0, so P is no shorter after all. With a negative edge, the remainder can be negative and the invariant fails.

The lazy-deletion pattern is worth internalising: rather than a **decrease-key** operation (which needs an indexed heap, Chapter 13), push a new entry and discard stale ones on pop. The heap holds $O(E)$ entries instead of $O(V)$, but the constant factors are much better and the code is half the length.

WHERE THIS SHOWS UP IN ML | Dijkstra-shaped problems in ML systems

Best-first decoding. Replacing beam search's fixed width with a priority queue over partial hypotheses ordered by score gives A*-style decoding, which with an admissible heuristic returns the true best sequence.

Cheapest execution plan. Choosing among fused kernel implementations, or among quantisation levels per layer under a latency budget, is a shortest path over a layered DAG — and being a DAG, it is linear-time.

Graph-based retrieval. HNSW's search is a greedy variant of this loop with a bounded candidate heap; the “settled” set is the result heap.

Bellman–Ford and negative weights

Bellman–Ford relaxes *every* edge, $V-1$ times. After k rounds, every shortest path using at most k edges has been found; since a simple path uses at most $V-1$ edges, $V-1$ rounds suffice.

Bellman-Ford with negative-cycle detection

```

1  def bellman_ford(n, edges, src):
2      dist = [float("inf")] * n
3      dist[src] = 0.0
4      for _ in range(n - 1):
5          changed = False
6          for u, v, w in edges:
7              if dist[u] + w < dist[v]:
8                  dist[v] = dist[u] + w
9                  changed = True
10         if not changed:
11             break                    # early exit: converged
12         for u, v, w in edges:        # one more round
13             if dist[u] + w < dist[v]:
14                 return None         # still improving: negative
15                                     cycle
16     return dist

```

The negative-cycle check is not an afterthought — it is often the *point*. Detecting a negative cycle finds an arbitrage opportunity in a currency graph, an inconsistent set of difference constraints in a scheduler, and an unsatisfiable constraint system in a solver.

A*: Dijkstra with a conscience

A* orders the priority queue by $f(v)=g(v)+h(v)$, where g is the cost so far and h is an estimate of the remaining cost.

DEFINITION | Admissible and consistent heuristics

h is **admissible** if $h(v) \leq$ the true remaining cost for every v : it never overestimates.

Admissibility guarantees A* returns an optimal path.

h is **consistent** (monotone) if $h(u) \leq w(u,v) + h(v)$ for every edge. Consistency implies admissibility and additionally guarantees that each vertex is settled only once — so A* with a consistent heuristic is exactly Dijkstra on a reweighted graph, with the same complexity bound and a much smaller explored set.

With $h \equiv 0$, A* is Dijkstra. With a perfect h , it walks straight to the goal. Everything useful is in between, and the quality of h is the whole engineering problem: for road routing, straight-line distance divided by the maximum speed; for puzzles, the sum of Manhattan distances; for sequence decoding, an optimistic bound on the remaining log-probability.

A* over an implicit graph

```

1  import heapq
2
3  def astar(start, goal, neighbours, h):
4      g = {start: 0.0}
5      pq = [(h(start), 0.0, start, None)]
6      came = {}
7      while pq:
8          f, gv, v, parent = heapq.heappop(pq)
9          if v in came:
10             continue
11          came[v] = parent
12          if v == goal:
13              path, cur = [], v
14              while cur is not None:
15                  path.append(cur); cur = came[cur]
16              return path[::-1], gv
17          for u, w in neighbours(v):
18              ng = gv + w
19              if ng < g.get(u, float("inf")):
20                  g[u] = ng
21                  heapq.heappush(pq, (ng + h(u), ng, u, v))
22  return None, float("inf")

```

Note `neighbours` is a *function*, not a stored graph. A* is usually run over an implicit graph — puzzle states, decoder states, plan states — that is far too large to materialise. This is the same “generate the graph as you explore it” pattern as backtracking (Chapter 20).

Minimum spanning trees

An MST connects all vertices with minimum total edge weight. Both standard algorithms are greedy (Chapter 17) and both are correct for the same reason.

THEOREM | The cut property

For any partition of the vertices into two non-empty sets, the minimum-weight edge crossing the partition belongs to some MST. (With distinct weights, to *every* MST.)

Kruskal repeatedly adds the globally lightest edge that does not create a cycle — the cut is “the components at that moment”. Prim grows a single tree, repeatedly adding the lightest edge leaving it — the cut is “tree versus rest”.

Kruskal, with union-find from Chapter 15

```

1  def kruskal(n, edges):
2      dsu = DSU(n)
3      total, tree = 0.0, []
4      for w, u, v in sorted(edges):          # theta(E log E)
5          if dsu.union(u, v):                # theta(alpha(n))
6              tree.append((u, v, w))
7              total += w
8              if len(tree) == n - 1:
9                  break
10     return tree, total

```

Kruskal is $\Theta(E \log E)$, dominated by the sort; Prim with a binary heap is $\Theta(E \log V)$. Kruskal is simpler and parallelises better (sort then scan); Prim wins on dense graphs and when the edges are generated lazily.

WHERE THIS SHOWS UP IN ML | MSTs and single-linkage clustering

Single-linkage hierarchical clustering is *exactly* the MST: cutting the MST at all edges heavier than τ gives the single-linkage clusters at threshold τ . So you can compute a full single-linkage dendrogram for n points in $\Theta(n^2)$ with Prim on the complete distance graph — or much faster with a spatial index. Related: the minimum spanning tree of a k -nearest-neighbour graph underlies HDBSCAN, and MST-based clustering is the standard way to turn a near-duplicate similarity graph into groups when plain connected components (Chapter 15) over-merges.

COMMON PITFALL | Longest path is not shortest path with negated weights

Negating the weights and running a shortest-path algorithm fails, because the negation creates negative cycles and the longest simple path problem is NP-hard in general. The exception is a DAG: there are no cycles, so relaxation in topological order finds the longest path in $\Theta(V+E)$. This is exactly why critical-path analysis works on a schedule and not on a general graph.

EXERCISES

1. Implement 0-1 BFS with a deque (push-front for weight-0 edges, push-back for weight-1) and show it is $\Theta(V+E)$.

2. Construct a graph with one negative edge and no negative cycle on which Dijkstra returns a wrong answer. Verify Bellman–Ford gets it right.
3. Implement Dijkstra with an indexed heap supporting **decrease-key** and compare it to the lazy version on a graph with 10^6 edges. Explain the result in terms of constants.
4. Implement A* for the 15-puzzle with (a) misplaced-tile and (b) Manhattan distance heuristics. Compare expanded nodes. Prove both are admissible.
5. Show that if h is consistent then A* never re-expands a settled vertex. Then construct an admissible-but-inconsistent h that causes re-expansion.
6. Implement both Kruskal and Prim; verify they produce the same total weight on random graphs, and compare their times at densities $E=2V$ and $E=V^2/10$.
7. Build a single-linkage dendrogram of 5,000 points by computing the MST of the complete distance graph, and verify it matches `scipy.cluster.hierarchy.linkage(method="single")`.
8. Model “choose a quantisation level per layer to minimise latency subject to an accuracy floor” as a shortest path on a layered DAG, and solve it.

CHAPTER RECAP

- With weights, replace BFS’s queue with a priority queue: that is Dijkstra. Lazy deletion is simpler and usually faster than an indexed heap.
- Dijkstra requires non-negative weights; the settled-vertex invariant fails otherwise. Bellman–Ford handles negative weights in $\Theta(VE)$ and detects negative cycles, which is frequently the real goal.
- On a DAG, relaxing in topological order is $\Theta(V+E)$ and handles negative weights and longest paths.
- A* orders by $g+h$. Admissible h gives optimality; consistent h also gives single expansion. With $h=0$ it is Dijkstra.
- A* is usually run over an implicit graph generated on demand.
- Kruskal and Prim both follow from the cut property; Kruskal is sort-dominated and simple, Prim is better on dense graphs.
- Single-linkage clustering is the MST, cut at a threshold.

DAGs: Topological Order, Autograd and Computation Graphs

The graph your training loop is

CORE

PREREQS: Chapters 18, 22

Directed acyclic graphs are the best-behaved graphs: they have a linear order consistent with their edges, which makes dynamic programming over them trivial and shortest and longest paths equally easy. They are also the exact structure of a neural network's computation, so this chapter doubles as a proper explanation of how automatic differentiation works and where its memory goes.

WHAT YOU WILL LEARN

- Compute a topological order two ways and detect cycles while doing it
- Solve longest path, critical path and DP on a DAG in linear time
- Explain reverse-mode automatic differentiation as a reverse topological sweep
- Account precisely for training memory and understand gradient checkpointing
- Understand operator fusion and scheduling as graph transformations
- Recognise DAG scheduling in pipelines, build systems and model parallelism

Topological order

DEFINITION | Topological order

A linear ordering of a directed graph's vertices such that every edge (u, v) has u before v . It exists if and only if the graph is acyclic, and it is generally not unique.

Kahn's algorithm: peel off zero in-degree vertices

```

1  from collections import deque
2
3  def topo_sort(n, indptr, indices):
4      indeg = [0] * n
5      for v in range(n):
6          for u in indices[indptr[v]:indptr[v + 1]]:
7              indeg[u] += 1
8      q = deque(v for v in range(n) if indeg[v] == 0)
```

```

9     order = []
10    while q:
11        v = q.popleft()
12        order.append(v)
13        for u in indices[indptr[v]:indptr[v + 1]]:
14            indeg[u] -= 1
15            if indeg[u] == 0:
16                q.append(u)
17    if len(order) != n:
18        raise ValueError("graph has a cycle")    # free cycle detection
19    return order

```

$\Theta(V+E)$, and the length check at the end is a complete cycle detector: if any vertex never reaches in-degree zero, it is on or downstream of a cycle. The alternative is reverse DFS finishing order, which is equally valid; Kahn’s is easier to parallelise (all zero in-degree vertices are independent) and gives a natural notion of “levels”.

KEY INSIGHT | Any topological order is a valid evaluation order

This is why dynamic programming on a DAG needs no thought about ordering (Chapter 18, step 4): compute a topological order and evaluate in it. It is also why a build system, a data pipeline and a neural network forward pass are the same computation.

Linear-time DP on a DAG

Longest path in a DAG: the critical path of a schedule

```

1  def longest_path(n, indptr, indices, weights, order):
2      dist = [0.0] * n
3      parent = [-1] * n
4      for v in order:                                # dependencies come
5          first
6          for i in range(indptr[v], indptr[v + 1]):
7              u, w = indices[i], weights[i]
8              if dist[v] + w > dist[u]:
9                  dist[u] = dist[v] + w
10                 parent[u] = v
11     end = max(range(n), key=lambda v: dist[v])
12     path, cur = [], end
13     while cur != -1:
14         path.append(cur); cur = parent[cur]
15     return path[::-1], dist[end]

```

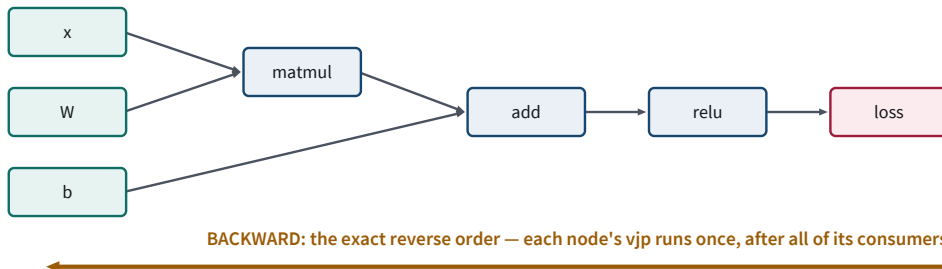
In a general graph, longest path is NP-hard. On a DAG it is one pass. That gap is why *acyclic* is such a valuable property and why so many systems go to trouble to guarantee it.

The **critical path** — the longest weighted path through a task graph — is the minimum possible makespan with unlimited parallelism. It tells a pipeline engineer exactly which tasks to optimise: shortening anything off the critical path changes nothing at all.

Automatic differentiation

A neural network's forward pass builds a DAG: leaves are inputs and parameters, internal nodes are operations, and the root is the loss.

FORWARD: topological order



The stored activations are what the reverse sweep needs; checkpointing trades some of them for recomputation.

Figure 24.1 A computation graph. The forward pass evaluates in topological order; the backward pass traverses the exact reverse, and each node's vector-Jacobian product runs once, after every node that consumes its output.

DEFINITION | Reverse-mode automatic differentiation

Let the computation be $L = f_k(f_{k-1}(\dots f_1(x)))$. Forward mode propagates $\partial \text{node} / \partial x$ from inputs to outputs; reverse mode propagates $\text{bar}v = \partial L / \partial \text{node}$ from the output back to the inputs. Each node needs a **vector-Jacobian product**: given $\text{bar}v$ for its output, produce $\text{bar}v$ for each input, without ever materialising the Jacobian.

For a function $\mathbb{R}^n \rightarrow \mathbb{R}^m$, forward mode costs $O(n)$ passes and reverse mode costs $O(m)$. Neural network training has $n = \text{millions of parameters}$ and $m = 1$ (a scalar loss), so reverse mode is cheaper by a factor of millions. That asymmetry — not any deep mathematics — is why backpropagation is the algorithm.

A minimal reverse-mode autodiff engine

```

1  class Var:
2      __slots__ = ("val", "grad", "_parents", "_backward")
3
4      def __init__(self, val, parents=(), backward=None):
5          self.val = val
6          self.grad = 0.0
7          self._parents = parents
8          self._backward = backward or (lambda g: ())
9
10     def __add__(self, other):
11         other = other if isinstance(other, Var) else Var(other)
12         return Var(self.val + other.val, (self, other), lambda g: (g, g))
13
14     def __mul__(self, other):
15         other = other if isinstance(other, Var) else Var(other)
16         return Var(self.val * other.val, (self, other),
17                 lambda g: (g * other.val, g * self.val))
18
19     def relu(self):
20         return Var(max(0.0, self.val), (self,),
21                 lambda g: (g * (1.0 if self.val > 0 else 0.0),))
  
```

```

22
23     def backward(self):
24         order, seen = [], set()
25         def visit(v):                                # DFS post-order =
26             topological
27             if id(v) in seen:
28                 return
29             seen.add(id(v))
30             for p in v._parents:
31                 visit(p)
32             order.append(v)
33         visit(self)
34         self.grad = 1.0
35         for v in reversed(order):                    # reverse topological sweep
36             for parent, g in zip(v._parents, v._backward(v.grad)):
37                 parent.grad += g                    # ACCUMULATE: a node may have
38                                                         # several consumers
39
40     def __radd__(self, o):
41         return self + o
42
43     def __rmul__(self, o):
44         return self * o

```

That is the whole idea. PyTorch’s `autograd` and JAX’s `grad` differ in engineering — tensors instead of scalars, C++ kernels, a tape or a traced program — but not in structure. Two details in the code are the ones people get wrong:

- **Accumulate, never assign.** A node feeding several consumers receives a gradient contribution from each, and they sum. Assigning instead of accumulating silently drops all but the last.
- **Reverse topological order, not reverse call order.** A node’s gradient is complete only after every consumer has contributed. That is why the sort is necessary and why a naive “call backward as you go” is wrong for any graph with a branch.

WHERE THIS SHOWS UP IN ML | Where training memory actually goes

Peak memory during training is

$$M = \underbrace{P(2+2+12)}_{\text{weights, grads, Adam states tensors)} + \underbrace{\sum_{\text{nodes}} \text{size(saved activations)}}_{\text{workspace}}$$

for a P -parameter model in mixed precision. The first term is fixed. The second grows with batch size $\times$ sequence length $\times$ depth, and it is the term you can actually control:

- **Gradient checkpointing** saves only segment boundaries and recomputes the rest: $\Theta(\sqrt{L})$ memory for one extra forward pass (Chapter 18).
- **Activation offload** moves saved tensors to host memory and back, trading PCIe bandwidth for capacity.
- **Fusion** removes intermediates entirely — a fused `bias + gelu + dropout` never writes the two intermediate tensors.
- **In-place operations** overwrite a tensor that is not needed for the backward pass. When it is needed, the framework raises an error; that error is a DAG check, which is why it can detect the problem at all.

Graph transformations

Modern compilers (`torch.compile`, XLA, TensorRT) operate on this DAG.

Operator fusion replaces a chain of elementwise nodes with a single kernel. The benefit is not fewer FLOPs — it is fewer round trips to memory, which for memory-bound kernels is the entire cost (Chapter 3). Fusing `x -> mul -> add -> gelu` turns four reads and four writes into one of each.

Common subexpression elimination merges identical nodes; the DAG makes this a hash of (op, inputs).

Scheduling chooses a specific topological order. Different valid orders have very different peak memory, because memory is determined by how long tensors stay live. Choosing the order that minimises peak live memory is NP-hard in general, and compilers use heuristics plus DP on segments.

Dead code elimination drops nodes not reachable backwards from the outputs — a reverse traversal.

KEY INSIGHT | Why `torch.compile` cares so much about graph breaks

A graph break — caused by data-dependent Python control flow, an unsupported operation, or a print statement — splits the DAG into independent pieces. Fusion, scheduling and memory planning cannot cross the boundary, so each break costs you the optimisations that span it. This is why “remove graph breaks” is the first thing a performance guide tells you, and why `torch.compile(fullgraph=True)` exists: it turns a silent slowdown into an error.

EXERCISES

1. Implement topological sort both ways (Kahn and reverse DFS finishing order) and verify they produce valid but generally different orders.
2. Compute the critical path of a 50-task pipeline with random durations and dependencies. Show that shortening a non-critical task does not change the makespan.
3. Extend the `Var` class with `exp`, `log`, `pow` and division, and verify all gradients against finite differences.
4. Construct a computation graph where a node has three consumers. Demonstrate that assigning rather than accumulating gradients gives a wrong answer, and quantify how wrong.
5. Implement forward-mode autodiff with dual numbers. For a function $\mathbb{R}^{1000} \rightarrow \mathbb{R}$, measure the cost of forward mode versus reverse mode and confirm the factor of 1000.
6. For a chain of L layers with equal activation sizes, derive the optimal number of checkpoint segments and show that the optimum is $\sqrt{L}$.
7. Take a small model, print its graph with `torch.fx` or `torch._dynamo.explain`, count the graph breaks, and remove them.
8. Given a DAG with tensor sizes on edges, write a program that evaluates peak live memory for a given topological order, and search over orders to find a better one.

CHAPTER RECAP

- A topological order exists exactly when the graph is acyclic; Kahn's algorithm produces one in $\Theta(V+E)$ and detects cycles for free.
- Any topological order is a valid evaluation order, which is why DP on a DAG needs no ordering argument and why longest path — NP-hard in general — is linear here.
- The critical path is the minimum makespan; optimising off it changes nothing.
- Reverse-mode autodiff is a reverse topological sweep of vector-Jacobian products. It is cheap for scalar losses because cost scales with the number of *outputs*, not inputs.
- Accumulate gradients across consumers and respect topological order; those are the two correctness requirements.
- Training memory is fixed optimiser state plus controllable activations; checkpointing, offload, fusion and in-place ops each attack the second.
- Compilers transform this DAG: fusion for memory traffic, CSE, scheduling for peak memory, dead-code elimination. Graph breaks cost you all of them.

Flows, Matchings and Assignment

The algorithm inside your object detector's loss function

ADVANCED

PREREQS: Chapters 17, 23

Network flow is the classical topic that looks furthest from machine learning and turns out to be inside it. The Hungarian algorithm assigns predictions to ground-truth objects in DETR-style detectors; bipartite matching balances experts in a mixture-of-experts layer; min-cut solves image segmentation exactly. This chapter develops the theory only as far as those applications need, and no further.

WHAT YOU WILL LEARN

- State the max-flow min-cut theorem and use it to reason about bottlenecks
- Implement Edmonds–Karp and Dinic’s algorithm
- Find maximum bipartite matchings with augmenting paths and Hopcroft–Karp
- Implement the Hungarian algorithm for minimum-cost assignment
- Recognise assignment problems in detection losses, MoE routing and tracking
- Know when to use a solver rather than writing one

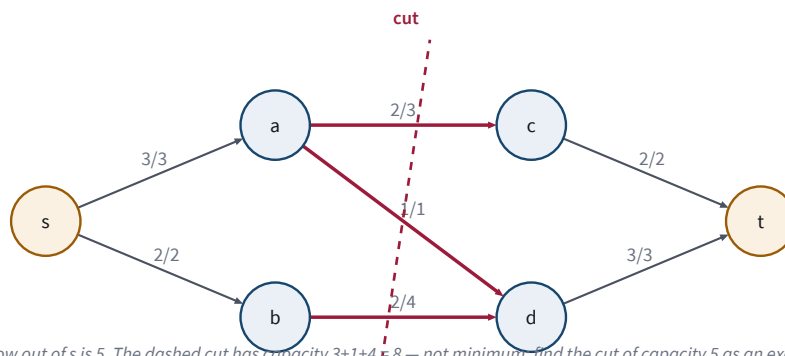
Flow networks

DEFINITION | Flow network and max-flow

A directed graph with a capacity $c(u,v) \geq 0$ on each edge, a source s and a sink t . A **flow** assigns $f(u,v) \leq c(u,v)$ to each edge such that every vertex other than s and t has equal in-flow and out-flow. The **value** of the flow is the net flow out of s ; max-flow asks to maximise it.

THEOREM | Max-flow min-cut

The maximum value of an s – t flow equals the minimum capacity of an s – t cut: a partition of the vertices with s on one side and t on the other, whose capacity is the total capacity of edges crossing forwards.



Flow out of s is 5. The dashed cut has capacity $3+1+4=8$ — not minimum, find the cut of capacity 5 as an exercise.

Figure 25.1 A flow network. The max-flow value equals the capacity of the *minimum* cut; finding one gives you the other, and the cut identifies exactly which edges are the bottleneck.

The theorem is the practically useful half: max-flow tells you the throughput, and the min-cut tells you *which resources to add*. After running a max-flow, the vertices reachable from s in the residual graph form one side of a minimum cut — a one-line BFS at the end.

Algorithms

Ford–Fulkerson repeatedly finds any augmenting path in the residual graph and pushes flow along it. With arbitrary path choice it can be slow (and with irrational capacities, non-terminating). **Edmonds–Karp** chooses the *shortest* augmenting path by BFS, giving $\Theta(VE^2)$. **Dinic’s algorithm** builds a level graph by BFS and then finds a blocking flow by DFS, giving $\Theta(V^2E)$ in general and $\Theta(E\sqrt{V})$ on unit-capacity graphs — which is the bipartite-matching case.

Dinic’s algorithm

```

1  from collections import deque
2
3  class Dinic:
4      def __init__(self, n):
5          self.n = n
6          self.g = [[] for _ in range(n)]      # adjacency of edge indices
7          self.edges = []                     # (to, capacity)
8
9      def add_edge(self, u, v, cap):
10         self.g[u].append(len(self.edges)); self.edges.append([v, cap])
11         self.g[v].append(len(self.edges)); self.edges.append([u, 0]) #
12         reverse
13
14     def _levels(self, s, t):
15         self.level = [-1] * self.n
16         self.level[s] = 0
17         q = deque([s])
18         while q:
19             v = q.popleft()
20             for i in self.g[v]:
21                 to, cap = self.edges[i]
22                 if cap > 0 and self.level[to] < 0:
23                     self.level[to] = self.level[v] + 1
24                     q.append(to)
25         return self.level[t] >= 0

```

```

26     def _dfs(self, v, t, pushed):
27         if v == t or pushed == 0:
28             return pushed
29         while self.it[v] < len(self.g[v]):
30             i = self.g[v][self.it[v]]
31             to, cap = self.edges[i]
32             if cap > 0 and self.level[to] == self.level[v] + 1:
33                 d = self._dfs(to, t, min(pushed, cap))
34                 if d > 0:
35                     self.edges[i][1] -= d
36                     self.edges[i ^ 1][1] += d      # XOR trick pairs the
37                     edges
38                     return d
39             self.it[v] += 1
40         return 0
41
42     def maxflow(self, s, t):
43         total = 0
44         while self._levels(s, t):
45             self.it = [0] * self.n
46             while True:
47                 pushed = self._dfs(s, t, float("inf"))
48                 if pushed == 0:
49                     break
50                 total += pushed
51         return total

```

The `i ^ 1` trick is worth noting: storing each edge immediately before its reverse makes the reverse edge's index the forward index with the low bit flipped. Small, but it removes a whole class of bookkeeping bugs.

Bipartite matching

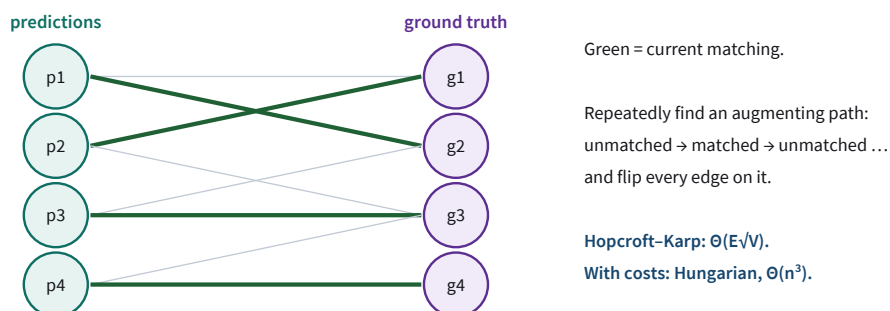


Figure 25.2 Maximum bipartite matching via augmenting paths. The classical formulation on the left is exactly the set-prediction problem on the right.

A matching is a set of edges with no shared endpoints. Maximum bipartite matching reduces to max-flow: add a source connected to every left vertex and a sink from every right vertex, all capacities 1. Dinic on this unit-capacity network is Hopcroft-Karp, running in $O(E\sqrt{V})$.

Kuhn's algorithm: simple augmenting-path matching

```

1  def bipartite_match(adj, n_left, n_right):
2      """adj[u] = list of right-vertices adjacent to left-vertex u."""
3      match_r = [-1] * n_right
4      def try_kuhn(u, seen):
5          for v in adj[u]:
6              if seen[v]:
7                  continue
8              seen[v] = True
9              if match_r[v] == -1 or try_kuhn(match_r[v], seen):
10                 match_r[v] = u          # augment along the path
11                 return True
12         return False
13     count = 0
14     for u in range(n_left):
15         if try_kuhn(u, [False] * n_right):
16             count += 1
17     return count, match_r

```

$\Theta(V E)$, perfectly adequate below a few thousand vertices, and about fifteen lines. Reach for Hopcroft–Karp only when profiling says to.

The assignment problem

Maximum matching asks *how many* pairs. The assignment problem asks for the *cheapest* perfect matching given an $n \times n$ cost matrix. The Hungarian (Kuhn–Munkres) algorithm solves it in $\Theta(n^3)$.

The idea: maintain dual variables (potentials) u_i on rows and v_j on columns with $u_i + v_j \leq C_{ij}$ for all pairs. Edges where the constraint is tight form the “equality subgraph”; find a maximum matching in it; if it is not perfect, adjust the potentials by the smallest slack, which adds at least one new tight edge. Repeat. When the matching is perfect, complementary slackness proves it is optimal.

Use the library, and know what it is doing

```

1  import numpy as np
2  from scipy.optimize import linear_sum_assignment
3
4  cost = np.random.rand(200, 200)
5  rows, cols = linear_sum_assignment(cost)          # Jonker-Volgenant, theta(
6  print(cost[rows, cols].sum())                    # provably minimal total
7  cost

```

WHERE THIS SHOWS UP IN ML | The Hungarian algorithm inside modern detectors

DETR and every set-prediction model after it face a problem: the model emits N object queries in no particular order, and the image has M ground-truth objects. To compute a loss you must decide which prediction corresponds to which object — and any fixed rule (by index, by confidence) makes the loss discontinuous and the training unstable.

The solution is a **bipartite matching loss**. Build an $N \times M$ cost matrix where C_{ij} combines classification cost and box distance for prediction i against object j , run `linear_sum_assignment`, and apply the loss only along the matched pairs. Unmatched predictions are trained toward “no object”.

This makes the loss permutation-invariant, removes the need for non-maximum suppression entirely, and is why DETR-family models are end-to-end. The cost is one $\Theta(N^3)$ assignment per image per step — with $N=100$ that is about a million operations, negligible beside the backbone.

WHERE THIS SHOWS UP IN ML | Three more matching problems you will meet

Mixture-of-experts routing. Top- k routing overloads popular experts. Balanced assignment — solving a transportation problem so each expert gets roughly equal load — is what Sinkhorn-based routers (BASE layers, Sinkhorn Transformers) do; Sinkhorn iteration is an entropic relaxation of the assignment problem that is differentiable and runs on a GPU.

Multi-object tracking. Associating detections in frame t with tracks from frame $t-1$ is exactly the assignment problem; SORT and DeepSORT call the Hungarian algorithm every frame with a cost built from IoU and appearance distance.

Optimal transport. Wasserstein distance between two discrete distributions is a transportation problem — assignment with fractional masses. The Sinkhorn algorithm solves the entropy-regularised version in $\Theta(n^2)$ per iteration, and appears in domain adaptation, in self-supervised clustering (SwAV), and in generative model evaluation.

KEY INSIGHT | Min-cut as exact inference

For an energy function over binary variables that is *submodular* — roughly, pairwise terms that prefer agreement — the global minimum can be found exactly by a single min-cut. This is the basis of graph-cut image segmentation: pixels are vertices, the source and sink are foreground and background, unary terms become terminal capacities, and smoothness terms become inter-pixel capacities. It is one of the few cases in vision where a global optimum over an exponential space is available in polynomial time, and it remains the method of choice for interactive segmentation.

COMMON PITFALL | Do not implement min-cost flow yourself

Max-flow and bipartite matching are reasonable to write. Min-cost flow and the assignment problem have subtle degenerate cases, and mature implementations (`scipy.optimize.linear_sum_assignment`, Google OR-Tools, LEMON) are both faster and correct. Write them once to understand them, then use the library.

EXERCISES

1. Find the minimum cut in the figure's network and verify it equals the max-flow value.
2. Implement Edmonds–Karp and Dinic, and compare them on a unit-capacity bipartite graph with 10,000 vertices.
3. Show that Kuhn's algorithm is $\Theta(VE)$ and construct a graph on which it hits that bound.
4. Implement a DETR-style matching loss: build the cost matrix from classification log-probabilities and generalised IoU, call `linear_sum_assignment`, and verify the loss is invariant to permuting the predictions.
5. Implement Sinkhorn iteration for entropic optimal transport and show that as the regularisation goes to zero the solution approaches the Hungarian assignment.
6. Model balanced expert routing as a transportation problem with capacity constraints, solve it, and compare expert load to plain top-1 routing.
7. Implement binary image segmentation via min-cut on a 200×200 image with user-supplied foreground and background scribbles.

CHAPTER RECAP

- Max-flow equals min-cut; the cut identifies the bottleneck edges, which is usually the actionable output.
- Edmonds–Karp is $\Theta(VE^2)$; Dinic is $\Theta(V^2E)$ generally and $\Theta(E\sqrt{V})$ on unit capacities, which covers bipartite matching.
- Maximum bipartite matching reduces to unit-capacity max-flow; Kuhn's fifteen-line algorithm is $\Theta(VE)$ and usually enough.
- The assignment problem — cheapest perfect matching — is solved in $\Theta(n^3)$ by the Hungarian algorithm; use `linear_sum_assignment`.
- Set-prediction losses (DETR), multi-object tracking and balanced MoE routing are all assignment problems; optimal transport is its fractional relaxation, solved by Sinkhorn.
- Submodular binary energies are globally minimised by one min-cut, which is how graph-cut segmentation works.

Graphs in Machine Learning

PageRank, spectral methods and the sampling that makes GNNs trainable

ADVANCED

PREREQS: Chapters 22--24

This chapter is about the graph algorithms that are themselves machine learning: random walks that compute importance, eigenvectors that reveal cluster structure, and message passing whose cost explodes unless you sample. The unifying observation is that all of them are sparse matrix operations, so the data structure from Chapter 22 — CSR — is what actually determines whether they run.

WHAT YOU WILL LEARN

- Derive PageRank as a random walk and implement it by power iteration
- Implement personalised PageRank and use it for recommendation and retrieval
- Understand spectral clustering and what the graph Laplacian's eigenvectors mean
- Explain the neighbour explosion in GNNs and the three families of fixes
- Implement neighbour sampling and mini-batch construction for a GNN
- Estimate the cost of a graph algorithm from V , E and the degree distribution

Everything is a sparse matrix product

A graph with adjacency matrix A turns graph algorithms into linear algebra:

Graph operation	Linear algebra	Cost
One step of BFS from a frontier x	$A^{\text{top}} x$ (boolean semiring)	$\Theta(E)$
Count paths of length k	A^k	$\Theta(kE)$ sparse, $\Theta(V^3 \log k)$ dense
Random walk step	$P^{\text{top}} x$ with $P = D^{-1}A$	$\Theta(E)$
GNN message passing layer	$\text{hat}A X W$	$\Theta(Ed + Vd^2)$
Spectral embedding	eigenvectors of L	$\Theta(Ek)$ per Lanczos step

KEY INSIGHT | Why CSR keeps coming back

A sparse matrix–vector product in CSR is: for each row, gather the entries its column indices point to and accumulate. That is a sequential scan over **indices** (good) plus a random gather from the vector (bad). The gather is the bottleneck, which is why graph algorithms are memory-bound, why vertex reordering (RCM, Gorder) can speed them up by 2–3x with no change to the algorithm, and why GPU implementations obsess over coalescing.

PageRank

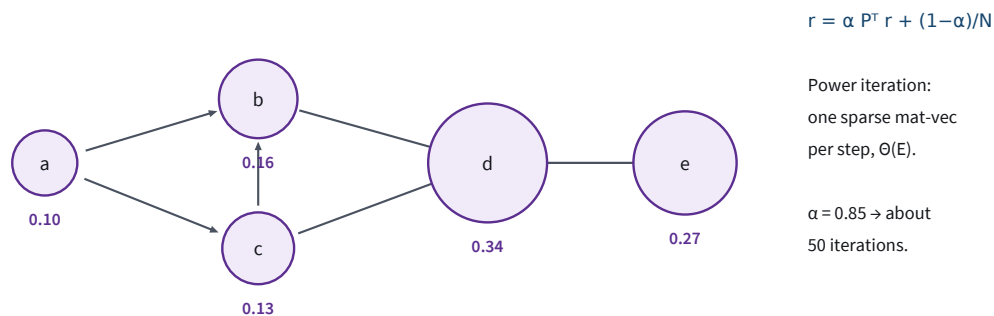


Figure 26.1 PageRank as a random surfer. Rank flows along out-edges and pools in well-connected regions; the damping term stops it draining into sinks.

Model a surfer who, at each step, follows a uniformly random out-link with probability α , and teleports to a uniformly random page with probability $1-\alpha$. The stationary distribution of this walk is PageRank:

$$r = \alpha P^{\text{top}} r + 1 - \alpha / N, P_{ij} = A_{ij} / \deg(i)$$

The teleport term does two jobs: it makes the chain irreducible and aperiodic (so a unique stationary distribution exists), and it makes the iteration contract at rate α , so the error falls by a factor of α per step. With $\alpha=0.85$, reaching 10^{-8} takes $\log(10^{-8})/\log(0.85) \approx 113$ iterations; in practice 50 is plenty.

PageRank by power iteration, with dangling-node handling

```

1  import numpy as np
2
3  def pagerank(indptr, indices, n, alpha=0.85, iters=60, tol=1e-9):
4      out_deg = np.diff(indptr).astype(np.float64)
5      dangling = out_deg == 0
6      r = np.full(n, 1.0 / n)
7      for _ in range(iters):
8          contrib = np.zeros(n)
9          w = np.where(dangling, 0.0, r / np.maximum(out_deg, 1))
10         np.add.at(contrib, indices, np.repeat(w, np.diff(indptr)))
11         leaked = r[dangling].sum() # mass with nowhere
12         to go
13         new = alpha * (contrib + leaked / n) + (1 - alpha) / n
14         if np.abs(new - r).sum() < tol:

```

```

14         return new
15     r = new
16     return r

```

Dangling nodes — vertices with no out-edges — are the detail everyone gets wrong. Their rank has nowhere to flow and simply disappears, so the vector stops summing to 1 and the result is quietly wrong. Redistributing that leaked mass uniformly is the standard fix.

WHERE THIS SHOWS UP IN ML | Personalised PageRank is a retrieval algorithm

Replace the uniform teleport vector with a distribution concentrated on a seed set S , and the stationary distribution measures *proximity to S* rather than global importance. This gives:

- **Recommendation.** Seed at a user's interacted items in a bipartite user-item graph; the top-ranked unseen items are recommendations. This is Pixie, the algorithm behind Pinterest's related-pins, which runs approximate personalised PageRank by *sampling* random walks rather than iterating the matrix — a few thousand short walks give a good top- k in milliseconds.
- **Graph-augmented retrieval.** Seed at the entities matched by a query in a knowledge graph, and rank documents by the resulting mass. This captures multi-hop relevance that vector similarity alone misses.
- **Local clustering.** The sweep cut over a personalised PageRank vector finds a good local cluster around a seed without touching the whole graph — $\Theta(1/\epsilon)$ work, independent of graph size.

Spectral methods

DEFINITION | Graph Laplacian

For an undirected graph with adjacency A and degree matrix D , the unnormalised Laplacian is $L=D-A$ and the symmetric normalised Laplacian is $L_{\text{sym}}=I-D^{-1/2}AD^{-1/2}$. Both are positive semi-definite; the multiplicity of eigenvalue 0 equals the number of connected components.

The key identity is

$$x^{\text{top}} L x = \sum_{(u,v) \in E} w_{uv} (x_u - x_v)^2$$

so the quadratic form measures how much x varies across edges. Eigenvectors with small eigenvalues are therefore *smooth* over the graph: they take similar values on connected vertices. The eigenvector of the second smallest eigenvalue — the **Fiedler vector** — is the smoothest non-trivial signal, and thresholding it gives a low-conductance cut.

Spectral clustering in a dozen lines

```

1 import numpy as np
2 from scipy.sparse import csr_matrix, diags
3 from scipy.sparse.linalg import eigsh
4 from sklearn.cluster import KMeans
5

```

```

6 def spectral_clusters(A, k):
7     d = np.asarray(A.sum(axis=1)).ravel()
8     d_inv_sqrt = diags(1.0 / np.sqrt(np.maximum(d, 1e-12)))
9     L = diags(np.ones(A.shape[0])) - d_inv_sqrt @ A @ d_inv_sqrt
10    vals, vecs = eigsh(L, k=k, sigma=0, which="LM") # k smallest
11    eigenpairs
12    U = vecs / np.maximum(np.linalg.norm(vecs, axis=1, keepdims=True),
13                          1e-12)
14    return KMeans(n_clusters=k, n_init=10).fit_predict(U)

```

Spectral clustering finds non-convex clusters that k -means on raw coordinates cannot, because it clusters in a space where the *graph* geometry is Euclidean. Its cost is dominated by the eigendecomposition: Lanczos needs $\Theta(Ek)$ per iteration, so it is practical to millions of edges and not to billions.

KEY INSIGHT | Spectral methods, graph signal processing and GNNs are one story

L 's eigenvectors are a Fourier basis for signals on the graph: small eigenvalues are low frequencies (smooth), large ones are high frequencies (oscillating). Filtering a graph signal means reweighting its spectrum. Evaluating a polynomial filter $\sum_k \theta_k L^k x$ needs only sparse mat-vecs — no eigendecomposition. A graph convolution is exactly such a polynomial filter, usually of degree 1, and stacking L layers gives a degree- L filter. That is the derivation of the GCN from spectral graph theory, and it explains why deep GCNs over-smooth: a high-degree low-pass filter drives every vertex toward the same value.

Message passing and the neighbour explosion

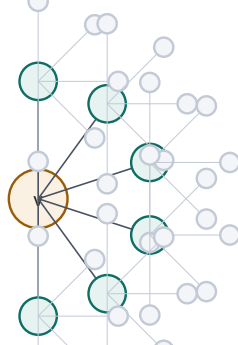
A graph neural network layer computes, for each vertex,

$$h_v^{(l+1)} = \sigma(W^{(l)} \cdot \text{aggregate}(\{h_u^{(l)} : u \in N(v)\} \cup \{h_v^{(l)}\}))$$

which in matrix form is $H^{(l+1)} = \sigma(\text{hat{A}} H^{(l)} W^{(l)})$: one sparse mat-mat product per layer, $\Theta(Ed + Vd^2)$.

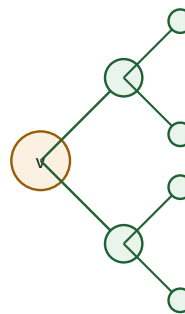
That is fine for full-graph training when the graph fits in memory. It is not fine for mini-batch training, because computing $h_v^{(L)}$ for a single vertex requires its entire L -hop neighbourhood.

FULL 2-HOP NEIGHBOURHOOD



average degree $d \rightarrow d^L$ nodes at depth L .
 $d = 100, L = 3$ is a million nodes per example.

SAMPLED: 2 NEIGHBOURS PER HOP



bounded fan-out $\rightarrow$ constant work
per example (GraphSAGE)

Figure 26.2 The neighbour explosion. With average degree d , an L -layer GNN touches d^L vertices per target. At $d = 100$ and $L = 3$ that is a million vertices — per example in the batch.

Three families of fixes:

Node sampling (GraphSAGE). Sample a fixed number of neighbours per hop: $s_1 \times s_2 \times \dots \times s_L$ vertices per target, bounded and tunable. Typical values are (25,10) for two layers. The estimator is biased for mean aggregation unless you reweight, but works well in practice.

Layer sampling (FastGCN, LADIES). Sample a fixed set of vertices *per layer* shared by the whole batch, with importance weights. Avoids the exponential blow-up entirely and gives lower variance than independent node sampling.

Subgraph sampling (Cluster-GCN, GraphSAINT). Partition the graph into dense clusters (with METIS) and train on whole clusters. Every edge inside the cluster is used with no sampling at all, so there is no neighbour explosion and the memory is predictable; the cost is that cross-cluster edges are dropped.

Neighbour sampling for a two-layer GNN

```

1  import numpy as np
2
3  def sample_block(indptr, indices, seeds, fanout, rng):
4      """One hop: for each seed, sample up to `fanout` neighbours."""
5      src, dst = [], []
6      for v in seeds:
7          nbrs = indices[indptr[v]:indptr[v + 1]]
8          if len(nbrs) > fanout:
9              nbrs = rng.choice(nbrs, fanout, replace=False)
10             src.append(nbrs)
11             dst.append(np.full(len(nbrs), v))
12     src = np.concatenate(src) if src else np.array([], dtype=np.int64)
13     dst = np.concatenate(dst) if dst else np.array([], dtype=np.int64)
14     frontier = np.unique(np.concatenate([src, np.asarray(seeds)]))
15     return frontier, src, dst
16
17 def make_minibatch(indptr, indices, targets, fanouts, rng):
18     blocks, seeds = [], np.asarray(targets)
19     for f in reversed(fanouts):          # build from the output
20         backwards
21         frontier, src, dst = sample_block(indptr, indices, seeds, f, rng)
22         blocks.append((src, dst, seeds))
23         seeds = frontier
24     return seeds, blocks[::-1]          # input frontier, then layers

```

COMMON PITFALL | Sampling changes what you compute

Neighbour sampling gives a *stochastic* estimate of the full-neighbourhood aggregation. With mean aggregation and uniform sampling the estimate is unbiased; with sum aggregation it is not, unless you rescale by $\deg(v)/s$. Attention-based aggregation (GAT) is biased under sampling because the softmax denominator is computed over the sample rather than the full neighbourhood. And at *inference* time you usually want the full neighbourhood, so a model trained with sampling and served without it sees a distribution shift. Decide deliberately, and evaluate the served configuration.

WHERE THIS SHOWS UP IN ML | A cost model for GNN training

For L layers, hidden dimension d , fanouts $s_1, \dots, s_L$ and batch size B :

- Vertices touched per batch: $B \prod_i s_i$ — this is the number that explodes.
- Feature-fetch bytes: $4d B \prod_i s_i$, usually the bottleneck, and it is a random gather from a feature table that may not fit in GPU memory.
- FLOPs: $\Theta(B d^2 \prod_i s_i)$.

The practical consequence is that GNN training is almost always *data-loading* bound, not compute bound, and the effective optimisations are feature caching for high-degree vertices, quantised feature storage, and historical embeddings (reuse a stale $h^{(l)}$ instead of recursing). Once again the algorithm's cost is determined by memory movement rather than arithmetic.

EXERCISES

1. Implement PageRank with and without dangling-node handling on a graph with sinks. Show that the vector stops summing to 1 without it.
2. Implement personalised PageRank two ways — power iteration and random-walk sampling — and compare the top-20 results and the runtimes on a graph with 10^6 edges.
3. Compute the Fiedler vector of a two-cluster graph and show that thresholding it at zero recovers the clusters. Then add noise edges and find the point at which it fails.
4. Verify $x^{\text{top}} L x = \sum_{(u,v)} (x_u - x_v)^2$ numerically, and use it to explain why low eigenvalues correspond to smooth signals.
5. Implement `make_minibatch` and measure the number of vertices touched for fanouts (25,10) versus full neighbourhoods on a graph with a power-law degree distribution.
6. Show empirically that sum aggregation under neighbour sampling is biased, and that rescaling by $\deg(v)/s$ removes the bias.
7. Implement Cluster-GCN sampling with a simple graph partitioner and compare memory and accuracy to neighbour sampling on the same model.
8. Reorder the vertices of a large graph by BFS order and measure the change in sparse mat-vec throughput. Explain it using Chapter 3.

CHAPTER RECAP

- Graph algorithms are sparse matrix operations; their cost is passes over the edge list and the random gathers those passes perform, so they are memory-bound.
- PageRank is the stationary distribution of a damped random walk, computed by power iteration in $\Theta(E)$ per step; handle dangling nodes or the mass leaks.
- Personalised PageRank measures proximity to a seed set and is a practical recommender and multi-hop retriever, often approximated by sampled walks.
- The graph Laplacian's small eigenvalues correspond to smooth signals; the Fiedler vector gives a good cut, and spectral clustering clusters in that embedding.
- A GCN layer is a degree-1 polynomial filter in the Laplacian; stacking layers raises the degree, which is why deep GCNs over-smooth.
- L -layer message passing touches d^L vertices per target; node, layer and subgraph sampling are the three fixes, each with different bias.
- GNN training is usually data-loading bound; feature caching and quantised storage matter more than kernel optimisation.

PART VI

Data Structures for Scale

When the data no longer fits — in cache, in RAM, on one machine — exactness becomes negotiable. These structures trade a little accuracy for orders of magnitude of capacity.

-
- 27 Segment Trees and Fenwick Trees
 - 28 Probabilistic Structures: Bloom, Count–Min, HyperLogLog
 - 29 MinHash, SimHash and LSH: Deduplicating a Training Corpus
 - 30 Suffix Structures, Substring Search and Contamination
 - 31 On-Disk Structures: B-Trees, LSM Trees and Feature Stores

Segment Trees and Fenwick Trees

Range queries on data that keeps changing

ADVANCED

PREREQS: Chapters 12, 13

A prefix-sum array answers range queries in constant time and forbids updates. A plain array allows updates and forces linear queries. Segment trees and Fenwick trees give you both in logarithmic time, and they are the structure behind weighted sampling with changing weights — which is exactly what prioritized experience replay needs.

WHAT YOU WILL LEARN

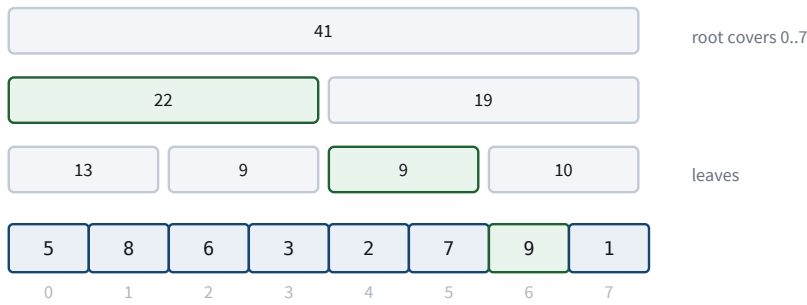
- Build a segment tree and implement range query and point update
- Implement a Fenwick tree and explain the lowest-set-bit arithmetic
- Choose between prefix sums, Fenwick trees and segment trees
- Add lazy propagation for range updates
- Implement $\Theta(\log n)$ weighted sampling with dynamic weights
- Build a prioritized replay buffer and understand its importance-sampling correction

The three-way tradeoff

Structure	Build	Range query	Point update	Range update	Memory
Plain array	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$	n
Prefix sums	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	n
Fenwick tree	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)^*$	n
Segment tree	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)^\dagger$	$2n-4n$
Sqrt decomposition	$\Theta(n)$	$\Theta(\sqrt{n})$	$\Theta(1)$	$\Theta(\sqrt{n})$	$n+\sqrt{n}$

Segment trees

query $\text{sum}(0..6) = 22 + 9 + 9$ — three nodes, not seven cells



Every node stores the aggregate of its range. Update a leaf $\rightarrow$ fix $\Theta(\log n)$ ancestors.

Figure 27.1 A segment tree. Each node stores the aggregate of a contiguous range; a query decomposes into $O(\log n)$ maximal nodes that tile it exactly.

Iterative segment tree: sum query, point update

```

1  class SegTree:
2      def __init__(self, data, op=lambda a, b: a + b, identity=0):
3          self.n = len(data)
4          self.op, self.id = op, identity
5          self.t = [identity] * (2 * self.n)
6          self.t[self.n:] = data
7          for i in range(self.n - 1, 0, -1):          # build bottom-up
8              self.t[i] = op(self.t[2 * i], self.t[2 * i + 1])
9
10         def update(self, i, value):                  # theta(log n)
11             i += self.n
12             self.t[i] = value
13             while i > 1:
14                 i >>= 1
15                 self.t[i] = self.op(self.t[2 * i], self.t[2 * i + 1])
16
17         def query(self, lo, hi):                      # [lo, hi), theta(
18             log n)
19             res_l, res_r = self.id, self.id
20             lo += self.n; hi += self.n
21             while lo < hi:
22                 if lo & 1:
23                     res_l = self.op(res_l, self.t[lo]); lo += 1
24                 if hi & 1:
25                     hi -= 1; res_r = self.op(self.t[hi], res_r)
26                 lo >>= 1; hi >>= 1
27             return self.op(res_l, res_r)

```

The iterative bottom-up form is shorter and roughly twice as fast as the recursive one, and it has a nice property: it works for *any associative operation*. Pass `min`, `max`, `gcd`, matrix multiplication, or “the maximum subarray sum of this range” (a four-field struct) and the structure is unchanged.

KEY INSIGHT | The requirement is associativity, not commutativity

Note that `query` keeps two accumulators, one from the left and one from the right, and combines them at the end. That is what makes the structure work for non-commutative operations such as matrix products and function composition. Get this wrong and it silently works for sums and fails for everything else.

Lazy propagation

To support *range* updates (“add 5 to every element in $[l, r]$ ”) without touching every leaf, store a pending update at each node and push it down only when a query needs to descend past it.

Lazy propagation: range add, range sum

```

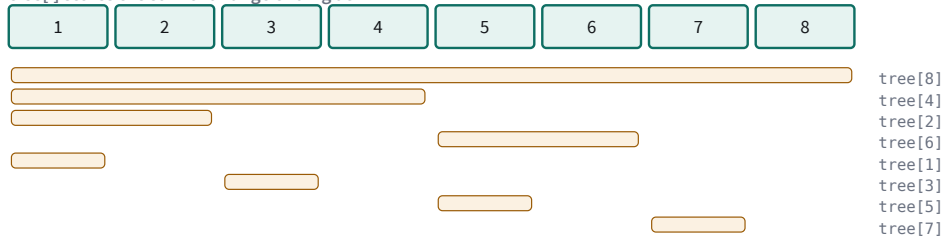
1  class LazySeg:
2      def __init__(self, n):
3          self.n = n
4          self.t = [0] * (4 * n)
5          self.lazy = [0] * (4 * n)
6
7      def _push(self, node, lo, hi):
8          if self.lazy[node]:
9              self.t[node] += self.lazy[node] * (hi - lo)
10             if hi - lo > 1:                # not a leaf: defer to
11                 children
12                 self.lazy[2 * node + 1] += self.lazy[node]
13                 self.lazy[2 * node + 2] += self.lazy[node]
14                 self.lazy[node] = 0
15
16      def update(self, l, r, val, node=0, lo=0, hi=None):
17          hi = self.n if hi is None else hi
18          self._push(node, lo, hi)
19          if r <= lo or hi <= l:
20              return
21          if l <= lo and hi <= r:
22              self.lazy[node] += val
23              self._push(node, lo, hi)
24              return
25          mid = (lo + hi) // 2
26          self.update(l, r, val, 2 * node + 1, lo, mid)
27          self.update(l, r, val, 2 * node + 2, mid, hi)
28          self.t[node] = self.t[2 * node + 1] + self.t[2 * node + 2]
29
30      def query(self, l, r, node=0, lo=0, hi=None):
31          hi = self.n if hi is None else hi
32          self._push(node, lo, hi)
33          if r <= lo or hi <= l:
34              return 0
35          if l <= lo and hi <= r:
36              return self.t[node]
37          mid = (lo + hi) // 2
38          return (self.query(l, r, 2 * node + 1, lo, mid)
39                  + self.query(l, r, 2 * node + 2, mid, hi))

```

Fenwick trees

A Fenwick tree (binary indexed tree) does less than a segment tree — prefix aggregates under an invertible operation — in half the memory and with a much smaller constant.

tree[i] stores the sum of a range ending at i



prefix_sum(i): i -= i & -i

update(i): i += i & -i

Both walk $\Theta(\log n)$ indices. One flat array, no pointers, half the memory of a segment tree.

Figure 27.2 The Fenwick tree's coverage pattern. `tree[i]` stores the aggregate of the range of length `i & -i` ending at `i`, so both query and update walk $O(\log n)$ indices by adding or removing the lowest set bit.

Fenwick tree, complete

```

1  class Fenwick:
2      def __init__(self, n):
3          self.n = n
4          self.t = [0.0] * (n + 1)           # 1-indexed
5
6      def update(self, i, delta):             # theta(log n)
7          i += 1
8          while i <= self.n:
9              self.t[i] += delta
10             i += i & -i                     # jump to the covering
11             parent
12
13     def prefix(self, i):                    # sum of [0, i), theta(log
14         n)
15         s = 0.0
16         while i > 0:
17             s += self.t[i]
18             i -= i & -i                     # strip the lowest set bit
19         return s
20
21     def range_sum(self, lo, hi):
22         return self.prefix(hi) - self.prefix(lo)
23
24     def find_prefix(self, target):
25         """Smallest i with prefix(i+1) > target. theta(log n), no binary
26         search over prefix() -- this walks the tree directly."""
27         pos, rest = 0, target
28         step = 1 << (self.n.bit_length())
29         while step:
30             nxt = pos + step
31             if nxt <= self.n and self.t[nxt] <= rest:
32                 pos = nxt
33                 rest -= self.t[nxt]
34             step >>= 1

```

```
33         return pos
```

`i & -i` isolates the lowest set bit: for `i = 12 = 0b1100` it gives `4 = 0b100`. Adding it moves to the next node that covers `i`; subtracting it moves to the previous disjoint block. Two lines of bit arithmetic replace an explicit tree.

KEY INSIGHT | `find_prefix` is the operation that matters for sampling

Searching for the smallest index whose prefix sum exceeds a target is the inverse-CDF sampling step. Doing it with `bisect` over `prefix()` costs $\Theta(\log^2 n)$; walking the tree as above costs $\Theta(\log n)$ and is the reason a Fenwick tree beats a sorted cumulative array when weights change.

Dynamic weighted sampling

Here is the problem this chapter exists for. You must sample an index with probability proportional to w_i , and the weights change after every sample.

Approach	Sample	Update	Notes
Linear scan	$\Theta(n)$	$\Theta(1)$	fine below a few thousand
Cumulative array + <code>searchsorted</code>	$\Theta(\log n)$	$\Theta(n)$	wrong choice if weights change
Alias table	$\Theta(1)$	$\Theta(n)$ rebuild	best for <i>static</i> weights (Chapter 34)
Fenwick / segment tree	$\Theta(\log n)$	$\Theta(\log n)$	the right answer when both happen

Table 27.1 ^{*}with a second tree, for range-add point-query. [†]with lazy propagation. If your data never changes, use prefix sums; if it changes constantly, use one of the trees.

Prioritized experience replay, the data-structure core

```
1  import numpy as np
2
3  class PrioritizedBuffer:
4      def __init__(self, capacity, alpha=0.6, eps=1e-6):
5          self.cap, self.alpha, self.eps = capacity, alpha, eps
6          self.sum_tree = Fenwick(capacity)      # sum of p_i^alpha
7          self.max_priority = 1.0
8          self.pos, self.size = 0, 0
9          self.data = [None] * capacity
10
11     def add(self, item, priority=None):
12         p = self.max_priority if priority is None else priority
13         i = self.pos
14         old = self.sum_tree.range_sum(i, i + 1)
15         self.sum_tree.update(i, (p + self.eps) ** self.alpha - old)
16         self.data[i] = item
17         self.pos = (i + 1) % self.cap           # ring buffer, Chapter 6
18         self.size = min(self.size + 1, self.cap)
19
20     def sample(self, batch, beta=0.4, rng=np.random):
```

```

21         total = self.sum_tree.prefix(self.cap)
22         idx, weights = [], []
23         for _ in range(batch):
24             u = rng.random() * total
25             i = self.sum_tree.find_prefix(u)          # theta(log n)
26             idx.append(i)
27             p_i = self.sum_tree.range_sum(i, i + 1) / total
28             weights.append((self.size * p_i) ** (-beta)) # IS correction
29         w = np.asarray(weights)
30         return idx, [self.data[i] for i in idx], w / w.max()
31
32     def update_priorities(self, idx, td_errors):
33         for i, err in zip(idx, td_errors):
34             p = (abs(err) + self.eps) ** self.alpha
35             old = self.sum_tree.range_sum(i, i + 1)
36             self.sum_tree.update(i, p - old)          # theta(log n)
37             self.max_priority = max(self.max_priority, abs(err) +
38                                     self.eps)

```

WHERE THIS SHOWS UP IN ML | Why the importance-sampling weight is not optional

Sampling transitions in proportion to p_i^α makes the gradient estimate *biased*: high-error transitions are over-represented relative to the distribution the loss is defined over. The correction is to weight each sampled transition's loss by $(1/Np_i)^\beta$, normalised by the maximum weight in the batch for stability. With $\beta=1$ the bias is fully removed; in practice β is annealed from about 0.4 to 1 over training, because early on the bias matters less than the variance reduction.

Omitting the correction is a common and silent bug: training still works, but it converges to a subtly different objective, and the effect looks like instability rather than like a bug.

COMMON PITFALL | Choosing α and the sampling distribution

$\alpha=0$ gives uniform sampling; $\alpha=1$ gives fully proportional sampling, which collapses onto a handful of transitions and starves the rest. The typical $\alpha \approx 0.6$ is a compromise nobody has improved on much. Also note that the *rank-based* variant — priority proportional to $1/\text{rank}$ — is more robust to outlier TD errors, at the cost of needing an order-maintaining structure instead of a sum tree.

WHERE THIS SHOWS UP IN ML | Other places these trees appear

Ranking and counting. “How many documents scored above x in the last hour?” with a stream of updates is a Fenwick tree over score buckets.

Online quantiles. A Fenwick tree over a fixed bucketisation gives approximate quantiles in $\Theta(\log B)$ per update and query — simpler than t-digest (Chapter 38) when the value range is known.

Sliding-window statistics. A segment tree over a window supports arbitrary associative aggregates with updates, where a monotonic deque (Chapter 6) only supports min and max.

Token budget accounting. Maintaining per-tenant usage with range queries in a multi-tenant serving system.

EXERCISES

1. Implement a segment tree for range minimum, and use it to answer 10^6 random range-min queries on an array of 10^6 elements. Compare to a sparse-table (which is $\Theta(1)$ per query but immutable).
2. Verify that your segment-tree query is correct for a non-commutative operation by using 2×2 matrix multiplication.
3. Implement a Fenwick tree and a prefix-sum array; measure query and update throughput for both at $n=10^6$, and find the update frequency at which the Fenwick tree wins.
4. Implement `find_prefix` with (a) `bisect` over `prefix()` and (b) the direct tree walk. Measure both at $n=10^6$ and confirm the $\log^2$ versus $\log$ difference.
5. Add lazy propagation for “assign a value to a range” (rather than “add”), and explain why the two compose differently.
6. Implement the full prioritized replay buffer and train a small DQN with and without the importance-sampling correction. Plot the learning curves.
7. Implement rank-based prioritized replay using a segment tree over rank buckets, and compare its robustness to an injected outlier TD error.

CHAPTER RECAP

- Prefix sums give $\Theta(1)$ queries and forbid updates; Fenwick and segment trees give $\Theta(\log n)$ for both.
- A segment tree works for any associative operation; keep separate left and right accumulators so non-commutative operations are correct.
- Lazy propagation defers range updates until a query forces them down, keeping range update and range query both $\Theta(\log n)$.
- A Fenwick tree is half the memory and a much smaller constant, for invertible prefix aggregates; `i & -i` is the whole trick.
- `find_prefix` walks the tree in $\Theta(\log n)$ and is the inverse-CDF sampling step — the reason to use a Fenwick tree for dynamic weighted sampling.
- Prioritized experience replay is a sum tree over priorities plus a ring buffer; the importance-sampling weight is required for an unbiased gradient.

Probabilistic Structures: Bloom, Count–Min, HyperLogLog

Trading a known, tunable error for orders of magnitude of memory

ADVANCED

PREREQS: Chapters 7, 21

When exact answers need more memory than you have, the question changes from “what is the answer?” to “how much error will I accept, and how little memory can I get away with?” Sketches answer that question with a tunable knob and a proof. They are the reason a single machine can deduplicate a trillion documents or count distinct users across a billion events.

WHAT YOU WILL LEARN

- Implement a Bloom filter and derive its optimal parameters
- Choose between Bloom, counting Bloom, cuckoo and quotient filters
- Implement a Count–Min sketch and state its error guarantee
- Implement HyperLogLog and explain why it needs only 12 KB
- Understand mergeability and why it matters for distributed aggregation
- Apply sketches to deduplication, heavy hitters and cardinality in real pipelines

Why sketches exist

An exact set of n 16-byte hashes needs $16n$ bytes plus hash-table overhead: about 40 GB for a billion items. A Bloom filter that answers membership with a 1% false-positive rate needs **10 bits per item** — 1.2 GB — regardless of how large the items are. That is a factor of 30, and it is the difference between a cluster and a laptop.

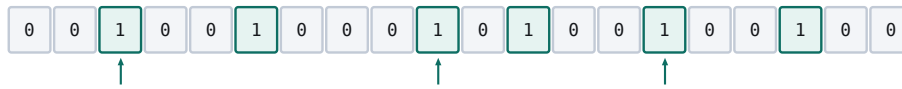
KEY INSIGHT | The three questions sketches answer

Membership: have I seen x ? (Bloom, cuckoo, quotient filters.) **Frequency:** how many times have I seen x ? (Count–Min, Count-Sketch.) **Cardinality:** how many *distinct* items have I seen? (HyperLogLog.)

Each gives up exactness in a specific, bounded direction, and each is *mergeable*: sketches computed on different shards combine into the sketch of the union. That property is what makes them work in a distributed pipeline.

Bloom filters

BIT ARRAY (m bits)



insert('cat'): set bits h_1, h_2, h_3

query('dog'): any 0 bit $\rightarrow$ definitely absent

query('emu'): all bits set $\rightarrow$ probably present (may be a false positive)

false positive rate $\approx (1 - e^{(-kn/m)})^k$, minimised at $k = (m/n) \ln 2$

$\rightarrow$ about 10 bits per element for a 1% error rate, whatever the element size

Figure 28.1 A Bloom filter. Insertion sets k bits; a query that finds any zero bit proves absence. False positives are possible; false negatives are not.

A Bloom filter with double hashing

```

1  import math
2  from bitarray import bitarray      # or use an int and bit operations
3
4  class BloomFilter:
5      def __init__(self, n, fp_rate=0.01):
6          self.m = max(8, int(math.ceil(-n * math.log(fp_rate) / math.log(
7              2) ** 2)))
8          self.k = max(1, int(round(self.m / n * math.log(2))))
9          self.bits = bitarray(self.m)
10         self.bits.setall(False)
11         self.count = 0
12
13     def _indices(self, item):
14         h = hash(item)                # use blake2b/xxhash if
15         persisted                     # persisted
16         h1, h2 = h & 0xFFFFFFFF, (h >> 32) | 1
17         for i in range(self.k):      # Kirsch-Mitzenmacher: two
18             hashes                    # hashes
19             yield (h1 + i * h2) % self.m  # simulate k independent ones
20
21     def add(self, item):
22         for i in self._indices(item):
23             self.bits[i] = True
24             self.count += 1
25
26     def __contains__(self, item):
27         return all(self.bits[i] for i in self._indices(item))

```

THE MATH | Deriving the optimal k and m

After inserting n items with k hashes into m bits, the probability a given bit is still zero is $(1-1/m)^{kn} \approx e^{-kn/m}$. A false positive requires all k bits to be set:

$$p \approx (1 - e^{-kn/m})^k$$

Differentiating with respect to k gives the optimum $k^* = m/n \ln 2 \approx 0.693 m/n$, at which $p = 2^{-k^*}$, equivalently

$$m = -n \ln p / (\ln 2)^2 \approx 1.44 n \log_2(1/p)$$

So **9.6 bits per item for 1%, 14.4 for 0.1%, 19.2 for 0.01%** — each additional decimal place of accuracy costs 4.8 bits per item. And note what is *absent* from the formula: the size of the items.

The Kirsch–Mitzenmacher trick in `_indices` is worth knowing: k independent hash functions can be simulated by $h_1 + i \cdot h_2$ for $i=0 \dots k-1$ with no measurable loss in the false-positive rate. One hash computation instead of k .

Filter	Bits/item at 1%	Deletes	Lookup	Notes
Bloom	9.6	no	k random probes	the baseline
Counting Bloom	38	yes	k probes	4-bit counters instead of bits
Cuckoo	9.0	yes	2 probes	better locality, can fail to insert
Quotient	10–12	yes	1 probe	cache-friendly, resizable, mergeable
Blocked Bloom	10.5	no	1 cache line	all k bits in one 64-byte block

WHERE THIS SHOWS UP IN ML | Bloom filters in the ML stack

Corpus deduplication. Stream documents, hash, query the filter, insert. A 1% false-positive rate discards 1% of *unique* documents, which for a web-scale corpus is entirely acceptable and saves 30x the memory.

Training–test contamination checks. Put every test-set n -gram in a filter and stream the training corpus past it; any hit is investigated exactly. The filter is a cheap pre-filter for an expensive exact check.

LSM-tree reads. Every SSTable carries a Bloom filter so a lookup can skip files that certainly do not contain the key. This is the single most important optimisation in an LSM store (Chapter 31).

Negative caching. “This embedding is definitely not in the cache” avoids a network round trip. False positives merely cost a wasted lookup.

COMMON PITFALL | Bloom filters have one-sided error — know which side

A Bloom filter never says “absent” about something present. It sometimes says “present” about something absent. So it is safe for *skipping work* and unsafe for *asserting existence*.

Using one to decide “this user already consented” is a correctness bug; using one to decide “this SSTable might contain the key, go and check” is correct by construction. Always ask which direction an error would take you.

Count–Min sketch

Bloom answers yes/no. Count–Min answers “how many”, using d rows of w counters and d independent hash functions.

h1	3	0	7	1	0	4	2	0	5	1
h2	0	6	2	0	9	1	3	0	0	4
h3	5	1	0	3	2	0	8	1	0	6
h4	2	0	4	0	1	7	0	3	5	0

estimate = min(7, 9, 8, 7) = 7

never underestimates;
overestimates with prob. δ

$w = \lceil e/\epsilon \rceil$, $d = \lceil \ln(1/\delta) \rceil$ gives error $\leq \epsilon N$ with probability $1 - \delta$, in $\Theta(wd)$ counters — independent of the number of distinct keys.

Figure 28.2 A Count–Min sketch. Each item increments one counter per row; the estimate is the minimum across rows, because collisions can only ever inflate a counter.

Count–Min sketch

```

1  import numpy as np
2
3  class CountMin:
4      def __init__(self, epsilon=0.001, delta=1e-4, seed=0):
5          self.w = int(np.ceil(np.e / epsilon))      # error <= eps * total
6          self.d = int(np.ceil(np.log(1 / delta)))    # with prob. 1 - delta
7          self.table = np.zeros((self.d, self.w), dtype=np.int64)
8          rng = np.random.default_rng(seed)
9          self.seeds = rng.integers(1, 2 ** 31, size=self.d)
10
11     def _cols(self, key):
12         h = hash(key)
13         return [(h ^ int(s)) % self.w for s in self.seeds]
14
15     def add(self, key, count=1):
16         for r, c in enumerate(self._cols(key)):
17             self.table[r, c] += count
18
19     def estimate(self, key):
20         return min(self.table[r, c] for r, c in enumerate(self._cols(
21             key)))
22
23     def merge(self, other):                # mergeable: just add
24         self.table += other.table
25         return self

```

THEOREM | Count–Min error bound

With $w = \lceil e/\epsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$, the estimate $\hat{f}_x$ satisfies $f_x \leq \hat{f}_x$ always, and $\hat{f}_x \leq f_x + \epsilon N$ with probability at least $1 - \delta$, where N is the total count of all items.

The error is relative to the *total* stream, not to the item’s own count. That is why Count–Min is excellent for heavy hitters — items whose counts are a significant fraction of N — and poor for rare items, whose true count may be dwarfed by ϵN . If you need accurate estimates of small counts, use a Count-Sketch (which uses signed updates and takes a median, giving a two-sided but much tighter bound for small items) or keep exact counts for the heavy hitters found by the sketch.

WHERE THIS SHOWS UP IN ML | Count–Min in practice

Tracking the most frequent tokens, queries, user agents or feature values in a stream too large to count exactly; detecting hot keys in a sharded cache or a parameter server; estimating n -gram frequencies for a language model without storing the full table; monitoring per-tenant request rates. In each case the pattern is: sketch everything, and keep an exact heap of the top k discovered heavy hitters.

HyperLogLog

Counting *distinct* items exactly requires storing them. HyperLogLog estimates the count using a single insight: if you hash items uniformly, the maximum number of leading zeros you observe tells you roughly how many distinct items there were — seeing a hash beginning with 20 zeros suggests about 2^{20} distinct items.

HyperLogLog in forty lines

```

1  import math
2
3  class HyperLogLog:
4      def __init__(self, p=14):                # 2^14 = 16384 registers
5          self.p = p
6          self.m = 1 << p
7          self.registers = bytearray(self.m)    # 6 bits each would suffice
8          self.alpha = {4: 0.673, 5: 0.697, 6: 0.709}.get(
9              p, 0.7213 / (1 + 1.079 / self.m))
10
11     def add(self, item):
12         h = hash(item) & ((1 << 64) - 1)
13         idx = h >> (64 - self.p)                # first p bits pick a
14         register
15         rest = (h << self.p) & ((1 << 64) - 1) # remaining bits
16         rank = 1
17         while rest and not (rest >> 63):        # count leading zeros + 1
18             rest <<= 1
19             rank += 1
20         if rank > self.registers[idx]:
21             self.registers[idx] = rank
22
23     def count(self):
24         raw = self.alpha * self.m ** 2 / sum(2.0 ** -r for r in
25             self.registers)
26         zeros = self.registers.count(0)
27         if raw <= 2.5 * self.m and zeros:        # small-range correction
28             return int(round(self.m * math.log(self.m / zeros)))
29         return int(round(raw))

```

```

28
29     def merge(self, other):                                # mergeable: element-wise
30         max
31         self.registers = bytearray(max(a, b) for a, b in
32                                     zip(self.registers, other.registers))
33
34         return self

```

The construction: split the hash into a register index and a tail; store the maximum rank per register; and combine the registers with a *harmonic* mean (which suppresses the influence of outlier registers, the key improvement of HyperLogLog over LogLog). The standard error is $1.04/\sqrt{m}$, so $m=2^{14}$ registers at 6 bits each — **12 KB** — gives about 0.81% error for cardinalities from zero to 10^9 .

KEY INSIGHT | Mergeability is the property that makes sketches distributed

Bloom filters merge with OR. Count–Min merges with addition. HyperLogLog merges with element-wise maximum. In every case, merging sketches of disjoint shards yields exactly the sketch of the union — so you can compute a global answer by a tree reduction (Chapter 16), with no coordination and no data movement beyond the sketches themselves. This is why they are the native aggregation primitive in Spark, Druid, Redis and BigQuery.

COMMON PITFALL | The union of HyperLogLogs is free; the intersection is not

HLL merges give you $|A \cup B|$ exactly as well as it gives you either. To get $|A \cap B|$ you must use inclusion–exclusion, $|A| + |B| - |A \cup B|$, and the errors *add* while the answer may be small — so the relative error on the intersection can be enormous. For intersections use a different structure (MinHash, Chapter 29, whose whole point is estimating Jaccard similarity) or keep exact sets.

Sketch	Answers	Memory	Error	Merge
Bloom filter	membershi p	$1.44n \log_2(1/p)$ bits	one-sided, p	OR
Count–Min	frequency	$e/\epsilon \ln 1/\delta$ counters	$+\epsilon N$, one-sided	add
Count-Sketch	frequency	same	$\pm \epsilon n$	f\
HyperLogLo g	cardinality	m registers of 6 bits	$1.04/\sqrt{m}$	max
t-digest	quantiles	$\Theta(\delta)$ centroids	tight at the tails	merge centroids
MinHash	Jaccard	k hashes per set	$1/\sqrt{k}$	min

Table 28.1 The filter family. Blocked Bloom trades a slightly worse rate for a single cache miss per query — usually the right trade in a hot path (Chapter 3).

EXERCISES

1. Implement the Bloom filter and measure the empirical false-positive rate for $p \in \{0.1, 0.01, 0.001\}$ at $n=10^6$. Compare to the formula.

2. Show empirically that the Kirsch–Mitzenmacher double-hashing trick gives the same false-positive rate as k independent hashes.
3. Implement a blocked Bloom filter (all k bits within one 64-byte block) and measure the lookup throughput against the plain version. Quantify the accuracy cost.
4. Build a Count–Min sketch over a Zipf-distributed stream of 10^8 events. Measure the relative error for the top 10, top 1000 and tail items, and explain the pattern.
5. Implement the heavy-hitters pattern: a Count–Min sketch plus a size- k min-heap of candidates. Compare its top-100 to the exact answer.
6. Implement HyperLogLog and plot the relative error against true cardinality from 10^2 to 10^9 . Identify the range where the small-range correction matters.
7. Estimate $|A \cap B|$ by inclusion–exclusion on HLLs where $|A|=|B|=10^8$ and $|A \cap B|=10^4$. Report the relative error and explain it.
8. Design a memory budget: deduplicate 10 billion documents on a machine with 32 GB of RAM. State your structure, its parameters, its error rate, and what that error costs you.

CHAPTER RECAP

- Sketches trade a bounded, tunable error for memory that depends on the accuracy target rather than on the data size.
- Bloom filters cost $1.44n \log_2(1/p)$ bits with $k=(m/n)\ln 2$ hashes. Their error is one-sided: safe for skipping work, unsafe for asserting existence.
- Count–Min never underestimates and overestimates by at most ϵN with probability $1-\delta$; it is good for heavy hitters and poor for rare items.
- HyperLogLog estimates cardinality from the maximum leading-zero rank per register, with error $1.04/\sqrt{m}$ — about 0.8% in 12 KB for any cardinality up to 10^9 .
- Mergeability (OR, add, max) lets sketches be combined by tree reduction across shards with no coordination.
- Unions of HLLs are accurate; intersections by inclusion–exclusion are not.

MinHash, SimHash and LSH: Deduplicating a Training Corpus

Finding near-duplicates among ten billion documents

ADVANCED

PREREQS: Chapters 15, 28

Exact deduplication is a hash set. Near-duplicate detection — documents differing by a header, a licence block, or one paragraph — cannot use a hash set, because hashing destroys similarity by design. Locality-sensitive hashing is the repair: a family of hashes engineered so that similar inputs collide on purpose. This is the algorithm behind every serious corpus-cleaning pipeline.

WHAT YOU WILL LEARN

- Explain why near-duplicate detection is not a hash-set problem
- Implement shingling and MinHash, and prove the collision identity
- Tune the banding parameters to hit a target similarity threshold
- Implement SimHash for cosine similarity and know when to prefer it
- Build a complete corpus deduplication pipeline and estimate its cost
- Understand LSH as the ancestor of modern vector indexes

The problem

A web-scale corpus is 30–60% near-duplicate by document count. Duplicates waste training compute, inflate benchmark scores through contamination, and increase verbatim memorisation. Removing them measurably improves models per unit of compute.

Comparing all pairs of 10^{10} documents is 5×10^{19} comparisons — not “slow”, but impossible. We need a method that only compares documents that have a real chance of being similar.

DEFINITION | Locality-sensitive hash family

A family H of hash functions is (s_1, s_2, p_1, p_2) -sensitive for a similarity measure sim if, for h drawn at random from H :

- $\text{sim}(x, y) \geq s_1 \Rightarrow \Pr[h(x) = h(y)] \geq p_1$
- $\text{sim}(x, y) \leq s_2 \Rightarrow \Pr[h(x) = h(y)] \leq p_2$

with $s_1 > s_2$ and $p_1 > p_2$. In words: similar items collide often, dissimilar items collide rarely. An ordinary hash function has $p_1 = p_2 = 0$ for $x \neq y$, which is exactly what makes it useless here.

Step 1: shingling

Represent each document as a *set*, so that similarity becomes set similarity.

Shingling: a document becomes a set of hashed n-grams

```

1  import hashlib
2
3  def shingles(text, k=9, word_level=True):
4      """k-shingles as 64-bit hashes. k=5..10 words, or 8..12 characters."""
5      units = text.split() if word_level else list(text)
6      out = set()
7      for i in range(len(units) - k + 1):
8          piece = " ".join(units[i:i + k]) if word_level else "".join(
9              units[i:i + k])
10         out.add(int.from_bytes(
11             hashlib.blake2b(piece.encode("utf-8"), digest_size=8).digest(
12                 ),
13             "big"))
14     return out

```

The choice of k is a real decision. Too small and unrelated documents share shingles by chance (every English document contains “of the”); too large and a single word change destroys many shingles. Word-level $k=5$ to 9 is standard for prose.

DEFINITION | Jaccard similarity

$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$. Two documents with identical shingle sets have $J=1$; disjoint sets give $J=0$. Empirically $J > 0.8$ means near-duplicate for prose, $J > 0.5$ means substantial overlap.

Step 2: MinHash

Storing shingle sets is expensive — a 10 KB document has thousands of shingles. MinHash compresses each set to a fixed-length signature that preserves Jaccard similarity.

THEOREM | The MinHash identity

Let π be a random permutation of the universe, and define $h_\pi(A) = \min_{a \in A} \pi(a)$. Then

$$\Pr[h_\pi(A) = h_\pi(B)] = J(A, B)$$

PROOF

Consider the elements of $A \cup B$. Under a random permutation, each is equally likely to be the one with the smallest π -value. That minimum element yields $h_{\pi}(A) = h_{\pi}(B)$ exactly when it lies in $A \cap B$, and otherwise the minima differ. The probability of that event is $|A \cap B|/|A \cup B| = J(A, B)$.

So with K independent hash functions, the fraction of the K signature positions on which two documents agree is an unbiased estimate of J , with standard error $\sqrt{J(1-J)/K} \leq 1/(2\sqrt{K})$. $K=128$ gives about 4.4% error; $K=256$ gives 3%.

MinHash signatures, vectorised

```

1  import numpy as np
2
3  MERSENNE = (1 << 61) - 1
4
5  class MinHasher:
6      def __init__(self, num_perm=128, seed=0):
7          rng = np.random.default_rng(seed)
8          self.a = rng.integers(1, MERSENNE, size=num_perm, dtype=np.uint64)
9          self.b = rng.integers(0, MERSENNE, size=num_perm, dtype=np.uint64)
10         self.num_perm = num_perm
11
12         def signature(self, shingle_hashes):
13             if not shingle_hashes:
14                 return np.full(self.num_perm, MERSENNE, dtype=np.uint64)
15             x = np.fromiter(shingle_hashes, dtype=np.uint64)
16             # (a*x + b) mod p, for all permutations at once: (n, num_perm)
17             h = (np.outer(x, self.a) + self.b) % MERSENNE
18             return h.min(axis=0) # one min per
19                                permutation
20
21         @staticmethod
22         def estimate(sig_a, sig_b):
23             return float((sig_a == sig_b).mean())

```

Universal hashing $h(x) = (ax + b) \bmod p$ with a Mersenne prime is the standard substitute for a true random permutation: cheap, and provably good enough for this bound.

Step 3: banding into an LSH index

SIGNATURE (one column per hash function)

doc A	17	3	41	8	22	9	30	12
doc B	17	3	41	8	91	4	30	55
doc C	62	77	5	19	40	2	88	6

band 1
band 2
band 3
band 4

Two documents land in the same bucket if ANY band matches entirely.

$P(\text{candidate}) = 1 - (1 - s^r)^b$ — an S-curve whose threshold is about $(1/b)^{1/r}$.

Figure 29.1 MinHash signatures split into b bands of r rows. Two documents become candidates if any band matches exactly, which turns similarity search into hash-table lookups.

Split the K -element signature into b bands of r rows ($K=br$). Hash each band; two documents are *candidates* if they share a bucket in any band.

THE MATH | The S-curve

If two documents have Jaccard similarity s :

- one row agrees with probability s ;
- a whole band of r rows agrees with probability s^r ;
- no band agrees with probability $(1-s^r)^b$;
- so $\Pr[\text{candidate}] = 1 - (1-s^r)^b$.

This is an S-curve with its steep region near $s^* \approx (1/b)^{1/r}$. Choosing b and r is choosing your similarity threshold.

For $K=128$: $(b, r) = (16, 8)$ gives $s^* \approx 0.69$; $(32, 4)$ gives $s^* \approx 0.42$; $(8, 16)$ gives $s^* \approx 0.88$. Fewer, longer bands means a higher, sharper threshold.

The LSH index

```

1  from collections import defaultdict
2
3  class LSHIndex:
4      def __init__(self, num_perm=128, bands=16):
5          assert num_perm % bands == 0
6          self.bands, self.rows = bands, num_perm // bands
7          self.tables = [defaultdict(list) for _ in range(bands)]
8
9      def add(self, doc_id, signature):
10         for i, table in enumerate(self.tables):
11             band = signature[i * self.rows:(i + 1) * self.rows].tobytes()
12             table[band].append(doc_id)
13
14     def candidates(self, signature):
15         out = set()
16         for i, table in enumerate(self.tables):
17             band = signature[i * self.rows:(i + 1) * self.rows].tobytes()
18             out.update(table.get(band, ()))
19         return out

```

Step 4: clustering

Candidate pairs are verified (recompute the MinHash estimate, or the exact Jaccard if you kept the shingles) and then turned into groups with union-find (Chapter 15). Keep one representative per cluster.

The complete pipeline

```

1  def deduplicate(docs, num_perm=128, bands=16, threshold=0.8):
2      hasher = MinHasher(num_perm)
3      index = LSHIndex(num_perm, bands)
4      sigs = {}
5      pairs = []
6      for doc_id, text in docs:                                # one streaming
7          pass
8          sig = hasher.signature(shingles(text))
9          for other in index.candidates(sig):
10             if MinHasher.estimate(sig, sigs[other]) >= threshold:
11                 pairs.append((doc_id, other))
12             index.add(doc_id, sig)
13             sigs[doc_id] = sig
14         dsu = DSU(len(sigs))
15         for a, b in pairs:
16             dsu.union(a, b)
17         keep = {}
18         for doc_id in sigs:
19             keep.setdefault(dsu.find(doc_id), doc_id)        # one per cluster
20         return set(keep.values())

```

PERFORMANCE NOTE | Cost of deduplicating a billion documents

Signatures: 128×8 bytes = 1 KB per document, so 1 TB for 10^9 documents — too much for memory, fine on disk or sharded across a cluster. Use 4-byte hashes and $K=128$ to get 512 bytes each.

The LSH index holds b entries per document; with $b=16$ that is 1.6×10^{10} entries. The standard approach is to run one band at a time: sort all documents by their band hash (an external sort, Chapter 42), and emit candidate pairs from each equal run. That is 16 passes of an external sort and needs no random-access index at all — which is exactly how the published pipelines (C4, The Pile, RefinedWeb, FineWeb) do it.

Total: about 20 CPU-hours per billion documents for signatures, plus the sorts.

Comfortably feasible; the naive pairwise comparison is not.

SimHash: LSH for cosine similarity

MinHash targets Jaccard similarity on sets. For *vectors* and cosine similarity, the right family is random hyperplanes.

THEOREM | The random-hyperplane identity

For a random unit vector r , define $h_r(x) = \text{sign}(r \cdot x)$. Then

$$\Pr[h_r(x) = h_r(y)] = 1 - \theta(x, y) / \pi$$

where θ is the angle between x and y .

SimHash: 64 random hyperplanes, one 64-bit fingerprint

```

1  import numpy as np
2
3  def simhash(vectors, bits=64, seed=0):
4      rng = np.random.default_rng(seed)
5      R = rng.standard_normal((vectors.shape[1], bits)).astype(np.float32)
6      signs = (vectors @ R) > 0 # (n, bits) booleans
7      weights = (1 << np.arange(bits, dtype=np.uint64))
8      return (signs * weights).sum(axis=1) # pack into uint64
9
10 def hamming(a, b):
11     return bin(int(a) ^ int(b)).count("1")

```

Two documents are near-duplicates if their fingerprints differ in few bits. Finding all pairs within Hamming distance d of each other among billions of fingerprints is itself an indexing problem: the standard solution splits the 64 bits into $d+1$ blocks, since two fingerprints within distance d must agree exactly on at least one block — the pigeonhole principle, and the same banding idea as MinHash.

Method	Similarity	Signature	Best for
MinHash + LSH	Jaccard	K integers (512 B–1 KB)	documents as shingle sets
SimHash	cosine	64 bits	documents as tf-idf or embedding vectors
SimHash on embeddings	cosine	64–256 bits	semantic near-duplicates
Exact hash	equality	16 bytes	byte-identical duplicates
Suffix automaton	substring	index	long exact substrings, Chapter 30

WHERE THIS SHOWS UP IN ML | Deduplication decisions that matter more than the algorithm

Granularity. Document-level deduplication misses boilerplate repeated across otherwise different pages. Paragraph or line-level deduplication (as in RefinedWeb and FineWeb) removes far more redundancy but risks fragmenting documents; the usual compromise is to remove lines appearing in more than N documents.

Which copy to keep. Longest, highest-quality-score, or earliest-crawled. This matters more for final model quality than the threshold does.

Cross-split contamination. Run deduplication *between* training and evaluation sets as well as within training. Reporting the overlap you found is now standard practice in model cards.

Transitivity. Union-find merges $A-B$ and $B-C$ into one cluster even if A and C are unrelated (Chapter 15). At scale this produces occasional enormous clusters. Cap cluster size, or use connected components of the *mutual* top- k graph instead.

KEY INSIGHT | LSH is the ancestor of every vector index

The idea — hash so that near things collide, then search only within a bucket — is the direct ancestor of the inverted-file index (partition space, probe a few cells) in Chapter 33. LSH has been largely superseded for high-dimensional nearest-neighbour search by graph methods such as HNSW, because LSH needs many tables to achieve good recall and graph methods get there with less memory. But for *deduplication*, where you want all pairs above a threshold rather than the k nearest to a query, LSH is still the right tool and is not close to being displaced.

EXERCISES

1. Verify the MinHash identity empirically: generate random sets with known Jaccard similarity and check that the signature agreement matches, and that the error scales as $1/\sqrt{K}$.
2. Plot the S-curve $1-(1-s^r)^b$ for $(b,r) \in \{(16,8), (32,4), (8,16)\}$ and mark the thresholds. Choose (b,r) for a target threshold of 0.75 with $K=200$.
3. Implement the full pipeline and run it on 100,000 documents with injected near-duplicates. Report precision and recall against ground truth as a function of (b,r) .
4. Implement the band-sort variant that avoids holding the index in memory, and verify it produces the same candidate pairs.
5. Implement SimHash over tf-idf vectors and compare its precision/recall to MinHash at the same signature size in bytes.
6. Implement the pigeonhole block search for all fingerprint pairs within Hamming distance 3 among 10^6 64-bit fingerprints.
7. Demonstrate the transitive-merging problem: construct documents A, B, C with $J(A,B)=J(B,C)=0.85$ and $J(A,C)=0.3$, and show they end up in one cluster. Propose and implement a mitigation.

CHAPTER RECAP

- Ordinary hashing destroys similarity; LSH families are engineered so that similar inputs collide with high probability.
- Shingling turns documents into sets so that similarity becomes Jaccard.
- MinHash compresses a set to K numbers with $\Pr[\text{collision}]=J$, so signature agreement estimates Jaccard with error $\approx 1/(2\sqrt{K})$.
- Banding $K=br$ gives a candidate probability $1-(1-s^r)^b$, an S-curve with threshold $\approx (1/b)^{1/r}$; choosing b and r is choosing the threshold.
- Candidate pairs are verified and clustered with union-find; the band-sort variant scales to billions without an in-memory index.
- SimHash uses random hyperplanes for cosine similarity and gives a 64-bit fingerprint; pigeonhole blocking finds close pairs.
- The hard decisions are granularity, which copy to keep, cross-split contamination and transitive over-merging — not the hashing.

Suffix Structures, Substring Search and Contamination

Indexing every substring of a corpus at once

ADVANCED

PREREQS: Chapters 9, 14

A trie indexes a set of strings. A suffix array indexes every substring of one string — which turns out to be the right tool for finding long exact repeats in a training corpus, detecting benchmark contamination, and measuring how much a model could be memorising. This chapter builds the structures and the pipeline that uses them at terabyte scale.

WHAT YOU WILL LEARN

- Build a suffix array and an LCP array, and use them for substring search
- Implement KMP and understand its failure function as prefix knowledge
- Find the longest repeated substring and all long repeats in a corpus
- Build an exact substring-matching pipeline for contamination detection
- Compare suffix arrays, suffix automata and n -gram indexes
- Quantify memorisation and contamination in a training set

Exact substring search: KMP

Before the index, the single-pattern scan. Naive search is $\Theta(nm)$ because a mismatch throws away everything learned. Knuth–Morris–Pratt keeps it.

KMP: build the failure function, then scan without backtracking

```

1  def kmp_failure(pat):
2      """fail[i] = length of the longest proper prefix of pat[:i+1]
3          that is also a suffix of it."""
4      fail = [0] * len(pat)
5      k = 0
6      for i in range(1, len(pat)):
7          while k and pat[i] != pat[k]:
8              k = fail[k - 1]          # fall back within the
9              pattern
10             if pat[i] == pat[k]:
11                 k += 1
12             fail[i] = k
13     return fail

```

```

13
14 def kmp_search(text, pat):
15     if not pat:
16         return []
17     fail, hits, k = kmp_failure(pat), [], 0
18     for i, ch in enumerate(text):
19         while k and ch != pat[k]:
20             k = fail[k - 1]
21         if ch == pat[k]:
22             k += 1
23         if k == len(pat):
24             hits.append(i - k + 1)
25             k = fail[k - 1] # continue, allowing overlaps
26     return hits

```

$\Theta(n + m)$, and the text pointer never moves backwards — which means it works on a stream. The failure function is the same idea as Aho–Corasick’s failure links (Chapter 14): a mismatch tells you how far you may skip, because you know what you have already matched.

Suffix arrays

DEFINITION | Suffix array and LCP array

For a string S of length n , the **suffix array** SA is the permutation of $0 \dots n-1$ that lists the starting positions of all suffixes in lexicographic order. The **LCP array** gives, for each adjacent pair in SA , the length of their longest common prefix.

TEXT: b a n a n a \$

Suffixes are sorted, so every occurrence of a pattern is a contiguous run: two binary searches, $\Theta(m \log n)$.

$LCP[i]$ = shared prefix with the previous suffix — gives longest repeats for free.

i	suffix	SA[i]	LCP
0	\$	6	0
1	a\$	5	0
2	ana\$	3	1
3	anana\$	1	3
4	banana\$	0	0
5	na\$	4	0
6	nana\$	2	2

Figure 30.1 The suffix array of “banana\$”. Because the suffixes are sorted, every occurrence of a pattern occupies a contiguous block, findable by two binary searches.

Suffix array by prefix doubling: $O(n \log^2 n)$

```

1 def suffix_array(s):
2     n = len(s)
3     sa = list(range(n))
4     rank = [ord(c) for c in s]
5     k = 1
6     while True:
7         key = lambda i: (rank[i], rank[i + k] if i + k < n else -1)
8         sa.sort(key=key) # theta(n log n) per round

```

```

9      new_rank = [0] * n
10     for i in range(1, n):
11         new_rank[sa[i]] = new_rank[sa[i - 1]] + (key(sa[i]) != key(
12             sa[i - 1]))
13     rank = new_rank
14     if rank[sa[-1]] == n - 1:           # all ranks distinct: done
15         return sa
16     k <= 1                               # compare twice as far next
17     time
18
19 def lcp_array(s, sa):
20     """Kasai's algorithm: theta(n)."""
21     n = len(s)
22     rank = [0] * n
23     for i, p in enumerate(sa):
24         rank[p] = i
25     lcp, h = [0] * n, 0
26     for i in range(n):
27         if rank[i] > 0:
28             j = sa[rank[i] - 1]
29             while i + h < n and j + h < n and s[i + h] == s[j + h]:
30                 h += 1
31             lcp[rank[i]] = h
32             if h:
33                 h -= 1           # amortised: h drops by at
34                                 most 1
35         else:
36             h = 0
37     return lcp

```

Prefix doubling is $\Theta(n \log^2 n)$ with a comparison sort, or $\Theta(n \log n)$ with radix sort. Linear-time constructions exist (SA-IS, DC3) and are what production libraries use; for corpus work, `libdivsufsort` or SA-IS on a memory-mapped file is the standard choice.

Kasai's algorithm for LCP is a small gem: the amortised argument is that h increases at most n times in total and decreases by at most one per iteration, so the whole thing is linear.

Query	With suffix array	Cost
Does P occur?	binary search for the block	$\Theta(m \log n)$
How many times?	size of the block	$\Theta(m \log n)$
All occurrences	the block's entries	$\Theta(m \log n + occ)$
Longest repeated substring	$\max(\text{LCP})$	$\Theta(n)$ after construction
All repeats of length $\geq L$	runs where $\text{LCP} \geq L$	$\Theta(n)$
Longest common substring of two texts	concatenate with separators, scan LCP	$\Theta(n)$

Contamination and memorisation

This is the application that matters for ML, and it is a direct use of the table above.

WHERE THIS SHOWS UP IN ML | Detecting benchmark contamination at corpus scale

The question: does any test-set example appear verbatim in the training corpus? At 10^{12} tokens of training text and 10^5 test examples, you cannot run 10^5 separate searches.

The standard pipeline:

1. **Concatenate** the training corpus with a unique separator byte and build a suffix array over it. For 1 TB with 40-bit indices this is about 5 TB on disk — built in shards and merged.
2. For each test example, **binary search** the suffix array for every 50-token window. Any hit is a candidate contamination.
3. **Verify and measure**: report the fraction of each test example covered by matches of length $\geq L$ (typically 50 tokens).

The cheaper approximation that most pipelines actually run first: build a Bloom filter (Chapter 28) or a hash set of all n -gram hashes of the test sets, stream the training corpus past it, and exact-check the hits. That is one pass, no index, and it answers the same question for a fixed n .

WHERE THIS SHOWS UP IN ML | Measuring memorisation

The same structure answers “how much of the training data can the model reproduce?”

Prompt the model with the first k tokens of a training document, sample a continuation, and check whether the continuation appears in the corpus by substring search. The fraction of prompts for which a 50-token verbatim match occurs is the standard extractable-memorisation metric.

A related corpus-only question — *how much of the corpus is exactly duplicated?* — is answered by scanning the LCP array for runs of long common prefixes, which is what the “deduplicating training data makes language models better” line of work does. Note that this is an *exact* substring method and is complementary to the *approximate* similarity of Chapter 29: LSH finds documents that are 85% similar, suffix structures find the 50-token spans they share.

Suffix automata and the alternatives

A **suffix automaton** (DAWG) is the minimal deterministic automaton recognising all suffixes of S . It has at most $2n-1$ states and $3n-4$ transitions, is built online in $\Theta(n)$, and answers “is P a substring?” in $\Theta(m)$ with no binary search — but it is a pointer structure with poor locality, so at corpus scale suffix arrays usually win despite the log factor.

Structure	Build	Query	Memory	Notes
Suffix array	$\Theta(n)$ (SA-IS)	$\Theta(m \log n)$	$4-5n$ bytes	the practical default
SA + LCP interval tree	$\Theta(n)$	$\Theta(m)$	$\approx 9n$ bytes	when query time dominates

Structure	Build	Query	Memory	Notes
Suffix tree	$\Theta(n)$	$\Theta(m)$	20–40n bytes	rarely worth the memory
Suffix automaton	$\Theta(n)$	$\Theta(m)$	$\approx 10n$ bytes	online construction
FM-index (BWT)	$\Theta(n)$	$\Theta(m)$	$\approx nH_k$ bytes	compressed; genomics standard
n -gram hash index	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$	only for fixed n

Table 30.1 What a suffix array answers. Adding an LCP-interval tree brings pattern search down to $\Theta(m + \log n)$, and an FM-index compresses the whole structure to near the entropy of the text.

COMMON PITFALL | Tokens or bytes?

Contamination measured over *characters* and over *tokens* give different answers, and neither is obviously right. Character-level matching catches paraphrase-free copies with different tokenisation; token-level matching aligns with what the model actually saw. Report which you used — published contamination numbers are not comparable without it. Normalise first (Chapter 5), or trivial whitespace differences will hide real matches.

EXERCISES

1. Implement KMP and verify the failure function on “ababcbab”. Explain what `fail[7]` means in terms of the string.
2. Build the suffix array for a 1 MB text by prefix doubling and by `sorted(range(n), key=lambda i: s[i:])`. Explain the memory blow-up of the second and measure it.
3. Implement Kasai’s algorithm and prove the amortised bound.
4. Use the LCP array to find the longest repeated substring of a book-length text, and all repeats of length ≥ 100 .
5. Find the longest common substring of two documents by concatenating them with a separator and scanning the LCP array with a bitmask of which document each suffix came from.
6. Build an n -gram hash index for $n=13$ over 100 MB of text and use it to find all test-set windows present in it. Compare its memory and query time to a suffix array.
7. Implement the contamination pipeline end-to-end on a small corpus with deliberately injected test examples, and report the coverage metric.
8. Estimate the disk, memory and wall-clock cost of building a suffix array over 1 TB of tokenised text, given a machine with 256 GB of RAM.

CHAPTER RECAP

- KMP scans in $\Theta(n+m)$ without backtracking; its failure function is the same “reuse what you already matched” idea as Aho–Corasick.
- A suffix array sorts all suffixes, so every occurrence of a pattern is a contiguous block found by two binary searches: $\Theta(m \log n)$.
- Kasai’s algorithm builds the LCP array in linear time, and the LCP array gives longest repeats, all long repeats, and longest common substrings.
- Contamination detection and memorisation measurement are substring-search problems; a Bloom filter over fixed-length n -grams is the cheap first pass, a suffix array the general one.
- Suffix arrays beat suffix trees and automata in practice on memory and locality; FM-indexes compress to near text entropy.
- Exact substring methods (this chapter) and approximate similarity (Chapter 29) answer different questions; a full pipeline uses both.

On-Disk Structures: B-Trees, LSM Trees and Feature Stores

When the data structure is larger than memory

ADVANCED

PREREQS: Chapters 12, 28

Every structure so far assumed the data fits in RAM. When it does not, the cost model changes: a random read costs 100 microseconds instead of 100 nanoseconds, and the block is the unit of everything. This chapter covers the two storage engines the world runs on, the amplification tradeoffs that distinguish them, and how they show up in feature stores and vector databases.

WHAT YOU WILL LEARN

- Reason with the external-memory cost model and the block size B
- Explain why B+ trees dominate read-heavy on-disk indexing
- Understand LSM trees, compaction, and the read/write/space amplification triangle
- Choose between B-tree and LSM storage for a given workload
- Understand skip lists and why memtables use them
- Map these structures onto feature stores, vector databases and checkpoint storage

The external-memory model

DEFINITION | The external-memory (I/O) model

Memory holds M items; disk is unbounded; data moves in blocks of B items. The cost of an algorithm is the number of block transfers. This is the right model whenever the data exceeds memory — and, with B reinterpreted as a cache line, it is also the right model for cache behaviour (Chapter 3).

The model immediately changes what is optimal:

Operation	RAM model	External-memory model
Scan n items	$\Theta(n)$	$\Theta(n/B)$

Operation	RAM model	External-memory model
Sort n items	$\Theta(n \log n)$	$\Theta(n/B \log_{M/B} n/B)$
Search in a sorted structure	$\Theta(\log_2 n)$	$\Theta(\log_B n)$
Binary search on disk	$\Theta(\log_2 n)$ I/Os	terrible: each probe is a block
B-tree search	—	$\Theta(\log_B n)$ I/Os

KEY INSIGHT | $\log_B$ versus $\log_2$ is the whole story

$\log_2(10^9)=30$. $\log_{200}(10^9)=3.9$. At 80 microseconds per random read, that is 2.4 ms versus 0.3 ms. Every on-disk structure in this chapter exists to make the base of that logarithm as large as possible, by matching the node size to the block size.

B+ trees

A B+ tree (Chapter 12) stores keys in internal nodes and *all* values in the leaves, with the leaves linked into a list.

Property	Consequence
Node = one page (4–16 KB)	branching factor 200–800
Height 3–4 at 10^9 keys	3–4 random reads per point lookup
All values in linked leaves	range scans are sequential
Updates in place	one random write per update
Node fill 50–70%	30–50% space overhead

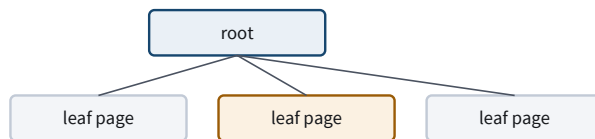
Table 31.1 The same problems, re-costed. Note the sorting bound: with $M=10^{10}$ bytes and $B=10^6$, $\log_{M/B}$ is about 1, so external sorting is essentially two passes over the data.

PERFORMANCE NOTE | Why the top levels are free

A B+ tree’s root and second level are tiny — for a 4 KB page size and 10^9 keys, the top two levels are a few megabytes — so they stay permanently in the buffer pool. The effective cost of a lookup is therefore one or two disk reads, not four. This is why “the index fits in memory, the data does not” is such a common and comfortable operating point.

LSM trees

An LSM (log-structured merge) tree accepts a very different tradeoff: never update in place, only append.

B-TREE — update in place

one random write per update
 reads: one page, $\Theta(\log_B n)$
 space amplification: low

LSM TREE — append, then compact

sequential writes only
 reads may touch every level
 → a Bloom filter per SSTable
 compaction merges downward

Write-heavy → LSM. Read-heavy with range scans → B-tree.

Figure 31.1 The two write paths. A B-tree does one random write per update. An LSM tree buffers in memory, flushes sequentially, and merges in the background — so all writes are sequential, at the cost of reads that may touch several levels.

Write path. New records go into an in-memory *memtable*. When it fills, it is written out as an immutable sorted file (an SSTable) — one sequential write. Background **compaction** merges SSTables, discarding superseded versions.

Read path. Check the memtable, then each level in order, newest first. Every SSTable has a Bloom filter (Chapter 28) so most levels can be skipped without any I/O, and a sparse index so the right block can be found with one read.

A minimal LSM structure, to fix the idea

```

1  import bisect
2
3  class MiniLSM:
4      def __init__(self, memtable_limit=1000):
5          self.memtable = {}                # a skip list in reality
6          self.levels = []                 # each: sorted list of (
7              k, v)
8          self.limit = memtable_limit
9
10         def put(self, key, value):
11             self.memtable[key] = value      # theta(1), no disk I/O
12             if len(self.memtable) >= self.limit:
13                 self._flush()
14
15         def delete(self, key):
16             self.memtable[key] = TOMBSTONE  # deletion is also a
17                 write
18
19         def _flush(self):
20             run = sorted(self.memtable.items()) # one sequential write
21             self.levels.insert(0, run)
22             self.memtable = {}
23             self._maybe_compact()
24
25         def get(self, key):
26             if key in self.memtable:         # newest first
27                 v = self.memtable[key]
  
```

```

26         return None if v is TOMBSTONE else v
27     for run in self.levels:
28         i = bisect.bisect_left(run, (key,))
29         if i < len(run) and run[i][0] == key:
30             v = run[i][1]
31             return None if v is TOMBSTONE else v
32     return None
33
34     def _maybe_compact(self):
35         while len(self.levels) > 4:          # merge the two oldest
36             a, b = self.levels.pop(), self.levels.pop()
37             merged, i, j = [], 0, 0
38             while i < len(a) or j < len(b):    # k-way merge, Chapter 13
39                 if j >= len(b) or (i < len(a) and a[i][0] <= b[j][0]):
40                     if not merged or merged[-1][0] != a[i][0]:
41                         merged.append(a[i])
42                     i += 1
43                 else:
44                     if not merged or merged[-1][0] != b[j][0]:
45                         merged.append(b[j])
46                     j += 1
47             self.levels.append(merged)
48
49     TOMBSTONE = object()

```

Two details that matter in real systems: **deletion is a write** (a tombstone that must survive until every older copy has been compacted away), and **compaction is where the CPU and I/O budget actually goes** — a write-heavy LSM store can spend more I/O on compaction than on the original writes.

The amplification triangle

DEFINITION | The three amplifications

Write amplification: bytes written to disk per byte of user data. A B-tree writes a whole page per update; a levelled LSM rewrites each record once per level, so $\approx L \times$ the fan-out ratio.

Read amplification: blocks read per lookup. B-tree: the tree height. LSM: potentially one per level, reduced to near 1 by Bloom filters.

Space amplification: bytes stored per byte of live data. B-tree: 1.4–2x from partial page fill. Levelled LSM: about 1.1x. Tiered LSM: 2–10x, because superseded versions linger.

You cannot minimise all three. Levelled compaction (RocksDB default, LevelDB) minimises space and read amplification at the cost of write amplification; tiered compaction (Cassandra, ScyllaDB) minimises write amplification at the cost of the other two.

Workload	Choose	Why
Read-heavy, range scans	B+ tree	low read amplification, sequential leaves
Write-heavy, point lookups	LSM (levelled)	sequential writes, Bloom-filtered reads

Workload	Choose	Why
Append-only time series	LSM (tiered)	writes dominate; old data rarely read
Small dataset, in memory	neither	use a hash map
Immutable after build	sorted file + sparse index	no update machinery needed at all

Table 31.2 The B+ tree’s profile. Its strength is predictable, low read amplification; its weakness is that every update is a random write.

Skip lists, and why memtables use them

A memtable must be sorted (so it can be flushed as an SSTable), support concurrent writes, and be fast. A balanced tree needs rebalancing with locks; a skip list does not.

DEFINITION | Skip list

A sorted linked list with additional “express lane” levels. An element is promoted to level $i+1$ with probability p (usually $1/2$ or $1/4$), so the expected height is $\Theta(\log n)$ and search, insert and delete are all $\Theta(\log n)$ expected.

Skip list search: the shape that makes it worth knowing

```

1  def search(head, key):
2      node = head
3      for level in range(len(head.forward) - 1, -1, -1):  # top lane down
4          while node.forward[level] and node.forward[level].key < key:
5              node = node.forward[level]                # skip ahead
6      node = node.forward[0]
7      return node if node and node.key == key else None

```

Skip lists are the memtable in RocksDB and LevelDB, the sorted set in Redis, and a common choice in lock-free concurrent structures — because an insert touches $\Theta(\log n)$ pointers and each can be published with a single atomic compare-and-swap, with no rotation and no global restructuring. They are the practical answer to “I need a balanced tree but I do not want to write one”.

WHERE THIS SHOWS UP IN ML | Where these structures appear in an ML system

Feature stores. The offline store is columnar files (Parquet) for training-time scans; the online store is a key-value store — usually LSM (RocksDB, Cassandra, DynamoDB) — for point lookups at serving time. The hard part is not the storage engine but *point-in-time correctness*: joining features as they were at the label’s timestamp, which is a range query on a (entity, timestamp) key and therefore wants sorted storage.

Vector databases. The vectors live in a flat memory-mapped file or an HNSW graph (Chapter 33); the *metadata and filters* live in a B-tree or LSM store; and the mapping from external id to internal offset is a hash index. A “vector database” is a vector index plus a conventional storage engine.

Checkpoints and datasets. Model checkpoints and shard files are write-once, read-many blobs: no index structure needed, just object storage and an offset table. WebDataset’s tar shards plus an index file are exactly the “sorted file + sparse index” row of the table above.

Embedding tables larger than memory. Billion-scale embedding tables in recommendation systems are stored in an LSM key-value store with an in-memory cache of hot ids — which is why cache policy (Chapter 6) matters so much to their throughput.

COMMON PITFALL | Random reads dominate; measure IOPS, not bandwidth

An NVMe drive advertising 7 GB/s delivers that on sequential reads. Random 4 KB reads give perhaps 1 million IOPS, which is 4 GB/s only if fully queued — and a single-threaded synchronous read loop will see 10–20 thousand IOPS, two orders of magnitude worse. If your structure does random reads, you must issue them concurrently (io_uring, async I/O, or many threads). This is the dominant consideration for disk-resident vector indexes such as DiskANN (Chapter 33), whose entire design is about minimising the *number* of random reads per query.

EXERCISES

1. Compute the height of a B+ tree over 10^{10} keys with 8-byte keys and 16 KB pages. How many of its levels fit in 1 GB of buffer pool?
2. Implement external merge sort for a file of 10^8 integers with a 100 MB memory budget. Count the passes and verify the $\log_{M/B}$ prediction.
3. Implement **MiniLSM** with Bloom filters per run and measure the reduction in blocks touched per lookup.
4. Compute the write amplification of a levelled LSM with 4 levels and a fan-out of 10. Compare with tiered compaction.
5. Implement a skip list with insert, delete and search. Measure the height distribution and confirm $\Theta(\log n)$ expected.
6. Benchmark random 4 KB reads on your machine, single-threaded synchronous versus 32-deep asynchronous. Report the IOPS ratio.
7. Design the storage layer for an online feature store serving 100,000 lookups per second over 500 GB of features with a 20 ms p99 budget. State the engine, the cache, and the expected hit rate you would need.

CHAPTER RECAP

- In the external-memory model cost is block transfers; $\log_B n$ replaces $\log_2 n$, which is why node size should match page size.
- B+ trees give 3–4 reads per lookup, sequential range scans, and one random write per update; their top levels stay cached.
- LSM trees make all writes sequential by buffering in a memtable and flushing immutable sorted runs, at the cost of reads that may touch several levels — mitigated by a Bloom filter per SSTable.
- Write, read and space amplification form a triangle; levelled compaction favours space and reads, tiered favours writes.
- Skip lists give $\Theta(\log n)$ expected operations with no rebalancing, which makes them the standard concurrent memtable.
- Feature stores, vector databases and embedding tables are all built on these engines; write-once artefacts need none of them.
- Random-read throughput is limited by IOPS and queue depth, not bandwidth.

PART VII

Algorithms at the Heart of Modern ML

The algorithms that make embeddings searchable, samplers fast, decoders cheap and tree models accurate. This is the part no general algorithms textbook contains.

-
- 32 Nearest-Neighbour Search and the Curse of Dimensionality
 - 33 Vector Search in Production: IVF, PQ, HNSW, DiskANN
 - 34 Sampling Algorithms
 - 35 Decoding Algorithms
 - 36 KV Caches as Data Structures
 - 37 Tree Learners: Decision Trees, Histograms and GBDT
 - 38 Streaming and Online Algorithms

Nearest-Neighbour Search and the Curse of Dimensionality

Why the obvious data structures stop working above twenty dimensions

ADVANCED

PREREQS: Chapters 12, 21

Nearest-neighbour search is the operation underneath retrieval, recommendation, clustering and deduplication. In two dimensions it is a solved problem with a beautiful data structure. In seven hundred and sixty-eight dimensions — which is where your embeddings live — that data structure is slower than a linear scan, for reasons worth understanding precisely.

WHAT YOU WILL LEARN

- State the exact and approximate nearest-neighbour problems and their cost
- Implement a k-d tree and understand its pruning rule
- Explain the curse of dimensionality quantitatively, not just as a slogan
- Know when brute force is genuinely the right answer
- Understand distance metrics and their normalisation traps
- Set up the approximate methods that Chapter 33 develops

The problem and the honest baseline

DEFINITION | Nearest-neighbour search

Given a set X of n points in $\mathbb{R}^d$ and a query q , find the k points of X minimising a distance $D(q, \cdot)$. The **approximate** version asks only for points within a factor $(1+\epsilon)$ of the true nearest distance, or — more usefully in practice — for a set whose *recall* against the true top- k exceeds a target.

The baseline is brute force, and it is much better than people expect, because it is one matrix multiply.

Brute-force top-k: the baseline you must beat

```
1 import numpy as np
2
3 def brute_force_topk(X, Q, k=10, normalised=True):
```

```

4      """X: (n, d) database. Q: (m, d) queries. Returns (m, k) indices."""
5      if normalised:                                # cosine == inner product
6          scores = Q @ X.T                            # (m, n) -- one GEMM
7      else:                                          # squared L2 without
8          materialising diffs
9          scores = -(np.einsum("ij,ij->i", Q, Q)[: , None]
10                     - 2 * Q @ X.T
11                     + np.einsum("ij,ij->i", X, X)[None, :])
12      idx = np.argpartition(-scores, k, axis=1)[: , :k] # theta(nm)
13      rows = np.arange(len(Q))[: , None]
14      return idx[rows, np.argsort(-scores[rows, idx], axis=1)]

```

PERFORMANCE NOTE | When brute force is the right answer

Cost is $\Theta(nmd)$ FLOPs, and a modern CPU does about 10^{11} FLOP/s with a good BLAS; a GPU does 10^{14} .

- $n=10^5$, $d=768$, one query: 77 MFLOP, about 1 ms on CPU. **Use brute force.**
- $n=10^6$, $d=768$, one query: 770 MFLOP, about 8 ms on CPU, 0.1 ms on GPU. **Still fine**, especially batched.
- $n=10^8$, $d=768$: 77 GFLOP per query, and 300 GB of memory. **Now you need an index.**

The memory limit usually bites before the compute limit. Note also that the identity in the code — expanding $|q-x|^2$ — is what turns a distance computation into a GEMM; computing differences explicitly would allocate an (m, n, d) tensor and exhaust memory (Chapter 4).

k-d trees

A k-d tree partitions space by axis-aligned splits: at depth ell , split on axis $ell \bmod d$ at the median.

k-d tree with the pruning rule that makes it work

```

1  import numpy as np
2
3  class KDTree:
4      def __init__(self, points, indices=None, depth=0, leaf_size=16):
5          indices = np.arange(len(points)) if indices is None else indices
6          self.axis = depth % points.shape[1]
7          self.leaf = len(indices) <= leaf_size
8          if self.leaf:
9              self.indices = indices
10             return
11         order = indices[np.argsort(points[indices, self.axis])]
12         mid = len(order) // 2
13         self.split = points[order[mid], self.axis]
14         self.point_idx = order[mid]
15         self.left = KDTree(points, order[:mid], depth + 1, leaf_size)
16         self.right = KDTree(points, order[mid + 1:], depth + 1, leaf_size)
17
18     def query(self, points, q, k, heap=None):
19         import heapq
20         heap = [] if heap is None else heap

```

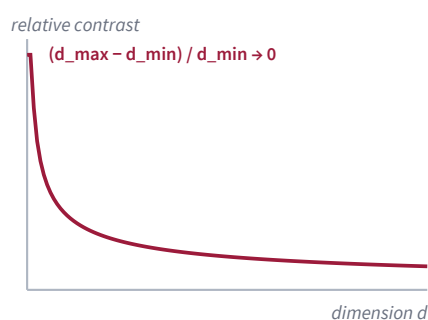
```

21         if self.leaf:
22             for i in self.indices:
23                 d = -np.sum((points[i] - q) ** 2)
24                 if len(heap) < k:
25                     heapq.heappush(heap, (d, int(i)))
26                 elif d > heap[0][0]:
27                     heapq.heapreplace(heap, (d, int(i)))
28             return heap
29         d = -np.sum((points[self.point_idx] - q) ** 2)
30         if len(heap) < k:
31             heapq.heappush(heap, (d, int(self.point_idx)))
32         elif d > heap[0][0]:
33             heapq.heapreplace(heap, (d, int(self.point_idx)))
34         diff = q[self.axis] - self.split
35         near, far = (self.left, self.right) if diff < 0 else (self.right,
36             self.left)
37         near.query(points, q, k, heap)
38         # THE PRUNING RULE: can anything on the far side beat the worst
39         # so far?
40         if len(heap) < k or diff ** 2 < -heap[0][0]:
41             far.query(points, q, k, heap)
42         return heap

```

The pruning test is the whole algorithm: if the squared distance from the query to the splitting *plane* already exceeds the distance to the current k -th best point, the entire far subtree can be skipped. In two dimensions this fires almost always and the query is $\Theta(\log n)$. In high dimensions it almost never fires.

The curse of dimensionality, quantitatively



d	fraction of a unit cube within 0.1 of its surface
2	36%
10	89%
50	99.5%
100	99.997%

Everything is near the boundary, so a tree's pruning test almost never fires: k -d trees degrade to a full scan above $d \approx 20$.

Figure 32.1 Two views of the same phenomenon. Left: the relative gap between the nearest and farthest neighbour shrinks as d grows. Right: in high dimensions nearly all of a cube's volume is near its surface, so a query's neighbourhood almost always straddles a split.

Three facts, each sufficient on its own to kill spatial trees.

1. Distance concentration. For i.i.d. random points, the ratio

$$\max_i |q - x_i| - \min_i |q - x_i| / \min_i |q - x_i| \rightarrow 0 \text{ as } d \rightarrow \infty$$

at rate roughly $1/\sqrt{d}$. At $d=1000$ the nearest and farthest points differ by a few percent, so “nearest” becomes a statement about noise. (Real embeddings are not i.i.d. — they lie on

a much lower-dimensional manifold — which is exactly why nearest-neighbour search still works at all.)

2. Volume concentrates at the surface. The fraction of a unit cube within ε of its boundary is $1 - (1 - 2\varepsilon)^d$. At $\varepsilon = 0.05$ and $d = 100$, that is 99.997%. A query ball therefore almost always crosses many splitting planes.

3. The pruning test fails. A k-d tree must examine both sides whenever the query ball crosses the split, and by fact 2 it nearly always does. The expected number of leaves examined grows roughly as 2^d until it saturates at “all of them”. Empirically, k-d trees beat a linear scan up to about $d = 20$ and lose beyond it — and they lose while also adding a pointer structure with terrible cache behaviour (Chapter 3).

Structure	Good to about	Fails because
k-d tree	$d \approx 20$	axis-aligned splits; pruning stops firing
Ball tree	$d \approx 30$	tighter bounds, but same concentration
Cover tree	intrinsic dimension	good when data lies on a low-dim manifold
VP tree	$d \approx 30$	metric-only; same limits
R-tree	$d \approx 10$	designed for spatial extents, not points
Brute force	any d	$\Theta(nd)$ always, but cache-perfect and vectorised
LSH / IVF / PQ / HNSW	any d	approximate, Chapter 33

KEY INSIGHT | The one-sentence version

There is no known exact nearest-neighbour structure that beats a linear scan in high dimensions, and there are theoretical reasons (hardness results based on the Strong Exponential Time Hypothesis) to believe none exists. So high-dimensional nearest-neighbour search is *necessarily* approximate — which is why every vector database exposes a recall knob.

Intrinsic dimension

The saving grace is that real data is not uniformly distributed in $\mathbb{R}^{768}$. Embeddings lie near a manifold of much lower *intrinsic* dimension — typically 10–50 for text and image embeddings — and all practical methods exploit that.

Estimating intrinsic dimension (two-NN estimator)

```

1 import numpy as np
2
3 def intrinsic_dimension(X, sample=5000, rng=np.random):
4     """Facco et al. two-NN estimator: robust and needs only the two
5     nearest neighbours of each sampled point."""
6     idx = rng.choice(len(X), size=min(sample, len(X)), replace=False)
7     S = X[idx]
8     d2 = ((S[:, None, :] - X[None, :, :]) ** 2).sum(-1)
```

```

9      d2.sort(axis=1)
10     r1, r2 = np.sqrt(d2[:, 1]), np.sqrt(d2[:, 2])    # skip self at index
        0
11     mu = r2 / np.maximum(r1, 1e-12)
12     mu = np.sort(mu)
13     F = np.arange(1, len(mu) + 1) / len(mu)
14     x = np.log(mu[:-1])
15     y = -np.log(1 - F[:-1])
16     return float(np.dot(x, y) / np.dot(x, x))        # slope through
        origin

```

If the estimate comes back near 10, an index will work very well; if it comes back near 100, expect to need many more probes for the same recall. It is worth measuring before choosing an index configuration.

Metrics and their traps

Metric	Formula	Notes
Euclidean (L2)	$\ q - x\ $	$\ q - x\ $
Squared L2	$\ q - x\ ^2$	$\ q - x\ ^2$
Cosine	$1 - \frac{q \cdot x}{\ q\ \ x\ }$	$\frac{q \cdot x}{\ q\ \ x\ }$
Inner product	$-q \cdot x$	not a metric: no triangle inequality
Manhattan (L1)	$\sum_i q_i - x_i $	$\sum_i q_i - x_i $
Hamming	bits differing	for binary codes

Table 32.1 Exact spatial indexes and where they stop. Above roughly 30 dimensions, the only options are brute force and approximation.

COMMON PITFALL | Maximum inner product search is not nearest-neighbour search

If you normalise vectors, cosine similarity and Euclidean distance induce the same ranking ($\|q - x\|^2 = 2 - 2q \cdot x$ for unit vectors), and every index works. If you do *not* normalise — because vector norm encodes something, as it does in some recommender models — then maximum inner product search (MIPS) is a genuinely different problem. It has no triangle inequality, so tree pruning and graph-based methods lose their correctness argument. The standard fix is a transformation that converts MIPS into ordinary nearest-neighbour search by appending a component that absorbs the norm. Check your index's documentation: many silently assume normalised vectors.

WHERE THIS SHOWS UP IN ML | Practical guidance

- **Normalise once, at write time.** Store unit vectors, use inner product, and never think about it again.
- **Use float32, or less.** float64 doubles bandwidth for no recall benefit (Chapter 3). float16 or int8 halves or quarters it again with a small, measurable recall cost.
- **Measure recall, not “approximate-ness”.** The useful metric is recall@ k against a brute-force ground truth on a sample of a few thousand queries. Compute it whenever you change any index parameter.
- **Rerank.** Retrieve 5–10x more candidates with the cheap index, then rescore them exactly with the full-precision vectors. This recovers almost all the recall lost to quantisation at a negligible cost, and it is the single most valuable trick in the area.

EXERCISES

1. Measure brute-force query time for $n \in \{10^4, 10^5, 10^6\}$ at $d=768$ on CPU and on GPU, batched and unbatched. Find your crossover into “needs an index”.
2. Implement the k-d tree and measure the fraction of leaves visited as d grows from 2 to 64 at fixed n . Plot it against a linear scan.
3. Verify distance concentration: for $d \in \{2, 10, 100, 1000\}$, sample 10^4 uniform points and plot the distribution of $(d_{\max} - d_{\min})/d_{\min}$.
4. Estimate the intrinsic dimension of a real embedding set (any sentence-transformer output) and compare it to the ambient dimension.
5. Show that for unit vectors, ranking by squared L2 and by inner product give identical orderings. Then construct non-unit vectors where they differ.
6. Implement rerank-after-retrieve on top of a quantised index and measure the recall gained as a function of the over-retrieval factor.
7. Show empirically that a k-d tree over 20-dimensional data lying on a 3-dimensional manifold performs like 3-dimensional data, not 20-dimensional.

CHAPTER RECAP

- Brute-force search is one GEMM and is the right answer up to roughly 10^6 vectors; memory usually binds before compute.
- k-d trees prune by comparing the distance to the splitting plane against the current k -th best; this test stops firing above about 20 dimensions.
- The curse of dimensionality is three concrete facts: distances concentrate, volume moves to the surface, and pruning therefore fails.
- No exact high-dimensional structure beats a linear scan, so practical methods are approximate and expose a recall knob.
- Real embeddings have low intrinsic dimension, which is why approximate methods work at all; measure it before tuning an index.
- Normalise at write time and use inner product; use squared L2 for ranking; MIPS on unnormalised vectors is a different problem.
- Over-retrieve and rerank with exact vectors: the cheapest large recall win available.

Vector Search in Production: IVF, PQ, HNSW, DiskANN

The algorithms inside every vector database

FRONTIER

PREREQS: Chapter 32

This is the chapter that explains what your vector database is actually doing. Four ideas — partition the space, compress the vectors, navigate a graph, and put it on disk — combine into every index you will meet. Understanding them turns a set of opaque parameters into a set of decisions you can reason about.

WHAT YOU WILL LEARN

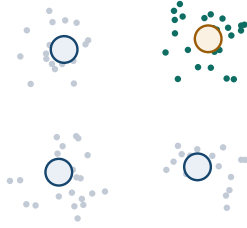
- Build and tune an IVF index and reason about `nlist` and `nprobe`
- Explain product quantisation, its memory arithmetic and its asymmetric distance computation
- Implement HNSW search and explain the role of M and `efSearch`
- Choose an index for a given corpus size, memory budget and recall target
- Understand filtered search, updates and deletion — the parts that are actually hard
- Reason about disk-resident indexes and the IOPS constraint

The four ideas

Idea	Mechanism	Buys you	Costs you
Partition	cluster the space, probe a few cells (IVF)	scan 1% of the data	recall at cell boundaries
Compress	quantise vectors to bytes (PQ, SQ, binary)	4–64x memory	precision
Navigate	greedy walk on a proximity graph (HNSW)	$\Theta(\log n)$ hops, high recall	memory, build time
Externalise	keep vectors on SSD, graph in RAM (DiskANN)	10–100x capacity	one random read per hop

IVF: partition the space

IVF — cluster into nlist cells

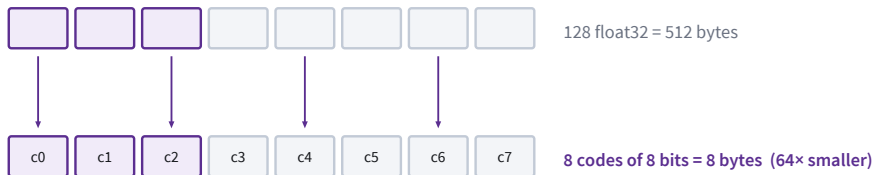


Build: k-means over a sample → nlist centroids;
assign every vector to its nearest cell.

Query: find the nprobe nearest centroids, then
scan only those cells.

$nlist = \sqrt{n}$, $nprobe = 8-32$ typically scans
about 1% of the database.

PQ — split $d = 128$ into $m = 8$ sub-vectors of 16 dims



Each sub-vector is replaced by the index of its nearest of 256 centroids. Distances come from a precomputed 8×256 lookup table: 8 table reads and 8 adds per candidate, with no multiplications at all.

Figure 33.1 Top: an inverted file index clusters the vectors and probes only the cells nearest the query. Bottom: product quantisation splits each vector into sub-vectors and replaces each with a one-byte codebook index.

An IVF index, in full

```

1  import numpy as np
2
3  class IVFIndex:
4      def __init__(self, nlist=1024, seed=0):
5          self.nlist, self.seed = nlist, seed
6
7      def train(self, X, iters=20):
8          rng = np.random.default_rng(self.seed)
9          self.centroids = X[rng.choice(len(X), self.nlist,
10             replace=False)].copy()
11          for _ in range(iters):
12              # Lloyd's algorithm
13              assign = self._assign(X)
14              for c in range(self.nlist):
15                  members = X[assign == c]
16                  if len(members):
17                      self.centroids[c] = members.mean(axis=0)
18          return self
19
20      def _assign(self, X, nprobe=1):
21          sims = X @ self.centroids.T
22          # unit vectors
23          assumed
24          if nprobe == 1:
25              return sims.argmax(axis=1)
26          return np.argpartition(-sims, nprobe, axis=1)[:nprobe]
27
28      def add(self, X, ids):
29          assign = self._assign(X)
30          self.lists = [[] for _ in range(self.nlist)]
31          self.vecs = [[] for _ in range(self.nlist)]
32          for i, c in enumerate(assign):

```

```

29         self.lists[c].append(ids[i])
30         self.vecs[c].append(X[i])
31         self.vecs = [np.asarray(v) if v else np.zeros((0, X.shape[1]),
32                 dtype=X.dtype) for v in self.vecs]
33         self.lists = [np.asarray(l, dtype=np.int64) for l in self.lists]
34         return self
35
36     def search(self, q, k=10, nprobe=8):
37         cells = np.argpartition(-(q @ self.centroids.T), nprobe)[:nprobe]
38         cand_ids, cand_scores = [], []
39         for c in cells:
40             if len(self.lists[c]):
41                 cand_ids.append(self.lists[c])
42                 cand_scores.append(self.vecs[c] @ q)
43         if not cand_ids:
44             return np.array([], dtype=np.int64)
45         ids = np.concatenate(cand_ids)
46         scores = np.concatenate(cand_scores)
47         top = np.argpartition(-scores, min(k, len(scores) - 1))[:k]
48         return ids[top[np.argsort(-scores[top])]]

```

Tuning. Set $nlist \approx \sqrt{n}$ (so each cell holds about $\sqrt{n}$ vectors) — 4096 for a million, 65536 for a hundred million. `nprobe` then trades recall for latency: 1 is fast and poor, 8–32 is the usual operating range, and `nprobe = nlist` degenerates to brute force.

COMMON PITFALL | The boundary problem

A vector near a cell boundary may be the true nearest neighbour of a query in an adjacent cell that you did not probe. This is the *only* source of recall loss in a plain IVF index, and it is why recall improves so smoothly with `nprobe`. Training the centroids on a representative sample (100–200 vectors per centroid is plenty) matters more than the number of k -means iterations.

Product quantisation: compress the vectors

An IVF index still stores every vector at full precision — 768 float32 values, 3 KB each, so 300 GB for 10^8 vectors. Product quantisation makes that 8 bytes.

DEFINITION | Product quantisation

Split each d -dimensional vector into m sub-vectors of d/m dimensions. Run k -means with 2^b centroids (usually $b=8$, so 256) independently on each sub-space. Represent a vector by the m codes of its nearest sub-centroids: mb bits total. The effective codebook size is $2^{mb} - 2^{64}$ for $m=8, b=8$ — from only $m \cdot 2^b = 2048$ stored centroids.

Product quantisation with asymmetric distance computation

```

1  import numpy as np
2
3  class PQ:
4      def __init__(self, m=8, nbits=8, seed=0):
5          self.m, self.k = m, 1 << nbits
6          self.seed = seed
7

```

```

8     def train(self, X, iters=25):
9         n, d = X.shape
10        self.dsub = d // self.m
11        rng = np.random.default_rng(self.seed)
12        self.codebooks = np.zeros((self.m, self.k, self.dsub),
13        dtype=np.float32)
14        for j in range(self.m):
15            sub = X[:, j * self.dsub:(j + 1) * self.dsub]
16            C = sub[rng.choice(n, self.k, replace=False)].copy()
17            for _ in range(iters):
18                a = np.argmin(((sub[:, None, :] - C[None, :, :]) **
19                2).sum(-1),
20                axis=1)
21                for c in range(self.k):
22                    members = sub[a == c]
23                    if len(members):
24                        C[c] = members.mean(axis=0)
25            self.codebooks[j] = C
26        return self
27
28    def encode(self, X):
29        codes = np.empty((len(X), self.m), dtype=np.uint8)
30        for j in range(self.m):
31            sub = X[:, j * self.dsub:(j + 1) * self.dsub]
32            d2 = ((sub[:, None, :] - self.codebooks[j][None]) ** 2).sum(
33            -1)
34            codes[:, j] = d2.argmin(axis=1)
35        return codes # (n, m) bytes
36
37    def distance_table(self, q):
38        """ADC: precompute q's distance to every sub-centroid. theta(
39        k*d)."""
40        return np.stack([
41            ((q[j * self.dsub:(j + 1) * self.dsub][None] -
42            self.codebooks[j]) ** 2).sum(-1)
43            for j in range(self.m)]) # (m, k)
44
45    def search(self, codes, q, k=10):
46        table = self.distance_table(q) # once per query
47        d2 = table[np.arange(self.m), codes.T].sum(axis=0) # m adds per
48        vector
49        top = np.argpartition(d2, k)[:k]
50        return top[np.argsort(d2[top])]

```

KEY INSIGHT | Asymmetric distance computation is the whole point

The query is *not* quantised. We build an $m \times 256$ table of distances from the query's sub-vectors to every sub-centroid, then each database vector's distance is m table lookups and m additions — 8 operations instead of 768 multiply-adds, and the data read per vector is 8 bytes instead of 3072. It is roughly 100x less memory traffic, which on memory-bound hardware (Chapter 3) is roughly a 100x speedup. Keeping the query unquantised is what makes the estimate accurate enough to be useful.

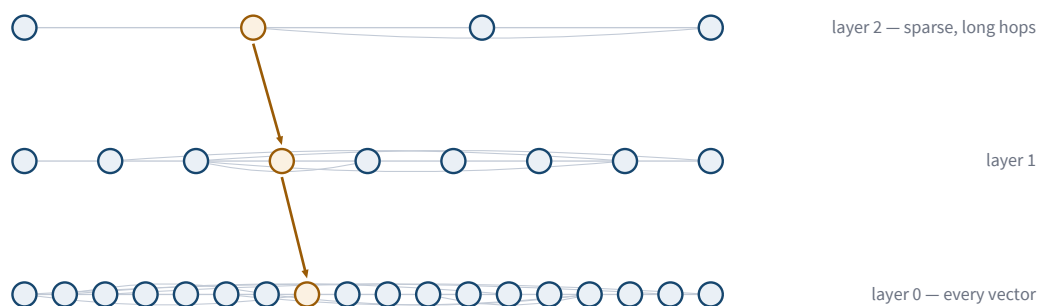
PERFORMANCE NOTE | The memory arithmetic that drives index choice

For 10^8 vectors of dimension 768:

Storage	Bytes/vector	Total	Recall@10 (typical)
float32	3072	307 GB	1.00 (exact)
float16	1536	154 GB	~1.00
int8 (scalar quant.)	768	77 GB	0.97–0.99
PQ, m=64, 8 bits	64	6.4 GB	0.90–0.95
PQ, m=16, 8 bits	16	1.6 GB	0.70–0.85
Binary (1 bit/dim)	96	9.6 GB	0.60–0.80 (rerank!)

Every row below float16 should be paired with reranking: retrieve 5k candidates with the compressed representation, then rescore those with full-precision vectors fetched from disk. Recall usually returns to within a point or two of exact.

HNSW: navigate a graph



Enter at the top, greedily descend to the closest node, drop a layer, repeat: $\Theta(\log n)$ hops.

Figure 33.2 HNSW. Higher layers are sparse and have long-range links for fast global movement; layer 0 contains every vector with short links for local refinement. A search descends layer by layer.

DEFINITION | Hierarchical navigable small world

A multi-layer proximity graph. Each vector appears in layer 0 and, with probability p^{ell} , in layers 1...ell. Within a layer, each node links to about M approximate nearest neighbours, pruned by a *diversity* heuristic that keeps links pointing in different directions. Search starts at the top layer's entry point and greedily descends.

HNSW search: two heaps and a greedy walk

```

1 import heapq
2
3 def search_layer(graph, vectors, q, entry_points, ef, layer):
4     """Returns the ef closest nodes found in this layer."""

```

```

5     visited = set(entry_points)
6     candidates = []                                # min-heap by distance: what to
        expand
7     results = []                                    # max-heap by distance: what to
        keep
8     for ep in entry_points:
9         d = dist(q, vectors[ep])
10        heapq.heappush(candidates, (d, ep))
11        heapq.heappush(results, (-d, ep))
12    while candidates:
13        d, c = heapq.heappop(candidates)
14        if d > -results[0][0] and len(results) >= ef:
15            break                                    # the nearest candidate is
        worse than
16                                                # our worst keeper: stop
17        for nbr in graph[layer][c]:
18            if nbr in visited:
19                continue
20            visited.add(nbr)
21            dn = dist(q, vectors[nbr])
22            if len(results) < ef or dn < -results[0][0]:
23                heapq.heappush(candidates, (dn, nbr))
24                heapq.heappush(results, (-dn, nbr))
25                if len(results) > ef:
26                    heapq.heappop(results)
27    return [(-d, i) for d, i in results]
28
29 def hnsw_search(graph, vectors, q, k, ef_search, entry, max_layer):
30     ep = [entry]
31     for layer in range(max_layer, 0, -1):
32         ep = [max(search_layer(graph, vectors, q, ep, 1, layer))[1]]
33     found = search_layer(graph, vectors, q, ep, max(ef_search, k), 0)
34     return [i for _, i in sorted(found)[:k]]

```

Two heaps: a min-heap of candidates to expand, and a bounded max-heap of the `ef` best found so far. The termination test — “the closest unexpanded candidate is farther than our worst keeper” — is what makes it terminate quickly. This is best-first search (Chapter 23) with a bounded frontier.

Parameters. M (links per node, 16–64) controls memory and recall: memory is $\approx 8Mn$ bytes for the graph alone, so $M=32$ on 10^8 vectors is 25 GB *before* the vectors. `efConstruction` (100–500) controls build quality and build time; `efSearch` ($\geq k$, typically 50–200) is the runtime recall/latency knob.

COMMON PITFALL | HNSW's real costs

Memory. The graph is often larger than compressed vectors. HNSW with $M=32$ plus PQ codes can be dominated by the graph.

Build time. $\Theta(n \log n)$ distance computations with a large constant; hours for 10^8 vectors.

Deletion. There is no clean removal — deleting a node breaks the connectivity its links provided. Implementations use tombstones and periodic rebuilds. If your corpus has high churn, this is the constraint that decides your architecture.

Filtered search. “Nearest neighbours where `tenant_id = 42`” is genuinely hard: filtering *after* search returns too few results, and filtering *during* the graph walk can disconnect the graph so the walk gets stuck. The practical answers are partitioned indexes per high-cardinality filter value, or pre-filtering into a candidate id set and brute-forcing when it is small. Always check how your database implements this before designing around it.

Choosing an index

Situation	Index	Why
$n < 10^5$, any d	brute force	exact, simple, fast enough
$n < 10^7$, memory plentiful	HNSW (flat)	best recall/latency, no training
$n < 10^7$, memory tight	IVF + PQ	10–60x compression
$n > 10^8$, memory tight	IVF-PQ, or HNSW + PQ	partition and compress
$n > 10^9$, one machine	DiskANN / SPANN	graph in RAM, vectors on SSD
High churn (frequent deletes)	IVF	rebuilding one cell is cheap
Heavy metadata filtering	partitioned IVF	filter selects cells
GPU available, batch queries	brute force or IVF-flat on GPU	GEMM throughput wins

Table 33.1 Every production index is a combination of these four. **IVF4096, PQ64** is partition + compress; **HNSW32, SQ8** is navigate + compress; DiskANN is navigate + externalise.

WHERE THIS SHOWS UP IN ML | DiskANN and the IOPS constraint

DiskANN keeps a graph in memory (with PQ codes for cheap approximate distance ranking) and full-precision vectors on SSD. Each hop of the search does one random SSD read of one node's neighbours and vector. A query making 100 hops therefore needs 100 random reads; at 100 microseconds each *sequentially* that is 10 ms, which is too slow — so the reads must be issued concurrently, with a deep queue, which is exactly Chapter 31's point about IOPS versus bandwidth.

The design consequence is that DiskANN's graph is built to be *short*: a single flat graph with careful long-range pruning rather than HNSW's layers, tuned to minimise the number of hops rather than the work per hop. When the bottleneck is random reads, the algorithm optimises for read count.

EXERCISES

1. Build an IVF index over 10^6 random 128-dimensional vectors. Plot `recall@10` against `nprobe` for `nlist` $\in \{256, 1024, 4096\}$ and find the knee.

2. Implement PQ and measure the recall loss as m goes from 4 to 64 at $d=128$. Then add reranking with the exact vectors and re-measure.
3. Verify the asymmetric-distance speedup: time PQ search against exact search on the same candidates, and attribute the gap to memory traffic.
4. Implement `search_layer` and measure the number of distance computations per query against `efSearch`. Plot it against recall.
5. Compute the total memory of HNSW($M=32$) plus PQ($m=64$) for 10^8 vectors at $d=768$, and compare to IVF-PQ with the same PQ settings.
6. Implement filtered search three ways — post-filter, pre-filter with brute force, and a partitioned index — and compare recall and latency as the filter selectivity goes from 100% to 0.1%.
7. Build an index, delete 30% of the vectors with tombstones, and measure how recall and latency degrade before a rebuild.
8. Given 500M vectors at $d=1024$, a 64 GB RAM budget, a 10 ms p99 target and a 0.9 recall@10 requirement: choose an index, state every parameter, and justify the memory arithmetic.

CHAPTER RECAP

- Every production vector index combines four ideas: partition, compress, navigate, externalise.
- IVF clusters the space and probes `nprobe` cells; set $nlist \approx \sqrt{n}$ and tune `nprobe` for recall. Its only recall loss is at cell boundaries.
- PQ splits vectors into sub-vectors and stores one byte per sub-vector, giving 64x compression; asymmetric distance computation keeps the query unquantised and turns search into table lookups and additions.
- HNSW is best-first search on a layered proximity graph with two heaps; M sets memory and recall ceiling, `efSearch` is the runtime knob.
- HNSW's real costs are memory, build time, deletion and filtered search — not query latency.
- Index choice is decided by whether the data fits in memory and how much it changes.
- Disk-resident indexes optimise for the *number* of random reads, because IOPS and queue depth are the binding constraint.
- Over-retrieve and rerank with exact vectors after any compression.

Sampling Algorithms

Drawing from a distribution, fast, correctly, and at the right time

ADVANCED

PREREQS: Chapters 21, 27

Sampling appears at every layer of a machine learning system: choosing examples for a batch, choosing negatives for a contrastive loss, choosing a token to emit. Doing it naively costs a linear scan per draw; doing it well costs constant time. This chapter collects the handful of algorithms that make the difference, including the Gumbel trick that quietly underlies a surprising amount of modern practice.

WHAT YOU WILL LEARN

- Implement inverse-CDF, alias-method and Gumbel-max sampling and know their costs
- Choose the right method given whether weights are static or dynamic
- Implement Gumbel top- k for sampling without replacement
- Implement negative sampling and understand the unigram^{3/4} distribution
- Implement stratified, balanced and curriculum samplers correctly
- Avoid the classic bias bugs in distributed and multi-worker sampling

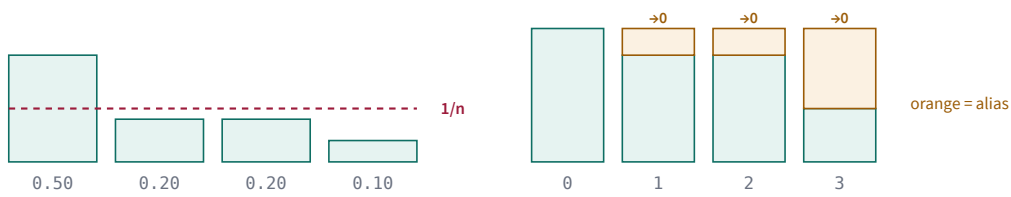
The core problem

Draw $i \in \{1, \dots, n\}$ with probability $p_i \propto w_i$.

Method	Preprocess	Per sample	Weights may change?
Linear scan	none	$\Theta(n)$	yes
Inverse CDF (prefix sums + binary search)	$\Theta(n)$	$\Theta(\log n)$	no (rebuild is $\Theta(n)$)
Fenwick tree	$\Theta(n)$	$\Theta(\log n)$	yes, $\Theta(\log n)$ per update
Alias method	$\Theta(n)$	$\Theta(1)$	no
Gumbel-max	none	$\Theta(n)$	yes (and needs no normalisation)
Gumbel top- k	none	$\Theta(n)$ for k items	yes

The alias method

ORIGINAL PROBABILITIES

ALIAS TABLE — every column has height $1/n$ 

```
sample: i = randint(n); u = random()
        return i if u < prob[i] else alias[i]
```

$O(n)$ to build, $O(1)$ per sample, forever — the right structure when weights are static.

Figure 34.1 The alias method. Redistribute probability mass so that every one of the n columns has height exactly $1/n$, each holding at most two outcomes. A draw is then one integer and one float.

Vose's alias method: $O(n)$ build, $O(1)$ sample

```
1 import numpy as np
2
3 class AliasSampler:
4     def __init__(self, weights, seed=0):
5         w = np.asarray(weights, dtype=np.float64)
6         n = len(w)
7         self.n = n
8         self.prob = w * n / w.sum()          # mean is now 1
9         self.alias = np.zeros(n, dtype=np.int64)
10        small = [i for i in range(n) if self.prob[i] < 1.0]
11        large = [i for i in range(n) if self.prob[i] >= 1.0]
12        while small and large:
13            s, l = small.pop(), large.pop()
14            self.alias[s] = l
15            self.prob[l] -= 1.0 - self.prob[s]  # l absorbs s's deficit
16            (small if self.prob[l] < 1.0 else large).append(l)
17        for i in small + large:
18            self.prob[i] = 1.0                  # floating-point residue
19        self.rng = np.random.default_rng(seed)
20
21    def sample(self, size=1):
22        i = self.rng.integers(0, self.n, size=size)
23        u = self.rng.random(size=size)
24        return np.where(u < self.prob[i], i, self.alias[i])
```

The invariant: every column has total height 1, consisting of some of outcome i and the rest of `alias[i]`. Pick a column uniformly, then pick within it with one coin flip. Two random numbers, two array reads, no search.

WHERE THIS SHOWS UP IN ML | Where the alias method belongs

Negative sampling in word2vec, contrastive learning and recommender training: the noise distribution is fixed for an epoch, and you draw billions of samples from it. Building an alias table once and sampling in $\Theta(1)$ is the difference between the sampler being negligible and being the bottleneck.

Static class-balanced sampling: weights are $1/\text{class frequency}$ and do not change.

Data mixture sampling: choosing which corpus a batch comes from, according to fixed mixture weights.

It is the *wrong* choice whenever weights change — prioritized replay, active learning, curriculum learning — because every change costs a $\Theta(n)$ rebuild. That is Chapter 27's Fenwick tree.

The Gumbel trick

THEOREM | Gumbel-max

Let $g_1, \dots, g_n$ be i.i.d. samples from the standard Gumbel distribution, $g_i = -\log(-\log u_i)$ with $u_i \sim \text{Uniform}(0,1)$. Then

$$\arg \max_i (\log w_i + g_i) \sim \text{Categorical}(w_i / \sum_j w_j)$$

This is remarkable in three ways. It needs no normalisation — unnormalised log-weights (logits) are enough. It is a pure **argmax**, so it vectorises perfectly on a GPU. And its extension is immediate:

THEOREM | Gumbel top-k

Taking the **top** k of $\log w_i + g_i$ gives a sample of k distinct items drawn **without replacement** from the same categorical distribution.

Gumbel-max and Gumbel top-k

```

1  import numpy as np
2
3  def gumbel_sample(logits, rng=np.random):
4      g = -np.log(-np.log(rng.random(logits.shape) + 1e-20) + 1e-20)
5      return int(np.argmax(logits + g))
6
7  def gumbel_topk(logits, k, rng=np.random):
8      """k distinct items, without replacement, in one vectorised pass."""
9      g = -np.log(-np.log(rng.random(logits.shape) + 1e-20) + 1e-20)
10     perturbed = logits + g
11     idx = np.argpartition(-perturbed, k)[:k]
12     return idx[np.argsort(-perturbed[idx])]
```

WHERE THIS SHOWS UP IN ML | Gumbel in practice

Temperature sampling from a language model is exactly Gumbel-max on **logits / temperature** — which is what every framework's **multinomial** does under the hood on a GPU, because **argmax** is cheaper than a cumulative sum plus a search.

Sampling without replacement at scale: choosing k distinct items from a billion-item catalogue for a slate, in one vectorised pass.

Gumbel-softmax (concrete) relaxation: replacing **argmax** with a temperature-scaled **softmax** makes the sample differentiable, which is how discrete latent variables, hard attention and differentiable architecture search are trained.

Weighted reservoir sampling (Chapter 21) is the Gumbel trick in disguise: the key $u^{1/w}$ is a monotone transform of $\log w + g$, which is why it produces the same distribution.

Negative sampling

Training a model to distinguish observed pairs from noise requires drawing negatives, and the noise distribution matters more than its implementation.

The unigram^{0.75} distribution and the sampler

```

1  import numpy as np
2
3  def build_negative_sampler(counts, power=0.75, seed=0):
4      """counts: array of item frequencies in the corpus."""
5      w = np.asarray(counts, dtype=np.float64) ** power
6      return AliasSampler(w, seed=seed)
7
8  def sample_negatives(sampler, positives, num_neg, max_tries=4):
9      """Draw num_neg negatives per positive, rejecting accidental
10         positives."""
11      pos = set(positives.tolist())
12      out = sampler.sample(size=len(positives) * num_neg).reshape(
13          len(positives), num_neg)
14      for _ in range(max_tries):
15          bad = np.isin(out, list(pos))
16          if not bad.any():
17              break
18          out[bad] = sampler.sample(size=int(bad.sum()))
19      return out

```

The exponent 0.75 is empirical and comes from word2vec: raising counts to a power below 1 flattens the distribution, so rare items are sampled more often than their frequency would suggest and common items less. Pure unigram (power=1) oversamples stop-words and gives weak gradients; uniform (power=0) samples implausible negatives that are too easy.

COMMON PITFALL | False negatives and sampling bias

Randomly drawn negatives are sometimes actually positive — an item the user would have liked, a document that does answer the query. The rejection loop above handles *known* positives; unknown ones are an inherent bias. Two standard mitigations: *in-batch negatives* (use other examples in the batch, which is free and makes negatives harder as batch size grows) and *hard-negative mining* (retrieve near-misses with the current model, which is much more informative and much more prone to false negatives). The usual production compromise is in-batch negatives plus a small number of mined hard negatives with a similarity ceiling that filters out likely positives.

Samplers that appear in every dataloader

Three correct samplers

```

1  import numpy as np
2
3  def stratified_indices(labels, n_per_class, rng=np.random):
4      """Exactly n_per_class from each class -- not an expectation."""
5      out = []
6      for c in np.unique(labels):
7          idx = np.flatnonzero(labels == c)
8          take = rng.choice(idx, size=min(n_per_class, len(idx)),
9                               replace=False)
10         out.append(take)
11     out = np.concatenate(out)
12     rng.shuffle(out)
13     return out
14
15 def class_balanced_weights(labels, beta=0.999):
16     """Effective-number reweighting: softer than 1/frequency."""
17     counts = np.bincount(labels)
18     eff = (1.0 - np.power(beta, counts)) / (1.0 - beta)
19     w = 1.0 / np.maximum(eff, 1e-12)
20     return (w / w.sum())[labels]
21
22 def distributed_shard(indices, rank, world_size, epoch, seed=0):
23     """Every rank gets a disjoint, deterministic shard of a common
24     shuffle."""
25     rng = np.random.default_rng(seed + epoch) # SAME seed on all
26     ranks
27     perm = rng.permutation(len(indices))
28     pad = (-len(perm)) % world_size # pad so shards are
29     equal
30     if pad:
31         perm = np.concatenate([perm, perm[:pad]])
32     return indices[perm[rank::world_size]]

```

WATCH OUT | Three sampling bugs that are hard to see

Different seeds per rank. If each rank shuffles independently, the ranks see overlapping data and the effective epoch is wrong. Seed the *shuffle* identically on every rank and shard the result; seed *augmentation* differently per rank.

Unequal shard sizes. If the dataset does not divide by `world_size`, one rank finishes early and the all-reduce hangs or silently drops a batch. Pad the permutation (as above) or drop the remainder — but do it explicitly.

Worker seeds that repeat. PyTorch dataloader workers each get a derived seed, but a `Dataset` that calls the global `random` module without reseeding per worker will produce *identical* augmentations in every worker. Verify by printing a hash of the first batch with `num_workers` set to 1 and to 4.

WHERE THIS SHOWS UP IN ML | Sampling in RL and curriculum learning

Prioritized replay needs dynamic weights: Fenwick tree (Chapter 27). **Curriculum learning** raises the weight of harder examples over time: dynamic weights again, but updated in bulk once per epoch, so a rebuilt alias table is fine. **Active learning** scores the entire pool with the current model and samples the top of it: not a sampling-structure problem at all but a top- k problem (Chapter 13). Identifying which of the three you have determines the data structure.

EXERCISES

1. Implement the alias method and verify the empirical distribution against the target with a chi-squared test at $n=1000$ and 10^7 draws.
2. Compare sampling throughput for linear scan, `np.random.choice` with `p`, inverse CDF with `searchsorted`, and the alias method at $n \in \{10^3, 10^5, 10^7\}$.
3. Verify the Gumbel-max theorem empirically for $n=5$ with skewed weights, using 10^6 samples.
4. Verify that Gumbel top- k gives the same distribution as k sequential draws without replacement, for $n=6$, $k=3$.
5. Show that weighted reservoir sampling's key $u^{1/w}$ induces the same ordering as $\log w + g$ with g Gumbel.
6. Implement the unigram^{0.75} negative sampler and plot the sampling probability against rank for power $\in \{0, 0.5, 0.75, 1\}$ on a Zipf distribution.
7. Construct the “different seed per rank” bug in a two-process script and measure how much of the dataset each rank actually sees per epoch.
8. Implement a curriculum sampler whose weights depend on a per-example difficulty updated every epoch. Choose the data structure and justify it.

CHAPTER RECAP

- The choice of sampling algorithm is decided by whether weights change between draws: alias for static, Fenwick tree for dynamic, Gumbel when you want vectorisation and no normalisation.
- The alias method redistributes mass so every column has height $1/n$, giving $\Theta(1)$ sampling after $\Theta(n)$ setup.
- Gumbel-max samples by **argmax** of perturbed logits, needs no normalising constant, vectorises perfectly, and its top- k form samples k items without replacement in one pass.
- Gumbel-softmax makes the sample differentiable, which is how discrete latents are trained.
- Negative sampling uses unigram^{0.75}; the real difficulties are false negatives, and in-batch plus lightly mined hard negatives is the standard compromise.
- Distributed sampling bugs — per-rank seeds, unequal shards, repeated worker seeds — are silent and change what your model trains on.

Decoding Algorithms

Greedy, beam, nucleus, speculative and constrained

FRONTIER

PREREQS: Chapters 13, 20, 34

A language model produces a probability distribution over the next token. Turning a sequence of such distributions into text is a search problem, and the algorithm you choose changes latency, cost, and what the model appears to be capable of. This chapter treats decoding as what it is — search with a bounded budget — and covers the algorithms that a serving system actually runs.

WHAT YOU WILL LEARN

- Distinguish the search objective from the sampling objective and know when each is right
- Implement beam search with length normalisation and early stopping
- Implement top- k , top- p and min- p sampling efficiently
- Explain speculative decoding and prove that it preserves the output distribution
- Implement constrained decoding with a finite automaton over tokens
- Reason about the cost of each method in tokens, memory and latency

Decoding is bounded search

At each step the model gives $p(x_t \mid x_{<t})$ over a vocabulary of $V \approx 10^5$. Generating T tokens means choosing a path through a tree with V^T leaves. Every decoding algorithm is a policy for exploring a vanishing fraction of it.

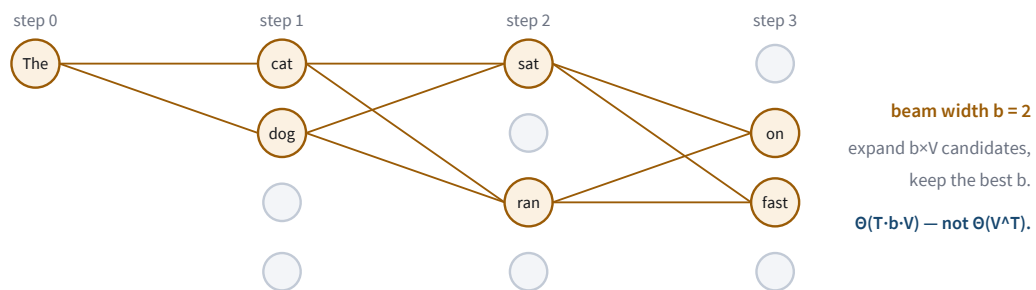
Algorithm	Explores	Objective	Cost per token
Greedy	1 path	locally most likely	1 forward pass
Beam search	b paths	approximately most likely sequence	b (batched)
Pure sampling	1 random path	a sample from the model	1
Top- k / top- p	1 truncated-random path	a sample from a truncated model	1
Speculative	1 path, verified	identical to the base method	$\approx 1/\gamma$

Algorithm	Explores	Objective	Cost per token
Constrained	1 path in a language	best/sample within a grammar	1 + mask

KEY INSIGHT | Likelihood is not quality

Beam search maximises sequence log-probability, and for open-ended generation that is the wrong objective: the highest-probability continuation of almost any prompt is bland, repetitive, and often degenerate. Humans do not produce maximum-likelihood text — real text sits in a band of moderate surprisal. This is why sampling methods dominate for chat and creative generation, while beam search remains standard for translation, speech recognition and constrained tasks where a single correct answer exists.

Beam search



Greedy is $b = 1$. Exhaustive is $b = V^T$. Beam search is the bounded-memory middle.

Figure 35.1 Beam search with width 2. At each step all $b \times V$ continuations are scored and the best b are kept — a bounded-width breadth-first search with a priority queue.

Beam search with length normalisation	
1	<code>import numpy as np</code>
2	
3	<code>def beam_search(step_fn, start, eos, beam=4, max_len=64, alpha=0.7):</code>
4	<code> """step_fn(tokens) -> log-probability vector over the vocabulary."""</code>
5	<code> beams = [(0.0, [start], None)] # (logprob, tokens, state)</code>
6	<code> finished = []</code>
7	<code> for _ in range(max_len):</code>
8	<code> cand = []</code>
9	<code> for score, toks, state in beams:</code>
10	<code> logp = step_fn(toks) # (V,) log-probabilities</code>
11	<code> top = np.argsort(-logp)[-beam:] # theta(V)</code>
12	<code> for t in top:</code>
13	<code> cand.append((score + float(logp[t]), toks + [int(t)],</code>
14	<code> state))</code>
15	<code> cand.sort(key=lambda c: -c[0])</code>
16	<code> beams = []</code>
17	<code> for score, toks, state in cand[:beam]:</code>
18	<code> if toks[-1] == eos:</code>
19	<code> lp = ((5 + len(toks)) / 6) ** alpha # GNMT penalty</code>
	<code> finished.append((score / lp, toks))</code>

```

20         else:
21             beams.append((score, toks, state))
22         if not beams:
23             break
24         # early stop: no unfinished beam can beat the best finished one
25         if finished and beams[0][0] < max(f[0] for f in finished):
26             break
27     if not finished:
28         finished = [(s, t) for s, t, _ in beams]
29     return max(finished)[1]

```

Why length normalisation. Log-probabilities are negative, so every extra token lowers the score and raw beam search is systematically biased toward short outputs. Dividing by $((5+|y|)/6)^{\alpha}$ with $\alpha \in [0.6, 1.0]$ is the standard correction.

Why only top- b per beam. Expanding all $b \times V$ candidates and sorting is $\Theta(bV \log(bV))$; since at most b continuations of any one beam can survive, taking each beam's top b first reduces it to $\Theta(bV)$ (Chapter 13).

COMMON PITFALL | Beam search's failure modes

Larger beams often produce *worse* output — the “beam search curse”. With $b=100$ a translation model tends to emit empty or very short strings, because those genuinely have higher probability under the model. Beam search also amplifies repetition, since repeated text is high-probability. Standard patches: length normalisation, coverage penalties, n -gram blocking, and diverse beam search. All of them are admissions that the objective is wrong.

Sampling: top- k , top- p , min- p

The three truncation schemes

```

1  import numpy as np
2
3  def top_k_filter(logits, k):
4      if k <= 0 or k >= len(logits):
5          return logits
6      kth = np.partition(logits, -k)[-k]           # theta(V)
7      return np.where(logits < kth, -np.inf, logits)
8
9  def top_p_filter(logits, p):
10     order = np.argsort(-logits)                  # theta(V log V)
11     probs = np.exp(logits[order] - logits[order].max())
12     probs /= probs.sum()
13     cum = np.cumsum(probs)
14     cutoff = int(np.searchsorted(cum, p) + 1)     # smallest set
15     >= p
16     out = np.full_like(logits, -np.inf)
17     out[order[:cutoff]] = logits[order[:cutoff]]
18     return out
19
20 def min_p_filter(logits, min_p):
21     """Keep tokens whose probability is at least min_p * max probability.

```

```

21     Adapts to the distribution's sharpness, unlike a fixed k or p."""
22     m = logits.max()
23     thresh = m + np.log(min_p)
24     return np.where(logits < thresh, -np.inf, logits)
25
26 def sample(logits, temperature=1.0, rng=np.random):
27     z = logits / max(temperature, 1e-6)
28     g = -np.log(-np.log(rng.random(z.shape) + 1e-20) + 1e-20) # Gumbel
29     return int(np.argmax(z + g))

```

Method	Truncates to	Adapts to sharpness?	Cost
Temperature	nothing	—	$\Theta(V)$
Top- k	fixed count	no	$\Theta(V)$ with partition
Top- p (nucleus)	smallest set with mass $\geq p$	yes	$\Theta(V \log V)$ sort
Min- p	tokens above $p \cdot p_{\max}$	yes	$\Theta(V)$
Typical / η	tokens near the entropy	yes	$\Theta(V \log V)$

Table 35.1 Decoding algorithms. Only speculative decoding changes the cost without changing the output distribution; everything else is a different objective.

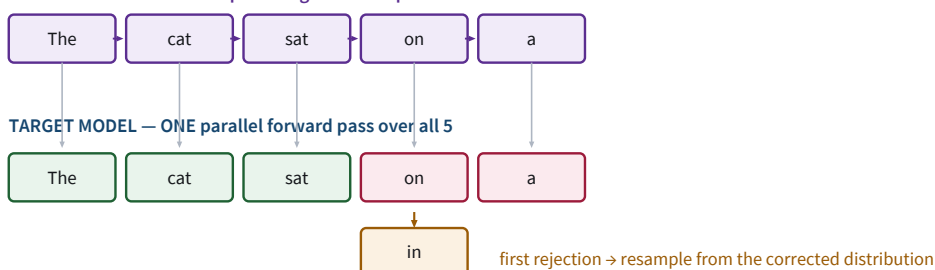
PERFORMANCE NOTE | Decoding overhead is not negligible

At $V=128,000$, a full sort per token costs roughly 2 million comparisons. Multiply by 100 tokens per second per stream and 64 concurrent streams and the sampler is consuming real CPU. Production servers keep this on the GPU (`torch.topk` plus a masked softmax) and prefer $\Theta(V)$ schemes. Repetition penalties, which need a scan over the generated prefix, are another silent cost — keep a set of emitted tokens rather than rescanning.

Speculative decoding

Generating one token requires reading every model weight from memory (Chapter 3), so decoding is bandwidth-bound and the GPU's arithmetic units are nearly idle. Speculative decoding exploits that: use a small draft model to guess several tokens, then verify them all in *one* forward pass of the large model.

DRAFT MODEL — $k = 5$ cheap autoregressive steps



Accepted 3 + 1 resampled = 4 tokens for the cost of one target pass plus 5 cheap ones. Output distribution is provably unchanged.

Figure 35.2 Speculative decoding. A cheap draft model proposes γ tokens; one parallel pass of the target model scores all of them; a modified rejection-sampling rule accepts a prefix and resamples at the first rejection.

Speculative decoding, the acceptance rule

```

1  import numpy as np
2
3  def speculative_step(target_logprobs, draft_logprobs, draft_tokens, rng):
4      """target_logprobs, draft_logprobs: (gamma+1, V) from one parallel
5      pass.
6      Returns the accepted tokens. Output distribution == target model's."""
7      accepted = []
8      for i, t in enumerate(draft_tokens):
9          p = np.exp(target_logprobs[i, t])          # target prob of the
10             draft
11          q = np.exp(draft_logprobs[i, t])          # draft prob of the
12             draft
13          if rng.random() < min(1.0, p / q):
14              accepted.append(t)                    # accept
15          else:
16              resid = np.maximum(np.exp(target_logprobs[i])
17                                 - np.exp(draft_logprobs[i]), 0.0)
18              resid /= resid.sum()                  # the corrected residual
19              accepted.append(int(rng.choice(len(resid), p=resid)))
20              return accepted                        # stop at first
21             rejection
22      bonus = int(np.argmax(target_logprobs[len(draft_tokens)]))
23      accepted.append(bonus)                        # free extra token
24      return accepted

```

THE MATH | Why the output distribution is exactly preserved

Consider one position with target distribution p and draft q . The draft proposes $x \sim q$; we accept with probability $\min(1, p(x)/q(x))$. The probability of emitting x by acceptance is $q(x) \min(1, p(x)/q(x)) = \min(q(x), p(x))$. The probability of rejecting is $1 - \sum_y \min(q(y), p(y))$, and on rejection we sample from the normalised residual $\max(p - q, 0) / \sum_y \max(p(y) - q(y), 0)$, whose normaliser equals that same rejection probability. So the total probability of emitting x is

$$\min(q(x), p(x)) + \max(p(x) - q(x), 0) = p(x)$$

exactly. The draft model affects *speed only* — never the distribution. This is why speculative decoding is safe to enable by default, unlike every other acceleration in this chapter.

Expected speedup. If the per-token acceptance rate is α and the draft length is γ , the expected number of tokens per target pass is $1 - \alpha^{\gamma+1} / 1 - \alpha$. With $\alpha=0.8$ and $\gamma=4$ that is 3.36 tokens per target pass; net speedup 2–3x once the draft model’s own cost is included. Variants replace the draft model with n -gram lookup from the prompt (very cheap, high acceptance on summarisation and code editing), with extra prediction heads (Medusa), or with a tree of candidates verified at once.

Constrained decoding

To guarantee valid JSON, a regular expression, or a value from a fixed set, compile the constraint into a finite automaton over *tokens* and mask everything that would leave its valid transitions.

Grammar-constrained decoding via token masks

```

1  class FSAConstraint:
2      def __init__(self, transitions, start, accepting, vocab_size):
3          self.transitions = transitions          # state -> {token:
4              next_state}
5          self.state = start
6          self.accepting = accepting
7          self.V = vocab_size
8
9      def mask(self):
10         import numpy as np
11         m = np.full(self.V, -np.inf, dtype=np.float32)
12         allowed = self.transitions.get(self.state, {})
13         if allowed:
14             m[list(allowed.keys())] = 0.0
15         return m                                # add to logits before
16             softmax
17
18     def advance(self, token):
19         self.state = self.transitions[self.state][token]
20
21     def can_stop(self):
22         return self.state in self.accepting

```

Two subtleties decide whether this works in practice.

Token–character mismatch. The grammar is over characters; the model emits tokens, and a token may span a grammar boundary. The automaton must therefore be built over *token* transitions, which means precomputing, for each state, the set of tokens whose character expansion keeps the automaton alive. This is a product construction between the tokenizer’s trie (Chapter 14) and the grammar’s automaton, and it is precomputed once per grammar.

Dead ends. If a valid prefix has no valid continuation, the decoder is stuck. The automaton must be trimmed to its *co-accessible* states — those from which an accepting state is reachable — before use. That trimming is a reverse reachability computation on the automaton graph (Chapter 22).

WHERE THIS SHOWS UP IN ML | What constrained decoding costs and does not cost

The mask is precomputed per state, so the runtime cost is one lookup and one vector add per token: negligible. What it *does* cost is distributional distortion — renormalising over a subset changes the model’s probabilities, and the constrained output may be far less likely than the model’s natural output. If a model wants to say “I don’t know” and your schema forbids it, you get a confident wrong answer instead. Constrain the *format*, and be careful about constraining the *content*.

EXERCISES

1. Implement beam search and show that increasing b from 1 to 50 on a translation model first improves and then degrades quality. Plot it.
2. Show that without length normalisation, beam search's output length shrinks as b grows. Quantify the effect.
3. Implement top- k , top- p and min- p , and compare the number of surviving tokens per step on peaked versus flat distributions.
4. Measure the cost of the full sort in top- p at $V=128,000$ and compare to min- p . Extrapolate to 1000 tokens per second.
5. Implement the speculative acceptance rule and verify empirically that the output distribution matches direct sampling from the target, using a chi-squared test over a small vocabulary.
6. Derive the expected tokens per target pass and verify it by simulation for $\alpha \in \{0.5, 0.7, 0.9\}$ and $\gamma \in \{2, 4, 8\}$. Find the optimal γ given a draft-to-target cost ratio of 0.1.
7. Implement n -gram (prompt-lookup) speculative decoding and measure its acceptance rate on a summarisation task versus a free-form chat task.
8. Build a token-level automaton for a small JSON schema and show that trimming non-co-accessible states is necessary by constructing a dead-end case.

CHAPTER RECAP

- Decoding is bounded search over a V^T tree; each algorithm is a policy for exploring a tiny fraction of it.
- Beam search approximates the most likely sequence and needs length normalisation; larger beams can make open-ended output worse, because likelihood is the wrong objective there.
- Top- k , top- p and min- p truncate before sampling; min- p adapts to sharpness at $\Theta(V)$ rather than $\Theta(V \log V)$.
- Sampling via Gumbel-max avoids normalisation and vectorises.
- Speculative decoding provably preserves the target distribution, because the accept/residual rule sums to $p(x)$ exactly; it converts bandwidth-bound decoding into compute-bound verification.
- Constrained decoding masks logits using a token-level automaton built as a product of the tokenizer trie and the grammar; trim to co-accessible states or the decoder can get stuck.

KV Caches as Data Structures

Paging, prefix trees and eviction inside an inference server

FRONTIER

PREREQS: Chapters 6, 14, 31

The key-value cache is the single largest consumer of memory in LLM serving, and managing it is a pure data-structures problem — one whose best-known solution is a direct port of operating-system virtual memory. This chapter works through the memory arithmetic, the paging scheme, the prefix tree that makes shared prompts free, and the scheduling that ties them together.

WHAT YOU WILL LEARN

- Compute KV cache size exactly and see why it dominates serving memory
- Explain the fragmentation problem in contiguous allocation
- Understand paged attention as virtual memory for the cache
- Implement a radix tree over token prefixes for cache sharing
- Understand continuous batching and its scheduling decisions
- Reason about eviction, recomputation and offloading

The memory arithmetic

For a transformer with L layers, H key/value heads, head dimension d_h , batch B , sequence length S , in 2-byte precision:

$$\text{KV bytes} = 2 \times 2 \times B \times S \times L \times H \times d_h$$

(the outer 2 for keys and values, the inner 2 for bytes per element).

THE MATH | Worked example

A 70-billion-parameter model with $L=80$, 64 attention heads of $d_h=128$ and 8 key/value heads (grouped-query attention):

$$2 \times 2 \times 1 \times 4096 \times 80 \times 8 \times 128 = 1.34 \text{ GB per sequence}$$

At batch 32 that is 43 GB — on an 80 GB device that already holds 140 GB of weights in fp16, which is why the model must be sharded before you even begin. Without grouped-query attention ($H=64$) it would be 10.7 GB per sequence: 8x more. GQA exists almost entirely for this reason.

Knob	Effect on KV cache
Grouped-query attention ($H_{kv} < H_q$)	linear reduction, 4–8x typical
Multi-query attention ($H_{kv} = 1$)	maximal reduction, some quality cost
KV quantisation to int8 / fp8	2x
Sliding-window attention (window w)	caps S at w
Cross-layer sharing	linear in the number of shared layers
Longer context	linear — and this is the growth direction

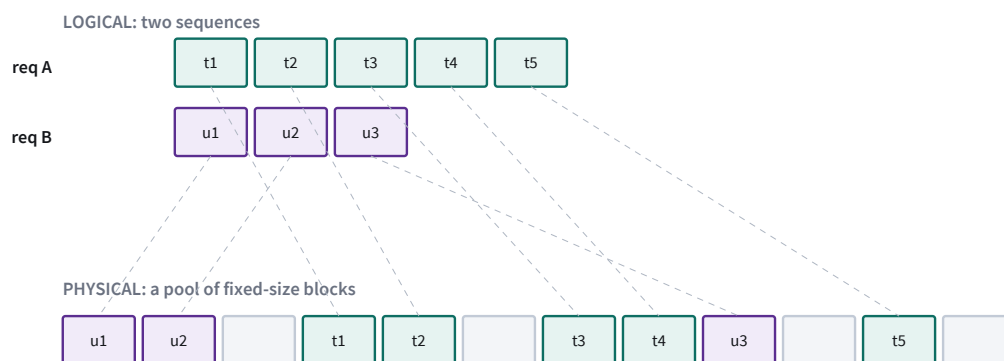
Fragmentation, and why contiguity is the enemy

The naive implementation allocates a contiguous buffer per request, sized to `max_tokens`. That produces three kinds of waste:

- **Internal fragmentation.** A request that generates 100 tokens out of a 2048 reservation wastes 95% of its allocation.
- **External fragmentation.** Freed blocks of varying sizes leave holes too small for the next request.
- **Reservation waste.** Space reserved for tokens not yet generated cannot be used by anyone else.

Measured in the vLLM paper, contiguous allocation wasted 60–80% of KV memory. Since KV memory bounds the batch size, and batch size determines throughput, that waste is a direct multiple on serving cost.

Paged attention



A block table per request maps logical positions to physical blocks — exactly OS virtual memory.

No contiguity requirement → no fragmentation waste, and shared prefixes can share blocks.

Figure 36.1 Paged attention. Each request’s logical token sequence is mapped through a block table to fixed-size physical blocks that need not be contiguous — precisely the page-table mechanism of an operating system.

A block-based KV cache manager

```

1  class BlockManager:
2      """Fixed-size blocks of `block_size` tokens; a block table per
   request."""

```

```

3
4     def __init__(self, num_blocks, block_size=16):
5         self.block_size = block_size
6         self.free = list(range(num_blocks))      # free list
7         self.tables = {}                        # req_id -> [block ids]
8         self.refcount = {}                     # block id -> users
9
10    def allocate(self, req_id, num_tokens):
11        need = (num_tokens + self.block_size - 1) // self.block_size
12        if len(self.free) < need:
13            return False                        # trigger preemption
14        blocks = [self.free.pop() for _ in range(need)]
15        for b in blocks:
16            self.refcount[b] = 1
17        self.tables[req_id] = blocks
18        return True
19
20    def append_token(self, req_id):
21        """Grow by one token; allocate a block only when the last one
22        fills."""
23        table = self.tables[req_id]
24        used = self.used_tokens.get(req_id, 0)
25        if used % self.block_size == 0:
26            if not self.free:
27                return False
28            b = self.free.pop()
29            self.refcount[b] = 1
30            table.append(b)
31        self.used_tokens[req_id] = used + 1
32        return True
33
34    def fork(self, parent_id, child_id):
35        """Copy-on-write: a beam or a parallel sample shares the
36        prefix."""
37        blocks = self.tables[parent_id]
38        for b in blocks:
39            self.refcount[b] += 1
40        self.tables[child_id] = list(blocks)
41
42    def free_request(self, req_id):
43        for b in self.tables.pop(req_id, []):
44            self.refcount[b] -= 1
45            if self.refcount[b] == 0:
46                self.free.append(b)

```

The design: fixed-size blocks (16 tokens is standard) eliminate external fragmentation entirely; internal fragmentation is bounded by half a block per request; and a reference count per block gives copy-on-write sharing for free. The attention kernel is modified to gather keys and values through the block table rather than assuming contiguity — which costs a little indirection and buys a 2–4x larger batch.

KEY INSIGHT | The same idea, three times

Fixed-size blocks with an indirection table is *the* answer to fragmentation, and you have now seen it in three places: the ragged values-plus-offsets layout of Chapter 4, the page table of an operating system, and the block table here. Whenever a system must store many variable-length things in a fixed pool, this is the structure.

Prefix sharing with a radix tree

In a chat service, every request shares a long system prompt, and successive turns of one conversation share the whole history. Recomputing that KV cache is pure waste — prefill is compute-bound and linear in prefix length.

A radix tree (Chapter 14) over token sequences, with KV blocks attached to its nodes, makes the sharing automatic.

A radix tree over token prefixes

```

1  class RadixNode:
2      __slots__ = ("children", "tokens", "blocks", "refs", "last_used")
3      def __init__(self, tokens=(), blocks=()):
4          self.children = {}          # first token -> RadixNode
5          self.tokens = tuple(tokens) # the edge label
6          self.blocks = list(blocks)  # KV blocks for these tokens
7          self.refs = 0
8          self.last_used = 0.0
9
10 class PrefixCache:
11     def __init__(self):
12         self.root = RadixNode()
13
14     def match(self, tokens):
15         """Longest cached prefix: returns (matched_len, blocks, node)."""
16         node, i, blocks = self.root, 0, []
17         while i < len(tokens):
18             child = node.children.get(tokens[i])
19             if child is None:
20                 break
21             k = 0
22             while (k < len(child.tokens) and i + k < len(tokens)
23                   and child.tokens[k] == tokens[i + k]):
24                 k += 1
25             blocks.extend(child.blocks[:k])
26             i += k
27             if k < len(child.tokens):
28                 break          # partial edge match: stop
29             here
30             node = child
31         return i, blocks, node
32
33     def insert(self, tokens, blocks, node=None, start=0):
34         node = node or self.root
35         if start >= len(tokens):
36             return node
37         child = RadixNode(tokens[start:], blocks[start:])

```

```

37         node.children[tokens[start]] = child
38         return child

```

Eviction. LRU over *leaves only*: an internal node’s blocks are in use by every descendant, so evicting it would corrupt them. Maintain a reference count and an LRU list of evictable leaves — which is the LRU cache of Chapter 6 with a tree-shaped validity constraint.

WHERE THIS SHOWS UP IN ML | What prefix caching is worth

On real chat traffic, cache hit rates of 50–90% are typical, because system prompts are shared and multi-turn conversations re-send their history. Since prefill cost is linear in the uncached prefix length, a 90% hit rate turns a 4000-token prefill into a 400-token one — roughly a 10x reduction in time-to-first-token for that request, and a large increase in the number of concurrent conversations a server can hold.

This is why API providers offer discounted “cached input” pricing: the mechanism is exactly the structure above, and the discount reflects a real cost saving.

Continuous batching

Static batching — collect B requests, run them together to completion — wastes enormous capacity because requests finish at different lengths and the whole batch waits for the longest.

Continuous batching (also called in-flight batching) runs the scheduler *per decoding step*:

1. Finished sequences leave the batch immediately and free their blocks.
2. Waiting requests are admitted if there are enough free blocks for their prompt.
3. When memory runs short, a request is **preempted** — either swapped out to host memory, or dropped and recomputed later (recompute is usually cheaper, since prefill is fast and PCIe is slow).
4. Prefill and decode are interleaved, or separated onto different devices (disaggregated serving), because they have opposite bottlenecks: prefill is compute-bound, decode is bandwidth-bound.

The scheduler’s core decision

```

1  def schedule(running, waiting, block_mgr, max_batch):
2      """One step of continuous batching."""
3      running = [r for r in running if not r.finished]
4      for r in list(running):
5          if not block_mgr.append_token(r.id):          # out of memory
6              victim = max(running, key=lambda x: x.remaining_estimate)
7              block_mgr.free_request(victim.id)
8              victim.preempted = True                  # recompute later
9              running.remove(victim)
10             waiting.insert(0, victim)
11     while waiting and len(running) < max_batch:
12         r = waiting[0]
13         matched, blocks, _ = prefix_cache.match(r.tokens)
14         if block_mgr.allocate(r.id, len(r.tokens) - matched):
15             r.cached_prefix = matched

```

```

16         running.append(waiting.pop(0))
17     else:
18         break # no room; wait
19     return running, waiting

```

The scheduling policy is a real choice with real tradeoffs: first-come first-served is fair and has poor tail latency; shortest-remaining-first maximises throughput and starves long requests; a priority queue by deadline (Chapter 13) gives predictable service levels. It is the same scheduling problem as any queueing system, and the same answers apply.

COMMON PITFALL | Eviction and recomputation are not equivalent

Swapping a KV cache to host memory over PCIe at 25 GB/s takes about 54 ms for a 1.34 GB sequence. Recomputing the prefill for 4096 tokens on a modern accelerator takes perhaps 200 ms but uses no PCIe bandwidth and no host memory. Which is better depends on how loaded the interconnect is and whether the request will resume soon. Most servers default to recomputation for short prefixes and swapping for long ones — and the threshold is worth measuring on your own hardware rather than accepting the default.

EXERCISES

1. Compute the KV cache size for Llama-3-8B ($L=32$, $H_q=32$, $H_{kv}=8$, $d_h=128$) at context lengths 4K, 32K and 128K in fp16 and in int8.
2. Simulate contiguous allocation with request lengths drawn from a log-normal distribution and measure the fragmentation waste. Then simulate 16-token paging and measure it again.
3. Implement `BlockManager` including copy-on-write `fork`, and use it for beam search with width 4 sharing a common prefix. Measure the memory saved.
4. Implement `PrefixCache` and measure the hit rate on a synthetic chat workload with a shared 2000-token system prompt and 5-turn conversations.
5. Implement leaf-only LRU eviction over the radix tree and show that naive LRU can evict an internal node still in use.
6. Simulate static versus continuous batching with a mix of short and long requests. Report throughput and p99 latency for both.
7. Compare preemption policies: swap-to-host versus recompute, as a function of prefix length. Find the crossover for a given PCIe bandwidth and prefill throughput.
8. Design a scheduler that guarantees a p99 time-to-first-token under 500 ms while maximising throughput. State the data structures.

CHAPTER RECAP

- KV cache size is $4 \cdot B \cdot S \cdot L \cdot H_{\text{kv}} \cdot d_h$ bytes in fp16, and it dominates serving memory; grouped-query attention, KV quantisation and sliding windows are all attacks on this formula.
- Contiguous per-request allocation wastes 60–80% of KV memory to internal, external and reservation fragmentation.
- Paged attention stores the cache in fixed-size blocks with a per-request block table — OS virtual memory — eliminating external fragmentation and enabling copy-on-write sharing.
- A radix tree over token prefixes makes shared system prompts and multi-turn histories free to reuse; eviction must be LRU over leaves only.
- Continuous batching schedules per decoding step, admitting and preempting requests as blocks free up; prefill and decode have opposite bottlenecks.
- Preemption by recomputation and by swapping have different costs; the crossover depends on prefix length and interconnect load.

Tree Learners: Decision Trees, Histograms and GBDT

The algorithms that still win on tabular data

ADVANCED

PREREQS: Chapters 9, 12, 17

Gradient-boosted decision trees remain the strongest method for tabular data, and their speed comes almost entirely from data-structure engineering rather than from statistics. This chapter builds a decision tree from first principles, then shows the three ideas — histogram binning, histogram subtraction and leaf-wise growth — that made LightGBM and XGBoost orders of magnitude faster than their predecessors.

WHAT YOU WILL LEARN

- Implement decision-tree induction with an exact split finder
- Derive the gain formula used by gradient boosting
- Implement histogram-based split finding and explain its complexity
- Understand histogram subtraction, leaf-wise growth and EFB
- Handle categorical features and missing values correctly
- Reason about inference cost and why tree models are fast to serve

Decision-tree induction

A decision tree is built greedily (Chapter 17): at each node, choose the split that maximally reduces impurity, then recurse. Finding the *globally* optimal tree is NP-hard, so every practical learner is greedy and accepts the suboptimality.

Exact split finding: the $O(n \log n)$ baseline

```

1  import numpy as np
2
3  def best_split_exact(X, g, h, lam=1.0, min_child=1.0):
4      """X: (n, d) features. g, h: gradient and hessian per sample.
5      Returns (feature, threshold, gain)."""
6      G, H = g.sum(), h.sum()
7      parent = G * G / (H + lam)
8      best = (None, None, 0.0)
9      for j in range(X.shape[1]):
10         order = np.argsort(X[:, j], kind="stable")         # theta(n log n)

```



```

11     gs, hs = np.cumsum(g[order]), np.cumsum(h[order])
12     xs = X[order, j]
13     valid = np.flatnonzero(xs[1:] != xs[:-1])      # no split
14     inside ties
15     for i in valid:
16         GL, HL = gs[i], hs[i]
17         GR, HR = G - GL, H - HL
18         if HL < min_child or HR < min_child:
19             continue
20         gain = GL * GL / (HL + lam) + GR * GR / (HR + lam) - parent
21         if gain > best[2]:
22             best = (j, 0.5 * (xs[i] + xs[i + 1]), gain)
23     return best

```

THE MATH | Where the gain formula comes from

Gradient boosting fits each tree to a second-order Taylor expansion of the loss. With g_i and h_i the first and second derivatives at the current prediction, the optimal constant output for a leaf containing set I is

$$w^* = -\sum_{i \in I} g_i / \sum_{i \in I} h_i + \lambda$$

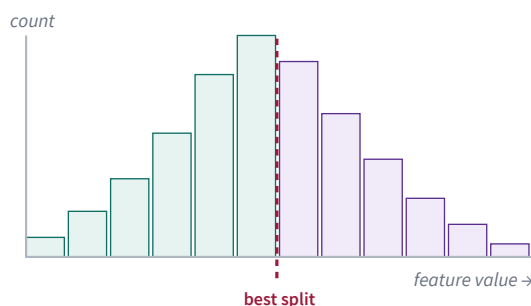
and the loss reduction achieved by that leaf is $-1/2(\sum g_i)^2 / \sum h_i + \lambda$. Splitting a node into L and R therefore gains

$$\text{gain} = 1/2[G_L^2/H_L + \lambda + G_R^2/H_R + \lambda - (G_L + G_R)^2/(H_L + H_R) + \lambda] - \gamma$$

The only quantities needed are the *sums* G and H on each side — which is exactly why prefix sums over a sorted feature give every candidate split in one pass, and why histogram binning works.

The exact method costs $\Theta(nd \log n)$ per level (the sorts dominate), or $\Theta(nd)$ if you presort once and keep index lists. For $n=10^7$ and $d=100$ that is still enormous, and it is where histograms come in.

Histogram-based split finding



Pre-bin every feature once:
 $\Theta(n \log n)$ at the start,
 then $\Theta(\#bins)$ per split.

Naive exact splitting is
 $\Theta(n)$ per feature per node.

Histogram subtraction:
 $\text{hist}(\text{right}) = \text{hist}(\text{parent})$
 $\quad - \text{hist}(\text{left})$
 — half the work, free.

Figure 37.1 Histogram-based splitting. Feature values are bucketed once into at most 256 bins; finding the best split then scans bins rather than samples.

Histogram split finding

```

1     import numpy as np
2

```

```

3  def build_bins(X, max_bins=255):
4      """Quantile-bin each feature once, before training starts."""
5      edges = []
6      for j in range(X.shape[1]):
7          qs = np.quantile(X[:, j], np.linspace(0, 1, max_bins + 1)[1:-1])
8          edges.append(np.unique(qs))
9      binned = np.empty(X.shape, dtype=np.uint8)
10     for j in range(X.shape[1]):
11         binned[:, j] = np.searchsorted(edges[j], X[:, j], side="right")
12     return binned, edges
13
14 def node_histogram(binned, rows, g, h, n_bins=256):
15     """Accumulate (sum g, sum h, count) per bin per feature: theta(|rows|
16         * d)."""
17     d = binned.shape[1]
18     hist_g = np.zeros((d, n_bins))
19     hist_h = np.zeros((d, n_bins))
20     for j in range(d):
21         np.add.at(hist_g[j], binned[rows, j], g[rows])
22         np.add.at(hist_h[j], binned[rows, j], h[rows])
23     return hist_g, hist_h
24
25 def best_split_hist(hist_g, hist_h, lam=1.0, min_child=1.0):
26     """theta(d * n_bins) -- independent of the number of samples."""
27     best = (None, None, 0.0)
28     for j in range(hist_g.shape[0]):
29         GL = np.cumsum(hist_g[j])
30         HL = np.cumsum(hist_h[j])
31         G, H = GL[-1], HL[-1]
32         GR, HR = G - GL, H - HL
33         parent = G * G / (H + lam)
34         gain = GL ** 2 / (HL + lam) + GR ** 2 / (HR + lam) - parent
35         gain[(HL < min_child) | (HR < min_child)] = -np.inf
36         b = int(np.argmax(gain))
37         if gain[b] > best[2]:
38             best = (j, b, float(gain[b]))
39     return best

```

Three consequences, and they compound:

- 1. Binning is done once.** $\Theta(nd \log n)$ at the start, then never again. Feature values become `uint8`, so the training matrix shrinks by 4x (from float32) — a direct bandwidth win (Chapter 3).
- 2. Split finding is $\Theta(d \cdot \text{bins})$, not $\Theta(nd)$.** Once the histogram is built, finding the best split does not look at the data at all.
- 3. Histogram subtraction halves the work.** Build the histogram of the *smaller* child by scanning its rows, then obtain the larger child's for free:

$$\text{hist}(\text{right}) = \text{hist}(\text{parent}) - \text{hist}(\text{left})$$

Since you always scan the smaller side, the total work per level is at most half the node's rows, and the whole tree costs $\Theta(nd)$ rather than $\Theta(nd \cdot \text{depth})$.

KEY INSIGHT | The accuracy cost of binning is close to zero

Quantising a feature to 255 bins loses almost nothing, because a decision tree only ever uses a threshold — it cannot exploit resolution finer than the gap between adjacent training values in a region. Empirically, 255 bins matches exact splitting to within noise on almost every dataset, while running 10–50x faster. This is a rare case where an approximation is essentially free.

Leaf-wise versus level-wise growth

Strategy	Grows	Used by	Behaviour
Level-wise (depth-first by level)	all nodes at depth ℓ	XGBoost (default)	balanced, more robust, more wasted splits
Leaf-wise (best-first)	the single highest-gain leaf anywhere	LightGBM	fewer leaves for the same loss; deeper, can overfit

Leaf-wise growth is maintained with a priority queue of (leaf, best gain), which is the heap of Chapter 13: pop the best leaf, split it, push its two children. That is the entire control structure of LightGBM’s tree builder.

Categorical features and missing values

Categoricals. One-hot encoding a 10,000-category feature creates 10,000 nearly-empty binary features, and a tree needs depth 10,000 to isolate one category. The correct treatment is the *optimal partition* of categories into two groups — which is exponential in general, but Fisher’s theorem says that for a convex loss, sorting categories by $\Sigma g / \Sigma h$ and taking a contiguous split of that ordering is optimal. So: sort the bins by their gradient statistics, then run the same prefix-sum scan. $\Theta(k \log k)$ instead of $\Theta(2^k)$.

Missing values. Rather than imputing, learn a default direction: during split finding, try sending all missing values left and all right, and keep whichever gives more gain. This costs one extra pass over the histogram and handles missingness as signal rather than noise — which matters, because missingness is very often informative.

WHERE THIS SHOWS UP IN ML | Why GBDT still wins on tabular data

Neural networks have strong inductive biases for images, audio and text — locality, translation invariance, sequence structure. Tabular data has none of those: columns are heterogeneous, unordered, differently scaled, and often have sharp thresholds (“age > 65”). Trees represent axis-aligned thresholds natively, are invariant to monotone transformations of any feature, handle missing values and categoricals without preprocessing, and need no normalisation. That combination is hard to beat, and repeated benchmark studies continue to find GBDT at or above the best deep models on medium-sized tabular datasets — at a fraction of the training cost.

PERFORMANCE NOTE | Inference is where trees really shine

A trained GBDT with 500 trees of depth 6 evaluates 500 root-to-leaf paths of 6 comparisons: 3000 comparisons and 500 additions per prediction — perhaps 2 microseconds. Compare with a small MLP: hundreds of thousands of multiply-accumulates. This is why trees dominate latency-critical ranking and fraud systems.

The engineering is all about branch prediction and cache: a naive pointer tree is a chain of dependent, unpredictable loads. Production implementations flatten trees into arrays, evaluate several trees in an interleaved loop so independent loads overlap, and compile to branchless arithmetic on the comparison results. QuickScorer and Treelite reach 5–10x over naive traversal by exactly the techniques of Chapter 3.

EXERCISES

1. Implement exact split finding and histogram split finding, and compare training time and final accuracy on a dataset with 10^6 rows.
2. Implement histogram subtraction and measure the reduction in histogram build time. Confirm the “always scan the smaller child” rule matters.
3. Vary `max_bins` from 8 to 1024 and plot validation accuracy against training time. Find the knee.
4. Derive the optimal leaf value w^* and the gain formula from the second-order Taylor expansion of squared loss and of logistic loss.
5. Implement leaf-wise growth with a priority queue and compare tree shape and validation loss to level-wise at equal leaf count.
6. Implement Fisher’s sorted-category split for a 5,000-category feature and compare to one-hot encoding in accuracy and training time.
7. Implement the learned default direction for missing values and construct a dataset where missingness is informative. Compare to mean imputation.
8. Flatten a trained tree ensemble into arrays and implement a branchless batched evaluator. Measure predictions per second against a recursive traversal.

CHAPTER RECAP

- Tree induction is greedy: pick the best split by gain, recurse. Globally optimal trees are NP-hard.
- The gradient-boosting gain formula needs only Σg and Σh on each side, which is why prefix sums and histograms both work.
- Histogram binning makes split finding $\Theta(d \cdot \text{bins})$ instead of $\Theta(nd)$, shrinks the feature matrix to `uint8`, and costs almost no accuracy.
- Histogram subtraction gives the larger child's histogram for free; always scan the smaller child.
- Leaf-wise growth is best-first search with a priority queue; it needs leaf count and minimum-samples controls rather than depth alone.
- Categoricals are split optimally by sorting on $\Sigma g / \Sigma h$ (Fisher), and missing values get a learned default direction.
- Inference is thousands of comparisons, so tree models are extremely fast to serve — provided the trees are flattened and the loads overlap.

Streaming and Online Algorithms

One pass, bounded memory, and an answer at every moment

ADVANCED

PREREQS: Chapters 11, 28

Production ML systems are streams: events arrive, predictions are made, metrics must be current, and nothing can be re-read. This chapter covers the algorithms that compute statistics, quantiles and drift in one pass with bounded memory — including the numerically stable updates that stop your monitoring dashboard reporting negative variances.

WHAT YOU WILL LEARN

- Implement numerically stable streaming mean, variance and covariance
- Implement exponentially weighted statistics and know their effective window
- Implement t-digest for accurate tail quantiles with mergeable state
- Detect distribution drift with a two-window test
- Understand the competitive-ratio framing of online algorithms
- Recognise which monitoring computations are safe to distribute

Why streaming is a different discipline

A streaming algorithm sees each item once, uses memory sub-linear in the stream length, and must have an answer available at any moment. That third requirement is what separates it from a batch job.

Statistic	Exact in one pass?	Memory
Count, sum, mean	yes	$\Theta(1)$
Variance, covariance	yes (Welford)	$\Theta(1)$
Min, max	yes	$\Theta(1)$
Any quantile	no	$\Omega(n)$ for exact
Approximate quantile ($\pm\epsilon$)	yes	$\Theta(1/\epsilon)$
Distinct count	no	$\Omega(n)$ exact; $\Theta(1)$ approximate (HLL)
Top- k frequent	no	$\Theta(k)$ approximate (Space-Saving)
Median of a window	yes	$\Theta(w)$ (two heaps, Chapter 13)

Stable moments

Welford's algorithm: numerically stable, mergeable

```

1  class RunningStats:
2      __slots__ = ("n", "mean", "m2")
3
4      def __init__(self):
5          self.n, self.mean, self.m2 = 0, 0.0, 0.0
6
7      def push(self, x):
8          self.n += 1
9          delta = x - self.mean
10         self.mean += delta / self.n
11         self.m2 += delta * (x - self.mean)      # note: uses the NEW mean
12
13     @property
14     def variance(self):
15         return self.m2 / (self.n - 1) if self.n > 1 else 0.0
16
17     def merge(self, other):                      # Chan's parallel update
18         if other.n == 0:
19             return self
20         n = self.n + other.n
21         delta = other.mean - self.mean
22         self.m2 += other.m2 + delta * delta * self.n * other.n / n
23         self.mean += delta * other.n / n
24         self.n = n
25         return self

```

WATCH OUT | The naive variance formula is genuinely broken

$\text{Var} = 1/n \sum x_i^2 - \bar{x}^2$ subtracts two nearly equal large numbers. For values around 10^6 with a standard deviation of 1, the result in float64 loses most of its significant digits and frequently comes out *negative*. Welford's update never forms those large intermediates and is accurate to within a few units in the last place. Use it; it is the same number of operations.

Note also `merge`: Welford's state combines exactly, so per-shard statistics reduce to a global answer in a tree (Chapter 16). Not every statistic has this property, and the ones that do are the ones you can compute across a fleet.

Exponentially weighted statistics for a drifting stream

```

1  class EWMA:
2      """alpha in (0,1); effective window is about 1/alpha samples."""
3      def __init__(self, alpha=0.01):
4          self.alpha = alpha
5          self.mean = None
6          self.var = 0.0
7
8      def push(self, x):
9          if self.mean is None:
10             self.mean = float(x)
11             return
12         delta = x - self.mean

```

```

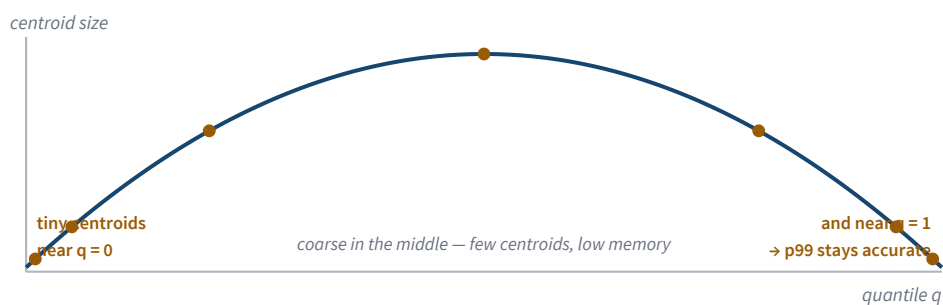
13         self.mean += self.alpha * delta
14         self.var = (1 - self.alpha) * (self.var + self.alpha * delta *
    ↵           delta)

```

An EWMA needs $\Theta(1)$ memory and forgets smoothly, which is what you want for a metric on a non-stationary stream. Its effective window is $1/\alpha$ samples: $\alpha=0.01$ averages over roughly the last 100 observations. Choosing α is choosing how quickly your monitor reacts and how noisy it is, and it should be stated in the alert definition.

Quantiles: t-digest

Tail quantiles are what monitoring actually needs — p99 latency, p999 score — and they are exactly where naive approaches fail.



Merging two digests merges centroid lists — so p99 across a fleet is a tree reduction, not a global sort.

Figure 38.1 t-digest. A scale function makes centroids small near $q=0$ and $q=1$ and large in the middle, so the tails stay accurate while total memory stays bounded.

DEFINITION | t-digest

A set of *centroids*, each with a count and a mean. A centroid at quantile q may absorb at most $4n\delta q(1-q)$ points, where δ is a compression parameter. This is small near the tails and large in the middle, so relative error at $q=0.999$ stays tiny while memory stays $\Theta(\delta)$. Digests merge by concatenating centroid lists and re-compressing.

A simple t-digest

```

1  import bisect
2
3  class TDigest:
4      def __init__(self, compression=100):
5          self.delta = compression
6          self.centroids = []          # sorted list of [mean, count]
7          self.n = 0
8
9      def push(self, x, w=1):
10         self.n += w
11         i = bisect.bisect_left([c[0] for c in self.centroids], x)
12         best, best_d = None, float("inf")
13         for j in (i - 1, i):          # nearest centroid on either side
14             if 0 <= j < len(self.centroids):
15                 d = abs(self.centroids[j][0] - x)
16                 if d < best_d and self._can_absorb(j, w):

```



```

17         best, best_d = j, d
18     if best is None:
19         self.centroids.insert(i, [float(x), w])
20     else:
21         m, c = self.centroids[best]
22         self.centroids[best] = [m + (x - m) * w / (c + w), c + w]
23     if len(self.centroids) > 20 * self.delta:
24         self._compress()
25
26     def _can_absorb(self, j, w):
27         before = sum(c[1] for c in self.centroids[:j])
28         q = (before + self.centroids[j][1] / 2) / max(self.n, 1)
29         limit = 4 * self.n * q * (1 - q) / self.delta
30         return self.centroids[j][1] + w <= max(limit, 1)
31
32     def _compress(self):
33         import random
34         pts = [(m, c) for m, c in self.centroids]
35         random.shuffle(pts)
36         self.centroids, self.n = [], 0
37         for m, c in pts:
38             self.push(m, c)
39
40     def quantile(self, q):
41         if not self.centroids:
42             return float("nan")
43         target = q * self.n
44         acc = 0.0
45         for m, c in self.centroids:
46             if acc + c / 2 >= target:
47                 return m
48             acc += c
49         return self.centroids[-1][0]

```

Method	Memory	Tail accuracy	Mergeable
Store everything, sort	$\Theta(n)$	exact	yes (concatenate)
Fixed histogram	$\Theta(B)$	poor unless range is known	yes
Reservoir sample	$\Theta(k)$	sampling error at the tail	no (needs care)
P ² algorithm	$\Theta(1)$ per quantile	poor on skew	no
GK (Greenwald–Khanna)	$\Theta(1/\epsilon \log \epsilon n)$	uniform ϵ	yes
t-digest	$\Theta(\delta)$	excellent at the tails	yes
DDSketch	$\Theta(1/\gamma \log \text{range})$	relative error guarantee	yes

Table 38.1 What one pass can and cannot give you. The exactness boundary is worth knowing: quantiles and distinct counts *cannot* be exact in sub-linear memory, which is a theorem, not an implementation gap.

Drift detection

A two-window drift test

```

1  import numpy as np
2  from collections import deque
3
4  class DriftDetector:
5      """Compare a reference window to a recent window."""
6      def __init__(self, ref_size=10_000, cur_size=1_000, bins=32):
7          self.ref = deque(maxlen=ref_size)
8          self.cur = deque(maxlen=cur_size)
9          self.bins = bins
10
11     def push(self, x):
12         self.cur.append(x)
13         if len(self.ref) < self.ref.maxlen:
14             self.ref.append(x)
15
16     def psi(self):
17         """Population stability index: sum (p - q) * log(p / q)."""
18         if len(self.cur) < self.cur.maxlen:
19             return 0.0
20         edges = np.quantile(self.ref, np.linspace(0, 1, self.bins + 1))
21         edges[0], edges[-1] = -np.inf, np.inf
22         p, _ = np.histogram(self.ref, bins=edges)
23         q, _ = np.histogram(self.cur, bins=edges)
24         p = (p + 1) / (p.sum() + self.bins)      # Laplace smoothing
25         q = (q + 1) / (q.sum() + self.bins)
26         return float(((p - q) * np.log(p / q)).sum())

```

PSI above 0.1 conventionally indicates moderate drift and above 0.25 significant drift. The alternatives are a Kolmogorov–Smirnov test (distribution-free, but sensitive to sample size, so it flags trivial drift in large windows), and ADWIN, which adaptively resizes the window and has a false-positive guarantee.

COMMON PITFALL | Drift detection on a schedule is drift detection on the wrong data

Comparing “today” to “the training set” flags seasonality as drift every Monday. Comparing to “last week” misses slow degradation entirely. And feature drift is not the same as *performance* drift — features can move a long way with no accuracy loss, and accuracy can fall with no feature drift at all (label shift, concept drift). Monitor both, and alert on the one you actually care about: the metric that affects users.

Online algorithms and competitive ratios

DEFINITION | Competitive ratio

An online algorithm, which must decide irrevocably without seeing the future, is c -competitive if for every input its cost is at most c times the cost of the optimal offline algorithm that knows the entire input in advance.

This framing turns several practical questions into provable statements. Caching: LRU is k -competitive for a cache of size k , and no deterministic policy does better; randomised marking is $O(\log k)$ -competitive. The ski-rental problem (buy or keep renting?) has a 2-competitive deterministic solution and an $e/(e-1) \approx 1.58$ randomised one — and it is the exact shape of “keep a GPU instance warm or pay cold-start latency?”, “keep a KV cache or recompute it?”, and “hold a connection open or reconnect?”.

WHERE THIS SHOWS UP IN ML | The streaming pieces of an ML system

Online feature computation. Rolling counts and rates per entity, computed with EWMA or a windowed aggregate, must match exactly between training and serving or you get training–serving skew. The usual bug is computing the feature in batch SQL for training and in a stream processor for serving, with subtly different window semantics.

Metric monitoring. Throughput, latency percentiles and error rates from t-digests merged across replicas.

Online learning. SGD is itself an online algorithm; bandit algorithms (UCB, Thompson sampling) have regret bounds that are the online-algorithm framing of exploration.

Streaming inference. Sliding-window attention with a ring-buffer KV cache (Chapters 6 and 36) is a streaming algorithm: bounded memory, one pass, answer available at every step.

EXERCISES

1. Show that the naive variance formula returns a negative value for 10^6 samples drawn from $N(10^8, 1)$ in float32, and that Welford’s does not.
2. Implement Welford’s **merge** and verify it exactly matches a single-pass computation over the concatenated data.
3. Derive the effective window of an EWMA with parameter α , and verify it by measuring the lag of the mean after a step change.
4. Implement t-digest and measure the relative error at $q \in \{0.5, 0.9, 0.99, 0.999\}$ for a log-normal stream, as a function of **compression**.
5. Merge 100 t-digests computed on disjoint shards and compare the merged quantiles to the exact ones on the full data.
6. Implement Space-Saving for top- k heavy hitters and compare it to a Count–Min sketch plus heap (Chapter 28) at equal memory.
7. Implement PSI and KS drift tests, and show that KS flags drift on a distribution that has not meaningfully changed when the window is large.
8. Formulate “keep the model warm on a GPU or evict it and pay the load time” as ski-rental. State the 2-competitive policy and simulate it against the offline optimum on a realistic request trace.

CHAPTER RECAP

- Streaming algorithms see each item once, use sub-linear memory, and must have an answer at any moment.
- Welford's algorithm gives stable mean and variance and merges exactly across shards; the naive $E[x^2] - E[x]^2$ formula is numerically broken.
- EWMA gives $\Theta(1)$ forgetting with effective window $1/\alpha$; state that window in your alert definitions.
- Exact quantiles need linear memory; t-digest keeps small centroids at the tails for excellent p99 accuracy in bounded, mergeable state. DDSketch gives a relative-error guarantee.
- Drift detection needs the right reference window and should distinguish feature drift from performance drift.
- The competitive ratio formalises online decisions: LRU is k -competitive; ski-rental is 2-competitive deterministically and $e/(e-1)$ randomised, and describes several real caching decisions.

PART VIII

Parallel, Distributed and Hardware-Aware

One core is a special case. Modern ML runs on thousands of them, and the winning algorithm is the one that moves the least data, not the one that does the fewest operations.

-
- 39 Parallel Primitives: Scan, Reduce, Sort
 - 40 GPU Algorithm Design
 - 41 Distributed Algorithms: All-Reduce, Sharding, Consistent Hashing
 - 42 External Memory and the Data Loading Pipeline

Parallel Primitives: Scan, Reduce, Sort

Six operations from which every parallel algorithm is built

FRONTIER

PREREQS: Chapters 16, 39

Parallel programming looks like an open-ended discipline and is mostly not: a small set of primitives — map, reduce, scan, compaction, sort, gather/scatter — composes into the overwhelming majority of parallel algorithms, and every GPU library implements exactly these. This chapter derives them, analyses them in the work/depth model, and shows where each appears in an ML stack.

WHAT YOU WILL LEARN

- Analyse parallel algorithms with work, depth and Brent's theorem
- Implement parallel reduction and understand tree contraction
- Derive the Blelloch scan and its $\Theta(n)$ work, $\Theta(\log n)$ depth
- Use scan to build compaction, partition, histogram and radix sort
- Recognise these primitives inside dataloaders, tokenizers and attention kernels
- Reason about Amdahl's law and where parallel speedup actually stops

Work and depth

DEFINITION | Work and depth

The **work** W is the total number of operations, and the **depth** (or span) D is the length of the longest chain of dependencies — the time with unlimited processors. Parallelism is W/D .

Brent's theorem. With P processors, the running time is at most $W/P + D$. So an algorithm is efficient if its work matches the best sequential algorithm (it is *work-efficient*) and its depth is small.

This is the right model because it separates two questions: are you doing more total work than necessary, and is there a serial bottleneck?

Primitive	Work	Depth	Example use
Map	$\Theta(n)$	$\Theta(1)$	elementwise ops, embedding lookup

Primitive	Work	Depth	Example use
Reduce	$\Theta(n)$	$\Theta(\log n)$	loss sum, norm, softmax denominator
Scan (prefix sum)	$\Theta(n)$	$\Theta(\log n)$	offsets, compaction, sorting
Compaction / filter	$\Theta(n)$	$\Theta(\log n)$	sparse masking, candidate pruning
Sort (radix)	$\Theta(kn)$	$\Theta(k \log n)$	top-k, grouping, MoE routing
Gather / scatter	$\Theta(n)$	$\Theta(1)$	embedding lookup and its gradient
Segmented scan	$\Theta(n)$	$\Theta(\log n)$	ragged batches, CSR row sums

Reduction

Tree reduction: the shape, in one function

```

1  def tree_reduce(xs, op):
2      """Work  $\theta(n)$ , depth  $\theta(\log n)$ ."""
3      cur = list(xs)
4      while len(cur) > 1:
5          nxt = [op(cur[i], cur[i + 1]) for i in range(0, len(cur) - 1, 2)]
6          if len(cur) % 2:
7              nxt.append(cur[-1])
8          cur = nxt
9      return cur[0]

```

each level runs fully in parallel

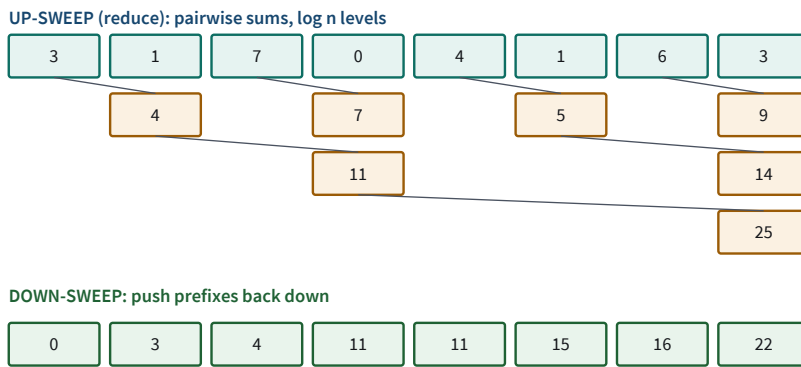
On a GPU this is done within a warp with shuffle instructions (no shared memory at all), then across warps in shared memory, then across blocks with atomics or a second kernel. The three-level structure — warp, block, grid — is the standard shape of every reduction kernel you will read.

COMMON PITFALL | Floating-point reduction is not associative

$(a+b)+c \neq a+(b+c)$ in floating point, so a tree reduction gives a different answer from a sequential one, and a *different* answer on a different number of threads. This is the main source of “why is my loss not bit-identical across runs?” and why `torch.use_deterministic_algorithms(True)` makes some kernels slower: it forces a fixed reduction order. Tree reduction is usually *more* accurate than sequential summation (error grows as $\log n$ rather than n), but it is not reproducible unless the order is pinned.

Scan

The prefix sum looks inherently sequential — each output depends on the one before it — and is not.



exclusive prefix sum

$\Theta(n)$ total work, $\Theta(\log n)$ depth — the primitive behind compaction, sorting, histograms and ragged offsets.

Figure 39.1 The Blelloch scan. An up-sweep builds a reduction tree in place; a down-sweep pushes prefixes back down. Both phases are $\Theta(n)$ work and $\Theta(\log n)$ depth.

Blelloch exclusive scan

```

1  def blelloch_scan(a):
2      """Exclusive prefix sum. Work theta(n), depth theta(log n)."""
3      n = len(a)
4      x = list(a)
5      step = 1
6      while step < n:                                # up-sweep: build the reduce
7          tree
8          for i in range(step * 2 - 1, n, step * 2):    # parallel
9              x[i] += x[i - step]
10         step *= 2
11     total = x[n - 1]
12     x[n - 1] = 0                                     # clear the root
13     step //= 2
14     while step >= 1:                                  # down-sweep: distribute
15         prefixes
16         for i in range(step * 2 - 1, n, step * 2):    # parallel
17             left = x[i - step]
18             x[i - step] = x[i]
19             x[i] += left
20         step //= 2
21     return x, total
  
```

The naive parallel scan (Hillis–Steele) is simpler but does $\Theta(n \log n)$ work — fine when n is small and processors are idle, wasteful otherwise. Blelloch is work-efficient, which matters when the scan is over millions of elements. Real implementations use a *decoupled look-back* single-pass scan that achieves near-memory-bandwidth throughput.

What scan gives you

Stream compaction. Keep the elements passing a predicate, contiguously.

Compaction: the canonical scan application

```

1  def compact(xs, keep):
2      flags = [1 if k else 0 for k in keep]
  
```



```

3     offsets, total = blleloch_scan(flags)          # where each survivor goes
4     out = [None] * total
5     for i, f in enumerate(flags):                  # fully parallel scatter
6         if f:
7             out[offsets[i]] = xs[i]
8     return out

```

Radix sort. One pass per digit: compute a histogram of digit values, scan it to get bucket offsets, scatter. Entirely built from reduce, scan and scatter — which is why GPU sorting is radix sorting (Chapter 9).

Ragged offsets. Given per-sequence lengths, the scan gives the `offsets` array of Chapter 4 — the layout behind nested tensors, CSR graphs and paged KV caches.

Segmented scan. A scan that restarts at segment boundaries, computing per-sequence cumulative sums in one pass. This is how a batched softmax denominator, a per-row CSR sum, and a per-sequence cumulative reward are all computed without a loop over sequences.

WHERE THIS SHOWS UP IN ML | The primitives inside your stack

`torch.cumsum` is a scan. `torch.nonzero` is compaction (and it is synchronising, because the output size is data-dependent — the usual cause of an unexplained stall in a training loop). `torch.topk` is a radix select. `scatter_add` for embedding gradients is a scatter with atomics. **MoE routing** sorts tokens by expert id (radix sort), scans to find each expert's offsets, and gathers — exactly the compaction pattern. The softmax denominator is a segmented reduction. Recognising them tells you the cost and where the synchronisation points are.

Amdahl, Gustafson, and where speedup stops

THE MATH | Amdahl's law

If a fraction s of a program is inherently serial, the speedup with P processors is

$$S(P) = 1/s + (1-s)/P \leq 1/s$$

With $s=0.05$, the speedup is capped at 20 no matter how many processors you add.

Gustafson's rejoinder is that in practice we scale the *problem* with the machine, so the serial fraction shrinks — which is exactly what happens when you raise the batch size along with the GPU count.

The serial fraction in ML training is rarely arithmetic. It is data loading, the optimiser step, checkpointing, and the synchronisation barrier at each all-reduce. Those are what limit scaling, which is why the next chapters are about moving data rather than doing arithmetic.

PERFORMANCE NOTE | The parallel primitives you should never hand-write

`cub::DeviceScan`, `cub::DeviceRadixSort`, `cub::DeviceReduce`, and their Thrust and PyTorch wrappers are tuned to within a few percent of memory bandwidth by people who have measured every variant. A hand-written scan is typically 3–10x slower. Write your own only to fuse a primitive with surrounding work so that intermediates never reach memory — which is the one thing a library cannot do for you, and exactly what Chapter 40 is about.

EXERCISES

1. Compute work, depth and parallelism for: tree reduction, Hillis–Steele scan, Blelloch scan, and bitonic sort.
2. Implement both scans and verify their work counts empirically by instrumenting the operation count.
3. Implement stream compaction with a scan and compare it to a sequential filter for $n=10^7$.
4. Implement radix sort using only histogram, scan and scatter. Verify it is stable.
5. Implement segmented scan and use it to compute per-sequence cumulative sums for a ragged batch, without a Python loop.
6. Demonstrate non-associativity: sum 10^7 float32 values sequentially and by tree reduction, and compare both to a float64 reference. Which is more accurate?
7. Measure the serial fraction of a small training loop by timing with 1, 2, 4 and 8 dataloader workers, and fit Amdahl’s law.
8. Show that `torch.nonzero` synchronises, by timing a loop with and without it under `torch.cuda.synchronize` instrumentation. Propose an alternative that keeps a fixed output size.

CHAPTER RECAP

- Work is total operations, depth is the longest dependency chain; Brent’s theorem gives time $\leq W/P+D$.
- Map, reduce, scan, compaction, sort and gather/scatter compose into essentially every parallel algorithm, and all have $\Theta(\log n)$ depth or better.
- Tree reduction is the shape of every GPU reduction kernel: warp, block, grid.
- Floating-point reduction is not associative, so results depend on thread count; determinism costs speed.
- The Blelloch scan is work-efficient with logarithmic depth, and scan is the building block of compaction, radix sort, ragged offsets and segmented aggregation.
- Amdahl’s law caps speedup at $1/s$; in ML the serial fraction is usually data loading and synchronisation, not arithmetic.
- Use the library primitives; write your own only to fuse them with neighbouring work.

GPU Algorithm Design

Tiling, coalescing, and why FlashAttention does more arithmetic

FRONTIER

PREREQS: Chapters 3, 39

A GPU changes the cost model so thoroughly that algorithms which are optimal on a CPU can be badly wrong on it. The unit of execution is a warp of threads that must agree on their branches; the unit of memory is a 128-byte transaction that must be requested by consecutive threads; and the scarce resource is bandwidth, not arithmetic. This chapter derives the design rules and works through FlashAttention as the canonical example.

WHAT YOU WILL LEARN

- Describe the GPU execution and memory model accurately enough to reason about cost
- Apply the four rules: coalesce, tile, avoid divergence, occupy
- Compute arithmetic intensity and place a kernel on the roofline
- Derive FlashAttention as an IO-aware algorithm and explain online softmax
- Understand why fusion and quantisation are the main serving levers
- Know which algorithmic choices to reconsider on a GPU

The execution model in one page

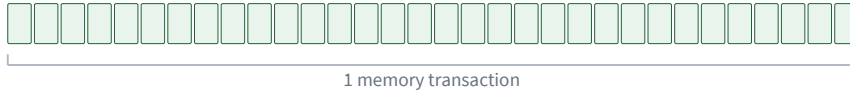
A GPU has thousands of simple cores arranged in streaming multiprocessors. Threads execute in **warps** of 32 that share a program counter, so if threads in a warp take different branches, both branches run with the inactive threads masked — the cost is the *sum* of the paths, not the maximum.

Level	Size (typical)	Latency	Shared by
Registers	64 K per SM	1 cycle	one thread
Shared memory / L1	100–228 KB per SM	~30 cycles	one block
L2	40–60 MB	~200 cycles	whole device
HBM	40–192 GB	~400 cycles	whole device
Host RAM over PCIe	TB	~10 μ s	everything

The four rules

1. Coalesce. Consecutive threads must read consecutive addresses, so the hardware merges 32 requests into one transaction.

COALESCED: 32 consecutive threads → one 128-byte transaction



STRIDED: thread i reads element $32i \rightarrow 32$ separate transactions



Identical arithmetic. On a memory-bound kernel this is the entire performance difference.

Figure 40.1 Coalesced versus strided access. The arithmetic is identical; the memory traffic differs by 32x. This single property decides the performance of most memory-bound kernels.

This is why layout matters so much (Chapter 3): a kernel over a (batch, seq, dim) tensor should have its fastest-varying thread index on **dim**, and if your data is laid out the other way you must transpose it first. It is also why **NHWC** beats **NCHW** for convolutions on tensor cores.

2. Tile. Load a block of data into shared memory once and reuse it many times.

naive matmul: $\Theta(n^3)$ HBM reads $\rightarrow$ tiled with tile b : $\Theta(n^3/b + n^2)$

Each element of a $b \times b$ tile is used b times while resident. With $b=64$ that is a 64x reduction in memory traffic for identical arithmetic. Every fast GEMM, convolution and attention kernel is tiled.

3. Avoid divergence. Within a warp, branches serialise. Replace `if (x > 0) y = a; else y = b;` with arithmetic: `y = (x > 0) * a + (x <= 0) * b`, or use predication. Data-dependent loop bounds are worse — if one thread iterates 100 times and its 31 neighbours iterate once, the warp takes 100 iterations. This is why ragged workloads must be sorted or padded into uniform buckets before a kernel runs.

4. Occupy. Each SM can host many warps concurrently and switches between them for free, which is how it hides memory latency. Occupancy is limited by registers per thread and shared memory per block. Using too much of either reduces the number of resident warps, and past a point the SM has nothing to run while waiting on memory. Maximum occupancy is not the goal — a register-heavy kernel with 50% occupancy often beats a lean one with 100% — but zero slack is fatal.

The roofline

THE MATH | Where your kernel lives

Plot achievable FLOP/s against arithmetic intensity I (FLOPs per byte of HBM traffic):

$$\text{attainable} = \min(\text{peak FLOP/s}, I \times \text{bandwidth})$$

The ridge point is at $I^* = \text{peak FLOP/s} / \text{bandwidth}$, about 100–500 on modern accelerators.

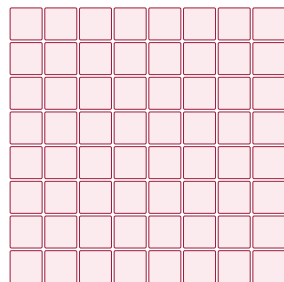
- $I \ll I^*$: memory-bound. Reducing FLOPs does nothing; reduce bytes.
- $I \gg I^*$: compute-bound. Use tensor cores, lower precision, better tiling.

Elementwise ops have $I \approx 0.1$. LayerNorm and softmax, $I \approx 1$. Attention during decoding, $I \approx 1$. Batched GEMM with large matrices, $I = \Theta(n)$. This is why almost everything except the big matmuls is memory-bound.

FlashAttention

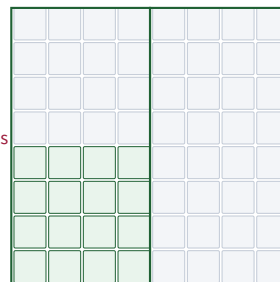
Standard attention computes $S = QK^{\text{top}} (n \times n)$, then $P = \text{softmax}(S)$, then $O = PV$. For $n=8192$ the score matrix is 268 million float16 values — 537 MB — written to HBM and read back twice.

STANDARD: materialise the full $n \times n$ scores in HBM



$O(n^2)$ writes + $O(n^2)$ reads

FLASH: one tile at a time, in SRAM



online softmax keeps a running max and sum, so no tile ever needs the others.

More FLOPs, far fewer bytes moved — and the wall-clock time falls by 2–4× (Chapter 3).

Figure 40.2 FlashAttention. The $n \times n$ score matrix is never materialised: Q , K and V are tiled so each block of scores is computed, consumed and discarded inside SRAM, with an online softmax carrying a running maximum and sum.

The obstacle is the softmax: it needs a global maximum and a global sum over each row, and a tiled computation only sees part of a row at a time. The resolution is the **online softmax**, which is a streaming algorithm (Chapter 38) with a rescaling correction.

THE MATH | Online softmax

Process a row in blocks. Maintain the running maximum m and running sum ell and output accumulator O . On seeing block j with scores $s^{(j)}$:

$$\begin{aligned} m_{\text{new}} &= \max(m, \max_i s_i^{(j)}), \alpha = e^{m - m_{\text{new}}} \\ \text{ell}_{\text{new}} &= \alpha \text{ell} + \sum_i e^{s_i^{(j)} - m_{\text{new}}}, O_{\text{new}} = \alpha O + (e^{s^{(j)} - m_{\text{new}}}) V^{(j)} \end{aligned}$$

The factor α rescales everything accumulated so far to the new maximum. At the end, divide O by ell . The result is *exactly* the softmax — this is not an approximation — and it never needs more than one block of scores in memory at a time.

Online softmax attention, the algorithm (not the kernel)

```
1 import numpy as np
```

```

2
3 def flash_attention(Q, K, V, block=64):
4     """Mathematically identical to softmax(QK^T)V, with O(n) memory."""
5     n, d = Q.shape
6     0 = np.zeros((n, d), dtype=np.float32)
7     for i in range(0, n, block):          # tile over queries
8         q = Q[i:i + block]
9         m = np.full((len(q), 1), -np.inf, dtype=np.float32)
10        l = np.zeros((len(q), 1), dtype=np.float32)
11        acc = np.zeros((len(q), d), dtype=np.float32)
12        for j in range(0, n, block):      # tile over keys/values
13            k, v = K[j:j + block], V[j:j + block]
14            s = q @ k.T / np.sqrt(d)      # the tile, stays in SRAM
15            m_new = np.maximum(m, s.max(axis=1, keepdims=True))
16            alpha = np.exp(m - m_new)     # rescale what we have
17            pexp = np.exp(s - m_new)
18            l = alpha * l + pexp.sum(axis=1, keepdims=True)
19            acc = alpha * acc + pexp @ v
20            m = m_new
21        0[i:i + block] = acc / l
22    return 0

```

KEY INSIGHT | The lesson that generalises beyond attention

FlashAttention performs *more* floating-point operations than the standard implementation — the rescaling is extra work, and the backward pass recomputes the scores rather than storing them — and it is two to four times faster, with memory linear rather than quadratic in sequence length.

The reason is Chapter 3's thesis in its strongest form: on this hardware, the cost of an algorithm is the bytes it moves, and an algorithm that trades arithmetic for locality wins. When you are optimising a kernel, count HBM traffic, not FLOPs.

What to reconsider on a GPU

CPU intuition	GPU reality
Fewer operations is better	Fewer <i>bytes moved</i> is better
Branches are cheap	Divergent branches serialise the warp
Recomputation is waste	Recomputation is often cheaper than storing
Linked structures are fine	Pointer chasing has no parallelism
Sorting is $\Theta(n \log n)$	Radix sort is $\Theta(kn)$ and branch-free
<code>if</code> inside a loop is free	Sort or bucket first so warps are uniform
Small kernels are fine	Launch overhead is $\sim 5 \mu\text{s}$; fuse them
Dynamic shapes are fine	They trigger recompilation and sync points

Table 40.1 The GPU memory hierarchy. The ratio that matters: HBM bandwidth is 2–8 TB/s while compute is 100–1000 TFLOP/s, so the machine can do 100–500 FLOPs per byte fetched. A kernel with lower arithmetic intensity is bandwidth-bound, full stop.

WHERE THIS SHOWS UP IN ML | The three levers in serving, in order

1. **Batching.** Decoding is memory-bound (Chapter 36), so processing 32 sequences costs barely more than one — the weights are read once either way. Continuous batching is the single largest throughput lever.
2. **Quantisation.** Weights in int8 or fp8 halve or quarter the bytes read per token. Since decoding is bandwidth-bound, that is close to a proportional speedup.
3. **Fusion.** Each unfused elementwise operation is a full read and write of the activation tensor. Fusing a chain of ten into one kernel removes eighteen of the twenty HBM round trips. This is what `torch.compile`, Triton and TensorRT are for.

Notice that none of these reduces the arithmetic. All three reduce bytes moved.

EXERCISES

1. Write a coalesced and a strided CUDA (or Triton, or CuPy) kernel that sum an array, and measure the bandwidth of each.
2. Implement naive and tiled matrix multiplication and measure the achieved FLOP/s. Compute the HBM traffic of each and confirm the b -fold reduction.
3. Place five kernels on a roofline plot for your device: vector add, softmax, LayerNorm, a 4096^3 GEMM, and attention decoding at batch 1.
4. Implement `flash_attention` and verify it matches standard attention to float32 precision. Measure peak memory against sequence length for both.
5. Derive the online-softmax rescaling and prove it yields the exact softmax.
6. Construct a warp-divergent kernel (a data-dependent loop bound) and measure the slowdown against a sorted-input version.
7. Measure kernel launch overhead by timing 10^5 trivial kernels, and compute how many fused operations it takes to amortise it.
8. Take a small model, apply `torch.compile`, and count the kernels before and after with a profiler. Attribute the speedup to fusion versus other effects.

CHAPTER RECAP

- GPU threads run in warps of 32 sharing a program counter; divergent branches serialise.
- Consecutive threads must touch consecutive addresses, or memory traffic multiplies by up to 32.
- Tiling reuses loaded data from shared memory and is what makes GEMM, convolution and attention fast.
- Compute arithmetic intensity and place the kernel on the roofline before optimising; below the ridge point, only reducing bytes helps.
- FlashAttention never materialises the $n \times n$ scores, using an online softmax with a rescaling correction; it does more arithmetic and is several times faster.
- On a GPU, recomputation can be cheaper than storage, sorting is branch-free radix sort, and small kernels should be fused.
- The three serving levers — batching, quantisation, fusion — all reduce bytes moved, not operations.

Distributed Algorithms: All-Reduce, Sharding, Consistent Hashing

Algorithms whose cost is measured in messages

FRONTIER

PREREQS: Chapters 16, 39

Once a computation spans machines, the dominant cost is communication, and the algorithms that matter are the ones that minimise bytes on the wire. This chapter derives ring all-reduce — whose communication cost is independent of the number of workers, a genuinely surprising result — then covers the sharding and placement algorithms that hold a distributed training or serving system together.

WHAT YOU WILL LEARN

- Model communication cost with latency and bandwidth terms
- Derive ring all-reduce and prove its bandwidth optimality
- Choose among data, tensor, pipeline and expert parallelism by their communication patterns
- Implement consistent hashing with virtual nodes and bounded loads
- Understand gradient compression and asynchronous updates, and their costs
- Reason about stragglers, fault tolerance and checkpointing

The cost model

A message of N bytes between two nodes costs approximately

$$T = \alpha + N/\beta$$

with α the latency (1–10 μ s within a node over NVLink, 5–100 μ s across a network) and β the bandwidth (400–900 GB/s NVLink, 10–50 GB/s InfiniBand, 1–10 GB/s Ethernet).

Two regimes follow. For small messages, latency dominates and you should minimise the *number* of messages — batch them. For large messages, bandwidth dominates and you should minimise *bytes*. Gradient all-reduce is firmly in the second regime.

Ring all-reduce

Every data-parallel worker holds a gradient vector of N bytes; all must end with the sum. The naive designs are bad: a central parameter server receives PN bytes on one link, and naive all-to-all sends P^2 messages.

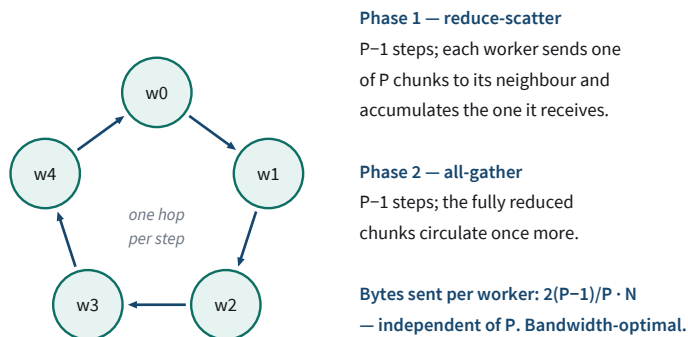


Figure 41.1 Ring all-reduce. Reduce-scatter leaves each worker with one fully reduced chunk; all-gather circulates those chunks. Each worker sends $2(P-1)/P \times N$ bytes regardless of P .

THE MATH | The derivation

Arrange P workers in a ring and split the gradient into P chunks of N/P bytes.

Reduce-scatter, $P-1$ steps. In step s , worker i sends chunk $(i-s) \bmod P$ to worker $i+1$ and adds the chunk it receives into its own. After $P-1$ steps, worker i holds the fully reduced chunk $(i+1) \bmod P$. Bytes sent per worker: $(P-1) \cdot N/P$.

All-gather, $P-1$ steps. Each worker passes its completed chunk around the ring. Bytes sent per worker: $(P-1) \cdot N/P$.

Total per worker: $\frac{2(P-1)PN}{P} \rightarrow 2N$ as P grows.

This is independent of P . Doubling the cluster does not increase the bytes each worker sends. The latency term does grow — $2(P-1)\alpha$ — which is why very large clusters use hierarchical or tree-based reductions that trade a little bandwidth for logarithmic latency.

Ring all-reduce, explicitly

```
1 def ring_allreduce(local, rank, world, send_rcv):
2     """local: list of P chunks. send_rcv(chunk, to, frm) -> received
3     chunk."""
4     P = world
5     for s in range(P - 1):
6         # reduce-scatter
7         send_idx = (rank - s) % P
8         rcv_idx = (rank - s - 1) % P
9         got = send_rcv(local[send_idx], (rank + 1) % P, (rank - 1) % P)
10        local[rcv_idx] = [a + b for a, b in zip(local[rcv_idx], got)]
11    for s in range(P - 1):
12        # all-gather
13        send_idx = (rank - s + 1) % P
14        rcv_idx = (rank - s) % P
15        got = send_rcv(local[send_idx], (rank + 1) % P, (rank - 1) % P)
16        local[rcv_idx] = got
17    return local
```

PERFORMANCE NOTE | Overlapping communication with computation

Gradients become available layer by layer during the backward pass, so the all-reduce for layer ℓ can start while layer $\ell-1$ is still computing. Bucketing (group small tensors into ~25 MB buckets to amortise latency) plus overlap hides most of the communication behind computation. This is what `DistributedDataParallel` does with its gradient hooks, and it is why DDP scales better than a manual “compute everything, then all-reduce” loop — the algorithm is the same, the *schedule* is not.

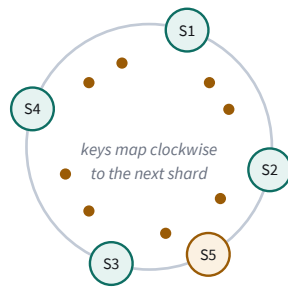
The four parallelisms

Strategy	Splits	Communicates	Per step	Best when
Data parallel	the batch	all gradients	$2N$ per worker	model fits on one device
Tensor parallel	each matrix	activations	2 all-reduces per layer	model too big for one device; fast interconnect
Pipeline parallel	the layers	boundary activations	small	many devices, slower interconnect
Expert parallel (MoE)	the experts	routed tokens	all-to-all	sparse models
ZeRO / FSDP	optimiser state, then params	params on demand	$\approx 1.5\times$ data parallel	memory-bound, want data-parallel simplicity

KEY INSIGHT | Pipeline bubbles are a scheduling problem

Splitting L layers across S stages means stage s is idle until stage $s-1$ produces its first output. With M micro-batches the bubble fraction is $(S-1)/(M+S-1)$: with $S=8$ and $M=8$ that is 47% idle. Raising M to 64 drops it to 10%. Interleaved (virtual-stage) schedules and zero-bubble variants reduce it further by reordering forward and backward work — which is a job-shop scheduling problem over a DAG (Chapter 24), solved once and compiled into the runtime.

Sharding and placement



Adding S5 moves only the keys
between S2 and S5 — about $1/N$.

$\text{hash}(\text{key}) \% N$ would move
 $(N-1)/N$ of every key.

Virtual nodes (100–200 per shard)
reduce load imbalance to $1/\sqrt{v}$.

Figure 41.2 Consistent hashing. Shards and keys are placed on a ring and each key belongs to the next shard clockwise, so adding a shard moves only the keys in one arc.

Consistent hashing with virtual nodes

```

1  import bisect, hashlib
2
3  class ConsistentHash:
4      def __init__(self, nodes=(), vnodes=150):
5          self.vnodes = vnodes
6          self.ring = []          # sorted hashes
7          self.owner = {}         # hash -> node
8          for n in nodes:
9              self.add(n)
10
11     def _h(self, key):
12         return int.from_bytes(
13             hashlib.blake2b(str(key).encode(), digest_size=8).digest(),
14             "big")
15
16     def add(self, node):
17         for i in range(self.vnodes):
18             h = self._h(f"{node}#{i}")
19             bisect.insort(self.ring, h)
20             self.owner[h] = node
21
22     def remove(self, node):
23         for i in range(self.vnodes):
24             h = self._h(f"{node}#{i}")
25             idx = bisect.bisect_left(self.ring, h)
26             if idx < len(self.ring) and self.ring[idx] == h:
27                 self.ring.pop(idx)
28                 del self.owner[h]
29
30     def lookup(self, key):
31         if not self.ring:
32             return None
33         h = self._h(key)
34         idx = bisect.bisect_right(self.ring, h) % len(self.ring)
35         return self.owner[self.ring[idx]]

```

Virtual nodes are what make it usable: with one point per shard the load imbalance is large (the arcs have very different lengths), and with v points the standard deviation of

load falls as $1/\sqrt{v}$. $v=100$ – 200 brings imbalance to a few percent.

Bounded-load consistent hashing adds a cap: if a key's target shard is already above $(1+\epsilon)$ times the average load, walk clockwise to the next one. This gives a hard load bound while still moving only $O(1/N)$ keys on a membership change, and it is what modern load balancers implement.

WHERE THIS SHOWS UP IN ML | Where sharding shows up

Sharded vector indexes (route a query to the shards whose centroids match), distributed KV caches keyed by conversation id (so a follow-up turn lands on the machine holding its prefix cache — Chapter 36), embedding tables split by id across parameter servers, and model replicas behind a router that must keep a session sticky. In every case the requirement is the same: stable placement under membership change.

Gradient compression, asynchrony, and stragglers

Compression. All-reduce moves $2N$ bytes per worker per step; shrinking N helps directly. Options: fp16 or bf16 gradients (2x, essentially free), 1-bit / sign SGD with error feedback (32x, needs the error-feedback term or it does not converge), top- k sparsification (100x, but the indices cost almost as much as the values unless you exploit structure), and low-rank factorisation (PowerSGD). All of them change the optimisation, so they must be evaluated on convergence, not only on throughput.

Asynchrony. Asynchronous SGD removes the barrier entirely, at the cost of stale gradients. It works at modest staleness and degrades badly beyond it. Most large-scale training is synchronous, because reproducibility and convergence are worth more than the tail-latency saving.

Stragglers. One slow worker stalls a synchronous step. Mitigations: backup workers (start $P+b$ and use the first P results), bounded-staleness protocols, and simply detecting the sick node. At scale, hardware failure is routine — the practical answer is frequent asynchronous checkpointing so that a failed step costs minutes, not hours.

COMMON PITFALL | Checkpointing is a distributed algorithm too

A naive checkpoint stops every worker, gathers all state to rank 0, and writes it: $\Theta(PN)$ bytes through one node, and every worker idle for the duration. The right design is *sharded* checkpointing — each worker writes its own shard in parallel to object storage — plus *asynchronous* writes (copy state to host memory, then write in the background) and a manifest that records the shard layout so the checkpoint can be loaded onto a different number of workers. Getting this wrong can cost 10% of total training time.

EXERCISES

1. Derive the total bytes sent by naive parameter-server all-reduce, tree all-reduce and ring all-reduce for $P=8, 64, 512$ and $N=1$ GB. Include the latency term.
2. Implement `ring_allreduce` over multiprocessing pipes and verify the result matches a serial sum. Measure the bytes sent.
3. For a 7B model in bf16 with 8-way data parallelism, compute the all-reduce time at 400 GB/s and at 25 GB/s. What fraction of a 200 ms step is that?
4. Compute the pipeline bubble fraction for $S \in \{4, 8, 16\}$ and $M \in \{S, 4S, 16S\}$. Plot it.
5. Implement consistent hashing and measure the fraction of keys that move when a shard is added, for $v \in \{1, 10, 100, 500\}$. Also measure load imbalance.
6. Implement bounded-load consistent hashing and show it caps load at $(1+\epsilon)$ times the mean while keeping key movement low.
7. Implement 1-bit compression with and without error feedback on a small training job and compare convergence.
8. Design the checkpointing scheme for a 70B model on 512 GPUs with a 5% overhead budget. State the sharding, the write path and the recovery path.

CHAPTER RECAP

- Communication costs $\alpha + N/\beta$: minimise message *count* when small, *bytes* when large.
- Ring all-reduce sends $2(P-1)/P \cdot N$ bytes per worker — independent of P — and is bandwidth-optimal; its latency term grows with P , which hierarchical schemes address.
- Bucketing and overlapping communication with the backward pass is a scheduling change that matters as much as the algorithm.
- Data, tensor, pipeline and expert parallelism differ in what they communicate; compose them so the heaviest traffic uses the fastest link.
- Pipeline bubbles are $(S-1)/(M+S-1)$ and are reduced by more micro-batches and better schedules.
- Consistent hashing with 100–200 virtual nodes gives stable placement with few-percent imbalance; bounded-load variants cap it hard.
- Gradient compression, asynchrony and straggler mitigation all trade convergence or reproducibility for speed; sharded asynchronous checkpointing is not optional at scale.

External Memory and the Data Loading Pipeline

The part of training that is not the model, and usually the bottleneck

FRONTIER

PREREQS: Chapters 31, 39

A surprising fraction of training jobs are limited by their input pipeline rather than by the accelerator, and the fix is almost never a bigger machine. This chapter covers external-memory algorithms, the storage layouts that make sequential reading possible, and how to diagnose and fix an input-bound job.

WHAT YOU WILL LEARN

- Apply the external-memory model to sorting, joining and shuffling
- Implement external merge sort and external joins
- Choose a storage layout and shard size for a training corpus
- Diagnose an input-bound training job and find the limiting stage
- Implement correct sharding, shuffling and resumption for a dataloader
- Understand prefetching, pinned memory and overlap

External-memory algorithms, briefly

Chapter 31 introduced the model: M items in memory, blocks of B items, cost measured in block transfers. Three algorithms cover most needs.

External merge sort. Read M items, sort in memory, write a run; repeat; then k -way merge the runs with a heap (Chapter 13). With $k=M/B$ merge fan-in, the number of passes is $\lceil \log_{M/B}(n/M) \rceil + 1$ — which for realistic values is 2. Sorting a terabyte with 100 GB of RAM is two passes over the data, not more.

External merge sort

```

1  import heapq, os, pickle, tempfile
2
3  def external_sort(iterable, key=None, chunk=1_000_000, tmpdir=None):
4      tmpdir = tmpdir or tempfile.mkdtemp()
5      runs, buf = [], []
6      def flush():
7          buf.sort(key=key)
8          path = os.path.join(tmpdir, f"run{len(runs)}.pkl")
9          with open(path, "wb") as fh:
```

```

10         for item in buf:
11             pickle.dump(item, fh)
12         runs.append(path)
13         buf.clear()
14     for item in iterable:
15         buf.append(item)
16         if len(buf) >= chunk:
17             flush()
18     if buf:
19         flush()
20
21     def reader(path):
22         with open(path, "rb") as fh:
23             while True:
24                 try:
25                     yield pickle.load(fh)
26                 except EOFError:
27                     return
28     yield from heapq.merge(*[reader(r) for r in runs], key=key)

```

External join. To join two datasets neither of which fits in memory: *sort-merge* (sort both by key, then one linear pass — best when the output must be sorted or the inputs already are) or *hash join* (partition both by `hash(key) % k` into k buckets small enough to fit, then hash-join each pair — usually faster). Both are two passes.

External shuffle. Chapter 21: scatter records into m random files, then shuffle each in memory. Two passes, and a genuinely uniform shuffle — unlike a buffered shuffle, which only randomises within a window.

Storage layout for training data

Layout	Read pattern	Random access	Good for
Many small files	random, one open per sample	yes	nothing at scale
Tar shards (WebDataset)	sequential within a shard	shard-level only	streaming from object storage
TFRecord / RecordIO	sequential	with an index	TensorFlow pipelines
Parquet	columnar, sequential per column	row-group level	tabular, feature stores
Arrow / memory-mapped	zero-copy, OS-paged	yes	local NVMe, repeated epochs
Raw binary + offsets	fully sequential	yes	tokenised text (one <code>uint16</code> array)

COMMON PITFALL | The small-files problem

One million JPEGs in a directory is a catastrophe on any networked or object store: each file costs a metadata lookup plus a round trip, so you get hundreds of files per second instead of hundreds of megabytes per second. Aggregate into shards of 100 MB to 1 GB — large enough to amortise the round trip, small enough that a worker can hold one and that the shard count exceeds the worker count by a healthy factor (aim for at least $10 \times \text{world_size} \times \text{num_workers}$, or some workers will idle).

Tokenised corpus as one memory-mapped array

```

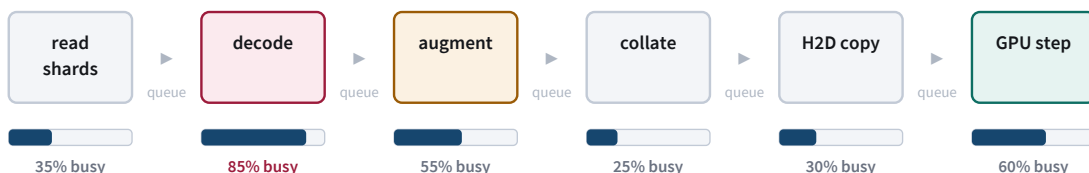
1  import numpy as np
2
3  class PackedTextDataset:
4      """The standard LLM pretraining layout: one flat uint16 token
5         array."""
6
7      def __init__(self, path, seq_len=2048):
8          self.tokens = np.memmap(path, dtype=np.uint16, mode="r")
9          self.seq_len = seq_len
10         self.n = (len(self.tokens) - 1) // seq_len
11
12     def __len__(self):
13         return self.n
14
15     def __getitem__(self, i):
16         lo = i * self.seq_len
17         x = np.asarray(self.tokens[lo:lo + self.seq_len], dtype=np.int64)
18         y = np.asarray(self.tokens[lo + 1:lo + 1 + self.seq_len],
19                        dtype=np.int64)
20         return x, y # no decode, no parse, no copy

```

No JSON parsing, no per-example allocation, and the OS page cache handles readahead. Sequences are packed end-to-end with document separators, which wastes nothing on padding (Chapter 4) at the cost of some cross-document attention — a tradeoff essentially every pretraining pipeline accepts.

Diagnosing an input-bound job

THE INPUT PIPELINE



The GPU is 60% busy, so the job is input-bound. Decode is the bottleneck at 85% — nothing else matters until it moves.

Prefetch depth hides jitter; it cannot raise the throughput of the slowest stage.

Figure 42.1 An input pipeline is a chain of stages joined by queues. Throughput is set by the slowest stage; everything else is idle capacity, and prefetching only hides jitter.

The diagnostic procedure, in order:

1. **Measure GPU utilisation over time.** Steady 95%+ means you are compute-bound; sawtooth or a low average means you are input-bound.
2. **Time the loader alone.** Iterate the dataloader with no model step. If that alone is slower than your step time, the loader is the ceiling.
3. **Time each stage.** Read, decode, augment, collate, host-to-device copy. One will dominate.
4. **Check the queues.** A full queue upstream of a stage means that stage is the bottleneck; an empty one means the stage before it is.

Symptom	Likely cause	Fix
Low GPU utilisation, high CPU	decode or augmentation	more workers; GPU decode (DALI, nvJPEG); cheaper augments
Low GPU and low CPU	I/O wait	larger shards; more prefetch; faster storage; compression
Sawtooth utilisation	insufficient prefetch	raise <code>prefetch_factor</code> ; pinned memory; non-blocking copies
First batch slow, rest fast	cold page cache	warm the cache; use a persistent worker pool
Slows down over epochs	memory growth or fragmentation	check for accumulating Python objects in the dataset
One rank always last	unequal shards or a slow disk	balance shards; check that node

Table 42.1 Storage layouts. The last row is what large-language-model pipelines actually use: the entire tokenised corpus as one flat array of token ids, memory-mapped, with document boundaries in a separate offsets array — the ragged layout of Chapter 4.

A correct, resumable, shard-based loader	
1	<code>import numpy as np</code>
2	
3	<code>class ShardedIterator:</code>
4	<code> """Deterministic, resumable, evenly divided across ranks and</code>
5	<code>workers."""</code>
6	
7	<code> def __init__(self, shards, rank, world, num_workers, seed=0):</code>
8	<code> self.shards = list(shards)</code>
9	<code> self.rank, self.world = rank, world</code>
10	<code> self.num_workers = max(1, num_workers)</code>
11	<code> self.seed = seed</code>
12	
13	<code> def epoch_plan(self, epoch):</code>
14	<code> rng = np.random.default_rng(self.seed + epoch) # SAME on all</code>
15	<code> ranks</code>
16	<code> order = rng.permutation(len(self.shards))</code>
17	<code> pad = (-len(order)) % (self.world * self.num_workers)</code>
	<code> if pad:</code>
	<code> order = np.concatenate([order, order[:pad]]) # equal-sized</code>

```

17         splits
18         return order
19
20     def for_worker(self, epoch, worker_id):
21         order = self.epoch_plan(epoch)
22         stride = self.world * self.num_workers
23         offset = self.rank * self.num_workers + worker_id
24         return [self.shards[i] for i in order[offset::stride]]
25
26     def state(self, epoch, worker_id, shard_index, offset_in_shard):
27         return dict(epoch=epoch, worker=worker_id,
28                     shard_index=shard_index, offset=offset_in_shard)

```

Three properties this gets right, all of which are commonly wrong: the shard permutation is identical on every rank so the split is disjoint; the plan is padded so every worker gets the same number of shards and no all-reduce hangs; and the state is small and explicit so a job can resume mid-epoch after a preemption without replaying data.

WHERE THIS SHOWS UP IN ML | The prefetch and copy path

The final stage is the host-to-device transfer, and it has its own rules:

- **Pinned (page-locked) host memory** lets the DMA engine copy without the CPU, roughly doubling PCIe throughput. `pin_memory=True`.
- **Non-blocking copies on a separate CUDA stream** overlap the transfer of batch $i+1$ with the compute of batch i . Without this the GPU idles during every transfer.
- **Prefetch depth** of 2–4 batches hides jitter. Deeper adds latency and memory without adding throughput — it cannot make a slow stage fast.
- **Do the cheap work on the GPU.** Normalisation, augmentation and even JPEG decode are often better on the accelerator, where they are nearly free, than on a saturated CPU.

PERFORMANCE NOTE | An arithmetic check worth doing before anything else

A training step that processes a batch of 256 images at $224 \times 224 \times 3$ bytes consumes 38 MB of decoded data. At 10 steps per second that is 385 MB/s of *decoded* pixels — which, from JPEGs at roughly 10:1 compression, is 38 MB/s from storage but requires decoding 2,560 images per second. A CPU core decodes perhaps 100 JPEGs per second, so you need 25 cores just for decode. That single calculation predicts most input-bound jobs, and it takes a minute.

EXERCISES

1. Implement external merge sort and sort a 10 GB file with a 1 GB memory limit. Count the passes and compare to the $\log_{M/B}$ prediction.
2. Implement a hash join and a sort-merge join over two 5 GB datasets and compare their I/O.
3. Pack a text corpus into a flat `uint16` array with an offsets file, and compare `PackedTextDataset` throughput to a JSON-lines loader.
4. Measure throughput for 100,000 small files versus 100 tar shards from local disk and from an object store.
5. Instrument a real dataloader to time each stage and produce the bar chart in this chapter's figure for your own pipeline.
6. Demonstrate the unequal-shard hang: run a two-process job where the shard count is not divisible by the world size, without padding.
7. Implement mid-epoch resumption and verify that resuming produces exactly the same remaining sample sequence as an uninterrupted run.
8. Measure host-to-device throughput with and without pinned memory, and with and without a separate copy stream. Report all four numbers.

CHAPTER RECAP

- External sorting, joining and shuffling are all two-pass algorithms at realistic memory sizes; the fan-in M/B is what keeps the pass count low.
- Aggregate training data into 100 MB–1 GB shards; the small-files pattern destroys throughput on any networked store.
- The standard LLM layout is one flat memory-mapped token array plus offsets, which removes parsing and allocation from the hot path entirely.
- Throughput is set by the slowest pipeline stage; prefetching hides jitter but cannot raise it. Diagnose by timing the loader alone and then each stage.
- A correct loader uses an identical shard permutation across ranks, pads to equal splits, and carries small explicit state for resumption.
- Pinned memory, a separate copy stream and modest prefetch depth are the transfer-path essentials; move cheap work onto the GPU.
- Do the decoded-bytes-per-second arithmetic before changing anything.

PART IX

Mastery

A method for attacking unseen problems, a catalogue of the patterns that cover most of them, a graded problem set with full solutions, and a study plan that fits around a job.

-
- 43 A Problem-Solving Framework
 - 44 The Pattern Catalogue
 - 45 Problem Set I: Foundations
 - 46 Problem Set II: Advanced and ML-Flavoured
 - 47 Study Plan and Reference

A Problem-Solving Framework

What to do in the ninety seconds after you read a problem

PRACTICE

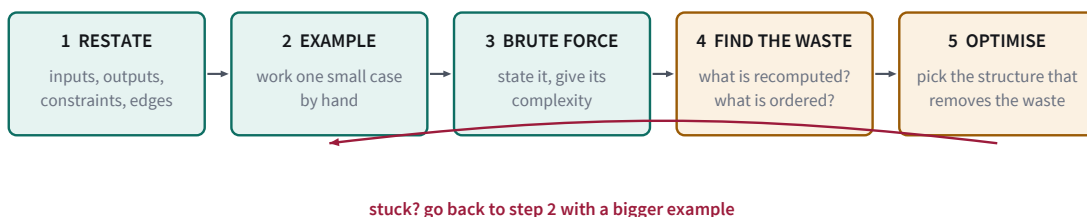
PREREQS: Parts I–VIII

Knowing algorithms and being able to solve problems are different skills, and the second one is a procedure rather than a talent. This chapter gives you that procedure, the questions that turn a vague problem into a precise one, the signals that point to a structure, and the debugging discipline that makes your answer actually correct.

WHAT YOU WILL LEARN

- Apply a five-step procedure to any unfamiliar problem
- Ask the clarifying questions that determine the whole solution
- Move systematically from a brute force to an optimal solution
- Recognise the signals that point to a particular data structure
- Verify a solution before running it, and debug it when it fails
- Communicate a solution the way a senior engineer does

The procedure



Steps 1–3 are not optional warm-up. A stated brute force is a correct answer you can improve; a half-remembered clever answer is neither.

Figure 43.1 The loop. Most people skip to step 5, produce something half-remembered, and get stuck. The first three steps take four minutes and make the rest tractable.

Step 1: restate the problem

Write down, in your own words: what the input is (types, ranges, whether it is sorted, whether values are distinct), what the output is (a value, an index, all solutions, any solution), and what constraints apply ($n \leq 10^5$ tells you $O(n \log n)$; $n \leq 20$ tells you 2^n is fine).

CHECKLIST | Questions that change the answer

- What is the size of n ? And of the values?
- Is the input sorted? Can I sort it, or is order meaningful?
- Are there duplicates? Negative numbers? Zeros? Empty input?
- Am I optimising time or memory? Is there a latency bound?
- Can I modify the input in place?
- Is this a one-off query or will there be many? (Decides whether to build an index.)
- Does the data change between queries? (Decides static versus dynamic structure.)
- Does the answer need to be exact? (Decides whether a sketch is allowed.)

The last three are the ones that distinguish a systems answer from an exercise answer, and they are the ones interviewers at good companies are actually listening for.

Step 2: work an example by hand

Take a small instance — five or six elements — and solve it on paper. Then take a slightly awkward one: empty, single element, all equal, already sorted, reverse sorted.

This does three things at once. It forces you to understand the problem. It usually reveals the structure of the answer. And it gives you your first test case.

Step 3: state the brute force

Say out loud what the exhaustive solution is and what it costs. “Try every pair: $O(n^2)$.” This is never wasted:

- It proves the problem is solvable and establishes correctness.
- It gives you something to test the fast version against.
- It exposes the waste that the fast version will eliminate.
- If you run out of time, you have a working answer.

Step 4: find the waste

This is the creative step, and it has only a few shapes.

What the brute force does	The waste	The fix
Recomputes the same subproblem	repeated work	memoisation / DP (Ch 18)
Rescans a prefix or window	repeated aggregation	prefix sums, sliding window (Ch 11)
Searches an unordered set	no structure to exploit	sort, then binary search (Ch 9, 10)
Searches a sorted set linearly	ignores order	binary search (Ch 10)

What the brute force does	The waste	The fix
Keeps everything to find the best few	excess state	heap (Ch 13)
Re-derives reachability	repeated traversal	union-find or precomputed components (Ch 15)
Compares all pairs	most pairs are irrelevant	hashing, LSH, spatial index (Ch 7, 29, 33)
Explores doomed branches	hopeless subtrees	pruning, branch and bound (Ch 20)
Stores exact counts of everything	more precision than needed	sketch (Ch 28)

Step 5: choose the structure

PHRASE IN THE PROBLEM	STRUCTURE TO REACH FOR
“top k”, “best so far”, “merge k streams”	heap
“contiguous” + “longest/shortest”	sliding window
“sorted” or “find the minimum x such that”	binary search
“all pairs” + “sorted”	two pointers
“how many ways”, “minimum cost to”	dynamic programming
“prefix”, “autocomplete”, “dictionary”	trie
“connected”, “groups”, “merge sets”	union-find
“shortest path” + weights	Dijkstra / BFS
“order with dependencies”	topological sort
“range sum” + updates	Fenwick / segment tree
“too big to store exactly”	sketch (Bloom/CMS/HLL)
“nearest” in high dimensions	ANN index

Figure 43.2 The phrase-to-structure lookup. It is not a substitute for understanding, but it is a fast way to generate candidates to evaluate.

Once you know the waste, the structure follows: something that makes the repeated operation cheap. Write the cost of the new solution *before* coding it, and check it against the constraints. If the new cost still does not fit, you have not found the real waste — go back to step 4.

Signals and what they mean

Signal	Likely approach
$n \leq 20$	exponential is fine: bitmask DP, backtracking
$n \leq 100$	$O(n^3)$: Floyd–Warshall, interval DP, matrix chain
$n \leq 5000$	$O(n^2)$: pairwise DP, simple graph algorithms

Signal	Likely approach
$n \leq 10^6$	$O(n \log n)$: sort, heap, binary search, divide and conquer
$n \leq 10^8$	$O(n)$: one pass, hashing, two pointers, counting sort
$n > 10^9$	sublinear or streaming: sketches, sampling, external memory
“ k -th” or “top k ”	heap or quickselect
“shortest / cheapest”	BFS, Dijkstra, DP
“how many ways”	DP, combinatorics
“can it be done in X ?”	binary search over the answer, greedy check
“maximum / minimum such that”	binary search over the answer
“next greater / previous smaller”	monotonic stack
“in place, $O(1)$ memory”	two pointers, cyclic sort, bit tricks
“stream, cannot store it”	reservoir, sketch, one-pass statistic

Table 43.1 The nine sources of waste. Nearly every optimisation in this book is one of these rows.

Verification before execution

Running the code is the *last* check, not the first.

CHECKLIST | Before you run it

1. **Trace your own example.** Step through the code on the instance from step 2. Most bugs surface here.
2. **Check the boundaries.** Empty, one element, two elements, all equal, maximum size. Off-by-one at both ends of every loop and slice.
3. **Check the invariant.** Every loop has one: “after iteration i , `dp[j]` holds ...”. State it and check it is preserved.
4. **Check integer and floating-point issues.** Overflow (in typed languages), division by zero, comparing floats for equality, and accumulating error in a long sum.
5. **Check the complexity you claimed.** Count the loops again. Look for a hidden $O(n)$ operation inside an $O(n)$ loop (`in` on a list, string concatenation, `insert(0, x)`).

When it does fail, debug by **bisection, not by staring**: find the smallest failing input (shrink it mechanically), then print the state at the midpoint of the computation and decide which half is wrong. A property-based test that compares against your brute force on random small inputs will find the bug in seconds, and you already wrote the brute force in step 3.

The test harness worth writing every time

```
1 import random
```

```

2
3 def test_against_brute(fast, brute, gen, trials=2000):
4     for _ in range(trials):
5         case = gen()
6         got, want = fast(*case), brute(*case)
7         if got != want:
8             print("MISMATCH on", case, "got", got, "want", want)
9             return case # return it for shrinking
10    print("ok:", trials, "random cases agree")
11    return None
12
13 gen = lambda: ([random.randint(-5, 5) for _ in range(random.randint(0,
14    8))],)

```

Small random inputs with a small value range — `-5..5`, length `0..8` — find far more bugs than large ones, because they produce duplicates, negatives, empties and ties, which is where the bugs live.

Communicating a solution

The difference between a correct answer and a good one is largely how it is presented.

1. **Restate the problem** to confirm the interpretation. Thirty seconds, and it has saved entire interviews.
2. **State the approach before coding.** “I will sort by finish time, then greedily take non-overlapping intervals; that is $O(n \log n)$ and optimal by an exchange argument.”
3. **State the complexity up front**, in time *and* space.
4. **Write clean code:** named variables, one idea per line, no cleverness that needs a comment to justify.
5. **Test it out loud** on your example from step 2.
6. **Name the tradeoffs you did not take.** “A hash map would be $O(1)$ lookup but 40 bytes per entry; at 200 million keys I would use a sorted array and `searchsorted` instead.”

Point 6 is the one that distinguishes a senior engineer. It shows you chose rather than defaulted.

WHERE THIS SHOWS UP IN ML | How this applies at work, not just in interviews

The same five steps, with different emphasis. Step 1 becomes *measure*: what is actually slow, at what n , and what is the budget? Step 3 becomes the baseline you must beat, and you should keep it as a correctness oracle. Step 4 becomes profiling rather than reasoning — the waste is often not where you think. And step 6 becomes the design document: the alternatives considered and why they were rejected, which is what makes a decision reviewable a year later.

EXERCISES

1. Take any problem you have solved recently and write out all five steps retrospectively. Which one did you skip?
2. For each row of the constraint table, find a problem from Chapters 45–46 that matches it.
3. Write a brute-force and an optimal solution to “longest substring without repeating characters” and run the test harness above on 5,000 random inputs.
4. Take a problem you have solved and list three clarifying questions whose answers would change your solution.
5. Deliberately introduce an off-by-one error in a binary search and find it using only the shrinking procedure. Time yourself.
6. Write the invariant for each loop in your last non-trivial function. If you cannot, that is where your next bug will be.

CHAPTER RECAP

- Restate, work an example, state the brute force, find the waste, choose the structure. The first three steps take four minutes and make the rest tractable.
- The clarifying questions that matter most are about scale, mutability, query count and required exactness.
- The brute force is never wasted: it proves correctness, exposes the waste, and becomes your test oracle.
- Nearly every optimisation removes one of nine kinds of waste; find the waste and the structure follows.
- The bound on n tells you the complexity class you are allowed before you have thought about the problem.
- Verify by tracing, boundaries, invariants and a recount of the complexity; debug by shrinking and bisection, not by staring.
- Communicate by stating the approach and complexity first, and by naming the tradeoffs you did not take.

The Pattern Catalogue

Thirty patterns that cover most of what you will be asked

REFERENCE

PREREQS: Parts I–VIII

This chapter is a reference, not a narrative. Each entry names a pattern, gives the signal that should make you think of it, a template compact enough to memorise, its complexity, and where it appears in real systems. Read it once end to end, then return to it when a problem looks familiar but you cannot place it.

WHAT YOU WILL LEARN

- Recognise thirty recurring problem shapes by their surface signals
- Reproduce a correct template for each from memory
- Know each pattern's complexity and its common failure mode
- Connect each to the chapter that develops it and the ML system that uses it

How to use this chapter

Each entry has the same five fields: **Signal** (what in the problem statement points here), **Idea** (one sentence), **Template**, **Cost**, and **In the wild**. Nothing here is new; everything refers back to a chapter that derives it.

Array and pointer patterns

1. Two pointers, converging

Signal. A pair with some property, in a *sorted* array. **Idea.** Move the pointer whose side of the comparison cannot help.

```
1 i, j = 0, len(a) - 1
2 while i < j:
3     s = a[i] + a[j]
4     if s == target: return i, j
5     if s < target: i += 1
6     else: j -= 1
```

Cost. $\Theta(n)$ after sorting. **Chapter 11. In the wild.** Merging sorted posting lists; intersecting candidate sets.

2. Two pointers, read and write

Signal. Remove, dedupe or partition *in place*. **Idea.** One cursor reads, one writes; the prefix before the write cursor is the answer.

```

1  w = 0
2  for r in range(len(a)):
3      if keep(a[r]):
4          a[w] = a[r]; w += 1
5  return w

```

Cost. $\Theta(n)$, $\Theta(1)$ extra. **Chapter 11. In the wild.** GPU stream compaction after a prefix sum (Chapter 39).

3. Sliding window, fixed size

Signal. A statistic over every window of size k . **Idea.** Add the entering element, remove the leaving one.

```

1  s = sum(a[:k]); best = s
2  for i in range(k, len(a)):
3      s += a[i] - a[i - k]
4      best = max(best, s)

```

Cost. $\Theta(n)$. **Chapter 11. In the wild.** Rolling metrics; n -gram shingles; sliding-window attention.

4. Sliding window, variable size

Signal. Longest or shortest *contiguous* run satisfying a monotone condition. **Idea.** Grow right always; shrink left only while invalid.

```

1  left = 0
2  for right in range(len(a)):
3      add(a[right])
4      while not valid():
5          remove(a[left]); left += 1
6      best = max(best, right - left + 1)

```

Cost. $\Theta(n)$ amortised. **Chapter 11. Failure mode.** The condition must be monotone in the window; with negative values a sum condition is not.

5. Prefix sums

Signal. Many range-sum queries on static data. **Idea.** $\text{pre}[i] = a[0] + \dots + a[i-1]$; range $[l, r)$ is $\text{pre}[r] - \text{pre}[l]$. **Cost.** $\Theta(n)$ build, $\Theta(1)$ query. **Chapters 4, 27. In the wild.** Ragged offsets; sampling by cumulative weight; IoU.

6. Difference array

Signal. Many range *updates*, one query at the end. **Idea.** `d[l] += v; d[r] -= v`, then prefix-sum once. **Cost.** $\Theta(1)$ per update, $\Theta(n)$ once. **Chapter 27. In the wild.** Accumulating attention masks; interval bookkeeping.

7. Monotonic stack

Signal. “Next greater”, “previous smaller”, largest rectangle. **Idea.** Keep a stack sorted; pop everything the new element dominates.

```
1 stack, out = [], [-1] * len(a)
2 for i, x in enumerate(a):
3     while stack and a[stack[-1]] < x:
4         out[stack.pop()] = i
5     stack.append(i)
```

Cost. $\Theta(n)$: each index pushed and popped once. **Chapter 6.**

8. Monotonic deque

Signal. Max or min over every sliding window. **Idea.** A monotonic stack with removal from the front when indices expire. **Cost.** $\Theta(n)$. **Chapter 6.**

9. Cyclic sort / index-as-hash

Signal. Values are a permutation of $1..n$; find missing or duplicate in $\Theta(1)$ space. **Idea.** Place each value at its own index, or use the sign of `a[abs(x)]` as a visited bit. **Cost.** $\Theta(n)$, $\Theta(1)$ extra.

Search and ordering patterns

10. Binary search on an array

Signal. Sorted data, a target or a boundary. **Idea.** Half-open interval, `lower_bound` / `upper_bound`. **Chapter 10.**

11. Binary search on the answer

Signal. “Minimum x such that ...”, “can it be done with capacity c ?” **Idea.** Find a monotone predicate and bisect the answer space.

```
1 lo, hi = 0, upper
2 while lo < hi:
3     mid = lo + (hi - lo) // 2
4     if feasible(mid): hi = mid
5     else: lo = mid + 1
```

Cost. $\Theta(\log(\text{range}) \times \text{cost of } \text{textit{feasible}})$. **Chapter 10. In the wild.** Calibration thresholds; maximum batch size; temperature for a target entropy.

12. Exponential (galloping) search

Signal. Unbounded input, or the answer is near the start. **Idea.** Double until you bracket, then binary search. $\Theta(\log i)$. **In the wild.** Timsort’s galloping merge; posting-list skipping.

13. Quickselect

Signal. The k -th element, or the top k unordered. **Idea.** Partition, recurse into one side only. $\Theta(n)$ expected. **Chapter 9.** Prefer `np.argpartition` when the data is an array.

14. Top- k with a bounded heap

Signal. Top k of a *stream*, or $k \ll n$ with bounded memory. **Idea.** A min-heap of size k ; the root is the weakest survivor. **Cost.** $\Theta(n \log k)$ time, $\Theta(k)$ space. **Chapter 13. In the wild.** Retrieval candidate sets; beam search; heavy hitters.

15. k -way merge

Signal. Merge k sorted streams. **Idea.** A heap of one element per stream. $\Theta(N \log k)$, $\Theta(k)$ memory. **Chapter 13. In the wild.** External sort; multi-shard retrieval.

Hashing and set patterns

16. Hash set for membership

Signal. “Have I seen this?”, “any duplicates?”, “complement exists?” **Idea.** Trade $\Theta(n)$ memory for $\Theta(1)$ lookup. **Chapter 7. Failure mode.** Memory at scale — 100 bytes per entry in Python.

17. Hash map for grouping

Signal. “Group by”, “anagrams”, “count occurrences”. **Idea.** A canonical key (sorted tuple, character count, normalised form). **Cost.** $\Theta(n \cdot |\text{key}|)$.

18. Prefix-sum plus hash map

Signal. Subarrays summing to k , with negative values allowed. **Idea.** Count previously seen prefix sums; `pre[r] - pre[l] == k` means `pre[l] == pre[r] - k`.

```
1 from collections import defaultdict
2 seen, s, count = defaultdict(int, {0: 1}), 0, 0
3 for x in a:
4     s += x
5     count += seen[s - k]
6     seen[s] += 1
```

Cost. $\Theta(n)$. **The standard replacement for a sliding window when values can be negative.**

19. Sketching

Signal. Too large to store exactly; approximate is acceptable. **Idea.** Bloom for membership, Count–Min for frequency, HyperLogLog for cardinality, MinHash for similarity. **Chapters 28, 29.**

Tree and graph patterns

20. DFS on a tree, combine upward

Signal. Any per-subtree quantity. **Idea.** Recurse on children, combine, return.

```

1  def go(node):
2      if node is None: return identity
3      return combine(node.val, go(node.left), go(node.right))

```

Cost. $\Theta(n)$. **Chapter 12.** Use an explicit stack when depth can exceed a few thousand.

21. BFS for shortest path

Signal. Fewest steps, unweighted. **Idea.** Queue, mark on push, parent array is the shortest-path tree. **Cost.** $\Theta(V + E)$. **Chapter 22.** Multi-source BFS seeds the queue with all sources.

22. Dijkstra / best-first

Signal. Cheapest path with non-negative weights. **Idea.** BFS with a priority queue and lazy deletion. $\Theta((V+E)\log V)$. **Chapter 23.**

23. Topological order

Signal. Dependencies, ordering, “is there a cycle?” **Idea.** Kahn’s algorithm; the length check detects cycles. $\Theta(V+E)$. **Chapter 24. In the wild.** Autograd; build systems; pipeline DAGs.

24. Union–find

Signal. “Connected”, “same group”, incremental merging. **Idea.** Union by rank plus path compression: $\Theta(\alpha(n))$. **Chapter 15. In the wild.** Deduplication clusters; Kruskal.

25. Trie

Signal. Prefixes, autocomplete, longest match, many patterns at once. **Idea.** Index by the string itself; add failure links for Aho–Corasick. **Chapter 14. In the wild.** Tokenizers; constrained decoding; prefix caches.

Dynamic programming patterns

26. Linear DP

Signal. `dp[i]` depends on a constant number of earlier positions. **Idea.** One array, one pass; often reducible to $\Theta(1)$ memory. **Chapter 18.** *Examples.* Climbing stairs, house robber, max subarray.

27. Two-sequence DP

Signal. Two strings or series compared. **Idea.** `dp[i][j]` from its three neighbours. $\Theta(nm)$, $\Theta(\min(n, m))$ memory with a rolling row. **Chapter 19.** *Examples.* Edit distance, LCS, DTW, alignment.

28. Knapsack DP

Signal. Choose a subset subject to a budget. **Idea.** `dp[capacity]`; iterate capacity *downwards* for 0/1, *upwards* for unbounded. **Chapter 18.** **In the wild.** Gradient checkpointing; context-window budgeting.

29. Interval DP

Signal. “Combine adjacent items”, “best way to split”. **Idea.** `dp[i][j] = min over k of dp[i][k] + dp[k][j] + cost`. $\Theta(n^3)$. **Chapter 18.** **In the wild.** Matrix-chain order; pipeline-stage partitioning.

30. Bitmask DP

Signal. $n \leq 20$ and subsets matter. **Idea.** `dp[mask][i]` = best over the visited set `mask` ending at `i`. $\Theta(2^n n^2)$. **Chapter 18.** *Examples.* TSP, assignment, scheduling.

Choosing between similar patterns

If you are tempted by	But also consider	Because
Sliding window	prefix sums + hash map	negative values break the window
Sorting	heap or quickselect	you may not need a total order
A hash map	a sorted array + <code>searchsorted</code>	memory at tens of millions of keys
DP over all states	greedy with an exchange proof	if a safe choice exists, DP is overkill
Recursion	an explicit stack	depth can exceed the limit
BFS on a grid	multi-source BFS	if several starts share an answer
A balanced tree	a sorted list of sorted lists	constants dominate in Python
Exact counting	a sketch	when the data does not fit
A new index	brute force	below 10^6 vectors, a GEMM wins

KEY INSIGHT | How to actually memorise this

Not by rereading. Take one pattern a day, implement it from memory in ten minutes, then find one problem in Chapters 45–46 that uses it and solve that problem *without* looking at the template. Thirty days, thirty patterns, and the recall is durable because it was reconstructed rather than recognised.

CHAPTER RECAP

- Thirty patterns cover the large majority of algorithmic problems; each has a surface signal, a short template and a known cost.
- Array patterns: two pointers (converging and read/write), fixed and variable windows, prefix and difference arrays, monotonic stack and deque, cyclic sort.
- Search patterns: binary search on an array and on the answer, galloping search, quickselect, bounded heaps, k -way merge.
- Hash patterns: membership, grouping by canonical key, prefix sums plus a map, and sketches when exactness is negotiable.
- Graph patterns: subtree DFS, BFS shortest paths, Dijkstra, topological order, union-find, tries.
- DP shapes: linear, two-sequence, knapsack, interval, bitmask.
- Always second-guess the first pattern that fits.

Problem Set I: Foundations

Thirty graded problems with complete solutions

PRACTICE

PREREQS: Chapters 1--21

Problems for Parts I through IV, graded from straightforward to hard, each with a full solution and an explanation of why the solution has the shape it does. Attempt each one before reading its solution; the value is entirely in the attempt, and a failed attempt followed by a solution teaches far more than a solution alone.

WHAT YOU WILL LEARN

- Apply the patterns of Chapter 44 to unfamiliar problems
- Recognise which technique a problem wants from its constraints
- Write correct boundary handling on the first attempt
- Build a reusable test harness habit

PRACTICE | How to work through this

Give each problem twenty minutes before looking. Write the brute force first, always. When you read a solution, close the book and reimplement it from memory the next day — that second pass is where the learning is. Problems marked dagger are the ones most likely to appear in an interview.

Arrays, hashing and windows

P1. Two sum dagger

Given an array and a target, return indices of two numbers summing to the target. Unsorted, one solution guaranteed.

SOLUTION

Brute force is $\Theta(n^2)$. The waste: for each x we rescan for $\text{target} - x$. A hash map makes that lookup $\Theta(1)$.

```

1 def two_sum(a, target):
2     seen = {}
3     for i, x in enumerate(a):
4         if target - x in seen:
5             return seen[target - x], i
6         seen[x] = i                # store AFTER checking: handles
7         x + x
8     return None

```

$\Theta(n)$ time, $\Theta(n)$ space. Storing after the check is what makes $\text{target} = 2x$ with a single x correctly return nothing.

P2. Longest substring without repeating characters dagger**SOLUTION**

Variable window with a hash map of last positions. The left edge jumps rather than crawling.

```

1 def longest_unique(s):
2     last, left, best = {}, 0, 0
3     for right, ch in enumerate(s):
4         if ch in last and last[ch] >= left:
5             left = last[ch] + 1        # jump past the previous
6             occurrence
7         last[ch] = right
8         best = max(best, right - left + 1)
9     return best

```

$\Theta(n)$. The $\text{last}[ch] \geq \text{left}$ guard matters: a character seen *before* the window started must not move the left edge backwards.

P3. Subarrays summing to k , with negatives

SOLUTION

A sliding window is wrong here (the sum is not monotone in the window). Prefix sums plus a hash map, pattern 18.

```

1  from collections import defaultdict
2
3  def count_subarrays(a, k):
4      seen = defaultdict(int)
5      seen[0] = 1                                # the empty prefix
6      s, count = 0, 0
7      for x in a:
8          s += x
9          count += seen[s - k]
10         seen[s] += 1
11     return count

```

$\Theta(n)$. The `seen[0] = 1` initialisation accounts for subarrays that start at index 0.

P4. Product of array except self

No division allowed, $\Theta(n)$ time.

SOLUTION

Two passes: prefix products left to right, then suffix products right to left, multiplied in place.

```

1  def product_except_self(a):
2      n = len(a)
3      out = [1] * n
4      pre = 1
5      for i in range(n):
6          out[i] = pre
7          pre *= a[i]
8      suf = 1
9      for i in range(n - 1, -1, -1):
10         out[i] *= suf
11         suf *= a[i]
12     return out

```

$\Theta(n)$ time, $\Theta(1)$ extra beyond the output. Avoiding division is not pedantry: it makes the solution correct when the array contains zeros.

P5. Maximum subarray sum (Kadane) dagger

SOLUTION

Linear DP: `best_ending_here` either extends the previous run or restarts.

```

1  def max_subarray(a):
2      if not a:
3          return 0
4      cur = best = a[0]
5      for x in a[1:]:
6          cur = max(x, cur + x)      # extend, or start fresh at x
7          best = max(best, cur)
8      return best

```

$\Theta(n)$, $\Theta(1)$. The recurrence is $f(i) = \max(a_i, f(i-1) + a_i)$ — pattern 26.

P6. Sliding-window maximum dagger

Maximum of every window of size k , in $\Theta(n)$.

SOLUTION

Monotonic deque of indices with decreasing values (pattern 8).

```

1  from collections import deque
2
3  def window_max(a, k):
4      dq, out = deque(), []
5      for i, x in enumerate(a):
6          while dq and a[dq[-1]] <= x:
7              dq.pop()          # x dominates everything
8              smaller
9          dq.append(i)
10         if dq[0] <= i - k:
11             dq.popleft()      # expired
12         if i >= k - 1:
13             out.append(a[dq[0]])
14     return out

```

Each index is pushed and popped once: $\Theta(n)$.

P7. Group anagrams

SOLUTION

Canonical key. Sorting each word is $\Theta(m \log m)$; a 26-element count tuple is $\Theta(m)$ and faster for long words.

```

1  from collections import defaultdict
2
3  def group_anagrams(words):
4      groups = defaultdict(list)
5      for w in words:
6          key = [0] * 26
7          for ch in w:
8              key[ord(ch) - 97] += 1
9          groups[tuple(key)].append(w)
10     return list(groups.values())

```

$\Theta(\sum |w_i|)$.

P8. Longest consecutive sequence

Unsorted array; find the longest run of consecutive integers, in $\Theta(n)$.

SOLUTION

Sorting is $\Theta(n \log n)$. Instead, start a run only from a value with no predecessor — which makes the total work linear.

```

1  def longest_consecutive(a):
2      s, best = set(a), 0
3      for x in s:
4          if x - 1 in s:
5              continue                # not the start of a run
6          y = x
7          while y + 1 in s:
8              y += 1
9          best = max(best, y - x + 1)
10     return best

```

The inner loop runs only for run starts, and each element is visited at most twice overall: $\Theta(n)$.

P9. Minimum window substring dagger

Shortest substring of S containing every character of T with multiplicity.

SOLUTION

Variable window plus a counter, with a scalar `missing` so validity is $\Theta(1)$ to check.

```

1  from collections import Counter
2
3  def min_window(s, t):
4      if not t or len(s) < len(t):
5          return ""
6      need = Counter(t)
7      missing = len(t)
8      best, left = (0, float("inf")), 0
9      for right, ch in enumerate(s):
10         if need[ch] > 0:
11             missing -= 1
12         need[ch] -= 1
13         while missing == 0:                # valid: try to shrink
14             if right - left < best[1] - best[0]:
15                 best = (left, right)
16             need[s[left]] += 1
17             if need[s[left]] > 0:
18                 missing += 1
19             left += 1
20     return "" if best[1] == float("inf") else s[best[0]:best[1] + 1]

```

$\Theta(|S|+|T|)$. `need` going negative for surplus characters is the trick that keeps `missing` correct.

P10. Merge intervals

SOLUTION

Sort by start, then sweep, extending the current interval while it overlaps.

```

1  def merge_intervals(intervals):
2      if not intervals:
3          return []
4      out = []
5      for lo, hi in sorted(intervals):
6          if out and lo <= out[-1][1]:
7              out[-1][1] = max(out[-1][1], hi)
8          else:
9              out.append([lo, hi])
10     return out

```

$\Theta(n \log n)$, dominated by the sort. Sorting by start is what makes a single pass sufficient.

Sorting, searching and selection

P11. k -th largest element dagger

SOLUTION

Three correct answers with different profiles.


```

1  import heapq, random
2
3  def kth_largest_heap(a, k):                # theta(n log k), theta(
4      k) space
5      h = []
6      for x in a:
7          if len(h) < k:
8              heapq.heappush(h, x)
9          elif x > h[0]:
10             heapq.heapreplace(h, x)
11     return h[0]
12
13 def kth_largest_select(a, k):              # theta(n) expected, in
14     place
15     a = list(a)
16     lo, hi, target = 0, len(a) - 1, len(a) - k
17     while True:
18         if lo == hi:
19             return a[lo]
20         p = random.randint(lo, hi)
21         a[p], a[hi] = a[hi], a[p]
22         pivot, i = a[hi], lo
23         for j in range(lo, hi):
24             if a[j] < pivot:
25                 a[i], a[j] = a[j], a[i]
26                 i += 1
27         a[i], a[hi] = a[hi], a[i]
28         if i == target:
29             return a[i]
30         lo, hi = (lo, i - 1) if target < i else (i + 1, hi)

```

Use the heap for streams and bounded memory, quickselect in memory, and `np.argpartition` for arrays.

P12. Search in a rotated sorted array dagger

SOLUTION

One half of any split is always sorted; decide which, then whether the target lies inside it.

```

1  def search_rotated(a, target):
2      lo, hi = 0, len(a) - 1
3      while lo <= hi:
4          mid = lo + (hi - lo) // 2
5          if a[mid] == target:
6              return mid
7          if a[lo] <= a[mid]:           # left half is sorted
8              if a[lo] <= target < a[mid]:
9                  hi = mid - 1
10             else:
11                 lo = mid + 1
12         else:                         # right half is sorted
13             if a[mid] < target <= a[hi]:
14                 lo = mid + 1
15             else:
16                 hi = mid - 1
17     return -1

```

$\Theta(\log n)$. With duplicates the `a[lo] <= a[mid]` test becomes ambiguous and the worst case degrades to $\Theta(n)$.

P13. Median of two sorted arrays

$\Theta(\log(\min(m, n)))$.

SOLUTION

Binary search the *partition point* of the smaller array so that the left parts of both arrays together hold exactly half the elements.

```

1  def median_two_sorted(a, b):
2      if len(a) > len(b):
3          a, b = b, a
4      m, n = len(a), len(b)
5      half = (m + n + 1) // 2
6      lo, hi = 0, m
7      while lo <= hi:
8          i = (lo + hi) // 2           # elements taken from a
9          j = half - i                 # elements taken from b
10         a_left = a[i - 1] if i else float("-inf")
11         a_right = a[i] if i < m else float("inf")
12         b_left = b[j - 1] if j else float("-inf")
13         b_right = b[j] if j < n else float("inf")
14         if a_left <= b_right and b_left <= a_right:
15             if (m + n) % 2:
16                 return max(a_left, b_left)
17             return (max(a_left, b_left) + min(a_right, b_right)) / 2
18         if a_left > b_right:
19             hi = i - 1
20         else:
21             lo = i + 1

```

The infinities remove every boundary special case — a sentinel technique worth reusing.

P14. Koko eating bananas dagger

Given pile sizes and H hours, find the minimum eating rate that finishes in time.

SOLUTION

Binary search on the answer (pattern 11). `feasible(rate)` is monotone.

```

1  import math
2
3  def min_rate(piles, H):
4      def hours(rate):
5          return sum(math.ceil(p / rate) for p in piles)
6      lo, hi = 1, max(piles)
7      while lo < hi:
8          mid = lo + (hi - lo) // 2
9          if hours(mid) <= H:
10             hi = mid
11          else:
12             lo = mid + 1
13      return lo

```

$\Theta(n \log(\max p))$. Recognising “minimum x such that” is the whole problem.

P15. Sort colours (Dutch national flag)

Three values, one pass, in place.

SOLUTION

Three pointers partition into `< 1, == 1, > 1`.

```

1  def sort_colours(a):
2      lo, i, hi = 0, 0, len(a) - 1
3      while i <= hi:
4          if a[i] == 0:
5              a[lo], a[i] = a[i], a[lo]; lo += 1; i += 1
6          elif a[i] == 2:
7              a[hi], a[i] = a[i], a[hi]; hi -= 1 # do NOT advance i
8          else:
9              i += 1
10     return a

```

Not advancing `i` after a swap with `hi` is the classic bug: the incoming element has not been examined yet.

P16. Count inversions

SOLUTION

Merge sort with a counter (Chapter 16).

```

1  def count_inversions(a):
2      def sort_count(x):
3          if len(x) <= 1:
4              return x, 0
5          m = len(x) // 2
6          left, cl = sort_count(x[:m])
7          right, cr = sort_count(x[m:])
8          out, i, j, cross = [], 0, 0, 0
9          while i < len(left) and j < len(right):
10             if left[i] <= right[j]:
11                 out.append(left[i]); i += 1
12             else:
13                 out.append(right[j]); j += 1
14                 cross += len(left) - i
15             out += left[i:] + right[j:]
16             return out, cl + cr + cross
17     return sort_count(list(a))[1]
```

$\Theta(n \log n)$. This is how Kendall's τ is computed efficiently.

Linked structures, stacks and heaps

P17. Reverse a linked list in groups of k

SOLUTION

Reverse each group with the three-pointer idiom, reconnecting as you go.

```

1  class ListNode:
2      __slots__ = ("val", "next")
3      def __init__(self, val=0, nxt=None):
4          self.val, self.next = val, nxt
5
6  def reverse_k_group(head, k):
7      def length(n):
8          c = 0
9          while n:
10             c += 1
11             n = n.next
12         return c
13
14     dummy = ListNode(0, head)
15     prev_group, remaining = dummy, length(head)
16     while remaining >= k:
17         cur = prev_group.next
18         nxt = cur.next
19         for _ in range(k - 1):           # rotate nxt to the
20             front
21             cur.next = nxt.next
22             nxt.next = prev_group.next
23             prev_group.next = nxt
24             nxt = cur.next
25         prev_group = cur
26         remaining -= k
27     return dummy.next

```

$\Theta(n)$ time, $\Theta(1)$ space. The “rotate to front” formulation avoids a separate reverse-and-splice step.

P18. LRU cache dagger

SOLUTION

Hash map plus doubly linked list with sentinels — the full implementation is in Chapter 6. The two points an interviewer listens for: the map stores *nodes*, not values, so splicing is $\Theta(1)$; and sentinels remove every edge case.

P19. Implement a queue with two stacks

SOLUTION

Push onto `inbox`; when `outbox` is empty, pour everything across.

```

1  class Queue:
2      def __init__(self):
3          self.inbox, self.outbox = [], []
4
5      def push(self, x):
6          self.inbox.append(x)
7
8      def pop(self):
9          if not self.outbox:
10             while self.inbox:
11                 self.outbox.append(self.inbox.pop())
12             return self.outbox.pop()
```

Amortised $\Theta(1)$: each element is moved at most once. The potential function $\Phi = |\text{inbox}|$ gives the proof (Chapter 2).

P20. Merge k sorted lists dagger**SOLUTION**

Heap of size k (pattern 15): $\Theta(N \log k)$.

```

1  import heapq
2
3  def merge_k_sorted(lists):
4      h = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
5      heapq.heapify(h)
6      out = []
7      while h:
8          val, li, idx = heapq.heappop(h)
9          out.append(val)
10         if idx + 1 < len(lists[li]):
11             heapq.heappush(h, (lists[li][idx + 1], li, idx + 1))
12     return out
```

Merging pairwise instead would be $\Theta(Nk)$; divide-and-conquer pairwise merging is also $\Theta(N \log k)$ and uses no heap.

P21. Find the median of a data stream dagger

SOLUTION

Two heaps, balanced (Chapter 13). `push` is $\Theta(\log n)$, `median` is $\Theta(1)$.

```

1  import heapq
2
3  class MedianFinder:
4      def __init__(self):
5          self.lo, self.hi = [], []          # max-heap (negated),
6              min-heap
7
8      def add(self, x):
9          heapq.heappush(self.lo, -heapq.heappushpop(self.hi, x))
10         if len(self.lo) > len(self.hi):
11             heapq.heappush(self.hi, -heapq.heappop(self.lo))
12
13     def median(self):
14         if len(self.hi) > len(self.lo):
15             return float(self.hi[0])
16         return (self.hi[0] - self.lo[0]) / 2.0

```

P22. Task scheduler with cooldown

Given task counts and a cooldown k , find the minimum number of time slots.

SOLUTION

A closed form beats simulation. The most frequent task determines the skeleton: $(\max - 1)$ full frames of length $k+1$, plus the tasks tying for the maximum.

```

1  from collections import Counter
2
3  def least_interval(tasks, k):
4      counts = Counter(tasks)
5      mx = max(counts.values())
6      ties = sum(1 for v in counts.values() if v == mx)
7      return max(len(tasks), (mx - 1) * (k + 1) + ties)

```

$\Theta(n)$. The `max` with `len(tasks)` handles the case where there are enough distinct tasks to fill every gap.

Trees and recursion**P23. Validate a binary search tree dagger**

SOLUTION

The classic wrong answer compares each node only with its children. The property is about entire subtrees, so carry bounds down.

```

1  def is_bst(node, lo=float("-inf"), hi=float("inf")):
2      if node is None:
3          return True
4      if not (lo < node.val < hi):
5          return False
6      return (is_bst(node.left, lo, node.val)
7              and is_bst(node.right, node.val, hi))

```

$\Theta(n)$. An in-order traversal checking that values strictly increase is an equally valid and equally short alternative.

P24. Lowest common ancestor**SOLUTION**

If both targets are in different subtrees, the current node is the answer.

```

1  def lca(node, p, q):
2      if node is None or node is p or node is q:
3          return node
4      left = lca(node.left, p, q)
5      right = lca(node.right, p, q)
6      if left and right:
7          return node                # p and q split here
8      return left or right

```

$\Theta(n)$. For repeated queries, preprocess with binary lifting for $\Theta(\log n)$ per query.

P25. Serialise and deserialise a binary tree**SOLUTION**

Pre-order with explicit null markers is sufficient and reconstructs the shape exactly.


```

1  def serialise(node, out=None):
2      out = [] if out is None else out
3      if node is None:
4          out.append("#")
5      else:
6          out.append(str(node.val))
7          serialise(node.left, out)
8          serialise(node.right, out)
9      return ",".join(out)
10
11 def deserialise(data, Node):
12     it = iter(data.split(","))
13     def build():
14         tok = next(it)
15         if tok == "#":
16             return None
17         node = Node(int(tok))
18         node.left = build()
19         node.right = build()
20         return node
21     return build()

```

$\Theta(n)$. Pre-order *without* null markers is not enough — it does not determine the shape.

P26. Diameter of a binary tree

SOLUTION

The answer at each node is `height(left) + height(right)`; compute heights and the best diameter in one pass.

```

1  def diameter(root):
2      best = 0
3      def height(node):
4          nonlocal best
5          if node is None:
6              return 0
7          l, r = height(node.left), height(node.right)
8          best = max(best, l + r)
9          return 1 + max(l, r)
10     height(root)
11     return best

```

$\Theta(n)$. Computing the height separately inside the recursion would be $\Theta(n^2)$ — a common mistake.

Dynamic programming and greedy

P27. Coin change (minimum coins) dagger

SOLUTION

Unbounded knapsack: iterate the amount *upwards*.

```

1 def coin_change(coins, amount):
2     INF = float("inf")
3     dp = [0] + [INF] * amount
4     for a in range(1, amount + 1):
5         for c in coins:
6             if c <= a and dp[a - c] + 1 < dp[a]:
7                 dp[a] = dp[a - c] + 1
8     return -1 if dp[amount] == INF else dp[amount]
```

$\Theta(\text{amount} \times |\text{coins}|)$. Greedy is wrong for general denominations — `coins = [1, 3, 4]`, `amount = 6` gives 4+1+1 greedily and 3+3 optimally.

P28. 0/1 knapsack dagger**SOLUTION**

Rolling array, inner loop *descending* so each item is used once.

```

1 def knapsack(weights, values, W):
2     dp = [0] * (W + 1)
3     for w, v in zip(weights, values):
4         for c in range(W, w - 1, -1): # DESCENDING
5             dp[c] = max(dp[c], v + dp[c - w])
6     return dp[W]
```

$\Theta(nW)$ time, $\Theta(W)$ space. Ascending would solve the unbounded version instead.

P29. Longest increasing subsequence dagger**SOLUTION**

The $\Theta(n \log n)$ version maintains `tails[k]` = the smallest possible tail of an increasing subsequence of length $k+1$.

```

1 from bisect import bisect_left
2
3 def lis(a):
4     tails = []
5     for x in a:
6         i = bisect_left(tails, x)
7         if i == len(tails):
8             tails.append(x)
9         else:
10            tails[i] = x
11    return len(tails)
```

`tails` is *not* an LIS — it is a set of best-possible tails — so recovering the actual subsequence needs a parent array alongside it.

P30. Jump game II

Minimum jumps to reach the end, where `a[i]` is the maximum jump from `i`.

SOLUTION

Greedy with a BFS-level interpretation: each “level” is the set of indices reachable in the same number of jumps.

```
1 def min_jumps(a):
2     jumps = end = farthest = 0
3     for i in range(len(a) - 1):
4         farthest = max(farthest, i + a[i])
5         if i == end:                                # exhausted the current
6             level
7             jumps += 1
8             end = farthest
9     return jumps
```

$\Theta(n)$. Seeing it as BFS on an implicit graph explains why greedy is correct here while it fails on the related “minimum coins” problem.

CHAPTER RECAP

- Every solution above came from the same procedure: brute force, identify the waste, choose the structure that removes it.
- The most frequent waste in this set was a repeated scan, removed by a hash map, a window, or a prefix sum.
- The second most frequent was excess ordering: needing the best k and computing a total order. Heaps and quickselect fixed it.
- Boundary handling — storing after checking, sentinels, infinities, not advancing after a swap — is where most incorrect submissions fail.
- Reimplement each solution from memory a day later. That is the exercise.

Problem Set II: Advanced and ML-Flavoured

Twenty-eight harder problems, most of them from real systems

PRACTICE

PREREQS: Chapters 22--42

The second problem set covers graphs, advanced dynamic programming, and the algorithms of Parts VI through VIII. Many of these are not interview questions — they are problems you will actually be handed, phrased the way they arrive. Solutions include the design reasoning, not just the code.

WHAT YOU WILL LEARN

- Solve graph, DP and systems problems end to end
- Translate a vague production requirement into a precise algorithmic one
- Choose data structures under real memory and latency budgets
- Justify approximation where exactness is unaffordable

Graphs

P31. Course schedule dagger

Given prerequisites, return a valid order or report a cycle.

SOLUTION

Kahn's algorithm; the length check is the cycle detector (Chapter 24).

```

1  from collections import deque, defaultdict
2
3  def course_order(n, prereqs):
4      adj, indeg = defaultdict(list), [0] * n
5      for course, need in prereqs:
6          adj[need].append(course)
7          indeg[course] += 1
8      q = deque(i for i in range(n) if indeg[i] == 0)
9      order = []
10     while q:
11         v = q.popleft()
12         order.append(v)
13         for u in adj[v]:
14             indeg[u] -= 1
15             if indeg[u] == 0:
16                 q.append(u)
17     return order if len(order) == n else []

```

$\Theta(V+E)$.

P32. Number of islands dagger**SOLUTION**

Connected components on an implicit grid graph. BFS/DFS is $\Theta(mn)$; union-find is the same asymptotically but wins when the grid is *streamed* or cells are added over time.

```

1  def num_islands(grid):
2      if not grid:
3          return 0
4      m, n, count = len(grid), len(grid[0]), 0
5      for i in range(m):
6          for j in range(n):
7              if grid[i][j] != "1":
8                  continue
9              count += 1
10             stack = [(i, j)]
11             grid[i][j] = "0"                # mark on push
12             while stack:
13                 r, c = stack.pop()
14                 for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)):
15                     nr, nc = r + dr, c + dc
16                     if 0 <= nr < m and 0 <= nc < n and grid[nr][nc]
17                         == "1":
18                         grid[nr][nc] = "0"
19                         stack.append((nr, nc))
19     return count

```

P33. Word ladder

Shortest transformation sequence between two words of equal length.

SOLUTION

BFS on an implicit graph. The key optimisation is the wildcard index: build buckets keyed by "h*t" so neighbours are $\Theta(L)$ to find instead of $\Theta(|\text{wordlist}| \cdot L)$.

```

1  from collections import deque, defaultdict
2
3  def word_ladder(begin, end, wordlist):
4      words = set(wordlist)
5      if end not in words:
6          return 0
7      buckets = defaultdict(list)
8      for w in words | {begin}:
9          for i in range(len(w)):
10             buckets[w[:i] + "*" + w[i+1:]].append(w)
11      seen, q = {begin}, deque([(begin, 1)])
12      while q:
13          w, d = q.popleft()
14          if w == end:
15              return d
16          for i in range(len(w)):
17              for nb in buckets[w[:i] + "*" + w[i+1:]]:
18                  if nb not in seen:
19                      seen.add(nb)
20                      q.append((nb, d + 1))
21      return 0

```

Bidirectional BFS roughly squares the reachable depth (Chapter 22).

P34. Network delay time

Time for a signal from one node to reach all others.

SOLUTION

Dijkstra; the answer is the maximum settled distance, or -1 if any node is unreachable.

```

1  import heapq
2  from collections import defaultdict
3
4  def network_delay(times, n, src):
5      adj = defaultdict(list)
6      for u, v, w in times:
7          adj[u].append((v, w))
8      dist = {}
9      pq = [(0, src)]
10     while pq:
11         d, v = heapq.heappop(pq)
12         if v in dist:
13             continue
14         dist[v] = d
15         for u, w in adj[v]:
16             if u not in dist:
17                 heapq.heappush(pq, (d + w, u))
18     return max(dist.values()) if len(dist) == n else -1

```

$\Theta((V+E)\log V)$.

P35. Cheapest flight with at most k stops

SOLUTION

Not Dijkstra — the constraint is on hop count, so the state is (node, hops). Bellman–Ford limited to $k+1$ rounds is cleaner and correct.

```

1  def cheapest_flight(n, flights, src, dst, k):
2      INF = float("inf")
3      dist = [INF] * n
4      dist[src] = 0
5      for _ in range(k + 1):
6          nxt = dist[:]                # copy: use only k-hop
7          for u, v, w in flights:
8              if dist[u] + w < nxt[v]:
9                  nxt[v] = dist[u] + w
10         dist = nxt
11     return -1 if dist[dst] == INF else dist[dst]
```

The copy is essential: without it, a path could use more than $k+1$ edges within one round.

P36. Accounts merge

Merge accounts sharing any email address.

SOLUTION

Union–find over accounts, keyed by email (Chapter 15).

```

1  def accounts_merge(accounts):
2      parent = list(range(len(accounts)))
3      def find(x):
4          while parent[x] != x:
5              parent[x] = parent[parent[x]]    # path halving
6              x = parent[x]
7          return x
8      owner = {}
9      for i, acc in enumerate(accounts):
10         for email in acc[1:]:
11             if email in owner:
12                 a, b = find(i), find(owner[email])
13                 parent[a] = b
14             else:
15                 owner[email] = i
16     groups = {}
17     for email, i in owner.items():
18         groups.setdefault(find(i), set()).add(email)
19     return [[accounts[i][0]] + sorted(e) for i, e in groups.items()]
```

This is exactly the deduplication-clustering pattern of Chapter 29.

P37. Alien dictionary

Infer the letter order from a sorted list of words.

SOLUTION

Build the constraint graph from adjacent word pairs, then topologically sort. The trap is the invalid case where a word is a strict prefix of its predecessor.

```

1  from collections import defaultdict, deque
2
3  def alien_order(words):
4      adj = defaultdict(set)
5      indeg = {c: 0 for w in words for c in w}
6      for a, b in zip(words, words[1:]):
7          if len(a) > len(b) and a.startswith(b):
8              return "" # invalid input
9          for x, y in zip(a, b):
10             if x != y:
11                 if y not in adj[x]:
12                     adj[x].add(y)
13                     indeg[y] += 1
14             break
15     q = deque(c for c in indeg if indeg[c] == 0)
16     out = []
17     while q:
18         c = q.popleft()
19         out.append(c)
20         for nxt in adj[c]:
21             indeg[nxt] -= 1
22             if indeg[nxt] == 0:
23                 q.append(nxt)
24     return "".join(out) if len(out) == len(indeg) else ""

```

P38. Critical connections (bridges)

Find every edge whose removal disconnects the graph.

SOLUTION

DFS with discovery times and low-link values. An edge (u, v) is a bridge when $\text{low}[v] > \text{disc}[u]$ — no back edge from v 's subtree reaches u or above.


```

1 def critical_connections(n, connections):
2     adj = [[] for _ in range(n)]
3     for u, v in connections:
4         adj[u].append(v)
5         adj[v].append(u)
6     disc, low, out = [-1] * n, [0] * n, []
7     timer = [0]
8     def dfs(u, parent):
9         disc[u] = low[u] = timer[0]
10        timer[0] += 1
11        for v in adj[u]:
12            if v == parent:
13                continue
14            if disc[v] == -1:
15                dfs(v, u)
16                low[u] = min(low[u], low[v])
17                if low[v] > disc[u]:
18                    out.append([u, v])
19            else:
20                low[u] = min(low[u], disc[v])
21        for i in range(n):
22            if disc[i] == -1:
23                dfs(i, -1)
24        return out

```

$\Theta(V+E)$. Note `low[u] = min(low[u], disc[v])` — not `low[v]` — for back edges; that is the classic bug.

Advanced dynamic programming

P39. Edit distance with traceback dagger

See Chapter 19 for the full derivation. The interview version wants the $\Theta(nm)$ table; the production version wants the banded variant and Myers' bit-parallel algorithm.

P40. Regular expression matching

Support `.` and `*`.

SOLUTION

Two-sequence DP where `*` means “zero occurrences” or “one more occurrence”.

```

1  def is_match(s, p):
2      m, n = len(s), len(p)
3      dp = [[False] * (n + 1) for _ in range(m + 1)]
4      dp[0][0] = True
5      for j in range(1, n + 1):
6          if p[j - 1] == "*":
7              dp[0][j] = dp[0][j - 2]          # x* matches empty
8      for i in range(1, m + 1):
9          for j in range(1, n + 1):
10             if p[j - 1] == "*":
11                 zero = dp[i][j - 2]
12                 more = dp[i - 1][j] and p[j - 2] in (s[i - 1], ".")
13                 dp[i][j] = zero or more
14             else:
15                 dp[i][j] = (dp[i - 1][j - 1]
16                             and p[j - 1] in (s[i - 1], "."))
17     return dp[m][n]

```

$\Theta(nm)$ — and note this is *why* an automaton-based matcher is $\Theta(nm)$ worst case with no backtracking (Chapter 5).

P41. Burst balloons

Maximise `nums[i-1] * nums[i] * nums[i+1]` summed over a bursting order.

SOLUTION

Interval DP with the reversal that makes it work: think about which balloon is burst **last** in an interval, because then its neighbours are the interval's boundaries.

```

1  def max_coins(nums):
2      a = [1] + list(nums) + [1]
3      n = len(a)
4      dp = [[0] * n for _ in range(n)]
5      for length in range(2, n):
6          for lo in range(0, n - length):
7              hi = lo + length
8              for k in range(lo + 1, hi):
9                  dp[lo][hi] = max(dp[lo][hi],
10                                   dp[lo][k] + a[lo] * a[k] * a[hi] +
11                                   dp[k][hi])
12     return dp[0][n - 1]

```

$\Theta(n^3)$. Thinking “first” instead of “last” gives a recurrence with non-local dependencies and does not work — a good illustration that the *choice of last decision* is the creative step in DP.

P42. Word break II

Return every sentence formed by splitting a string into dictionary words.

SOLUTION

Memoised recursion on the suffix, returning lists of completions. Bottom-up tabulation is awkward here because the output is exponential — the top-down version only explores reachable states.

```

1  from functools import lru_cache
2
3  def word_break(s, words):
4      wordset = frozenset(words)
5      @lru_cache(maxsize=None)
6      def go(i):
7          if i == len(s):
8              return [""]
9          out = []
10         for j in range(i + 1, len(s) + 1):
11             w = s[i:j]
12             if w in wordset:
13                 for rest in go(j):
14                     out.append(w if not rest else w + " " + rest)
15         return out
16     return go(0)

```

Add a feasibility pre-pass (a simple $\Theta(n^2)$ word-break DP) to avoid exponential work on strings with no valid split at all.

P43. Best time to buy and sell stock with k transactions

SOLUTION

Two-state DP over (day, transactions, holding).

```

1  def max_profit(prices, k):
2      if not prices or k == 0:
3          return 0
4      n = len(prices)
5      if k >= n // 2:                                     # unlimited: take
6          every rise
7          return sum(max(0, prices[i + 1] - prices[i]) for i in range(
8              n - 1))
9      buy = [float("-inf")] * (k + 1)
10     sell = [0] * (k + 1)
11     for p in prices:
12         for t in range(1, k + 1):
13             buy[t] = max(buy[t], sell[t - 1] - p)
14             sell[t] = max(sell[t], buy[t] + p)
15     return sell[k]

```

$\Theta(nk)$ time, $\Theta(k)$ space. The $k \geq n/2$ shortcut avoids a pointless $\Theta(n^2)$ when k is large.

P44. Longest palindromic substring

SOLUTION

Expand around each of the $2n-1$ centres: $\Theta(n^2)$ with $\Theta(1)$ space, and much faster in practice than the DP table. Manacher's algorithm achieves $\Theta(n)$ by reusing previously computed radii — the same “remember what you matched” idea as KMP (Chapter 30).

```

1  def longest_palindrome(s):
2      if not s:
3          return ""
4      best = (0, 1)
5      for centre in range(len(s)):
6          for lo, hi in ((centre, centre), (centre, centre + 1)):
7              while lo >= 0 and hi < len(s) and s[lo] == s[hi]:
8                  lo -= 1
9                  hi += 1
10             if hi - lo - 1 > best[1] - best[0]:
11                 best = (lo + 1, hi)
12     return s[best[0]:best[1]]

```

Systems and ML-flavoured problems

P45. Design a rate limiter

Allow N requests per user per minute, with a memory budget of 100 bytes per user and 10^7 users.

SOLUTION

A sliding-window log is exact and costs $\Theta(N)$ timestamps per user — too much. Two better options:

Token bucket. Per user store `(tokens: float32, last_refill: uint32)`: 8 bytes. On a request, refill by `elapsed * rate`, cap at the burst size, and spend one token if available. $\Theta(1)$ time, exact for the rate, approximate for the exact window boundary.

Sliding-window counter. Store the counts for the current and previous minute and interpolate: `estimate = prev * (1 - t) + cur`. 12 bytes, $\Theta(1)$, and within a few percent of exact.

```

1  import time
2
3  class TokenBucket:
4      __slots__ = ("rate", "burst", "tokens", "last")
5      def __init__(self, rate, burst):
6          self.rate, self.burst = rate, burst
7          self.tokens, self.last = float(burst), time.monotonic()
8
9      def allow(self, n=1.0):
10         now = time.monotonic()
11         self.tokens = min(self.burst, self.tokens + (now -
12             self.last) * self.rate)
13         self.last = now
14         if self.tokens >= n:
15             self.tokens -= n
16             return True
17         return False

```

Across a fleet, either shard users by consistent hashing so each user has one owner (Chapter 41), or accept per-replica limits at $1/R$ of the global rate.

P46. Deduplicate a 10-billion-document corpus on one machine

SOLUTION

Exact dedup with 16-byte digests needs 400 GB — it does not fit. The pipeline (Chapters 28, 29, 15):

1. **Exact duplicates:** a Bloom filter at 10 bits per document is 12.5 GB for a 1% false-positive rate, which discards 1% of unique documents — acceptable. Or, better, sort 8-byte digests externally (Chapter 42): two passes, exact, 80 GB on disk.
2. **Near-duplicates:** MinHash with $K=128$, banded into $b=16$ bands of $r=8$ (threshold ≈ 0.69). Process one band at a time as an external sort by band hash, emitting candidate pairs from equal runs — no in-memory index at all.
3. **Verify** candidates with the signature estimate; **cluster** with union-find over a 4-byte parent array (40 GB for 10^{10} documents — so shard by connected component or process in blocks).
4. **Cap cluster size** to avoid transitive over-merging.

Cost: roughly 200 CPU-hours for signatures plus 16 external-sort passes. State explicitly what the 1% false-positive rate costs you: 1% of unique documents dropped, which is a fair trade for a 30x memory reduction.

P47. Serve top- k retrieval over 500M vectors, 64 GB RAM, p99 < 20 ms

SOLUTION

Memory arithmetic first (Chapter 33). At $d=1024$, float32 is 2 TB: out of the question.

- **PQ with $m=64$, 8 bits:** 64 bytes per vector = 32 GB. Fits, leaving 32 GB.
- **IVF with $nlist = \sqrt{5 \times 10^8} \approx 22,000$** centroids: $22K \times 4 \text{ KB} = 90 \text{ MB}$, negligible.
- **HNSW would need $\approx 8Mn$ bytes = 128 GB** at $M=32$: does not fit alongside the codes. So IVF-PQ, not HNSW.
- **Query:** probe $nprobe = 32$ cells, about $32 \times 22,700 = 727,000$ candidates, each 64 table lookups and adds: about 46 million operations, a few milliseconds with SIMD.
- **Rerank:** fetch the top 200 full-precision vectors from an SSD-backed store (200 random reads, issued concurrently: about 1 ms at depth 32) and rescore exactly.
- **Budget:** 2 ms embed + 5 ms search + 1 ms rerank fetch + 1 ms rescore, leaving headroom for p99 jitter.
- **Validate:** measure recall@10 against brute force on 10,000 sampled queries; tune $nprobe$ until recall meets target, then re-check latency.

P48. Bound KV-cache memory for a serving fleet

Given a model and a GPU, how many concurrent sequences can you serve?

SOLUTION

Chapter 36's formula. For $L=80$, $H_{kv}=8$, $d_h=128$, fp16: $4 \times 80 \times 8 \times 128 = 327,680$ bytes per token, so 1.34 GB at 4096 tokens.

On an 80 GB device holding a 40 GB shard of weights, 35 GB remains after activations and workspace: **26 sequences** at full context. With 16-token paging and a realistic length distribution (mean 800 tokens), the *average* sequence needs 262 MB, so about 130 concurrent sequences — which is the argument for paging stated numerically. Add int8 KV quantisation for another 2x.

P49. Detect training–test contamination**SOLUTION**

Two-stage (Chapter 30). Stage 1: hash every 13-gram of every test example into a Bloom filter (a few GB), stream the training corpus past it, and collect hits. Stage 2: for each hit, verify exactly against the test set and measure the fraction of each test example covered by matches of at least 50 tokens.

Report: the fraction of test examples with any match, the distribution of coverage, and — crucially — the metric computed on the *clean* subset alongside the full-set number. Specify whether you matched tokens or characters; the numbers are not comparable otherwise.

P50. Choose a sampler for a changing curriculum

Weights over 50 million examples, updated after every epoch, sampled 10 million times per epoch.

SOLUTION

Weights change once per epoch, not per draw. So the alias table's $\Theta(n)$ rebuild is amortised over 10^7 draws — 50 million operations per epoch to build, 10^7 constant-time draws. That is the right choice.

If instead weights changed after *every draw* (prioritized replay), the rebuild would cost 5×10^{14} operations per epoch and you would need a Fenwick tree: $\Theta(\log n) \approx 26$ operations per draw and per update. Stating which regime you are in is the entire decision (Chapters 27, 34).

P51. Diagnose a training job at 40% GPU utilisation**SOLUTION**

Chapter 42's procedure, in order: time the dataloader alone; if it is slower than the step, it is the ceiling. Then time each stage. Then do the arithmetic — decoded bytes per second required versus what your CPU count can produce.

The common findings, in rough order of frequency: too few dataloader workers; JPEG decode on CPU (move to GPU with DALI/nvJPEG); small files on network storage (re-shard); no `pin_memory` or no separate copy stream; a synchronising call such as `.item()`, `.cpu()` or `torch.nonzero` inside the step; and `torch.compile` graph breaks. “Buy a bigger GPU” is on none of these lists.

P52. Design a feature store point-in-time join

Training rows are `(entity, label_time, label)`; features are `(entity, feature_time, value)`. Join each label with the most recent feature *strictly before* its label time, over 10^9 rows.

SOLUTION

This is an as-of join, and doing it wrong leaks future information into training — one of the most damaging bugs in applied ML.

Sort both sides by `(entity, time)`, then one merge pass with two pointers (Chapter 11): for each label, advance the feature pointer while `feature_time < label_time` and take the last one passed. Both sorts are external (Chapter 42), so the whole job is $\Theta(n \log_{M/B} n)$ — two passes each — plus one linear merge.

The correctness details that matter: strict inequality (a feature computed *at* the label time may already encode the label); a maximum staleness bound so a year-old feature is treated as missing; and identical logic in the serving path, which is what training-serving skew means in practice.

P53. Bound the memory of a streaming metrics service

Compute p50/p95/p99 latency per endpoint per minute, for 10,000 endpoints, across 200 replicas.

SOLUTION

Per endpoint per minute, a t-digest with compression 100 is about 100 centroids of 16 bytes: 1.6 KB. For 10,000 endpoints that is 16 MB per replica per minute — trivially affordable.

The property that makes it work is mergeability (Chapter 38): each replica keeps its own digests and the aggregator merges 200 of them, giving the exact same answer as a global computation would, with no global sort and no data movement beyond the digests.

Storing raw latencies instead would be $\Theta(\text{requests})$ and would require a global sort per minute.

P54. Pick a parallelism strategy

A 70B model, 64 GPUs with NVLink within 8-GPU nodes and 100 Gbit/s between nodes.

SOLUTION

Chapter 41. Weights plus optimiser state at $16P$ bytes is 1.1 TB, so the model cannot be data-parallel alone.

- **Tensor parallel within a node** (8-way): two all-reduces per layer, high volume, and NVLink at 400+ GB/s can carry it. Across nodes it could not.
- **Pipeline parallel across nodes** (say 4 stages): only boundary activations cross the slow link, which is small. Use enough micro-batches that the bubble fraction $(S-1)/(M+S-1)$ is under 10%: $S=4$ needs $M \geq 28$.
- **Data parallel across the remaining 2 replicas**, with ZeRO/FSDP sharding the optimiser state to fit.

The principle: the highest-volume communication rides the fastest link. State the bytes per step for each option and the decision makes itself.

CHAPTER RECAP

- Graph problems are almost always one of: BFS, Dijkstra, topological sort, union-find, or DFS with low-link values.
- Advanced DP is about choosing the right *last decision*; burst balloons is the canonical example of a problem that only works backwards.
- Systems problems are answered with arithmetic first: bytes, operations, and the budget. The data structure follows from the numbers.
- State what an approximation costs you, in the units the user cares about.
- Point-in-time correctness, contamination checks and train-serve consistency are algorithmic problems with correctness consequences far larger than a latency regression.

Study Plan and Reference

Sixteen weeks, and the tables worth keeping open

REFERENCE

PREREQS: none

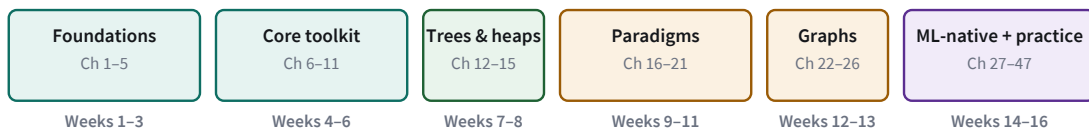
The last chapter is a plan and a reference. The plan is what this book is optimised for — sixteen weeks at four to six hours, half of them at a keyboard. The reference is the set of tables you will come back to long after you have read the prose: complexities, memory formulas, Python costs, and the cost model of a machine learning system.

WHAT YOU WILL LEARN

- Follow a week-by-week plan through the whole book
- Use the complexity tables as a working reference
- Estimate the memory and time cost of an ML system from its parameters
- Find the chapter that covers whatever you are currently fighting

The plan

A 16-WEEK PLAN AT 4–6 HOURS PER WEEK



Half the time on the keyboard. Every chapter: read, implement one structure from memory, do three exercises.

Interview sprint (6 weeks): Ch 2, 4, 6–13, 16–20, 22–24, then Part IX.

Reference use: Ch 2 and 3 once, properly; then jump to whatever you are fighting.

Figure 47.1 Sixteen weeks. The proportions reflect how much practice each part needs, not how many pages it has.

Weeks	Chapters	Deliverable each week
1	1–2	Measure your machine's ops/sec; complexity-annotate a file of your own code
2	3–4	Reproduce the cache-cliff experiment; convert a loop to NumPy and measure
3	5–6	Write a tokenizer trie; implement a ring buffer and an LRU from memory

Weeks	Chapters	Deliverable each week
4	7–8	Implement a hash map with resizing; convert three recursions to iteration
5	9–10	Implement merge, quick, radix sort; write <code>lower_bound</code> from memory
6	11 + P1–P10	Ten foundation problems, timed
7	12–13	Implement a BST and a binary heap; solve top- <i>k</i> four ways and benchmark
8	14–15	Build a trie tokenizer and a union–find dedup clusterer
9	16–17	Apply the master theorem to ten recurrences; prove two greedy algorithms
10	18	Five dynamic programs, each written top-down then bottom-up
11	19–21	Implement edit distance, Viterbi and reservoir sampling
12	22–24	Build a CSR graph; implement BFS, Dijkstra, topological sort, a tiny autograd
13	25–26	Implement a matching loss with <code>linear_sum_assignment</code> ; implement PageRank
14	27–31	Build a Fenwick tree, a Bloom filter and a MinHash deduplicator
15	32–36	Build an IVF-PQ index; implement beam search and a block-paged KV manager
16	37–42 + P31–P54	Histogram split finder; parallel scan; profile a real dataloader

PRACTICE | Three rules that make the plan work

Implement from memory. Read the chapter, close it, and write the structure. Getting stuck is the signal that you understood it less than you thought — and it is the moment the learning happens.

Keep a bug log. One line per bug: what it was, what the symptom was, and what would have caught it. After a month it becomes a personal checklist, and it is worth more than any generic list — including the ones in this book.

Measure something every week. A number you measured yourself is worth ten you read. The habit of reaching for a benchmark before an opinion is the most transferable thing this book teaches.

Complexity of data structures

Structure	Access	Search	Insert	Delete	Ordered?	Space	Chapter
Array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	no	n	4
Dynamic array	$\Theta(1)$	$\Theta(n)$	$\Theta(1)^*$	$\Theta(n)$	no	n	4
Sorted array	$\Theta(1)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	yes	n	10

Structure	Access	Search	Insert	Delete	Ordered?	Space	Chapter
Linked list	$\Theta(n)$	$\Theta(n)$	$\Theta(1)^{\text{ddagger}}$ ger	$\Theta(1)^{\text{ddagger}}$ ger	no	$2n$	6
Deque	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$ ends	$\Theta(1)$ ends	no	n	6
Hash table	—	$\Theta(1)^*$	$\Theta(1)^*$	$\Theta(1)^*$	no	$\approx 1.4n$	7
Binary heap	$\Theta(1)$ min	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$	partial	n	13
Balanced BST	—	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	yes	n	12
Skip list	—	$\Theta(\log n)^*$	$\Theta(\log n)^*$	$\Theta(\log n)^*$	yes	$\approx 2n$	31
B+ tree	—	$\Theta(\log_B n)$	$\Theta(\log_B n)$	$\Theta(\log_B n)$	yes	$\approx 1.5n$	12, 31
Trie	—	$\Theta(m)$	$\Theta(m)$	$\Theta(m)$	yes	shared prefixes	14
Union–find	—	$\Theta(\alpha(n))$	$\Theta(\alpha(n))$	—	no	n	15
Fenwick tree	—	$\Theta(\log n)$	$\Theta(\log n)$	—	prefix	n	27
Segment tree	—	$\Theta(\log n)$	$\Theta(\log n)$	—	range	$2n-4n$	27
Bloom filter	—	$\Theta(k)$	$\Theta(k)$	no	no	$1.44n \log_2 1/p$ bits	28
LSM tree	—	$\Theta(L)$ levels	$\Theta(1)$	$\Theta(1)$	yes	$\approx 1.1n$	31
HNSW	—	$\Theta(\log n)$ approx	$\Theta(\log n)$	tombstone	no	$\approx 8Mn$ bytes	33

Table 47.1 A week-by-week plan at four to six hours per week. The deliverable column is the point: reading without implementing produces recognition, not recall.

Complexity of algorithms

Algorithm	Best	Average	Worst	Space	Chapter
Binary search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$	10
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	9
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	9
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$	9
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(1)$	13

Algorithm	Best	Average	Worst	Space	Chapter
Timsort	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	9
Radix sort	$\Theta(dn)$	$\Theta(dn)$	$\Theta(dn)$	$\Theta(n+b)$	9
Quickselect	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$	$\Theta(1)$	9
BFS / DFS	—	$\Theta(V+E)$	$\Theta(V+E)$	$\Theta(V)$	22
Dijkstra (heap)	—	$\Theta((V+E) \log V)$	same	$\Theta(V)$	23
Bellman–Ford	$\Theta(E)$	$\Theta(VE)$	$\Theta(VE)$	$\Theta(V)$	23
Floyd–Warshall	$\Theta(V^3)$	$\Theta(V^3)$	$\Theta(V^3)$	$\Theta(V^2)$	23
Kruskal	—	$\Theta(E \log E)$	same	$\Theta(V)$	23
Topological sort	—	$\Theta(V+E)$	same	$\Theta(V)$	24
Dinic max-flow	—	$\Theta(V^2 E)$	same	$\Theta(V+E)$	25
Hungarian	—	$\Theta(n^3)$	same	$\Theta(n^2)$	25
Edit distance	—	$\Theta(nm)$	same	$\Theta(\min(n, m))$	19
Viterbi	—	$\Theta(TS^2)$	same	$\Theta(TS)$	19
CTC forward	—	$\Theta(TS)$	same	$\Theta(TS)$	19
KMP	—	$\Theta(n+m)$	same	$\Theta(m)$	30
Aho–Corasick	—	$\Theta(n + \sum m_i + z)$	same	$\Theta(\sum m_i)$	14
Suffix array (SA-IS)	—	$\Theta(n)$	same	$\Theta(n)$	30
FFT	—	$\Theta(n \log n)$	same	$\Theta(n)$	16
Matrix multiply	—	$\Theta(n^3)$	same	$\Theta(n^2)$	16
Strassen	—	$\Theta(n^{2.807})$	same	$\Theta(n^2)$	16
PageRank	—	$\Theta(E)$ / iteration	same	$\Theta(V)$	26

Table 47.2 * expected or amortised. ^{ddagger} given a reference to the node.

Python cost reference

Operation	Cost	Note
<code>x in list</code>	$\Theta(n)$	the most common accidental quadratic
<code>x in set/dict</code>	$\Theta(1)^*$	use this
<code>list.append</code>	$\Theta(1)^*$	amortised
<code>list.insert(0, x) / pop(0)</code>	$\Theta(n)$	use <code>deque</code>
<code>deque.appendleft / popleft</code>	$\Theta(1)$	
<code>list.sort()</code>	$\Theta(n \log n)$	Timsort, stable, adaptive

Operation	Cost	Note
<code>heapq.heappush / heappop</code>	$\Theta(\log n)$	min-heap only
<code>heapq.nlargest(k, xs)</code>	$\Theta(n \log k)$	in C
<code>s += t</code> in a loop	$\Theta(n^2)$	use <code>"".join</code>
<code>a[i:j]</code> (list or str)	$\Theta(j-i)$	allocates
<code>bisect.*</code>	$\Theta(\log n)$	on a sorted list
<code>np.argsort</code>	$\Theta(n \log n)$	pass <code>kind="stable"</code> for ties
<code>np.argpartition</code>	$\Theta(n)$	top- <i>k</i> without full ordering
<code>np.searchsorted</code>	$\Theta(m \log n)$	batched binary search
<code>dict</code> memory, int keys	≈ 100 B/entry	switch to arrays past 10^7
<code>np</code> array memory	<code>dtype</code> bytes/entry	8 for int64, 4 for float32
Python statement	$\approx 30\text{--}100$ ns	$\approx 10^7/\text{s}$
NumPy float add	≈ 0.3 ns	$\approx 3 \times 10^9/\text{s}$

Table 47.3 Reference complexities. “Same” means the average and worst coincide.

Hardware cost reference

Event	Latency	Relative
L1 cache hit	1 ns	1
Branch mispredict	5 ns	5
L2 cache hit	4 ns	4
L3 cache hit	15 ns	15
Main memory	90 ns	90
NVMe random read	80 μs	80,000
Same-datacentre round trip	500 μs	500,000
Disk seek (HDD)	5 ms	5,000,000
Cross-continent round trip	150 ms	150,000,000

Table 47.4 The Python numbers worth memorising. The last two lines explain most performance surprises in ML code.

ML system cost formulas

THE MATH | The formulas worth keeping**Model memory** (mixed-precision training, Adam), P parameters:

$$M \approx 16P \text{ bytes} = 2P(\text{bf16 weights}) + 2P(\text{grads}) + 12P(\text{fp32 master} + m + v)$$

KV cache, L layers, H_{kv} key/value heads, head dim d_h , batch B , length S , fp16:

$$\text{bytes} = 4 \cdot B \cdot S \cdot L \cdot H_{kv} \cdot d_h$$

Training FLOPs (Kaplan/Chinchilla approximation), N tokens:

$$C \approx 6PN(2PN \text{ forward} + 4PN \text{ backward})$$

Inference FLOPs per token: $\approx 2P$, plus attention $\approx 4LSd$.**Arithmetic intensity**: $I = \text{FLOPs/bytes moved}$; memory-bound if $I < \text{peak FLOP/s/}$ bandwidth (about 100–500).**Ring all-reduce bytes per worker**: $2(P-1)/P \cdot N$, independent of P .**Pipeline bubble fraction**: $(S-1)/(M+S-1)$ for S stages, M micro-batches.**Bloom filter**: $m = 1.44 n \log_2(1/p)$ bits, $k = (m/n) \ln 2$.**HyperLogLog**: standard error $1.04/\sqrt{m}$ registers.**MinHash**: standard error $\leq 1/(2\sqrt{K})$; LSH threshold $\approx (1/b)^{1/r}$.**Johnson–Lindenstrauss**: $m = O(\epsilon^{-2} \log n)$ dimensions preserve all pairwise distances.

The pitfalls index

Symptom	Likely cause	Chapter
Fine in dev, unusable in prod	invisible quadratic (<code>in list</code> , <code>+=</code> on strings)	2, 7
Same complexity, 10x slower	access pattern, not operation count	3
Out of memory in a distance computation	materialised broadcast	4
Dataloader results change with <code>num_workers</code>	view aliasing, or per-worker seeds	4, 21
Nondeterministic rankings	unstable sort, no tiebreaker	9
<code>RecursionError</code> on real data	unbalanced tree or deep list	8, 12
Dict of 100M keys killed by OOM	Python object overhead	7
Sharding reshuffles on restart	using <code>hash()</code> on strings	7
Sliding window gives wrong answers	negative values break monotonicity	11
Vector search recall silently poor	unnormalised vectors, or too few probes	32, 33
Deleting from an ANN index degrades it	tombstones, needs rebuild	33
Negative variance in a dashboard	naive $E[x^2] - E[x]^2$	38
Training hangs at an all-reduce	unequal shards per rank	42
GPU at 40% utilisation	input pipeline, not the model	42
Loss differs between runs with a fixed seed	float reduction order	39
Benchmark scores suspiciously high	test-set contamination	30
Model memorises verbatim	training-data duplication	29

Table 47.5 Latency numbers, normalised to an L1 hit. The two orders of magnitude between memory and NVMe, and the three between NVMe and a network hop, are the boundaries that structure every system.

Where to go next

- **For rigour:** Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, for proofs this book states; Kleinberg and Tardos, *Algorithm Design*, for the best treatment of exchange arguments and network flow.
- **For competitive practice:** the Competitive Programmer's Handbook, and daily problems. The patterns of Chapter 44 map onto them directly.
- **For systems:** Kleppmann, *Designing Data-Intensive Applications*, which is the operational counterpart to Part VI.
- **For the ML-native material:** the primary papers. FAISS, HNSW, DiskANN, ScaNN for retrieval; vLLM and SGLang for serving; FlashAttention and its successors for kernels; LightGBM and XGBoost for trees. They are readable, and by now you have the vocabulary.
- **For the habit:** pick one system you use — a vector database, a dataloader, a tokenizer — and read its source until you can name every data structure in it. That exercise, done three or four times, is worth more than any further reading.

CHAPTER RECAP

- Sixteen weeks, four to six hours a week, half of them at a keyboard. Implement from memory, keep a bug log, and measure something every week.
- The complexity tables are for reference; the ML formulas are for estimating before building.
- The latency table's boundaries — memory to NVMe to network — are what structure every system in this book.
- The pitfalls index maps symptoms to causes; most production problems in ML systems are on it.
- The most durable exercise is to read the source of a system you already use and name every structure in it.